Abstract

Building Knowledge-Enhanced AI Co-Scientists for Biomedical Discovery

Tianyu Liu
2026

Given the fast development of technology and automation systems, researchers are generating tons of data from scientific experiments to facilitate discoveries, especially in the areas of biology and medicine. However, large-scale biomedical data make it challenging for biologists or medical researchers to more effectively utilize these data with robust and powerful methods for novel and reproducible discoveries. Therefore, I pioneered the research for building Knowledge-Enhanced AI Co-Scientist System to overcome these challenges in biomedicine. In Chapter 1, I will show the importance of having a fair evaluation framework to rank the contributions of foundation models (FMs) and AI Scientists for biological data analysis and demonstrate the importance of prior knowledge. In Chapter 2, I will show the strength of leveraging knowledge to gain insights in analyzing multi-modal data for clinical and biomedical applications by building AI Bioinformaticians and biologists. Finally, in Chapter 3, I will discuss our progress of building a self-evolving AI Bioengineer for accelerating scientific discoveries which facilitate generation tasks in multiple areas to enclose this robust and extensive system.

Building Knowledge-Enhanced AI Co-Scientists for Biomedical Discovery

A Dissertation
Presented to the Faculty of the Graduate School
of
Yale University
in Candidacy for the Degree of
Doctor of Philosophy

by
Tianyu Liu

Dissertation Director: Hongyu Zhao

May 2026

Copyright © 2026 by Tianyu Liu
All rights reserved.

Acknowledgments

When I was a undergraduate, due to a complex set of circumstances, I once lost interests in research and no longer believed it could become part of my career. However, Prof. Hongyu Zhao truly changed everything. I am deeply grateful that he accepted me as his summer interns and further doctoral students, and has been willing to provide me with the most generous support and the best guidance. Not only did he provide me with the utmost support in my research direction and the greatest freedom to explore, but he also took a keen interest in my personal life and character development. He taught me how to become a researcher of integrity and rigor while maintaining a deep sense of humanistic care. I am deeply grateful to Prof. Zhao and delighted to have chosen him as my mentor. He is my role model, for sure.

I would also like to thank my committee members, including Prof. Rex Ying and Prof. Yuval Kluger. They also provided me with tremendous support during my doctoral studies. Prof. Ying supported me in publishing my first paper at a top-tier machine learning conference (NeurIPS) and provided me with the most generous support during my job search stage. Prof. Kluger's insightful questions always prompt me to think more deeply, and his humor is an indispensable part of our meetings.

I'm also delighted to be working alongside my labmates at Zhao Lab. Dr. Ning Sun is just like our parents in lab. She helps me a lot in developing a sound scientific research ethos. At the same time, her questions always pinpointed the pain points in my model design and spurred me to keep thinking. She has contributed so much to our lab, and I am truly grateful to her. Besides, I am thankful to my collaborators in the lab, including Dr. Yuge Wang (who also supervised me during my summer internship and worked as a perfect mentor), Dr. Jia Zhao, Dr. Gefei Wang, Dr. Lijun Wang, Dr. Yingxin Lin, Dr. Biqing Zhu, Dr. Tinyi Chu, Wangjie Zheng, Leqi Xu, Jiaqi Hu, Xiangyu Zhang, Lisa Chen, Yueqian Jing, Yiming Shi, Feiyang Wen, and Haoran Shao. I have collaborated with them on multiple projects, ranging from integrating single-cell multi-omic data to modeling electronic health records using language models. Their experience and knowledge are invaluable assets, and I am deeply grateful for their generous support in both research and life. I also extend my gratitude to the other members of our lab and wish everyone a bright future.

My co-authors have also been an integral part of my doctoral journey, and I am deeply grateful for their support and many of them become my good friends. Without them, I would not have been able to achieve what I have today. First and foremost, I would like to thank Prof. Xiao Luo and Prof. Wengong Jin. They are not only my reliable mentors in research but also great friends in life, embodying a genuine relationship of collaborators and friends. I would also like to thank my collaborators at Yale, such as Prof. Le Zhang, Prof. Hua Xu, Prof. Qingyu Chen, Tinglin Huang

(who is also a great player in League of Legends, along with Weikang), Allen Wang, etc. as well as outside Yale, including Yijia Xiao, Haixin Wang, Fan Zhang, Dr. Artur Szalata, Tong Ding, Chuhan Li, Dr. Simeng Han, Dr. Hanchen Wang, Dr. Kexin Huang, Prof. James Zou, all teammates in SAGA and Agent4BioMed, etc. I would also like to thank my mentors from industry, including Dr. Aviv Regev and Dr. Graham Heimberg. I spent three enjoyable months at Genentech, largely thanks to their excellent guidance.

I also thank my interns and mentees, including Wenxin Long, Zhiyuan Cao, Kexing Li, Sihan Jiang, and other outstanding young researchers. I feel fortunate to have worked alongside them and witnessed their growth. Many of them are still collaborating with me, which is what touches me most deeply.

Finally, I would like to thank my family for supporting my decision to pursue a doctoral degree. This has been a bumpy path, but they have always been by my side, giving me their strongest support. I am fortunate to have them along with me throughout my life. I love you all!

This is also for my grandma. I love her forever.

Contents

1 Evaluating the utilities of foundation models in single-cell data analysis 1
1.1 Introduction . . . 2
1.2 Results . . . 3
1.3 Discussion . . . 24
1.4 Methods . . . 28
1.5 Data availability and reproducibility . . . 36
1.6 Supplementary Figures . . . 38

2 Leveraging Multi-Modal Foundation Models for Analyzing Spatial Multi-Omic and Histopathology Data 83
2.1 Introduction . . . 84
2.2 Results . . . 86
2.3 Discussion . . . 97
2.4 Methods . . . 99
2.5 Tables . . . 106
2.6 Figure Legends . . . 106
2.7 Supplementary Figures . . . 113

3 Accelerating Scientific Discovery with Autonomous Goal-evolving Agents 131
3.1 Introduction . . . 133
3.2 Results . . . 135
3.2.1 SAGA for Antibiotic Design . . . 135
3.2.2 SAGA for Inorganic Materials Design . . . 138
3.2.3 SAGA for Functional DNA Sequence Design . . . 141
3.2.4 SAGA for Chemical Process Design . . . 144
3.3 Discussion . . . 147
3.4 Methods . . . 149

3.4.1 SAGA Framework 149
3.4.2 Task Configurations 151
3.4.3 Task Evaluations 153
S5 Implementation Details 155
S5.1 Framework Design 155
S5.2 Execution Workflow 158
S5.3 Experimental Setups 159
S5.4 Extensibility and Customization 160
S5.5 SAGA Reduces to the Optimizer agent (SAGA -Opt) 160
S6 Antibiotic Discovery 161
S6.1 Supplementary Figures 161
S6.2 Experimental Setups 165
S6.3 Additional Experimental Results 168
S7 Inorganic Materials Design 169
S7.1 Experimental Setups 169
S7.2 Material Property Evaluation 172
S8 Functional DNA Sequence Design 174
S8.1 Supplementary Figures 174
S8.2 Experimental Setups 178
S8.3 Additional Experimental Results 181
S9 Chemical Process Design 182
S9.1 Supplementary Figures 182
S9.2 Experimental Setups 185
S9.3 Challenges in chemical process design 187

List of Figures

1.1 Overview of single-cell FMs, landscape of scEval and factors affecting single-cell FMs. (a): Overview of single-cell FMs describing the typical structure of FMs and general tasks for single-cell data analysis. The right two blocks represent two types of downstream tasks. Yellow block: Sub-task 1, including Cell-type Annotation and Gene Function Prediction (top to bottom). Blue block: Sub-task 2, including Batch Effect Correction, Multi-omic Data Integration, Imputation (From left to right, top row), Perturbation Prediction, Gene Network Analysis, and Simulation (From left to right, bottom row). (b): The landscape of scEval shows the workflow of our systematic evaluation. Here we consider models based on fine-tuning design with settings S_1^f to S_n^f, and they have corresponding evaluation pipelines E_1^f to E_n^f. We also consider models based on zero-shot-learning design with settings S_1^z to S_m^z, and they have corresponding evaluation pipelines E_1^z to E_m^z. The reason for using the above notations to represent the process of evaluation is that different models can perform different tasks, while different tasks have different evaluation scenarios. (c): Factors which can affect the performance of single-cell FMs. These factors are summarized based on different designs of single-cell FMs. The known factors can be classified into four different types. Details of hyper-parameters can be found in original appendix. Details of initial settings can be found in original appendix. 4

1.2 Experimental results of single-cell FMs and benchmarking methods for cell-level tasks. (a): An overall assessment of raw data (The "Raw" label in this figure) and data after batch effect correction based on different methods. scGPT v1 represents the scGPT model with smaller pre-training datasets compared to scGPT. (b): An overall assessment of raw data and data after multi-omic data integration based on different methods. (c): Comparison among models in the Cell-type Annotation task. The scores on the left represent the average accuracy of different models across different datasets. (d): Ablation tests of the loss function components for cell-type annotation. The red components are significant (one-sided Wilcoxon Rank-sum Test, $p - value = 0.03, n = 11$). The panels (a) and (b) follow the default settings of funkyheatmap [174] for visualization. 7

1.3 Experimental results of single-cell FMs and benchmarking methods for gene-level tasks. (a): Comparisons among Geneformer, scGPT, and *vanilla* NN in the Gene Function Prediction task. (b): The effect of hyper-parameters including Loss weight, Bins, and Learning rate for scGPT and Geneformer in the Gene Function Prediction task (sample size $n = 3$ per box). (c): Correlation metrics of different methods. A higher correlation means lower rank and better performance. The numbers corresponding to the settings represent the average value across two datasets. (d): Dataset-level gene embeddings from scGPT colored by the marker genes of different cell types. (e): Dataset-level gene embeddings from Geneformer colored by the marker genes of different cell types. 14

1.4 Different comparison groups for model scaling analysis and stability analysis. (a): The model scale of different methods, defined by the number of parameters (unit: Million). (b): Accuracy of FMs and *vanilla* NN in Cell-type Annotation task. The dataset here is the Pancreas cross dataset. (c): Accuracy of FMs and SATURN in Cell-type Annotation task across species. (d): Overall score comparison including ResPAN, Novae and different settings of scGPT. The dataset here is the human spatial transcriptomic dataset. (e): Different batch correction scores of different models based on changing random seeds (left) and different average classification scores of different models based on changing random seeds (right). The bold black line represents the median value, while the length of each box can be interpreted as the variance level (sample size $n = 10$ per box). 22

1.6.1 Workflow of our evaluation. Here we considered ten single-cell FMs and generated their outputs based on their default usage settings. We also consider the evaluation of user accessibility for both open-source and closed-source tools. We then evaluated the outputs with task-specific metrics. Finally, we summarized our discoveries in this manuscript. 38
1.6.2 Statistics of pre-training datasets for different single-cell FMs. Here we consider summarizing the scale of pre-training datasets, the overlap for major diseases in the pre-training datasets, the overlap for major species in the pre-training datasets, and the overlap of multi-omics in the pre-training datasets, across all the methods. The scale of pre-training dataset is a continuous variable, whether other statistics are recorded based on a binary variable. 39
1.6.3 The overlap of major human tissues or organs across different single-cell FMs' pre-training datasets. Here we choose tissues or organs included by Geneformer as a baseline because it is published and it includes most of the major information for the cells from human. *: For scFoundation, we only record the major tissues or organs based on its manuscript since we did not find the data sources description from its manuscript and supplementary files. For CellPLM, we did not record the major tissues or organs since we did not find the data source description from its manuscript and supplementary files. . . . 40
1.6.4 S_{final} across different scFMs based on the zero-shot setting. 41
1.6.5 Compassion of zero-shot setting and fine-tuning setting of scGPT. (a) Comparison based on S_{final} for batch effect correction. (b) Comparison based on PCC for perturbation prediction. 42
1.6.6 Visualization of batch effect removal score and biology variation conservation score in the evaluation of batch effect correction task for each dataset. 43
1.6.7 Comparisons for different methods on preserving the trajectory information based on the trajectory score. 44
1.6.8 UMAPs for raw data and embeddings of scGPT after batch effect correction. 44
1.6.9 UMAPs for raw data and embeddings of scGPT after batch effect correction. 45
1.6.10 Benchmarking results for different hyper-parameters for batch effect correction (one-sided Wilcoxon Rank-sum Test, $p-value = 0.03, n = 11$). 46

1.6.11 Overall evaluation of different components of scGPT. (a): The final score of different settings across different datasets for batch effect correction. (b): The final score of different optimizers across different datasets for batch effect correction. (c): The final score of including different loss function terms across different datasets. The number of stars represents the significance level (one sided Wilcoxon Rank-Sum test, sample size $n = 12$, $***: p - value < 0.005, **: p - value < 0.05, *: p - value < 0.1$). The numbers on the left side of each sub-figure represent the average score across different datasets for one condition. We combine the datasets for batch effect correction and multi-omic data integration for testing. (d): The final score of different settings across different datasets for multi-omic data integration. (e): The final score of different optimizers across different datasets for multi-omic data integration. 47
1.6.12 Accuracy of different scFMs (zero-shot performances) for cell-type annotation. 48
1.6.13 Visualization of batch effect removal score and biology variation conservation score in the evaluation of multi-omic data integration task for each dataset. 49
1.6.14 UMAPs for raw data and embeddings of scGPT after multi-omic data integration. 50
1.6.15 Tuning parameters for multi-omic data integration. Sub-figures represent the score of scGPT under different hyper-parameters after training (sample size $n = 4$). 51
1.6.16 Results of cell-type annotation with the different modes of scGPT and Geneformer. (a) The performance of cell-type annotation of scGPT based on both fine-tuning mode and zero-shot learning mode. (b) The performance of cell-type annotation of Geneformer based on both fine-tuning mode and zero-shot learning mode. 52
1.6.17 UMAPs for ground truth cell types and prediction results based on scGPT. 53
1.6.18 UMAPs for ground truth cell types and prediction results based on scGPT. 54
1.6.19 Weighted F1 score across different methods for cell-type annotation. . 55
1.6.20 Tuning parameters for cell-type annotation. Sub-figures represent the score of scGPT under different hyper-parameters after training (sample size $n = 4$). 56

1.6.21 Results of different settings, running time, and memory usage for cell-type annotation task. (a): Accuracy of scGPT and scBERT for the Cell-type Annotation task across different datasets. (b): Scaled running time (up) and scaled memory usage (down) statistics for all three single-cell FMs. 57
1.6.22 Results of cell-type annotation with the different number of freezing layers based on scGPT. The x-axis represents the number of freezing layers and the y-axis represents the value of specific metrics (sample size $n = 4$). 58
1.6.23 Benchmarking results for different optimizers for cell-type annotation. Here each box contains the scores of fine-tuned scFMs across different datasets (sample size $n = 10$). 59
1.6.24 Tuning hyper-parameters for gene function prediction. Sub-figures represent the score of scGPT under different hyper-parameters after training (sample size $n = 3$). 60
1.6.25 Benchmarking results of different initial settings, optimizers, and loss components for gene function prediction. T1-T3 represent different gene prediction cases. (a): Results of adjusting initial settings. (b): Results of adjusting optimizers. (c): Ablation tests based on different loss components. 61
1.6.26 Comparison based on the zero-shot settings of different scFMs. (a) PCC based on the results from different scFMs across three datasets. (b) MSE based on the results from different scFMs across three datasets. 62
1.6.27 MSE across different models to evaluate the performances for perturbation prediction. The maximal value for display is 1 for better visualization, while the true MSE of tGPT is larger than 1e5 for both Norman dataset and Dixit dataset. scFoundation meets the OOT error. 63
1.6.28 Analysis of models for perturbation prediction based on DEGs. (a) MPC across different models to evaluate the performances for perturbation prediction. The averaged scores are reported in the figure. (b) MSE across different models to evaluate the performances for perturbation prediction. 64
1.6.29 Heatmap of $-log(p-value)$ to support the statistical analyses results of the perturbation prediction task. We conducted one-sided Wilcoxon Rank sums test based on different metrics, including (a) PCC (including both all genes and DEGs) and (b) MSE (including both all genes and DEGs), to examine the comparisons of different models. We mark the significance threshold 0.05 and corresponding $-log(p-value)$. . 65

1.6.30 MPC across different fine-tuned-based models and settings of scGPT to evaluate the performances for perturbation prediction. 66
1.6.31 Tuning hyper-parameters for perturbation prediction. Sub-figures represent the score of scGPT under different hyper-parameters after training (sample size $n = 3$). 67
1.6.32 Benchmarking results of different optimizers for perturbation prediction. 68
1.6.33 Examples of GCN inference results for the Immune Human Atlas (IHA) dataset. (a): Dataset-level gene embeddings from scGPT colored by Leiden clusters. (b): Dataset-level gene embeddings from Geneformer colored by Leiden clusters. (c): Cell-type-level gene embeddings from scGPT colored by the cell types. (d): Leiden cluster results based on the cell-type-level gene embeddings from scGPT. (e): Cell-type-level gene embeddings from Geneformer colored by the cell types. We omitted plotting the Leiden cluster results of Geneformer because of the plotting size limit. (f): Comparison of significant pathways ratio between scGPT and Geneformer for CD3-related gene sets. (g): An example of GCN for IHA based on scGPT. It is a network with CD3-related genes as major nodes. (h): An example of GCN for the IHA dataset based on Geneformer. 69
1.6.34 Results of overlap ratio by adjusting resolution based on the gene embeddings from (a) scGPT and (b) Geneformer. 70
1.6.35 Detailed pathway information. (a) The pathway enrichment information from scGPT. (b) The pathway enrichment information from Geneformer. 71
1.6.36 Examples of dataset-level GCNs from Geneformer based on HVGs. (a) Dataset-level gene embeddings from Geneformer colored by sources of marker genes. (b) Dataset-level gene embeddings from Geneformer colored by Leiden clusters. 72

1.6.37 Experimental results of the Imputation task and the Simulation task. The number of stars represents the significance level (one-sided Wilcoxon Rank-sum test, sample size $n = 8$, $***: p - value < 0.005, **: p - value < 0.05$). (a): Comparison of the average bio score between the raw data and imputed data by scGPT in the scRNA-seq imputation task. (b): Comparison of the average bio score, average correlation score, and average significance level score among Tangram, CellPLM, scGPT, and scGPT (zero-shots) in spatial transcriptomics imputation task. (c): Comparison of the average bio score among scDesign3, Splatter, and scGPT for simulation. (d): Gene-gene correlation heatmap from the raw HumanPBMC dataset. We select the subset of the top 100 highly variable genes. (e): Comparison of different simulation methods by correlation. The heatmap represents the top 100 highly variable genes (for raw and scDesign3) or the subset of the top 100 highly variable genes (for scGPT) based on the HumanPBMC dataset. The correlation "r" represents the Pearson correlation between the gene correlation of raw data and the gene correlation of simulation data. The white section represents NaN values caused by the problematic model outputs. 73
1.6.38 Deferentially expressed gene discovery based on results before imputation and after imputation. We used the Mouse scRNA-seq and the Mouse spatial transcriptomic datasets as examples. (a): Deferentially expressed genes by cell types for scRNA-seq data based on pre-imputation data (top) and post-imputation data (bottom). (b): Deferentially expressed genes by cluster types for spatial transcriptomic data based on pre-imputation data (top), post-imputation data based on zero-shot learning (middle), and post-imputation data based on fine-tuning (bottom). (c): Deferentially expressed genes by cluster types for spatial transcriptomic data based on Tangram. 74
1.6.39 Heatmaps for the gene-gene correlation of datasets simulated by Splatter, using the HumanPBMC dataset as an example. The correlation "r" represents the Pearson correlation between the gene correlation of raw data and the gene correlation of simulation data. (a): The heatmap for the simulation dataset with batch effect. (b): The heatmap for the simulation dataset without batch effect. 75
1.6.40 UMAPs for the simulation results with batch effect, using the HumanPBMC dataset as an example. 76
1.6.41 UMAPs for the simulation results without batch effect, using the HumanPBMC dataset as an example. 77
1.6.42 Proportion of orthologous genes between specified species and human. 78

1.6.43 Heatmap of $-log(p-value)$ to support the statistical analyses results of the stability examination. We conducted one-sided Wilcoxon Rank sums test based on different tasks, including (a) batch effect correction and (b) cell-type annotation, to examine the comparisons of different models. We mark the significance threshold 0.05 and corresponding $-log(p-value)$. 79
1.6.44 The number of stars in GitHub or likes in Huggingface for different single-cell FMs. 80
1.6.45 Preliminary results of parameter-efficient fine-tuning based on LoRA for Batch Effect Correction (the upper two figures) and Cell-type Annotation (the bottom two figures). (a): The comparison of scores for scGPT with/without LoRA. (b): The running time for scGPT with-/without LoRA. (c): The comparison of scores for scGPT with/without LoRA. (d): The running time for scGPT with/without LoRA. 81
1.6.46 Heatmaps for the ranks of different models across different datasets. (a): The heatmap of ranks based on the first method, known as adjusting the weights and computing the ranks. (b): The heatmap of ranks based on the second method, known as using the default weights for S_{bio} and S_{batch}. 82

2.6.1 The landscape of spEMO. (a) The zero-shot embedding framework of spEMO. Here we introduce embeddings from PFMs and LLMs to help the analysis of spatial multi-omic datasets, by transferring the selected dataset from the expression space to the embedding space. (b) The renewed framework with an expert-model-involved setting of spEMO. By integrating the image representations and expression profiles to domain-expert methods, spEMO is capable of performing more downstream applications. 106

2.6.2 Understanding the contribution of spEMO for spatial domain identification. (a) The evaluation results of different methods for domain identification of spatial transcriptomic datasets. Here spEMO represents the combination: SpaGCN+GPFM (n=12 per group; triangle, mean; center line, median; box limits, upper and lower quartiles; whiskers, up to 1.5×interquartile range; points, outliers). (b) The evaluation results of different PFMs as base models for domain identification of spatial transcriptomic datasets (n=12 per group; triangle, mean; center line, median; box limits, upper and lower quartiles; whiskers, up to 1.5×interquartile range; points, outliers). (c) The visualization of clustering performance based on the selected slide. The upper panel is colored by cortical layers, and the lower panel is colored by clustering labels. (d) The evaluation results of different settings for capturing spatial auto-correlated features (n=12 per group; triangle, mean; center line, median; box limits, upper and lower quartiles; whiskers, up to 1.5×interquartile range; points, outliers). 107
2.6.3 Clustering scores and visualization of embeddings colored by different metadata. (a) UMAP visualization based on the different embeddings colored by disease states. (b) UMAP visualization based on different embeddings colored by tissues. (c) Averaged clustering scores computed based on disease states (left panel) and tissues (right panel) across different embeddings. The direction of arrows indicates embeddings with a better performance. (d) Average rank of different PFMs by balancing the discrimination of diseases and tissues. 108
2.6.4 Results of utilizing image embeddings for possibly improving disease-state prediction. (a) Comparisons of different classifiers under the SampleCV mode using selected metrics (n=3 per group; triangle, mean; center line, median; box limits, upper and lower quartiles; whiskers, up to 1.5×interquartile range; points, outliers). (b) Comparisons of different input settings under the SampleCV mode using selected metrics. Here spEMO represents the combination: Pseudobulk+Virchow2+Oversam
(OS) (n=3 per group; triangle, mean; center line, median; box limits, upper and lower quartiles; whiskers, up to 1.5×interquartile range; points, outliers). (c) The confusion matrices of all validation sets under the SampleCV mode. (d) The C-index comparison across different input settings for survival prediction (n=5 per group; triangle, mean; center line, median; box limits, upper and lower quartiles; whiskers, up to 1.5×interquartile range; points, outliers). (e) The training loss curves of survival prediction between two modes with or without image embeddings. 109

2.6.5 The analysis of multicellular interaction based on predicted gene expression profiles. (a) The expression levels of spatially variable genes across the four samples. (b) The Leiden cluster labels computed based on gene expression profiles for sample TENX111. (c) The signal direction of cell-cell interaction CXCL13-CCR7 computed based on the sample TENX111. 110
2.6.6 Pipeline and results of generating medical reports with a multi-modal AI agent. (a) Workflow of using spEMO to generate medical reports and evaluate them from different perspectives. (b) Cosine similarity between the embeddings from generated reports with different base models and ground truth reports (n=991 per group; triangle, mean; center line, median; box limits, upper and lower quartiles; whiskers, up to 1.5×interquartile range; points, outliers). (c) BERT score between the generated reports with different base models and ground truth reports (n=991 per group; triangle, mean; center line, median; box limits, upper and lower quartiles; whiskers, up to 1.5×interquartile range; points, outliers). (d) MEDCON score between the generated reports and ground truth reports (n=991 per group; triangle, mean; center line, median; box limits, upper and lower quartiles; whiskers, up to 1.5×interquartile range; points, outliers). (e) Scores of human evaluation results under three different metrics between the human-generated reports and spEMO-generated reports ($n = 50$, standard deviation is annotated). (f) Scores of human evaluation results under three different metrics among generated reports with different input modality ($n = 50$, standard deviation is annotated). (g) Word cloud [151] of expert comments on the advantages of spEMO-generated reports in (e). (h) Word cloud of expert comments on the advantages of spEMO-generated reports with the Gene+Image mode in (f). The p-value is computed based on two-sided Wilcoxon rank-sum test. . . . 111

2.6.7 Evaluations and examples for the medical report generation task. The statistical test used here is two-side Wilcoxon Rank-sum test. (a) Evaluation results of completeness across different diseases. The statistics are: p-value=1.52e-5 for gene+image mode vs. image mode; p-value=7.63e-6 for gene+image mode vs. gene mode; p-value=7.63e-6 for gene mode vs. image mode. (b) Evaluation results of consistency for different modes. Each boxplot contains the PCCs from different disease (n=153 per group; triangle, mean; center line, median; box limits, upper and lower quartiles; whiskers, up to 1.5×interquartile range; points, outliers). The statistics are: p-value=1.21e-23 for gene+image mode vs. image mode; p-value=7.8e-27 for gene+image mode vs. gene mode; p-value=7.84e-27 for gene mode vs. image mode. (c) Medical report example from a COAD sample with gene+image mode. (d) Medical report example from a COAD sample with image mode. (e) Medical report example from a COAD sample with gene mode. . . . 112

1 Testing the contributions of image embeddings from different PFMs for spatial domain identification with the zero-shot inference mode (n=12 per group; triangle, mean; center line, median; box limits, upper and lower quartiles; whiskers, up to 1.5×interquartile range; points, outliers). Our metrics are NMI, ARI, ASW and their average value (Avg). 113

2 Testing the contributions of image embeddings from different PFMs with SpaGCN for spatial domain identification (n=12 per group; triangle, mean; center line, median; box limits, upper and lower quartiles; whiskers, up to 1.5×interquartile range; points, outliers). Our metrics are NMI, ARI, ASW and their average value (Avg). 115

3 Testing the contributions of the patch size of image used for generating embeddings. Here the selected PFM is GPFM (n=12 per group; triangle, mean; center line, median; box limits, upper and lower quartiles; whiskers, up to 1.5×interquartile range; points, outliers). Our metrics are NMI, ARI, ASW and their average value (Avg). 116

4 Testing the contributions of fine-tuning or zero-shot inference based on different PFMs for spatial domain identification (n=12 per group; triangle, mean; center line, median; box limits, upper and lower quartiles; whiskers, up to 1.5×interquartile range; points, outliers). Our metrics are NMI, ARI, ASW and their average value (Avg). 117

5 Spatial domain identification performances of the two Visium datasets from Squidpy. (a) Benchmarking results based on the Visium H&E dataset across different image features. (b) Benchmarking results based on the Visium fluo dataset across different image features. . . . 118

6 Results of spatial domain identification for spatial multi-omic data. (a) The evaluation results of different methods for domain identification of spatial multi-omic datasets. (b) The visualization of clustering performance based on the multi-omic slide. The left panel is colored by expert annotation, and the right panel is colored by clustering labels. 119
7 Comparisons of classifiers and settings for spot-type annotation (n=12 per group; triangle, mean; center line, median; box limits, upper and lower quartiles; whiskers, up to 1.5×interquartile range; points, outliers). (a) The evaluation of the performances of different classifiers. (b) The evaluation of the performances of different input settings based on LR. 120
8 Spot-type prediction performances of the two Visium datasets from Squidpy. (a) Benchmarking results by using Visium fluo as testing dataset across different input settings. (b) Benchmarking results by using Visium H&E as testing dataset across different input settings. . 121
9 UMAP visualization of sample embeddings generated by CHIEF. (a) Embeddings colored by disease states. (b) Embeddings colored by tissues. 122
10 Clustering performances across different labels and combinations. (a) Clustering performances of different PFMs and text embeddings based on disease labels. (b) Clustering performances of different PFMs and text embeddings based on tissue labels. 123
11 Embeddings from different resources colored by studies. (a) The UMAP based on embeddings from pseudo-bulk gene expression profiles. (b) The UMAP based on embeddings from PFMs. The study labels are shown below. 124
12 Boxplots with study-level cross-validation results for disease-state prediction and ablation studies for survival prediction. (a) Disease-state prediction performances by setting the cross-validation labels as studies (n=3 per group; triangle, mean; center line, median; box limits, upper and lower quartiles; whiskers, up to 1.5×interquartile range; points, outliers). (b) C-index for survival prediction based on different combinations (n=5 per group; triangle, mean; center line, median; box limits, upper and lower quartiles; whiskers, up to 1.5×interquartile range; points, outliers). 125

13 Comparisons between imbalance learning methods and raw methods. Here **raw** represents the default setting. We also tested clustering-based oversampling approach but met running errors due to small neighbor size (n=12 per group; triangle, mean; center line, median; box limits, upper and lower quartiles; whiskers, up to 1.5×interquartile range; points, outliers). (a) Classification results based on the SampleCV setting across different sampling methods. (b) Classification results based on the StudyCV setting across different sampling methods. 126
14 Comparisons between major voting methods and raw methods. Here **raw** represents the default setting (n=3 per group; triangle, mean; center line, median; box limits, upper and lower quartiles; whiskers, up to 1.5×interquartile range; points, outliers). (a) Classification results based on the SampleCV setting across different sampling methods. (b) Classification results based on the StudyCV setting across different sampling methods. 127
15 Comparisons between gene expression profiles and spEMO for understanding drug response for individuals. (a) Clustering results based on different input settings. (b) Prediction results based on different input settings. 128
16 Correlations of text embeddings based on the descriptions generated with different random seeds under the gene+image mode. We consider 10 different seeds for COAD report generation. 129
17 Comparison between expert-generated reports and spEMO-generated reports across five samples from different diseases based on (a) Cosine similarity in the embedding space, (b) BERT score, and (c) MEDCON score. 130

1 *The SAGA framework and the examples of scientific applications.* (a) Scientists constantly suffer from reward hacking issues, where optimization agents exploit the approximation error of objective functions and propose undesirable solutions with good scores. (b) Finding optimal objectives that bypass reward hacking issues is difficult due to the large search space of objectives and their relative weights. (c) We propose the SAGA framework to automatically discover optimal objectives and candidate solutions through a bi-level procedure. (d) SAGA operates at three different levels of automation, allowing scientists to steer the objective discovery process in various ways. (e) We apply SAGA to scientific design tasks related to chemistry, biology, and materials. (f) The SAGA framework is capable of implementing different objective functions across disciplines. 132

1 *Results for antibiotic design.* (a) Comparisons between SAGA and language model baselines. Candidates from all SAGA modes achieve the "drug likeness and activity sweet spot", whereas baselines struggle to balance biological objectives, especially KP activity, with chemical assessments like Drug likeness and Synthesizability. (b) Comparisons across SAGA iterations. Text annotations highlight specific agent feedback on objective evolution that drives the improvement in metric scores across iterations. The solid line means objectives address the evaluation metrics, and the dash line means the metric has not been addressed. (c) Distribution of drug likeness score. Most molecules from SAGA surpass the drug likeness threshold (red dashed line), while AlphaEvolve falls below it, demonstrating its critical misalignment with final objectives. (d) An example of the autopilot feedback loop. The analyzer identifies issues and the planner dynamically evolves objectives, shifting the distribution of the Top 100 Candidate more to the Desired Region of high activity and drug likeness. (e) T-SNE plot of SAGA molecules against known antibiotics. Numbered structures (1-4) are examples of molecules passing all evaluation metrics (Figure S6.2) that are likely function well in experiments. They contain novel scaffolds distinct from known antibiotic clusters. 136

2 *Results for inorganic materials design.* (a) Property distributions of generated structures from co-pilot across different iterations and from MatterGen (single) targeting only high magnetic density. (b) Number of stable and novel structures satisfying property requirements found by co-pilot and MatterGen (joint) within 200 DFT property calculations, for targets with magnetic density above 0.2 $Å^{-3}$ and HHI score below 1500. It also displays 3D visualizations of two crystal structures proposed by the co-pilot mode that satisfy the design goal. (c) Comparisons between different levels of SAGA and selected baselines on the design task of superhard materials for precision cutting. All evaluation metrics are normalized, with higher scores representing better performance. (d) Comparison of different SAGA modes over three iterations with the same held-out metrics. In each iteration, SAGA analyze the optimized crystal structures, propose new objectives, run property optimization, and select the best candidates across all current iterations. Text annotations highlight specific agent feedback on objective evolution that drives the improvement in metric scores across iterations. The solid line means objectives address the evaluation metrics, and the dash line means the metric has not been addressed. (e) An example of the autopilot feedback loop. SAGA identifies issues and dynamically evolves objectives, successfully proposed novel structures exhibiting high hardness, high elastic modulus, and thermodynamic stability. 139

3 *Results for functional sequence design.* (a) Comparisons between different levels of SAGA and selected baselines with evaluation metrics (both average score (higher is better) and scaled standard error are reported (lower is better)). Our selected task is to design HepG2 (a cell line of epithelial-like cells from liver) specific enhancers. (b) The comparisons of different iterations of two different levels with the same held-out metrics. Each iteration will create new objectives. The solid line means objectives address the evaluation metrics, and the dash line means the metric has not been addressed. (c) An example of SAGA to correct the issues in previous iterations. (d) UMAP visualization of enhancers designed by SAGA , SAGA-Opt, and from references. (e) HepG2-specific motif visualization. 142

4 *Results for chemical separation process design.* (a) Comparisons between different levels of SAGA and baseline RL (trained to maximize product purity) with evaluation metrics (higher is better). Our selected task is to design process flowsheets for separation of an azeotropic butanol/water mixture with varying feed compositions (between 2%/98%). (b) The comparisons of different iterations of the three SAGA levels ran for three iterations with the same evaluation metrics. In each iteration, SAGA will analyze the processes, create new objectives, run the RL optimization, and select the best candidates across all current iterations. (c) Exemplary designed process by baseline RL agent for separating a 50%/50% butanol-water mixture, demonstrating that maximizing the product purity only leads to full separation but also in applying unit operations that do not have an effect on the separation quality (marked in red), as the RL agent is not penalized for using unnecessary operations. (d) Workflow for using SAGA co-pilot with agent and user actions for two iterations, resulting in optimal process design for separating a 50%/50% butanol-water mixture. (e) Text description of an exemplary chemical process that is used by SAGA . 145
S5.1 The functional illustration of the four core modules. 156
S6.1 Additional benchmarks of antibiotic drug design. (a) Comparison of the property distributions of final candidates proposed by three SAGA levels and REINVENT4 [126]. (b) Pass rate for all final molecules proposed by SAGA and different baselines based on external evaluations. Definition of the property scores and pass rate can be found in Figure S6.2. 161
S6.2 Ablation studies of antibiotic design. (a) The change of scores from different objectives across all epochs in the optimization process, based on the optimizer only. (b) Examining the designed scorers based on random molecules ($n = 1000$). The output score correlation between human-implemented and implementer-implemented scoring functions. We report Pearson correlation, Spearman correlation and mean squared errors. 162
S6.3 Objectives proposed in early-stage iterations across three modes of SAGA 163
S6.4 Pass rate (proportion of molecules passing the metric-specific threshold) changes across iterations from different modes in SAGA 164
S7.1 Schematic diagram of the optimizer using LLM-based evolutionary algorithm. 170
S8.1 Illustration of analysis report generated by the analysis agent for two iterations. 174

S8.2 Objectives proposed in each iteration across three runs of the autopilot mode of SAGA 174
S8.3 Cross-modality prediction analysis. (a) Consistency between the MPRA and other sequence expression measurement, including CAGE-seq and DNase-seq. The prediction is performed with Enformer. (b) Comparison of predicted CAGE-seq and DNase-seq expression levels from the enhancers generated by two different methods. 175
S8.4 Comparison between SAGA and other baselines for (a) K562-specific enhancer design and (b) SKNSH-specific enhancer design. 176
S8.5 Ablation studies for functional DNA sequence design. (a) MPRA expression of generated DNA sequences across different levels in SAGA (b) Correlation between human-proposed HepG2 scoring function and implementer-proposed HepG2 scoring function. (c) Validation of using LLMs and evolutionary algorithm as the optimizer for optimizing the cell-type-specific expression levels. 177
S8.6 Comparison between SAGA and other baselines for promoter design across five different cell lines, and the variation is computed across five cell lines. The held-out metrics are consistent, while C1-C4 represents the -log(p-value) test result between the MPRA expression levels of the targeted cell line and the rest of cell lines. 178
S9.1 Different representations of an exemplary chemical process: (a) Visual representation that is created manually for illustration purposes; (b) matrix representation used within the RL optimizer [66], whereas the entries correspond to connection between the unit operations and product streams (colors correspond to (a); (c) text representation that is automatically generated based on (b) for use within SAGA ; (d) serialized process representation that is automatically generated from (b) and (c), and used to implement objectives by SAGA 183
S9.2 Exemplary plot for process candidate analysis automatically generated by SAGA 184

List of Tables

1.1 Overall comparisons of model capacity and performance. (a) Evaluations for the capacity of different single-cell FMs. We utilized different signs to denote the default settings we tested in scEval. Details are explained in the table legend. The icons are inspired by [244]. (b) Evaluations for both performance and usability of different single-cell FMs. We record the ranks of different methods across different tasks and user accessibility evaluations and compute the final overall rank. A lower rank means a better model. The top three methods are boldfaced. (c) User accessibility evaluation of different single-cell FMs. A smile face represents the selected single-cell FM can process the corresponding task by default. 3

1.2 Insights from the benchmarking results of scEval. We organize the table by tasks and model settings. 24

2.1 Evaluations of different PFMs for the multi-modal alignment task. Here in the bracket, we report the averaged metric scores and their standard deviation for each dataset, as well as a final average score computed based on all the datasets. The direction of the arrow indicates the relationship between higher scores and better (or lower) performance. ↓ means that a higher score corresponds to worse model performance, and ↑ means a higher score corresponds to better model performance. 114

Chapter 1

Evaluating the utilities of foundation models in single-cell data analysis

This paper is in press at Advanced Science
Liu, T., Li, K., Wang, Y., Li, H., and Zhao, H. (2023). Evaluating the utilities of foundation models in single-cell data analysis. bioRxiv, pages 2023–09. [124]

Abstract

Foundation Models (FMs) have made significant strides in both industrial and scientific domains. In this paper, we evaluate the performance of FMs for single-cell sequencing data analysis through comprehensive experiments across eight downstream tasks pertinent to single-cell data. Overall, the top FMs include scGPT, Geneformer, and CellPLM by considering model performances and user accessibility among ten single-cell FMs. However, by comparing these FMs with task-specific methods, we found that single-cell FMs may not consistently excel than task-specific methods in all tasks, which challenges the necessity of developing foundation models for single-cell analysis. In addition, we evaluated the effects of hyper-parameters, initial settings, and stability for training single-cell FMs based on a proposed **scEval** framework, and provide guidelines for pre-training and fine-tuning, to enhance the performances of single-cell FMs. Our work summarizes the current state of single-cell FMs, points to their constraints and avenues for future development, and offers a freely available evaluation pipeline to benchmark new models and improve method development.

1.1 Introduction

Single-cell sequencing technologies offer high-throughout observations into complex biological systems at the cell level with multimodal data [69, 176]. They help elucidate disease mechanisms and potential treatments [142, 198, 254]. In line with the central dogma of molecular biology, these technologies enable the characterization of various molecules, such as DNA (e.g. scDNA-seq) [54], RNA (e.g. scRNA-seq) [85, 260], and proteins (e.g. Cite-seq) [195]. Furthermore, single-cell sequencing can facilitate epigenetic studies, including chromatin accessibility (e.g. scATAC-seq) [35, 39] and methylation [135, 152]. These technologies have been rated as among the most impactful ones in recent years [57, 204].

With the development of single-cell technologies, expansive single-cell datasets have been collected, and they present challenges in integration, interpretation, and downstream utilization. Therefore, there is a need to build a Foundation Model (FM) which can benefit from prior knowledge for single-cell data analysis. In a similar motivation, natural language processing (NLP) also boasts extensive datasets, where FMs such as pre-trained Large Language Models (LLMs) have shown great success in addressing NLP tasks or multimodal tasks [259]. Numerous LLMs, including GPT-4 [156] and LLaMA [209], excel at diverse language-related tasks such as question answering and sentence generation, which has received widespread attention from both the AI community and society [206]. Moreover, these LLMs showcase impressive performance in zero-shot learning, thereby enabling them to address tasks beyond their original training scope, such as solving mathematical problems [86]. Similarly, a good FM in single-cell data analysis should also handle multiple tasks with a unified framework.

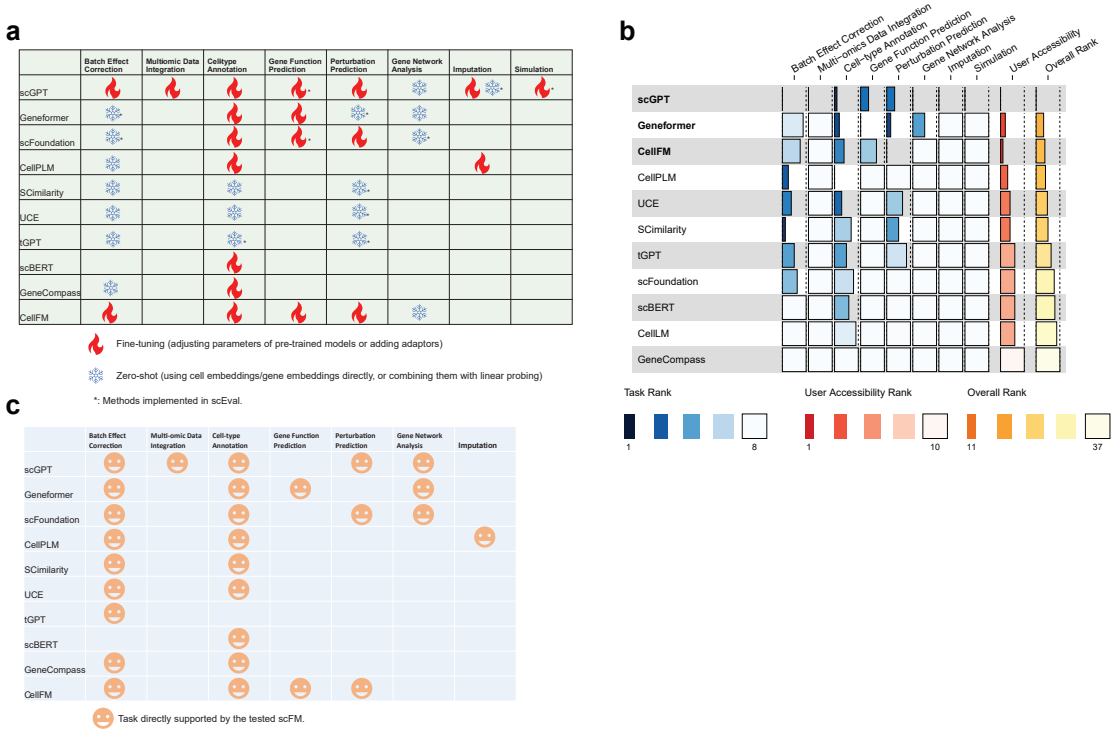
Indeed, notable parallels exist between studies based on single-cell sequencing data and those in NLP. Both fields leverage advanced technologies, such as transformer architectures, which have proven effective in handling a variety of tasks within each domain [29, 48, 107, 197]. Similarly, both disciplines emphasize the importance of analyzing intra-data and inter-data relationships, which in the context of single-cell analysis might involve interactions among genes or cells [7, 29, 104, 215]. Moreover, the concept of representation learning is central to both fields: effective embeddings of cells and genes can be developed using techniques analogous to those used for generating token embeddings in NLP [184]. Lastly, the success of both fields hinges on access to high-quality databases [76, 259], underscoring the need for careful data selection during model training. These synergies suggest that integrating methodologies from NLP may significantly advance the analysis and interpretation of single-cell data.

While FMs have seen marked success in the realms of DNA analysis [8] and biomedical NLP [189, 235], their application in single-cell research remains largely

uncharted. There is a limited number of robust pre-trained models (known as single-cell FMs) capable of managing multiple tasks in single-cell research. Some single-cell FMs focus on cell-type annotation or gene function prediction, including scBERT [240], CellLM [258], and Geneformer [205], while others aim to create an FM in this area that can handle multiple tasks, including scGPT [38], scFoundation [70], tGPT [186], GeneCompass [242], SCimilarity [74], UCE [171], CellFM [250], and CellPLM [225]. Details of these models can be found in original appendix. Furthermore, no studies to date have comprehensively evaluated the utility of these models and provided guidance for model training. Little has been done to compare NLP-focused LLMs with those used for single-cell research to gain insight into scaling laws and zero-shot (or few-shots) learning abilities of the latter.

Here we investigate the overlap of the pre-training stages of different single-cell FMs, and present a framework for assessing various single-cell FMs and tasks (shown in Figure 1.1 (a)), termed as Single-cell Large Language Model Evaluation (**scEval**), shown in Figure 1.1 (b). Using scEval, we not only compare different single-cell FMs across various datasets and tasks as a horizontal comparison but also identify critical parameters and strategies for the fine-tuning process of specific models as a vertical comparison. We also examine the potential contributions of model scaling of single-cell FMs, substantiating that the latter also possesses distinctive abilities.

1.2 Results



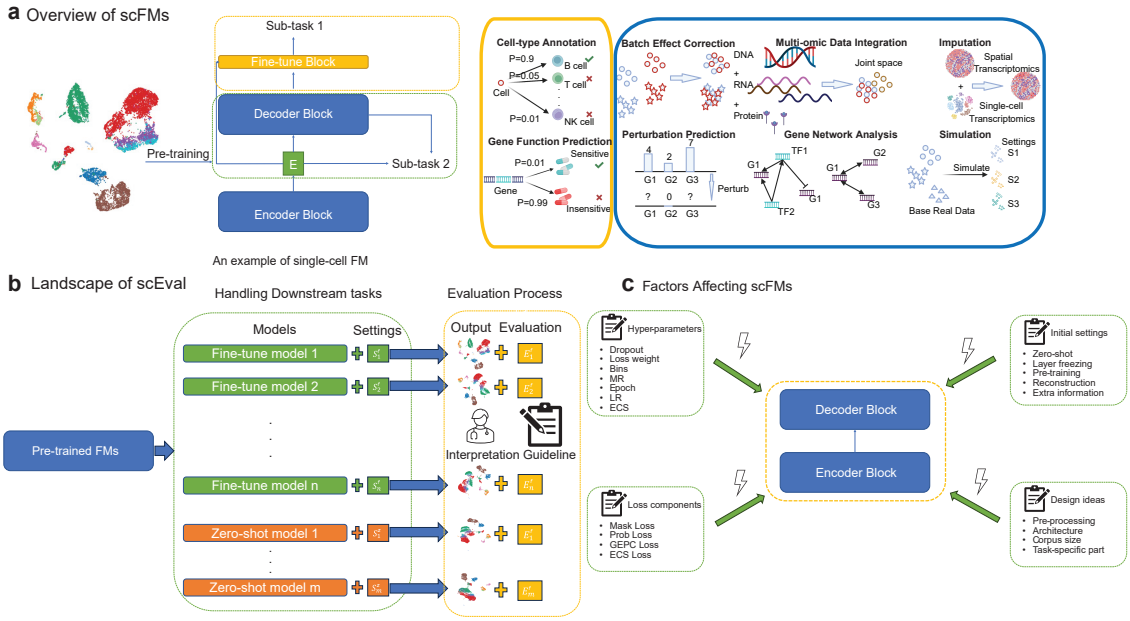


Figure 1.1: Overview of single-cell FMs, landscape of scEval and factors affecting single-cell FMs. (a): Overview of single-cell FMs describing the typical structure of FMs and general tasks for single-cell data analysis. The right two blocks represent two types of downstream tasks. Yellow block: Sub-task 1, including Cell-type Annotation and Gene Function Prediction (top to bottom). Blue block: Sub-task 2, including Batch Effect Correction, Multi-omic Data Integration, Imputation (From left to right, top row), Perturbation Prediction, Gene Network Analysis, and Simulation (From left to right, bottom row). (b): The landscape of scEval shows the workflow of our systematic evaluation. Here we consider models based on fine-tuning design with settings S_1^f to S_n^f, and they have corresponding evaluation pipelines E_1^f to E_n^f. We also consider models based on zero-shot-learning design with settings S_1^z to S_m^z, and they have corresponding evaluation pipelines E_1^z to E_m^z. The reason for using the above notations to represent the process of evaluation is that different models can perform different tasks, while different tasks have different evaluation scenarios. (c): Factors which can affect the performance of single-cell FMs. These factors are summarized based on different designs of single-cell FMs. The known factors can be classified into four different types. Details of hyper-parameters can be found in original appendix. Details of initial settings can be found in original appendix.

Table 1.1: Overall comparisons of model capacity and performance. (a) Evaluations for the capacity of different single-cell FMs. We utilized different signs to denote the default settings we tested in scEval. Details are explained in the table legend. The icons are inspired by [244]. (b) Evaluations for both performance and usability of different single-cell FMs. We record the ranks of different methods across different tasks and user accessibility evaluations and compute the final overall rank. A lower rank means a better model. The top three methods are boldfaced. (c) User accessibility evaluation of different single-cell FMs. A smile face represents the selected single-cell FM can process the corresponding task by default.

Overview of our evaluations. We evaluated the performance and user accessibility of eleven open-source single-cell FMs (scGPT, Geneformer, scBERT, CellLM, tGPT, SCimilarity, CellPLM, UCE, scFoundation, and CellFM, and GeneCompass) by assessing their outputs on eight tasks with 29 datasets with both zero-shot setting and possible fine-tuning setting. We did not evaluate all the models with all the datasets with the reasons provided in Supplementary files 2 and 3, but tried our best to implement the missing functions for evaluation. The tasks that can be performed for different models, the overall ranks, as well as user accessibility are summarized in Tables 1.2 (a), (b), and (c). We also compared their performance with state-of-the-art (SOTA) task-specific methods. Our workflow can be summarized in Appendix Figure S 1.6.1. The top three methods include scGPT, Geneformer, and CellFM by considering both usability and performance. For each task, we evaluate single-cell FMs based on their default settings for a fair comparison. Moreover, we discuss the effect of different parameter settings on model performance and investigate the contribution of different loss functions of scGPT and initial settings by ablation tests. We also consider the contribution of model scales to the performance of FMs. Finally, we evaluate the stability and usability of different single-cell FMs and make recommendations for preferred models. The detailed design of scEval is explained in the Methods section. To ensure fairness, we selected metrics for different tasks based on their benchmarking analysis, and we fixed all models to be from versions before December 1, 2025.

Comparisons of pre-training datasets for different single-cell FMs. In this section, we investigated and compared the pre-training steps of different single-cell FMs from three dimensions, including the scale of pre-training datasets, the diversity of pre-training datasets, and the major human tissues or organs overlap of pre-training datasets. Based on Appendix Figure S 1.6.2, we found that GeneCompass had the largest pre-training datasets, followed by scFoundation. However, GeneCompass has running issues, so we could not fully reproduce their results. Interestingly, the pre-training scales of UCE, scGPT, Geneformer, SCimilarity, and tGPT are comparable, which implies that having 20-30 million may be acceptable for constructing pre-training datasets. Moreover, SCimilarity had the largest diversity in the datasets

conditioned on diseases compared with other methods. Most of the single-cell FMs chose to include cancer cells in their pre-training process, which was possibly due to many single-cell studies focused on cancer [87]. UCE had the largest diversity in the datasets from different species, while the rest of the single-cell FMs focused more on human. In addition, few models considered the use of multi-omic data information, except tGPT, CellPLM, and UCE. However, in the pre-training process, UCE and tGPT omitted the extra information (for example, the spatial location of spatial transcriptomic data) provided by these datasets and treated the data the same as scRNA-seq data. Using different pre-training pipelines for multi-omic datasets might improve the performance of current models on addressing related tasks. In Appendix Figure S 1.6.3, we computed the overlap of major human tissues or organs used for pre-training across different models. Geneformer had the largest diversity in this comparison because we treat it as a reference method. UCE had the largest overlap compared with Geneformer. scBERT and CellLM shared major human tissues or organs because they both used PanglaoDB [58] for pre-training. Interestingly, scGPT did not use many types of tissues for pre-training. Considering its relatively better performance, including as many types of datasets as possible may not necessarily lead to better performance in our evaluation for selected tasks. Therefore, careful task-specific data collection and data ablation analysis is needed for the pre-training stage of new models.

Furthermore, we analyzed the relationship between these statistics and model performance. Here, we considered two major tasks, batch effect correction, and cell-type annotation, for our comparison. We compared four FMs for batch effect correction and seven FMs for cell-type annotation, thus offering statistical meaning to perform further investigation. For each task, we computed the Spearman correlation under three settings, including performance vs. pre-training data scale, performance vs. the number of major tissues for pre-training, and performance vs. the number of model parameters. However, all the six correlations computed based on the comparisons did not show statistical significance (p-value > 0.05). Therefore, the relationship between the performance of single-cell FMs on downstream tasks and their pre-training settings might be affected by many factors, including pre-training strategies, data-cleaning pipelines, rules of training-testing datasets splitting, and others. Therefore, we focus on task-driven analysis and offer guidelines for model development based on their performances on different tasks.

Finally, we include a table that represents whether our evaluation datasets are used for the pre-training of single-cell FMs with known pre-training information in Supplementary file 2. This table shows that most of the datasets were not included by all single-cell FMs, so we had a sparse table.

Evaluation based on cell-perspective tasks shows that single-cell FMs can reduce batch effect and annotate cell types.

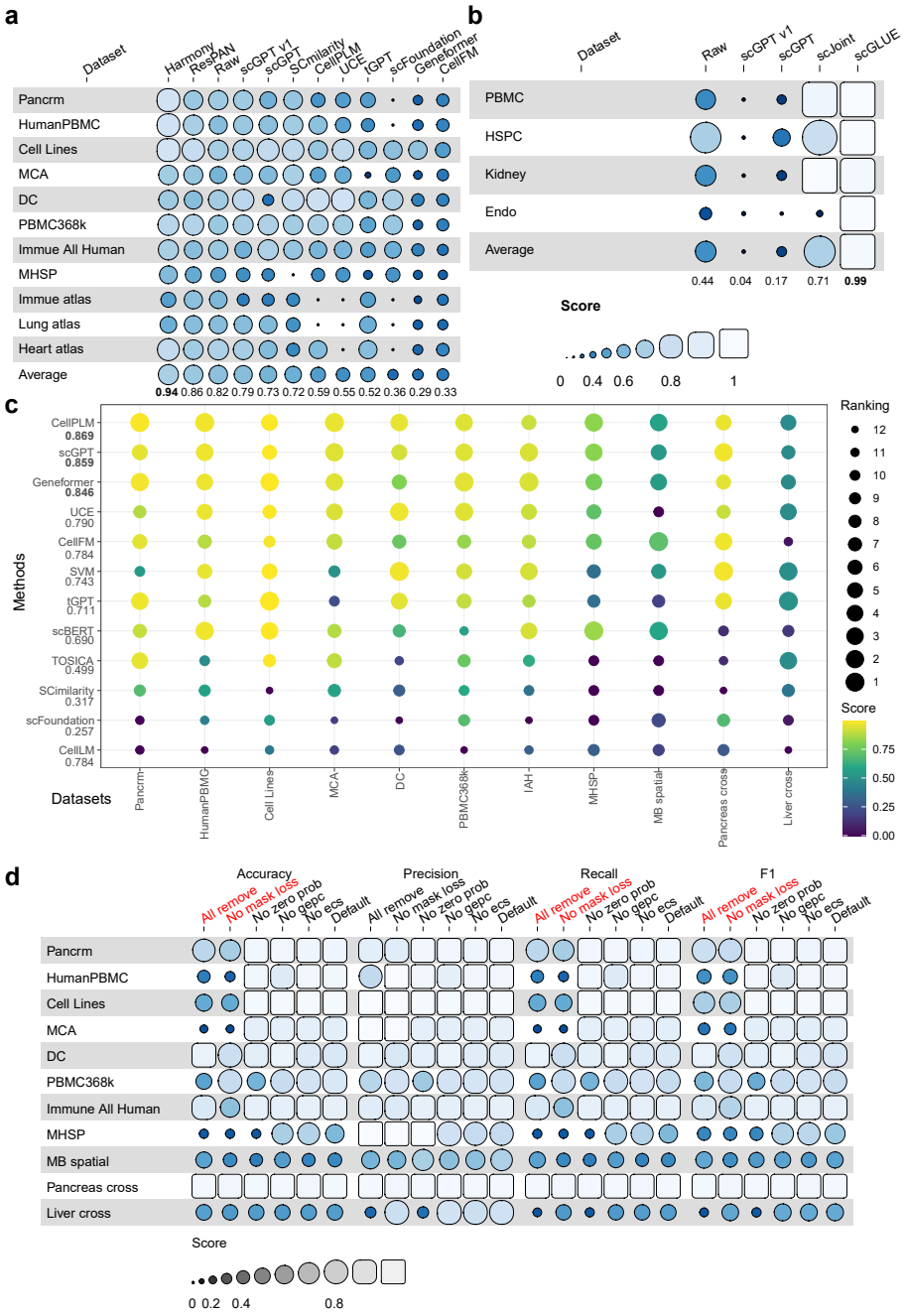


Figure 1.2: Experimental results of single-cell FMs and benchmarking methods for cell-level tasks. (a): An overall assessment of raw data (The "Raw" label in this figure) and data after batch effect correction based on different methods. scGPT v1 represents the scGPT model with smaller pre-training datasets compared to scGPT. (b): An overall assessment of raw data and data after multi-omic data integration based on different methods. (c): Comparison among models in the Cell-type Annotation task. The scores on the left represent the average accuracy of different models across different datasets. (d): Ablation tests of the loss function components for cell-type annotation. The red components are significant (one-sided Wilcoxon Rank-sum Test, $p - value = 0.03, n = 11$). The panels (a) and (b) follow the default settings of funkyheatmap [174] for visualization.

Batch Effect Correction. For this task, we intend to reduce batch effect of scRNA-seq datasets. We considered scGPT, tGPT, UCE, SCimilarity, CellPLM, Geneformer, scFoundation, CellFM, Harmony [101], and ResPAN [221] for this task. We also provided a detailed analysis of the influence of various hyper-parameters on the performance of scGPT on batch effect correction. The description of the evaluation metric S_{final} can be found in original appendix. We computed S_{final} based on the weighted average of metrics for evaluating the level of batch effect removal and the metrics for evaluating the level of biological variance conservation.

We first compared the performances of different single-cell FMs based on the zero-shot setting for this task, shown in Appendix Figure S 1.6.4. According to this figure, task-specific methods such as Harmony and ResPAN surpass the rest single-cell FMs, implying that the ability of FMs to remove the batch effect is limited under the zero-shot setting. Moreover, since scGPT also supports fine-tuning mode for correcting batch effect, we also compared the zero-shot setting and fine-tuning setting of scGPT for batch effect correction. According to Appendix Figure S 1.6.5 (a), the fine-tuning mode has a better overall score compared with the zero-shot mode, and thus fine-tuning scGPT can help on reducing batch effect in scRNA-seq data analysis.

We further compared different single-cell FMs based no their default setting. As shown in Figure 1.2 (a), scGPT outperformed ResPAN in one of the 11 datasets and outperformed Harmony in two of the 11 datasets, while Harmony had an overall best performance. In the comparison of FMs, scGPT v1 had the best performance, and scGPT v1 outperformed the scGPT model on average, raising the issue of the need for increasing the size of pre-training datasets for this task. Moreover, only Harmony and ResPAN achieved better S_{final} compared with the raw datasets on average. Therefore, the ability of FMs to remove the batch effect is limited. Moreover, these FMs had worse performance in reducing the batch effect for large-scale datasets. One reason is that their biology conservation scores were lower than those of raw datasets (shown in Appendix Figure S 1.6.6). UCE and scFoundation could not handle large-scale datasets due to running errors, which also raised problems in model accessibility. Appendix Figures S 1.6.8 and 1.6.9 show the Uniform Manifold Approximation and Projection (UMAP) [144] plots for the raw data and the scGPT results. We could still observe the batch effect in the output of scGPT for Pancrm, HumanPBMC, MCA, MHSP, DC, Lung atlas, Immune atlas, and Heart atlas datasets. We also evaluated the performances of single-cell FMs and task-specific methods for preserving the trajectory information during the batch effect correction process, shown in Appendix Figure S 1.6.7, and the metric used here is trajectory score. Based on this figure, Harmony still had the best performance, followed by scGPT.

We provide a detailed analysis of the impact of various hyper-parameters on the performance of scGPT in batch effect correction based on Appendix Figure S 1.6.10.

A smaller learning rate tended to lead to better performance across all datasets. The optimal number of training epochs varied across datasets, with a larger number of epochs being beneficial for most datasets. This result contradicts recent research advocating for a single-epoch training approach [238], suggesting that the optimal number of epochs might be context-dependent. Increasing the number of bins is generally associated with an increase in the final score. The impact of the mask ratio and dropout rate on model performance is unclear, suggesting that further investigation is needed to understand their influence. These observations may improve the application of scGPT for batch effect correction in single-cell data analysis and may also inform fine-tuning of other similar models.

Appendix Figure S 1.6.11 (a) presents the comparison of scores across different initial settings for the batch effect correction task using scGPT. We can see that scGPT is capable of performing zero-shot learning in batch effect correction. For the Cell Lines dataset, the zero-shot learning approach even achieved the highest score, indicating that it may be an effective method for certain datasets. Moreover, pre-training significantly contributes to the performance of scGPT in the batch effect correction task. Without pre-training, the model's performance notably decreased. Using cross-entropy as the loss function for gene expression reconstruction yielded better results than the mean square error (MSE) loss for most datasets. Freezing weights is not crucial for batch effect correction. Interestingly, the encoder structure appears to play a more important role in the training process, as freezing the encoder layer led to a larger decrease in the score. Incorporating the cell type information as a human label into the training process enhanced performance for most datasets. These observations suggest that each dataset may require unique tuning, underscoring the importance of adaptable methodologies in single-cell RNA-seq data analysis.

Appendix Figure S 1.6.11 (b) shows the performance metrics versus the choices of different optimizers for the batch effect correction. Adam and AdamW [98] had comparable performance in this task, while SGD [172], Sophia-G [120] (a novel optimizer which is designed for training FMs) and Lion [33] (an optimized version of Adam) were worse. Therefore, the optimizers in the Adam family are preferred.

Appendix Figure S 1.6.11 (c) illustrates the impact of different loss function components on the performance of batch effect correction using scGPT. Here we consider Mask Loss (It means we mask the gene expression levels of some genes and reconstruct them to compute the loss), Prob Loss (i.e. we predict whether some genes are expressed in some cells to compute the loss), GEPC Loss (i.e. we predict the gene expression levels based on the cell embeddings to compute the loss), ECS Loss (i.e. we bring similar cells closer together and push different cells farther apart in the embedding space to compute the loss), and one task-specific loss function, known as the gradient reverse loss (i.e. we reverse the gradient of batch label classifier to perform adversarial training). We compared the scGPT without certain loss

functions with its default mode to perform ablation tests. Using all components of the loss function did not always yield the best results, with the exceptions of the Pancrm, MCA, and MHSP datasets. Using only the gradient reverse loss function resulted in the worst performance. The GEPC Loss seemed to play a crucial role in the performance of the batch effect correction task. These results suggest the need for a careful composition of the loss function when training single-cell FMs for batch effect correction, with each loss function component contributing differently to model performance.

Multi-omic Data Integration. For this task, we intend to integrate unpaired scRNA-seq datasets with scATAC-seq datasets. We considered scGPT, scJoint [117], and scGLUE [23] for evaluation. We assessed the integration quality through the same score as Batch Effect Correction because these two tasks have similar targets, that is, integrating datasets from different domains while preserving biological variation. The results presented in Figure 1.2 (b) summarize the evaluation results for the multi-omics integration task. Overall, scGLUE surpassed scGPT and achieved the best performance. In addition, the Appendix Figure S 1.6.13 shows that the results of scGPT did not have better biology conservation scores than the raw condition across all the datasets. Therefore, there exists a performance gap between scGPT and task-specific models. Based on our analysis, scGPT still performed better than scGPT v1, which implied that larger pre-training datasets might help integrate multi-omic datasets. We also observed batch effect for the results of scGPT based on UMAP plots shown in Appendix Figure S 1.6.14. In conclusion, single-cell FMs did not show an advantage in handling datasets for multi-omic data integration.

We illustrate the effect of initial settings in Appendix Figure S 1.6.11 (d). Different from the case with batch effect correction, the cross-entropy loss function led to worse performance compared to the MSE loss for this task. Interestingly, pre-training did not significantly affect the performance for this task since training from scratch also had similar performance. The encoder part of the single-cell FM played a more important role than the decoder since we observed a larger performance drop by only freezing the encoder part. Including cell types or human labels in the training process proved beneficial, likely providing the model with more precise and useful information for the task. The zero-shot learning approach did not perform as well for this task as it did for batch effect correction. Therefore, we need more consideration for the design of single-cell FMs for this task.

We illustrate the evaluation metrics versus different parameter settings in Appendix Figure S 1.6.15. scGPT did not perform well on this task as shown by the low score (below 0.5) even if we tried to search for better hyper-parameters. Certain hyper-parameters affected the training process. A smaller weight for the loss function, a larger dropout rate, and more epochs improved the model's performance. The number of bins and mask ratio did not exhibit monotonicity, making it difficult

to draw conclusions. Setting a high learning rate and ECS weight decreased the performance of scGPT. Moreover, based on Appendix Figure S 1.6.11 (e), the optimizers in the Adam family are still preferred because of better performance, which is similar to what we found for the batch effect correction task. Since the patterns discovered in the multi-omic data integration task are not exactly the same as what we discovered in the batch effect correction task, we believed that specific design focusing on scATAC-seq data is required.

Cell-type Annotation. For this task, we intend to annotate cell types for scRNA-seq datasets. We considered all open-source FMs, TOSICA [29], and SVM_{rej} [1, 161] for this task. We assessed the performance of different single-cell FMs in assigning cell types based on the four metrics (Accuracy, Precision, Recall, and F1 score) discussed in original appendix.2. These metrics are widely used in the evaluation for cell-type annotation.

We first compared the performances of different single-cell FMs based on the zero-shot setting for this task, shown in Appendix Figure S 1.6.12. According to this figure, SCimilarity, CellPLM, and UCE have the overall highest annotation accuracy based on logistic regression. Considering SCimilarity was trained with cell-type labels, introducing cell states in the model training stage might help in performing this task and single-cell FMs have the potential to work as powerful tools for cell-type annotation. Moreover, since scGPT also supports fine-tuning mode for predicting cell types, we also compared the zero-shot setting and fine-tuning setting of scGPT for batch effect correction. In Appendix Figures S 1.6.16 (a) and (b), we explored the capacity of using cell embeddings from scGPT and Geneformer to annotate cell types (the zero-shot learning mode) and compared the results with the fine-tuning mode. We found that on average the fine-tuning mode surpassed the zero-shot learning mode for annotating cell types (p-value=1.9e-3 for scGPT, p-value=1.9e-3 for Geneformer, based on the Wilcoxon rank-sum test). Therefore, it is still necessary to consider fine-tuning single-cell FMs for cell-type annotation tasks.

We further compared different single-cell FMs based on their default setting. The UMAPs for the raw data and scGPT are shown in Appendix Figures S 1.6.17 and 1.6.18. Since these plots were generated based on the same cell embeddings, they could be used as visualizations for the observed and predicted distributions of cell types. Figure 1.2 (c) displays the Accuracy for different settings for these models. On average, models with pre-training performed better than those without pre-training. This observation and overall rank align well with results of weighted F1 score shown in Appendix Figure S 1.6.19. However, CellLM and scFoundation did not perform well across all the datasets, mainly because these two methods had running errors for Pancrm, HumanPBMC, PBMC368k, and Liver cross datasets, which raised the issue of the reliability and usability of these models again. The default setting of

SCimilarity also did not perform well in this task, which might be due to their pre-training process which contained too many sub-cell types and only utilized 10X sequencing data [74], and thus the default neighbor-based sub-cell-type annotation approach cannot work well in the real application. Moreover, for the intra-dataset prediction task, CellPLM, scGPT, and Geneformer were comparable although they had different pre-training settings. For the inter-dataset prediction task, CellPLM, scGPT, Geneformer, tGPT and UCE were better than scBERT. Therefore, different single-cell FMs also had large divergences in performance, but CellPLM, scGPT, and Geneformer had good performances across different datasets.

In Appendix Figure S 1.6.20, we compared the performance of models with different hyper-parameter settings. Higher loss weight, learning rate, ECS threshold, mask ratio, and smaller epochs tended to lead to worse performance of scGPT. There was little correlation between the number of bins and the performance of scGPT. We observed the consistency in the performance of different single-cell FMs under the condition of altering their shared hyper-parameters. For Geneformer and scBERT, a lower learning rate and higher epochs also tended to lead to better performance.

We also considered different initial settings for model training. The first setting is *Freeze all*, where we froze all the weights of pre-trained layers. The second setting is *Default*, where we used the default fine-tuning settings. The third setting is *From scratch*, where we did not use the pre-trained weights. Appendix Figure S 1.6.21 (a) shows the score versus initial settings across different datasets. Here we considered scGPT and scBERT. We omitted Geneformer because it requires pre-training weights as input. It can be seen that pre-training always improved results for scGPT, especially for the cross-dataset conditions. However, there was a little benefit of pre-training for scBERT. For both cases, freezing the pre-training layers and preventing them from being involved in the fine-tuning process was not recommended. In some cases, the fine-tuning performance of such freezing was worse than training from scratch. Transfer learning for different species is possible because, for the MCA dataset, pre-training based on human data can help predict cell types for the mouse. For the same type of GPU, the training process of scGPT was faster than scBERT and Geneformer, with more GPU memory usage, according to Appendix Figure S 1.6.21 (b).

In Appendix Figure S 1.6.22, we froze the front layers or scGPT (from 0 to 11) to investigate the relation between the number of freezing layers and annotation accuracy. We found that freezing layers had comparable or even slightly better performance than full-parameter fine-tuning. Although the metrics under different numbers of freezing layers are close and high, we still found that freezing five layers can lead the best performance. Therefore, we can freeze parts of layers for cell-type annotation based on single-cell FMs.

In Appendix Figure S 1.6.23, we show the performance of CellPLM, Geneformer,

scGPT, and scBERT based on different optimizers across four datasets. Overall, Adam, AdamW, and Lion were comparable. Sophia-G was worse than them but better than SGD, and they were both unstable. Moreover, Geneformer did not support Lion and Sophia-G as optimizers, and thus the optimizers in Adam family are more preferred in fine-tuning single-cell FMs.

Moreover, we explored the contribution of different loss function components towards the cell-type Annotation task based on ablation tests, and the results are shown in Figure 1.2 (d). Here we included three extra metrics, and details can be found in original appendix. Based on the Accuracy of Figure 1.2 (d), the inclusion of mask loss is important. Moreover, the default setting is generally good across different tasks. Based on precision and recall of Figure 1.2 (d), the effect of different loss function terms had less effect on precision and more effect on recall. Such a difference could affect the final F1 score. Removing the GEPC loss function terms improved the cell type prediction for the DC, MHSP, and MB Spatial datasets, and did not affect the prediction performance for the other datasets.

Therefore, most single-cell FMs can handle the cell-type annotation task with suitable pre-training data and model structure, but investigation for the pre-training framework is needed to understand their specific performance differences across all the datasets.

Evaluation based on gene-perspective tasks shows that single-cell FMs can handle tasks related to functions of genes.

Gene Function Prediction. For this task, we intend to predict the functions of genes. We considered Geneformer, scGPT, scFoundation, CellFM, and *vanilla* NN based on raw expression data and *vanilla* NN based on Gene2vec [49] for this task. scFoundation met running errors while generating the gene embeddings for datasets used in this task. We split the genes for training/testing and evaluated the results with the same four metrics as the Cell-type Annotation task because they are both classification problems. The results are shown in Figure 1.3 (a). On average, Geneformer and scGPT performed well in this task. Moreover, the accuracy scores of scGPT and *vanilla* NN based on Gene2vec were comparable, while there was a performance gap between single-cell FMs and *Vanilla* NN based on raw data. Therefore, using initial gene embeddings with prior information is meaningful for single-cell FMs.

Figure 1.3 (b) and Appendix Figure S 1.6.24 show the accuracy of different hyper-parameter settings. Smaller learning rate and loss weight tended to have more accurate results. Geneformer was more sensitive to Epoch compared to scGPT. For scGPT, pre-training contributed more than fine-tuning in this task as increasing epochs did not affect the model performance. Only tuning the number of bins, mask ratio, dropout rate, and ECS threshold did not affect the prediction results.

In Appendix Figure S 1.6.25 (a), we considered different initial settings for model

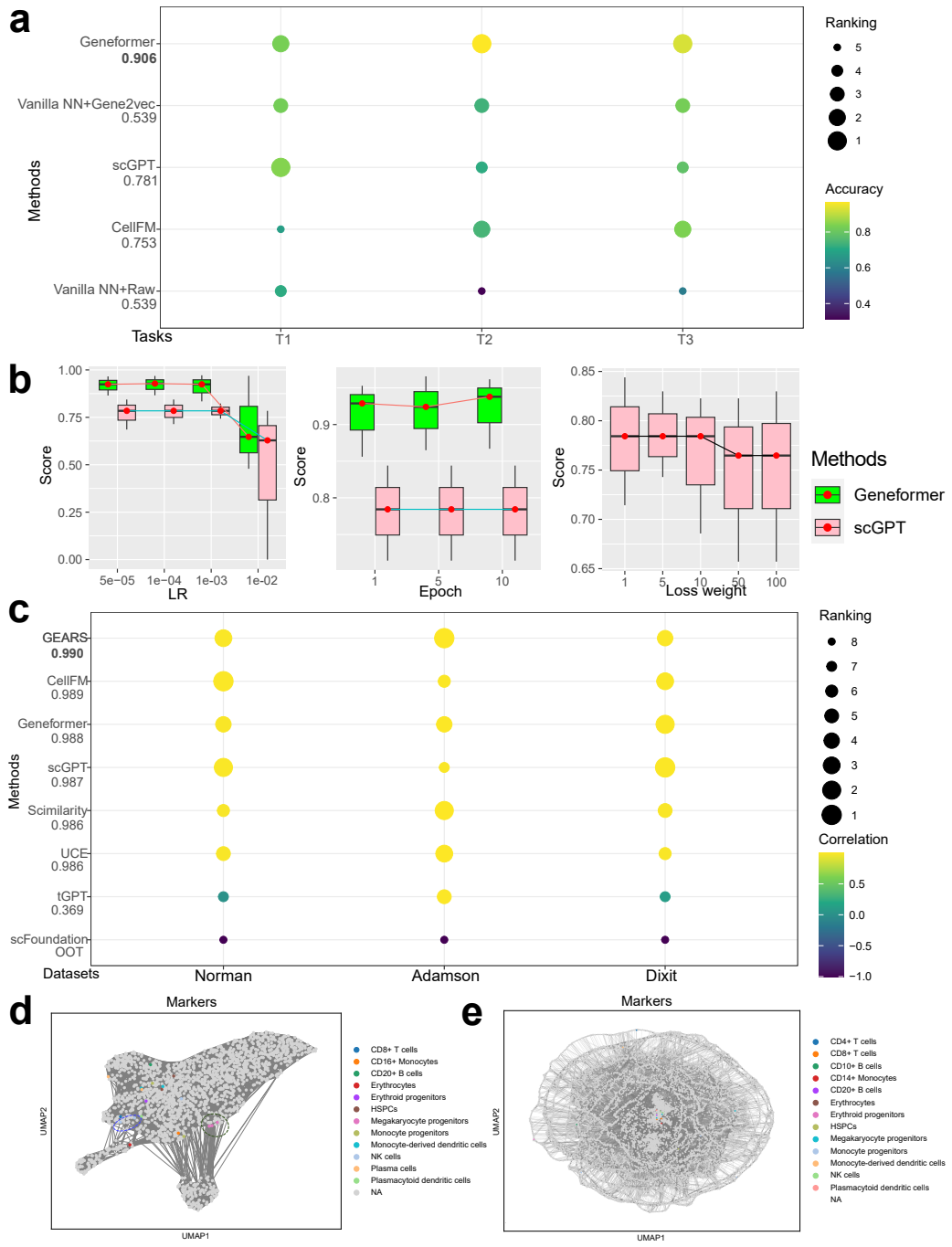


Figure 1.3: Experimental results of single-cell FMs and benchmarking methods for gene-level tasks. (a): Comparisons among Geneformer, scGPT, and *vanilla* NN in the Gene Function Prediction task. (b): The effect of hyper-parameters including Loss weight, Bins, and Learning rate for scGPT and Geneformer in the Gene Function Prediction task (sample size $n = 3$ per box). (c): Correlation metrics of different methods. A higher correlation means lower rank and better performance. The numbers corresponding to the settings represent the average value across two datasets. (d): Dataset-level gene embeddings from scGPT colored by the marker genes of different cell types. (e): Dataset-level gene embeddings from Geneformer colored by the marker genes of different cell types.

training. It can be seen that pre-training always improved performances of scGPT. Meanwhile, freezing the whole model did not affect the performance of scGPT.

Appendix Figure S 1.6.25 (b) shows the performance of scGPT based on different optimizers. Adam and AdamW were comparable, while Lion was worse than them but better than SGD and Sophia-G.

Appendix Figure S 1.6.25 (c) shows the ablation test results of scGPT for this task. There was no significant difference by comparing the default setting and those without certain components. Therefore, the task-specific loss function is the most important design for this task.

Perturbation Prediction. For this task, we intend to predict the gene expression levels under perturbations. We considered scGPT, Geneformer, tGPT, SCimilarity, UCE, scFoundation, CellFM, and GEARS [169] for this task. We implemented a linear-regression-based model to predict perturbed gene expression levels from cell embeddings for those FMs without this function. We used Mean Pearson Correlation (MPC) and Mean Squared Errors (MSE) as the metric to evaluate the performances across all genes, and details can be found in original appendix. Therefore, we can assess the prediction performance at the gene level for both perturbed genes and unperturbed genes. The datasets include two perturbation conditions: a single-gene perturbation and a double-gene perturbation, and two gene sets: all gene sets and Differentially Expressed Genes (DEGs).

We first compared the performances of different single-cell FMs based on the zero-shot setting for this task, shown in Appendix Figure S 1.6.26 (a) for MPC and (b) for MSE. According to these figures, Geneformer, scFoundation, and scGPT have the best perturbation prediction performance based on cell embeddings. Other methods such as tGPT show very high MSE for predicting perturbation effect based on the Norman and the Dixit datasets, implying that not every single-cell FM has good embeddings to perform this task. Moreover, since scGPT supports the fine-tuning mode for perturbation prediction, we also compare the MPC between the zero-shot mode and fine-tuning mode based on scGPT for perturbation prediction, shown in Appendix Figure S 1.6.5 (b). This figure implies that there is no big difference between predicting the perturbation effect directly from cell embeddings or with the fine-tuning setting based on scGPT, and thus cellular embeddings from these single-cell FMs might be helpful for developing tools focusing on addressing problems from perturbation prediction.

We further compared single-cell FMs and task-specific methods based on their default settings. Based on our experiments shown in Figure 1.3 (c) and Appendix Figure S 1.6.27, most of the FMs (except tGPT) and GEARS were comparable in this task. By evaluating the model performances based on DEGs shown in Appendix Figures S 1.6.28 (a)-(b), we find that GEARS still outperforms the rest of single-cell FMs, followed by scGPT. Geneformer, SCimilarity, and UCE have similar perfor-

mances. Moreover, the running time of scFoundation for this task was too long to finish and tGPT is not a suitable predictor, which suggest the challenges of deploying FMs in solving this task. Since GEARS has robust performance, the requirement of developing FMs for handling such task is not clear. To understand the contributions of single-cell FMs in this task, we further conducted statistical analyses by comparing the performances (PCC or MSE) of different methods with one-sided Wilcoxon Rank Sums test, and we visualize the $-log(p-value)$ in Appendix Figures S 1.6.29 (a)-(b). Here we found that GEARS single-cell FMs generally cannot surpass GEARS significantly, but fine-tuning models can generally benefit this task. Therefore, we believe that dataset-specific adaption is still meaningful for this task and task-specific expert models such as GEARS are recommended.

Appendix Figure S 1.6.30 also summarizes results for different initial settings of scGPT for the Norman, Adamson, and Dixit datasets. The default setting performed best for these datasets across different settings. This indicates that the initial configuration of scGPT works well for this task. The performance was comparable between training from scratch and training from pre-trained weights. Freezing the decoder part of the model performed better than freezing the encoder part, which implied that the encoder part is important for perturbation prediction. Interestingly, the ability of zero-shot learning of scGPT towards this task was not suitable. Therefore, the contribution of using non-perturbed data as pre-training datasets for perturbation prediction related to perturb-seq datasets is not significant. We may need to investigate more types of perturbation, including drug-level conditions or disease-level conditions, to check the importance of introducing prior information for perturbation analysis.

Regarding the effect of hyper-parameters, Appendix Figure S 1.6.31 shows that scGPT is sensitive to adjusting the learning rate and epochs. Decreasing learning rate and increasing the number of epochs improved MPC. Higher learning rate caused a running error for scGPT. We did not identify patterns for other hyper-parameters. Appendix Figure S 1.6.32 (a) shows that AdamW, Sophia-G, Adam, and Lion had comparable performance for scGPT in perturbation prediction. SGD could significantly reduce the performance of scGPT. Therefore, the use of single-cell FMs for perturbation prediction does not require a complicated design.

Gene Network Analysis. For this task, we intend to evaluate the quality of gene networks inferred from gene embeddings. We considered scGPT, Geneformer, CellFM, and scFoundation in this section. Firstly, we clarified the difference between Gene Regulatory Networks (GRNs) and Gene Co-expression Networks (GCNs), and the networks we evaluated here were GCNs. In general, GRNs characterize causal relationships among genes. The GRNs defined in scGPT are in fact GCNs, because the construction process is based on embedding similarity. For the inference of GCNs , two types of them can be defined: Type 1 GCN (Tissue-specific GCN)

and Type 2 GCN (Cell-type specific GCN). We used the Immune Human Atlas dataset to evaluate the performance of inferring these two types of GCNs. The known information including marker genes [132], cell types [132], and Reactome pathways [55] was utilized to evaluate the performance of scGPT on the GCN inferences. We also included scWGCNA [56] as a task-specific method for comparison.

Type 1 GCN is generated by applying the single-cell FM to the entire dataset under a zero-shot learning framework to create gene embeddings. The similarity is then computed to infer gene-gene relationships based on these embeddings. The quality of the GCN is evaluated based on the relationships between marker genes for different cell types.

Type 2 GCN is created by applying the single-cell FM to generate cell-type-specific gene embeddings under the zero-shot learning framework, and the same type of similarity is used to infer gene-gene relationships based on these embeddings [38]. The quality of this GCN is evaluated based on the Gene Ontology Enrichment Analysis (GOEA) [207] for specific gene sets. These GCNs can provide valuable insights into the understanding of gene interactions and regulation in specific tissues or cell types, which could have broad applications in biology and medicine [9, 183].

In the analysis of the Immune Human Atlas dataset, we initially focused on Type 1 GCN, with the results presented in Figures 1.3 (d) and (e). The neighboring relationships within this dataset are colored according to the distribution of marker genes. We collected marker genes based on the source paper of this dataset [132] and filtered them to have marker genes with similar expression patterns for each cell type. As the gene-gene relationship was determined based on k-nearest neighbors, it can be viewed as a form of gene co-expression relationship. From Figure 1.3 (d), only marker genes from two cell types showed the co-embedded and isolated relationship. They are Monocyte-derived dendritic cells and Megakaryocyte progenitors. Figure 1.3 (e) shows that the embeddings from Geneformer for maker genes were all co-embedded. Therefore, the embeddings of scGPT are better than the embeddings of Geneformer on preserving the cell-type-specific information. Appendix Figures S 1.6.33 (a) and (b), on the other hand, represent the cluster labels based on the Leiden clustering method [210]. These clusters can be interpreted as groups of genes that share common functions, or "gene co-function clusters". We first analyzed the cluster information from the gene embeddings from scGPT. For marker genes from other cell types, some of them are in different clusters shown in these two figures, and some genes are co-embedded with other cell types' marker genes. There are two isolated groups (9 and 12), but no marker genes are identified in these two groups. For the gene embeddings from Geneformer, we found that most of the marker genes from different cell types are embedded in the same cluster, while the rest of clusters did not contain much cell-type-related information. Therefore, the clustering result of scGPT was also more biologically meaningful than Geneformer, possibly due to

the number of selected genes. Geneformer generated all genes for embeddings by default, while scGPT only focused on highly variable genes.

We also evaluated and explored GCNs based on the human immunology system quantitatively, which is known for its complexity due to interactions among various cell types. The original analysis of GCN from [38] focused on HLA genes and CD genes. We did not analyze HLA genes because they are highly polymorphic and thus carry a higher risk for having errors of reads [17, 95, 147, 175], so the network result may not be reliable. Therefore, we selected genes co-embedded with CD3 genes, and utilized the embeddings of these genes to compute the GCNs. The results of scGPT and Geneformer for CD3-related genes are shown in Appendix Figures S 1.6.33 (g) and (h). The value above the edges represents the strength of correlation. To visualize the important interaction, we use the threshold of strong correlation from scGPT (0.4). A publicly accessible database containing pathway information- the Immune System R-HSA-168256 pathway from the Reactome 2022 database [55] - was used as a reference for validation. The GCN was constructed based on the correlation between the two genes. For the CD3-related genes, there was nearly no overlap between discovered pathways and the whole set (7/1943) for scGPT, Geneformer (8/1943), CellFM (0/1943), and scFoundation (0/1943), indicating poor inference of GCN. However, the results of scWGCNA have very high overlap (913/1943), and thus traditional method better captured pathway-specific information from genes. To ensure the conclusion is consistent for GCNs under different resolutions, we also adjusted the resolutions and Appendix Figures S 1.6.34 (a)-(b) show that scWGCNA's overlap was always higher than the overlap from other single-cell FMs. To explore the similarity of gene embeddings from singlecell FMs, we visualize the detailed pathway information from scGPT and Geneformer in Appendix Figures S 1.6.35 (a) and (b), and there is no overlapped pathways between the results from the two models. Moreover, for all the pathways involved in the selected gene set, we computed the ratio between the number of significant pathways and all pathways, and recorded the results in Appendix Figure S 1.6.33 (f). We found that gene embeddings from scGPT had a higher ratio, suggesting a better quality of scGPT output. These analyses highlight single-cell FMs might not contribute much to construct meaningful GCNs due to the insufficient quality of gene embeddings.

Appendix Figures S 1.6.33 (c) and (d) focus on gene embeddings categorized by cell types from scGPT, while Appendix Figure S 1.6.33 (e) shows the cell-type-specific gene embeddings from Geneformer. By treating cell types as observed labels, we also computed Normalized Mutual Information (NMI) and Adjusted Rand Index (ARI) for gene embeddings from different methods. Appendix Figures S 1.6.33 (c) and (d) show that gene embeddings of scGPT (NMI=0.049, ARI=0.035) from different cell types tended to be co-embedded and there was no apparent difference. The distribution of the remaining genes on the UMAP results was relatively random, and we

also observed a random distribution of gene embeddings from Geneformer (NMI=0, ARI=0). Furthermore, the NMI and ARI scores of two types of gene embeddings are also low. One reason could be that the quality of gene embeddings was unsatisfactory. Since scGPT and Geneformer adopt the zero-shot learning scheme embeddings, it does not incorporate cell-type-specific information for a specific dataset. The other reason might be that the complex biological network in the human immune system makes the communication between cell-cell or gene-gene difficult to decompose [42, 115]. Additional analysis is needed for generating gene embeddings. This analysis illustrates the complexity of cellular functionality and the difficulty of clearly defining these relationships based on gene embeddings. Despite these challenges, the scGPT model still demonstrates its potential in identifying functional similarities between different cell types.

In conclusion, our results highlight the importance of critical evaluation and cross-referencing in the development of GCNs inference, as well as the potential and limitations of using single-cell FMs for this purpose.

Evaluation based on Imputation and Simulation Analysis shows that further improvement of single-cell FMs is needed.

Imputation. For this task, we intend to impute gene expression profiles of different datasets. We considered imputation for two different types of datasets, known as scRNA-seq datasets and spatial transcriptomic datasets. To impute scRNA-seq datasets, we intend to evaluate the performances of single-cell FMs for filling technical zeros, which contain $\sim$ 20,000 genes. To impute spatial transcriptomics, we intend to evaluate the performances of single-cell FMs for filling $\sim$ 20,000 unobserved genes, which are observed in scRNA-seq data. We compared CellPLM, scGPT, and Tangram [11] in this task. The evaluation metrics for this task based on clustering and correlation are provided in original appendix. The evaluation of clustering performance and gene expression correlation can assess the preservation of biological variation after imputing. The imputation results for scRNA-seq are summarized in Appendix Figure S 1.6.37 (a), which implies that the imputation function of scGPT for scRNA-seq data introduced more noise into the original sequencing data, suggesting the unreliability of the decoder's output.

According to Appendix Figure S 1.6.37 (b), scGPT performed well in the spatial transcriptomic data imputation task compared to the SOTA spatial imputation method, Tangram [11, 113]. Moreover, CellPLM did not outperform Tangram evaluated based on cell clustering with biology conservation score and correlation. Its proportion of significant genes was lower than scGPT. Therefore, CellPLM might overfit scRNA-seq data under this task setting. Based on the evaluation of correlation and significance proportion, the imputation results of scGPT are better than the results of Tangram. Moreover, the scores of these two metrics based on the zero-shot learning version were even better than the pre-training version with scRNA-seq data.

However, based on the results of the average bio score evaluation, the raw data had better scores. This could be caused by the sources of the spatial clustering labels, which were generated from gene expression clusters rather than expert annotation. Such methods could introduce bias before and after imputation.

Appendix Figure S 1.6.38 shows the results of DEGs based on pre-imputation data and post-imputation data. Results in Appendix Figure S 1.6.38 (a) showed that scRNA-seq imputation was not reliable because the expression patterns of all genes were similar after imputation based on scGPT. However, based on Appendix Figure S 1.6.38 (b), no mitochondrial (MT) genes were included in the DEGs after imputation based on scGPT. These genes were identified as DEGs in the raw dataset. A high proportion of MT genes is indicative of low-quality data, which means cells with such patterns are apoptotic or lysing [133]. Therefore, the MT genes should be omitted in the downstream analysis by filtering[180, 229]. Moreover, based on Appendix Figures S 1.6.38 (b) and (c), the DEG patterns after imputation based on scGPT and Tangram are similar. Thus, scGPT has the potential to produce biologically meaningful imputation for spatial transcriptomic datasets.

Simulation Analysis. For this task, we intend to simulate synthetic scRNA-seq datasets. We considered scGPT, Splatter [248], and scDesign3 [193] for this task. We evaluated the output of scGPT against the output of scDesign3 and Splatter. For conditions incorporating batch effects, we employed the same metrics used in the evaluation of batch effect correction. In scenarios without batch effects, our metrics are primarily focused on assessing the preservation of biological information. As shown in Appendix Figures S 1.6.37 (c)-(e), scDesign3 outperformed scGPT and Splatter across two conditions of the simulation task. However, scGPT performed better than Splatter for simulating datasets without batch effect. In particular, scDesign3 had a more pronounced superiority in generating simulation data without batch effects, in comparison to scGPT. This is consistent with the results shown in Appendix Figures S 1.6.37 (d), (e) and Appendix Figure S 1.6.39. The gene-gene correlation from scDesign3 was also more similar to the gene-gene correlation of the raw data. The gene-gene correlation from scGPT has null value due to missing the gene expression certain genes, which is a problem caused by the decoder outputs from scGPT. Therefore, the simulation task needs to be improved for single-cell FMs as reference-based simulators.

In addition, we present UMAPs of the output produced by different methods in Appendix Figures S 1.6.40 and 1.6.41 that illustrate the advantage of scDesign3. The embeddings of scGPT with the no batch effect settings tended to preserve the batch effect, while the embeddings with batch effect tended to remove the batch effect. The embeddings of Splatter with no batch effect also mixed different cell-type information.

Model scaling contributes to single-cell FMs but their stability need

to be improved.

Exploring Model Scaling Analysis. In this section, we explored the contributions of model scaling based on CellPLM, scBERT, Geneformer, and scGPT. These methods have finetuned versions and do not suffer from running errors, and also demonstrate promising performances based on our previous evaluation. The scaling law implies that models with large-scale parameter size might have better performances for certain tasks [75, 96, 252]. Due to computational constraints, we are unable to test scaling effects within a single model family and instead investigate trends across models with different parameter scales and configurations. Here we considered three scenarios to investigate the possible contribution of model scaling: cross-data cell type prediction, cross-species analysis, and spatial transcriptomic analysis. Our hypothesis is that models have task-specific benefit from scaling the computational scales of models, which is summarized based on our previous observations and analyses.

Inspired by [70], we compared models with large parameter size of pre-training to models of small parameter size. For the task of cross-data cell type prediction, we compared other single-cell FMs with *Vanilla* neural networks (NNs) to identify any contributions of model scaling. For the task of cross-species analysis, we compared other single-cell FMs with expert model SATURN [170]. As for the task of spatial data batch effect correction, we examined the statistics derived from our correction evaluation to verify our hypothesis and the expert methods include ResPAN and Novae [15]. Novae is an FM pre-trained with spatial transcriptomics data and thus it is a meaningful baseline to evaluate the contribution of transferring knowledge from single-cell transcriptomics to analyze spatial transcriptomics.

In the first scenario, we compared single-cell FMs to *vanilla* neural networks (NNs) with smaller parameter counts. Figure 1.4(a) presents model sizes, and Figure 1.4(b) shows their performance. We observed that scGPT and Geneformer outperform *vanilla* NNs and scANVI [233], suggesting that, in some cases, models with parameter counts above 10 million may yield performance gains. However, this trend is not uniform: scANVI, which is a smaller yet specialized model, performs comparably or even better than some larger single-cell FMs (e.g., scBERT), indicating that architecture and task-specific design remain key performance factors. Thus, while model scale may contribute positively in certain scenarios, it is not the sole determinant of effectiveness.

In the second scenario, the desired contribution of model scaling mirrors that of the first task. To perform cross-species study, we only focus on orthologous genes across different species, which are extracted based on the Ensembl database [52] and the proportion is recorded in Appendix Figure S 1.6.42. Nonetheless, there was no enhancement in performance from Figure 1.4 (c). The cross-species cell-type annotation task is difficult, also suggested by [170, 194] as a representative task in

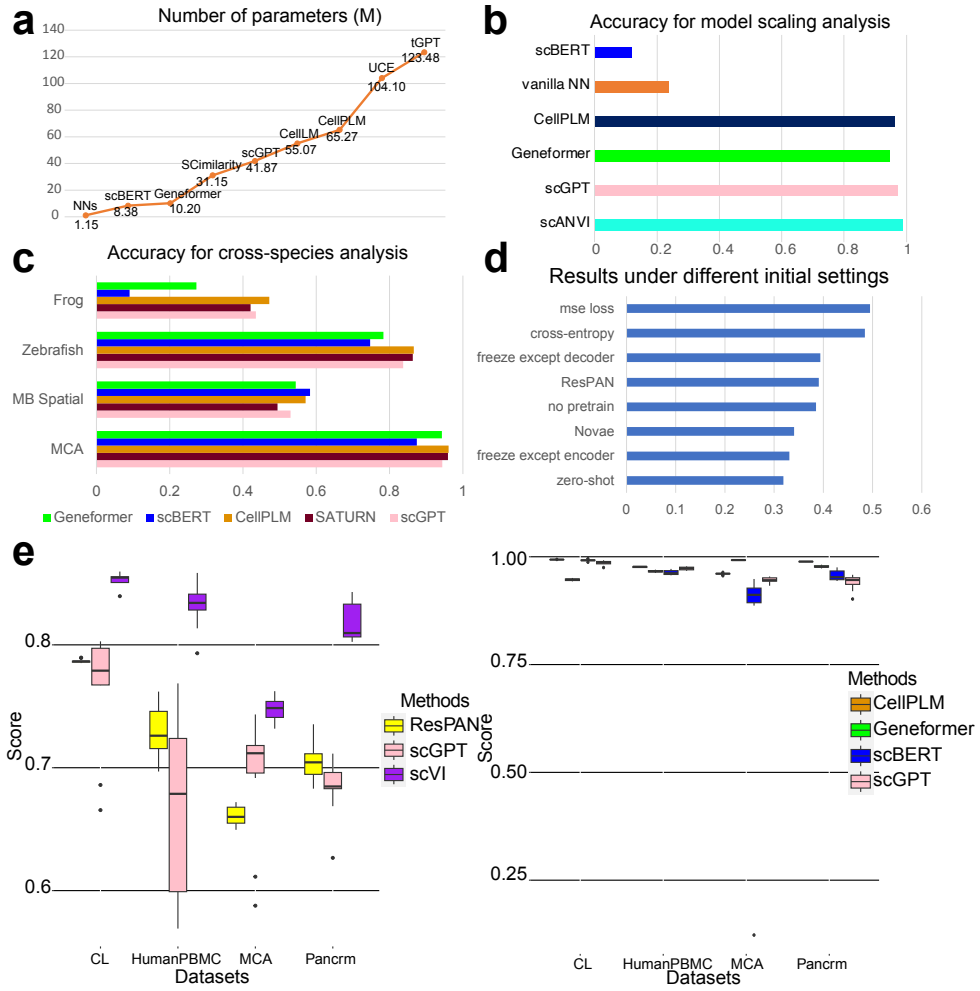


Figure 1.4: Different comparison groups for model scaling analysis and stability analysis. (a): The model scale of different methods, defined by the number of parameters (unit: Million). (b): Accuracy of FMs and *vanilla* NN in Cell-type Annotation task. The dataset here is the Pancreas cross dataset. (c): Accuracy of FMs and SATURN in Cell-type Annotation task across species. (d): Overall score comparison including ResPAN, Novae and different settings of scGPT. The dataset here is the human spatial transcriptomic dataset. (e): Different batch correction scores of different models based on changing random seeds (left) and different average classification scores of different models based on changing random seeds (right). The bold black line represents the median value, while the length of each box can be interpreted as the variance level (sample size $n = 10$ per box).

analyzing cross-species information transition. Figure 1.4 (c) shows that SATURN is a strong baseline and it outperforms single-cell FMs such as Geneformer and scBERT, while is comparable with scGPT. However, CellPLM can still outperform SATURN in most of the datasets used for cross-species analysis. Therefore, single-cell FMs including CellPLM and scGPT showed advantages in this task.

In the third scenario, we considered two possible tasks. Firstly, in line with the batch effect correction task, we postulated that using Human scRNA-seq data for pre-training could aid in batch effect correction for human spatial transcriptomic data. Secondly, resonating with the cell-type annotation task, we hypothesized that pre-training on Human scRNA-seq data might assist in cell-type annotation for mouse spatial transcriptomic data. Figure 1.4 (c) shows that we did not detect any contributions of model scaling in the MB Spatial data annotation task. Figure 1.4 (d) suggests the contribution of model scaling for batch effect correction. scGPT under different loss functions all outperform Novae, while Novae performs better than the zero-shot mode of scGPT in batch effect correction. Therefore, including single-cell transcriptomics and spatial transcriptomics data jointly for model training can contribute to spatial transcriptomics analysis in batch effect correction. The fine-tuning process appeared beneficial in reducing the batch effect inherent in the spatial data, whereas scenarios with model freezing except decoder yielded subpar results. The performance of scGPT in the integration of spatial data was better than that of ResPAN. Therefore, we observed the potential contribution of model scaling in the batch effect correction for spatial transcriptomic data.

Stability Analysis. To analyze the stability of single-cell FMs, we selected Batch Effect Correction and Cell-type Annotation as two representative tasks and varied the seeds from 0 to 9 of single-cell FMs to investigate the model stability. All of the methods used in this section are fixed with their default hyper-parameters. These two tasks are the main tasks in single-cell data analysis and have solid metrics for evaluation. Ideally, the results of different single-cell FMs should not vary substantially across different datasets. We also considered stability for other benchmarked tools. Our experiment results summarized in Figure 1.4 (e) showed that the stability for single-cell FMs is task-specific.

Based on the left panel of Figure 1.4 (e), the variance of scVI [128] and ResPAN was generally lower than that of scGPT, and scVI and ResPAN also had a higher score on average. Therefore, single-cell FMs were not as stable as SOTA deep-learning-based methods for the Batch Effect Correction task.

The right panel of Figure 1.4 (e) suggests that the variance of Geneformer and CellPLM was generally smaller than that of scGPT and scBERT. All four models had high median average scores. Moreover, the variance of scBERT was relatively large in the experiments based on the MCA dataset, which implies that single-cell FMs might fail under certain random seeds.

The statistical analyses of pairwise comparison in this section are shown in Appendix Figures S 1.6.43 (a)-(b), where expert models have significantly better performances in batch effect correction across different initialization stages and datasets, while CellPLM and Geneformer have generally good performances in annotating cell types across different initialization stages and datasets.

1.3 Discussion

Topic	Summary	Recommendation
Applications		
Batch Effect Correction	Overall, all scFMs performed worse than task-specific method.	Selecting task-specific method (Harmony) as a starting point.
Multi-omic Data Integration	Overall, all scFMs performed worse than task-specific methods.	Selecting task-specific method (GLUE) as a starting point, and including multimodal information in the pre-training stage of scFMs.
Cell-type Annotation	Overall, finetuned scFMs performed well in this task.	Exploring scGPT, CellPLM and Geneformer for annotating cell types, and we encourage researchers to include cell states in the pre-training stage.
Gene Function Prediction	Overall, finetuned scFMs performed well in this task.	Exploring Geneformer for predicting gene functions.
Perturbation Prediction	Overall, the performances of scFMs are close to task-specific methods.	Selecting task-specific method as a starting point, and including perturbation information in the pre-training stage of scFMs. Moreover, exploring the prediction based on zero-shot cell embeddings is also attractive.
Gene Network Analysis	scFMs are not good enough for modelling gene interaction networks.	Starting from gene expression profiles rather than gene embeddings for network inference and analysis.
Imputation	scFMs help on imputing spatial transcriptomics, while do not perform well in imputing scRNA-seq data.	Exploring the ability of scGPT for imputing spatial transcriptomics with a reference-free design.
Simulation	scFMs performed worse than task-specific methods for simulating scRNA-seq datasets.	Selecting task-specific method (scDesign3) as a starting point.
Properties		
Scaling Law	scFMs followed scaling law for cell-type annotation.	Investigating the setting of biological questions before enlarging model size.
Stability	scFMs performed inconsistently in batch effect correction, while performed consistently well in cell-type annotation.	Exploring different strategies for pre-training scFMs, and developing functions from cell embeddings or gene embeddings.
Training Strategies	Smaller learning rate (e.g. 1e-4) can improve model performance. During the finetuning process, freezing part of models can also improve efficiency.	Using small learning rate and exploring Parameter-Efficient Finetuning methods.
Optimizers	Optimizers in Adam family perform better based on our evaluation.	Using Adam or AdamW for model training.
Deployment		
Hard-Ware Requirement	We need at least one GPU with over 40 GB MEM for finetuning.	Using NVIDIA A100/A40/A6000 for deployment.
Model Development	scFMs vary greatly in the extent to which they are open source and the number of features they have by default.	Investigating scGPT, Geneformer and CellPLM for scFM development and deployment.

Table 1.2: Insights from the benchmarking results of scEval. We organize the table by tasks and model settings.

In this paper, we have evaluated the performance of ten single-cell FMs on eight different tasks for single-cell data analysis. The ranks of these single-cell FMs are shown in Tables 1.2 (a) and (b), where we not only considered the broader functions of the models, but also their usability. Based on our evaluation results, open-source models have higher ranks than closed-source models. Open-source models with tutorials are more friendly to researchers, and these models also receive a number of stars and likes based on Appendix Figure S 1.6.44. Tutorials associated with these models also enhance their accessibility to researchers.

Based on our experimental results, the performance and contribution of single-cell FMs are task-specific. For the Batch Effect Correction and Multi-omic Data

Integration tasks, single-cell FMs did not outperform task-specific SOTA methods. Moreover, the correction results of atlas-level datasets were worse than those of the raw datasets. For the Cell-type Annotation task, we found that CellPLM, scGPT, and Geneformer had comparable performance. Moreover, these methods were better than the other models in this task. For the Gene Function Prediction task, Geneformer also performed well and better than classical models for prediction. We also showed that the pre-trained model can have better performance in this task. For the Perturbation Prediction task, the function of model pre-training is not apparent, and GEARS is comparable with other single-cell FMs in this task, questioning the need of the necessity for developing single-cell FMs to understand perturbations. For the Gene Network Analysis task, we did not observe the great contribution of gene embeddings from single-cell FMs for either tissue-specific or cell-type-specific analysis. Constructing the cell-type-specific gene networks is still challenging for single-cell FMs reflected by the low clustering scores, and thus it is meaningful to explore methods for improving the function of network discovery. For the Simulation task, scGPT also did not perform very well. scDesign3, as a task-specific method, is better for simulating reference-based scRNA-seq datasets. For the Imputation task, we showed that scGPT did not perform well in the scRNA-seq imputation task, but it outperformed Tangram in the spatial transcriptomic data imputation task both under zero-shot learning and fine-tuning frameworks. Such a finding suggests that single-cell FMs can transfer knowledge across different omic data. Furthermore, we observed the contributions of large-scale parameters for single-cell FMs in specific tasks. For the cross-data and cross-species Cell-type Annotation tasks, the performance of single-cell FMs is much better than the performance of baselines with smaller parameter sizes. Moreover, scGPT can also be used to analyze spatial transcriptomic data for batch effect correction, which has attracted much attention in recent years [141]. In our stability analysis, we found that single-cell FMs were not very stable in the Batch Effect Correction task. Moreover, in the evaluation for the Cell-type Annotation task, we found that different single-cell FMs had different variance levels. The variance under different random seeds was driven by datasets. Considering the problems of stability, we still have difficulty refining it into a toolbox (for example, Seurat [71] or Scanpy [229]) with various functions.

Therefore, by considering the definition of a foundation model, there are evident limitations in their construction and training steps. A more comprehensive understanding of such FMs is necessary to address these issues. In this manuscript, we tried different approaches to discover insights from model components and training settings to possibly enhance the performances of single-cell FMs. Our conclusions, combined with the recommendations of models for different tasks, are summarized in Table 1.2. Considering the difference in resources required for zero-shot learning and fine-tuning, as well as the difference in model effectiveness, we suggest trying to apply

the model's zero-shot learning mode to obtain the embeddings required for the task at first, and then consider fine-tuning model for the given task if the performance of zero-shot learning mode is not satisfactory and we can obtain a good task-specific loss function. Based on our analyses, the task-specific loss function component is always the most important loss function component, and the second-most-important loss function is mask loss, which helps the model reconstruct the original expression values. The rest of loss functions are not as informative as the previously mentioned choices. We also explored more advanced parameter-efficient fine-tuning (PEFT) frameworks such as LoRA [78] for scGPT in Batch Effect Correction and Cell-type Annotation. Appendix Figure S 1.6.45 shows that LoRA slowed down the fine-tuning process and did not improve related scores. Thus, the training of single-cell FMs is more nuanced than that of general FMs. While it is important to consider the similarities between these two model types, we must also consider the differences rooted in the datasets and domain-specific knowledge.

The application of FMs to single-cell data remains a promising avenue of exploration, given its impressive performance in prediction tasks. Since much has been done to optimize general FMs [45, 80, 222, 236] (including efficient tuning, model compression, and other research directions), we focus specifically on how to better train and apply single-cell FMs. Here, we discuss several future directions:

In terms of pre-training preparation, we need high-quality data spanning different contexts, such as various cell types, disease states, genders, and even data from different species for pre-training datasets. High-quality pre-training datasets are important for general FMs; otherwise, the performance may be reduced [30, 92]. The qualification of pre-training data can also be verified with the online learning framework [24]. We can evaluate the performance of single-cell FMs on downstream tasks and then decide which datasets to include. With a better pre-training design, we may increase the scale of current models to a billion level. Moreover, the incorporation of biological information is crucial for the success of a FM in biology. Therefore, it is possible to incorporate other biological factors, including GRN and cell-cell interactions (CCI) [7] in the modeling process. With such information, we can also develop domain-specific or tissue-specific FMs for single-cell data analysis. Including extra labels in the pre-training stage might also improve the model's ability in identifying gene networks under specific context. Exploring the contribution of multimodal data to develop a multimodal FM is also a possible track. For example, incorporating text-based biological information [36] or multi-omic data with new tokens may help us further extend the functions of these FMs.

As for model training, both the pre-training and fine-tuning steps of the existing single-cell FMs need improvement. During model pre-training, we should focus on incorporating biological information or human feedback into the process, as opposed to just relying on the conventional masked token prediction task. For instance,

integrating cell-type or disease condition information into the pre-training step is an intriguing approach, and it allows experts to evaluate the quality of model output during the training process, which is also meaningful. Moreover, we should also consider the security or trustworthiness of FM training [116]. Using poisoned training single-cell datasets (for example, wrong data or make-up data) as an attack can test the robustness of single-cell FMs. For model fine-tuning, instruction tuning [223] is a potential direction to explore. In this context, cells could be considered as prompts, as described in scGPT. Another possible direction is to focus on generating unified embeddings for cells/genes and combining them with task-specific models for downstream applications, inspired by [70, 164].

For model evaluations, we need more effective methods to assess results for certain tasks. Our recommendations for researchers to consider as evaluations in developing new single-cell FMs are summarized in original appendix. For instance, we may include verification from biology experiments to avoid the harm of incorrect output of single-cell FMs (also known as FM hallucinations [201]). Furthermore, interpretability is also an important perspective we need to consider for a comprehensive evaluation. Methods such as SHAP [134] and Integrated Gradients [202] can interpret the contributions of genes during the pre-training or fine-tuning stage, which can be used to evaluate if scFMs are learning the correct biological rules or not. Moreover, analyzing the contributions of model scaling of single-cell FMs is also important to explore the breakthrough contribution and significance of a model, though we have not identified a task that can only be addressed by developing single-cell FMs in our evaluation. In addition, we should also account for the pre-training costs (such as training time and power consumption), and avoid developing a model that consumes large resources but has no significant improvement for downstream tasks.

When it comes to task selection, we should first define the tasks rigorously. For example, in the GRN inference task defined by scGPT, we cannot treat a co-expression network based on gene embeddings the same as a gene regulatory network. Also, from scGPT, using attention, we can infer gene-gene correlation strength with direction, but the relation between the correlation of features and the value in the attention map is in debate [12, 89, 227]. We should also consider more meaningful, hardcore, and challenging tasks related to single-cell and spatial data for single-cell FMs [84]. These tasks typically need prior information from large-scale transcriptomic data. For instance, tasks such as in-silico treatment analysis [205], complex perturbation analysis in the single-cell levels [129], and spatial data imputation [112] are difficult to handle without prior information, thus provides an opportunity for FMs to make a difference.

The study of benchmarking single-cell foundation models also has limitations. The first potential drawback is the timeliness of the results. The results of the comparison may change as the model is enhanced. The second drawback is the

validity of the dataset. Future base models may use the data in this paper for training, so new datasets will need to be found to evaluate the performance of new models. Finally, benchmarking studies should ideally have community support for reproducing results and extensions.

In summary, our goal in studying single-cell data-based FMs is to develop a large-scale model that is capable of performing multiple tasks with stable and reliable results. Such a model should also be user-friendly, with detailed tutorials and well-maintained websites. Our results indicate that there is much space for improving single-cell FMs. Although our evaluation is subject to the number of current single-cell FMs, the scale of current single-cell FMs, and the strategies of pre-training, we hope that our analysis can provide insights into the best practices and guide the development of future FMs for single-cell data analysis.

1.4 Methods

Problem definition. We consider a pre-trained FM, denoted as $\mathcal{M}(x, \theta)$, which is based on the single-cell dataset $\mathcal{D}$. Here, θ embodies the set of both model parameters (e.g., network weights) and hyper-parameters (e.g., epochs and learning rate). Different FMs have used distinct pre-training datasets. The model structure for the fine-tuning phase is defined as $\mathcal{M}'(x, \theta')$. Our objective is to ascertain the optimal set of θ' for various sub-tasks. Formally, we denote the loss for task k as $L_k(\cdot, \cdot)$, and use the evaluation dataset $\mathcal{D}_{\text{eval}} = \{x_i, y_i\}_{i=1}^n$, to assess L_k. Our primary goal is to find:

$$\theta^* = \text{Argmin}_{\theta'} \mathbf{E}_{x,y \in \mathcal{D}} \left[L_k(\mathcal{M}'(x, \theta'), y) \right].$$

Our second goal is to evaluate the performance of different single-cell FMs, that is, we intend to find:

$$\mathcal{M}^* = \text{Argmin}_{\mathcal{M}'} \mathbf{E}_{x,y \in \mathcal{D}} \left[L_k(\mathcal{M}'(x, \theta^*), y) \right].$$

Our third goal is to assess other abilities of single-cell FMs, including: 1. zero-shot learning; 2. model scaling [75, 96, 252]; 3. cross-species data analysis; 4. biological mechanism exploration; and 5. stability.

Parameters and tasks. Most single-cell FMs share the pre-training process. By considering the overlap among various single-cell FMs, we have selected scGPT, scBERT, and Geneformer as representative examples for our analysis. We also highlight the downstream tasks in Figure 1.1 (a). We focus on eight fine-tuning tasks in total: 1. Batch Effect Correction; 2. Multi-omic Data Integration; 3. Cell-type Annotation; 4. Gene Function Prediction; 5. Perturbation Prediction; 6. Gene Network Analysis; 7. Imputation; and 8. scRNA-seq Simulation. By compiling all

the hyper-parameters across different models, we present a list of factors that can affect the model performance categorized by types in Figure 1.1 (b). The definition of different hyper-parameters is discussed in original appendix. To analyze the effect of different hyper-parameters, initial settings, and optimizers, we selected representative datasets for different tasks. For Batch Effect Correction and Cell-type Annotation, we selected Pancrm, HumanPBMC, Cell Lines, and MCA because they cover various data types. Pancrm is from Pancreas tissue and has five batches. HumanPBMC is from PBMC and has nine cell types. Cell Lines has two cell types, as a binary label dataset. MCA is from *Mus musculus*. For Multi-omic Data Integration, we included all of the four datasets we used to analyze because their scales cover a large range. For Gene Function Prediction, we included all of the three datasets we used to analyze because they correspond to different sub-tasks. For Perturbation Prediction, we included all of the three datasets we used to analyze because their scales cover a large range and are from different sources.

Explanations of scEval. Here we introduce a framework known as scEval to evaluate the performance of different single-cell FMs on various tasks. The whole pipeline is highlighted in Figure 1.1 (a). Since most of the single-cell FMs choose to reconstruct the masked gene expression levels in the pre-training stage, we can use an encoder-decoder structure (similar to an auto-encoder [118]) as well as extra fine-tuning blocks to unify their architectures. Moreover, we split the eight tasks into two types. For tasks included in sub-task 1, we need an extra fine-tuning block to generate the results. Cell-type Annotation and Gene Function Prediction belong to this task type. For tasks included in sub-task 2, we rely on the outputs of the encoder part and decoder part to generate the results. The rest of the six tasks belong to this task type. The idea of scEval is inspired by benchmarking analysis in both single-cell area [112, 132, 211] and LLM area [10, 26]. According to Figure 1.1 (b), for each task, we collect the output of single-cell FMs under different settings. Each FM has its specific running settings, and their default modes can be classified into fine-tuning-based models and zero-shot-learning-based models, thus we propose a framework focusing on the outputs of different single-cell FMs to unify the evaluation process. To analyze the factors affecting single-cell FMs, we also adjusted the factors of different FMs and collected their results for evaluation. Moreover, scEval contains different evaluation pipelines for different tasks, and it can also be easily extended to evaluate more tasks by adding more functions.

Batch Effect Correction. Batch Effect Correction is an essential step following scRNA-seq data pre-processing. It primarily signifies the distribution disparity in scRNA-seq datasets originating from the same tissue, which can be attributed to various factors [110]. The reduction of batch effects is critical not only to allow researchers to discern genuine biological signals but also to facilitate integrated analyses across different studies. The challenge of this task arises from the need to

balance the removal of batch signals with the preservation of biological signals. We treat this task as a data integration problem.

For the Batch Effect Correction, the metrics we considered here were inspired by scIB [132], including Normalized Mutual Information (NMI), Adjusted Rand Index (ARI), and Cell-type Average Silhouette Width (cell-type ASW) for the biological conservation score; and batch Average Silhouette Width (batch ASW), Principal Component Regression (PCR), Graph Connectivity (GC) and kBET [20] for the batch effect correction score. We compute the weighted average of these metrics to represent the final batch effect correction score. Details of these metrics can be found in original appendix.1. Let S_{bio} donate the average biological conservation metric and S_{batch} donate the average batch effect correction metric as S_{batch}, the final model score is:

$$S_{\text{final}} = 0.6 \cdot S_{\text{bio}} + 0.4 \cdot S_{\text{batch}}.$$

Multi-omic Data Integration. Multi-omic Data Integration is a key step for multi-omic data analysis [197]. It is akin to an advanced form of batch effect correction. If unpaired multi-omic data are present, the objective is to map different datasets into a shared space for subsequent analysis. If paired multi-omic data are present, the goal is to assess whether the use of multi-omic data can contribute to learning a more comprehensive representation of the data. A significant challenge here is how to align omics at the feature level. For instance, the feature of the scRNA-seq data is a gene, the feature of the scATAC-seq data is a peak, and the feature of the protein data is a protein. The tokenization step can become complex given different modalities. We treat this task as a data integration problem. We used the same metrics for multi-omic data integration as those for Batch Effect Correction.

Cell-type Annotation. Cell-type Annotation is another key step following single-cell data pre-processing [82]. This step annotates each cell with its accurate cell-type label, which can be achieved through prior knowledge [166] or computational methods [165]. These annotated cell-type labels can provide essential biological information for further downstream analysis, such as cell-type specific network analysis. In addition, drug response prediction [70] or single-cell disease classification [167] can also be treated as a variation of this task. A common approach employed by single-cell FMs in dealing with the cell-type annotation task is to use single-cell datasets for model training and treat the unannotated datasets as testing datasets. The challenge lies in predicting or annotating a set of cells that originate from studies different from the training datasets. Differently, SCimilarity pre-trained the model with cell-type labels to allow query of cell types directly for testing datasets. Moreover, the existence of cells with novel cell types (which are not included in the training datasets) further complicates the problem. We treat this task as a multi-label classification problem. Regarding the choices of metrics for evaluation, we also computed

the Pearson correlation coefficients between accuracy and weighted F1-score for the top-tier methods in our evaluation (including scGPT, Geneformer, and CellPLM), the coefficients are all 0.99 with a very small p-value (p-value<2.2e-308). Therefore, accuracy is a suitable metric used in our comparison.

In this task, we chose datasets with batch effect in two different cases. The intra-dataset case allows batch intersection, which means that the training and testing datasets can contain cells from the same batch. Here the total dataset was split into approximately 70% as a training dataset and the rest as a testing dataset. The inter-dataset case is cross-batch (cross-data) annotation, which means that the training and testing datasets are from different sources. We consider two datasets from the same tissue in this setting. More specifically, we consider the Pancreas cross dataset from the Pancreas, and the Liver cross dataset from the Liver. The main score for evaluation here is accuracy, which is defined as:

$$S_{\text{celltype}} = \frac{\#\text{corrected classified cells}}{\#\text{Total cells}}.$$

We also consider Precision, Recall and F1 scores in the analysis for ablation test, and details can be found in original appendix.2. Moreover, except the general comparison, we considered four datasets for the effect of different hyper-parameters, initial settings and optimizers: Pancrm, HumanPBMC, Cell Lines, and MCA, which is from *Mus musculus* rather than *Homo sapiens*. Finally, we investigated the contribution of freezing layers for cell-type annotation by freezing different numbers (from 0 to 11) of forward layers of scGPT.

Gene Function Prediction. Gene Function Prediction is important to identify the properties of genes across different conditions [205]. There are approximately 20,000 protein-encoding genes for humans [177] and only some are annotated with functions. Accurate prediction of gene function can help us understand and infer the role of genes in biological systems. Here we consider three types of functions for genes. The first one is dosage-sensitive or non-sensitive. Some genes are dosage-sensitive, which means that they are significant in the analysis of Copy Number Variants (CNVs) related to genetic diagnosis. The second one is Bivalent versus non-methylated. Bivalent chromatin structure is important to identify key developmental genes in embryonic stem cells (ESCs). Therefore, identifying bivalently marked genes versus unmethylated genes is important. The third one is Bivalent versus Lys4-only methylated. Lys4-only-methylated genes are also different from bivalently marked genes. We compare the model output with the true gene labels. We treat this task as a binary classification problem. Here we used the same metrics as the cell-type annotation task. We used a public dataset [205] considering only labeled genes in the dataset for prediction and evaluation.

Perturbation Prediction. Perturbation Prediction [169] is a task based on gene editing and single-cell sequencing technologies. After silencing some genes, we can obtain unperturbed and perturbed gene expression levels by sequencing, which allows us to explore the interactions between genes. A well-known technique is Perturb-seq [47]. In perturbation prediction, we intend to predict the gene expression level after gene editing. Here a model may predict seen gene perturbation in the testing datasets (an easier one) or predict unseen gene perturbation in the testing datasets (a more difficult one). We treat this task as a regression problem. The metric we used here is MPC, and the details can be found in original appendix.3. In the perturbation prediction task, we construct the paired input-target datasets by selecting the cells with non-control guide identity and then randomly sample cells under the control condition, and then combine them as the training and testing datasets. Our Perturb-seq datasets are from GEARS [169], which contain cells with three conditions: control; one gene perturbation; and two genes perturbation. In the evaluation process, we combined case 2 and case 3. We also compared the MPC across all genes to avoid bias in the gene selection process.

Gene Network Analysis. Gene Network Analysis is a downstream task for single-cell datasets [72]. The objective is to infer specific gene networks (for example, Gene Regulatory Network (GRN) or Gene Co-expression Network (GCN)) from different datasets. A GRN can assist in understanding the regulatory relationships between genes and predicted perturbation outcomes. The challenge in this task stems from the Granger causal relationship or time-dependent correlation [44]. A GCN can be used to analyze genes with similar functions or uncover the characteristics of genes in some diseases [199]. GCN and GRN are two different tasks because correlation does not imply causal relation [160]. This limitation means that we cannot determine which genes are the "causes" of expression level changes in other genes only based on embeddings similarity or correlation. We treat this task as a network inference problem. In the gene network analysis task, we considered using the overlap between ground truth genes in certain pathways and inferred genes as one metric, and using the ratio of significant pathways related to inferred genes as the other metric. Details can be found in original appendix.4.

Imputation. Imputation is a filling task related to missing data. Generally, we have two targets: 1. Perform imputation for scRNA-seq data to reduce data noise and fill in technical zeros with biologically meaningful values [77, 114]. 2. Perform imputation for spatial transcriptomic data because of unseen or unmeasured genes [11, 113]. According to [140], current spatial imputation methods do not show strong performance across different datasets. Using single-cell FMs, we can either use zero-shot learning to impute the unseen genes, or fine-tune our model based on reference scRNA-seq with more genes to perform imputation. We treat this task as a matrix-completion problem. Details of the metrics we used here can be found

in original appendix.5. In the imputation task, we used two public datasets from the mouse tissue to analyze the performance of single-cell FMs. One dataset is a scRNA-seq dataset, and another one is a spatial transcriptomic dataset. For the imputation of the scRNA-seq dataset, we used the output of the model decoder as imputation results. To evaluate this task, we used the biology preservation score from the batch effect correction and compared it to the score from raw data. For the imputation of spatial transcriptomic data, we considered two different settings to perform imputation. The first setting uses scRNA-seq to perform fine-tuning and inference based on spatial transcriptomic data. The second setting uses a zero-shot learning framework to directly perform inference based on spatial transcriptomic data. We consider using the correlation between known raw gene expression and known imputed gene expression as a metric.

Simulation. scRNA-seq Simulation is a data generation task. Leveraging the generative pre-training process of scGPT, we can generate gene expressions based on real datasets. Since a prevalent issue with scRNA-seq data simulation is the considerable divergence between simulation datasets and real datasets [248], direct generation from real datasets is preferred.

By arranging different sequences of masking genes or altering different seeds, we can generate new simulated scRNA-seq datasets from real ones. By modifying scGPT, we fine-tuned the baseline model based on the reference scRNA-seq dataset with reconstruction loss and utilized the outputs of the decoder part to generate the simulation datasets. Such simulation datasets do not have exactly the same gene expression profiles compared with the reference scRNA-seq dataset, but they should preserve the biological variation information from the reference dataset. We utilized scGPT to simulate datasets with batch effect or without batch effect. To simulate datasets with batch effect, we removed the loss function for removing batch effect in the original pipeline. To simulate datasets without batch effect, we kept the original pipeline for the batch effect correction task but utilized the outputs of the decoder rather than the encoder. We did not incorporate extra information for implementing this function. The quality of our simulation datasets can be evaluated by comparing them with the outputs of current simulation methods. We treat this task as a data-generation problem.

We used the same metrics as Batch Effect Correction for evaluation. It is possible to produce diverse reconstruction outcomes from a single real dataset by varying the random seeds. This feature enables us to create simulated single-cell datasets. Notably, these generated datasets retain the same gene sets as their input counterparts. We generate the simulation results by multiplying the output of the gene expression decoder and Bernoulli decoder. We have the flexibility to generate datasets either with or without batch effects. If we intend to produce datasets with batch effects, the gradient reverse loss function is omitted. Conversely, to generate datasets without

batch effects, we either retain this function or utilize single-batch data.

Model Scaling. [75, 96, 252] argue that due to the extensive number of parameters, FMs can manage specific tasks that small-scale models cannot handle. Such an attribute refers to the scaling law. We hypothesize that single-cell FMs may also possess this capability. To test this, we devise different scenarios similar to instances of model scaling to assess the performance of single-cell FMs. Such scenarios include: Cross-dataset Cell-type Annotation, Cross-species Cell-type Annotation, and Spatial Transcriptomic Data Analysis.

Ablation tests. Given that there is no existing work investigating the significance of different loss function components of single-cell FMs, we conducted a comprehensive analysis of the impact of various loss function components. These include the Masked Gene Expression loss (Mask Loss), Zero Log Probability Loss (Prob Loss), Gene Expression Prediction for Cell Modelling Loss (GEPC Loss), and Elastic Cell Similarity Loss (ECS Loss). We retain the task-specific loss in the fine-tuning process as the baseline condition (All remove). Consequently, our null hypothesis (H_0) is that the removal of the component C will not degrade the performance, while the alternative hypothesis (H_1) is that the removal of component C will worsen the model's performance. Comparing the score after eliminating a specific component to the score from the default setting allows us to determine whether to reject the null hypothesis. The test we employed here includes both the paired Students' t-test and Wilcoxon Rank-sum test. The details of the different loss function components are shown below.

1. Mask Loss: In both pre-training and fine-tuning, we mask the expression levels of some genes, denoted as $\mathcal{M}_{\text{mask}}$, for gene expression prediction. The mask loss is motivated from this setting and works as follows,

$$\begin{aligned}\tilde{\boldsymbol{x}}^{(i)} &= \text{MLP}\left(\boldsymbol{h}_n^{(i)}\right), \\ \mathcal{L}_{\text{mask}} &= \frac{1}{|\mathcal{M}_{\text{mask}}|} \sum_{j \in \mathcal{M}_{\text{mask}}} \text{ce}\left(\tilde{x}_j^{(i)}, x_j^{(i)}\right),\end{aligned}$$

where $\tilde{\boldsymbol{x}}^{(i)}$ represents the predicted gene expression levels for cell i, while $\tilde{\boldsymbol{x}}_j^{(i)}$ represents the ground truth. $\boldsymbol{h}_n^{(i)}$ represents the embeddings for cell i with n genes. MLP means that we use linear Multi-Layer Perceptron (MLP) Neural Networks as the output of single-cell FMs under the setting of this loss function.

2. Prob Loss: Since single-cell data can be treated as count data, we can use the Bernoulli distribution to model the occurrence of the expression in masked genes and use the Maximum Likelihood Estimation (MLE) approach to estimate the parameters of Bernoulli distribution. Such loss function can be used to determine whether the given masked gene position carries zero expression levels or not. Prob Loss works as

follows:

$$\begin{aligned}
\text{Prob}_i &= \text{MLP}\left(\boldsymbol{h}_n^{(i)}\right), \\
\text{Dist}_i &= \textit{Bernoulli}(\text{Prob}_i), \\
\mathcal{L}_{\text{Prob}} &= -\text{LogProb}_i(x^{(i)} > 0),
\end{aligned}$$

where Prob_i represents the output of the model for the parameter estimation for cell i. The estimation is based on one MLP model. Dist_i represents the Bernoulli distribution based on Prob_i. LogProb_i is the log probability based on the relationship between gene expression levels of cell i and zero, which can be computed based on Dist_i.

3. GEPC Loss: This loss function is similar to Mask Loss, but now we predict the gene expression levels based on cell embeddings or cell representation. For cell i with gene j, we create a query vector $\boldsymbol{q}_j$ and represent the cell based on $\boldsymbol{h}^{(i)}c$. We can use the inner product between these two terms to predict gene expression levels. That is,

$$\begin{aligned}
\boldsymbol{q}_j &= \text{MLP}\left(\text{emb}_{\text{g}}\left(\boldsymbol{t}_g^{(i)}\right)\right), \\
\tilde{x}_j^{(i)} &= \boldsymbol{q}_j \cdot \boldsymbol{W}\boldsymbol{h}_c^{(i)}, \\
\mathcal{L}_{\text{GEPC}} &= \frac{1}{|\mathcal{M}_{\text{mask}}|} \sum_{j \in \mathcal{M}_{\text{mask}}} \text{ce}\left(\tilde{x}_j^{(i)}, x_j^{(i)}\right),
\end{aligned}$$

where $\text{emb}_{\text{g}}(\boldsymbol{t}_g^{(i)})$ represents the embeddings of the gene token g in cell i. We also use one MLP to generate the query embeddings. The following process is similar to the steps for $\mathcal{L}_{\text{mask}}$.

4. ECS Loss: This loss function is used to control the similarity of the embeddings of cells in the same batch, which is defined as:

$$\mathcal{L}_{\text{ECS}} = -\left(\text{CosSim}\left(\boldsymbol{h}_c^{(i)}, \boldsymbol{h}_c^{(i')}\right) - \beta\right)^2,$$

where CosSim represents the cosine similarity function, i and i' are the indices of the two cells. The idea of this loss function is to ensure the similarity between paired cells is higher than the predefined threshold β. Moreover, dissimilar pairs should be more dissimilar, respectively.

Task-specific fine-tuning process. For the experiments we have in the Results section, we load the pre-training weights based on the requirement of different single-cell FMs. The pre-training weights we used can be found in our GitHub folder. After the fine-tuning process, we recorded the related metrics and conducted more analysis. In the experiments for all tasks, we chose scGPT as a baseline and representative model for the following three reasons. Firstly, scGPT is an open-source single-cell FM with the largest datasets for pre-training, and it is well-defined with detailed tutorials.

In addition, the architecture of scGPT is easy to adjust and includes multiple loss function components. The functions of Gene Function Prediction, Imputation, and Simulation for scGPT were designed in scEval for evaluation. The settings of hyper-parameters for these tasks are transferred from the design for cell-type annotation and batch effect correction tasks. Moreover, different single-cell FMs have overlaps and unique terms in the pre-training and fine-tuning framework, but scGPT is the most general one. In each task, we also included task-specific methods as comparisons. Moreover, for Geneformer, scBERT, CellPLM, and CellLM, we also evaluated their performance based on shared hyper-parameters or optimizers with scGPT to verify if our rules found in scGPT can be extended for other single-cell FMs.

Statistical Analysis. scEval is an evaluation framework to benchmark the performances of scFMs across different tasks based on various datasets. In total, we consider eight tasks with 29 datasets. For data pre-processing, we refer model-specific processing. All single-cell data are processed with Scanpy. Only Geneformer is processed by ranking the gene expression for generating cell and gene representations. For discrete data comparisons, we employed the Wilcoxon Rank-Sum test for paired data and the Mann-Whitney U test for unpaired data. For continuous data comparison, we utilize Pearson Correlation Coefficient to measure data similarity. The significance level for all hypothesis tests was set at 0.05.

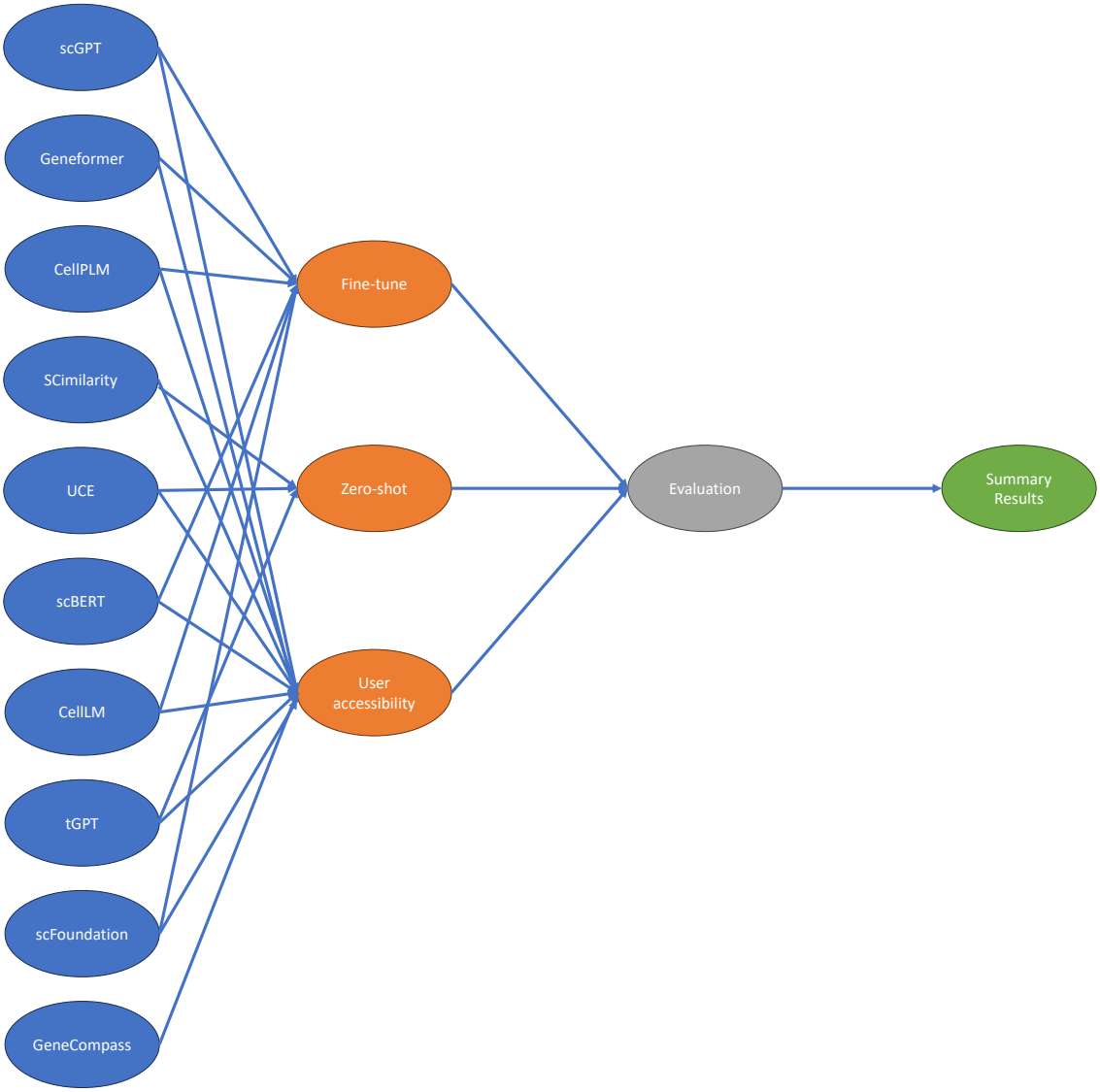
1.5 Data availability and reproducibility

We used the resources from the Yale High Performance Center (Yale HPC) to conduct all of the experiments. Our maximum running time for each dataset is 24 hours. The version of GPU we used is NVIDIA A100 (40 GB). The random seed of all experiments is the same as the default setting of the original papers. The information of datasets and the download link can be found in original appendix. This study includes no data deposited in external repositories. The selected cell lines are clean and contamination free. All single-cell data are publicly available. The datasets and computer code produced in this study are available in the following databases: Adamson data: Web Resource (`pert_data.load(data_name='adamson')`); Cell Lines data (RRID: CVCL_0065, CVCL_0063): GitHub (`https://github.com/JinmiaoChenLab/Batch-effect-removal-benchmarking/tree/master`); DC data: GitHub (`https://github.com/JinmiaoChenLab/Batch-effect-removal-benchmarking/tree/master`); Dixit data: Web Resource (`pert_data.load(data_name='dixit')`); Heart atlas data: Web Resource (`https://www.ebi.ac.uk/ena/browser/view/PRJEB39602`); Human spatial data: Web Resource (`http://research.libd.org/spatialLIBD/`); HumanPBMC data: GitHub (`https://github.com/JinmiaoChenLab/Batch-effect-removal-benchmarking/tree/master`); HSPC; Immune All Human

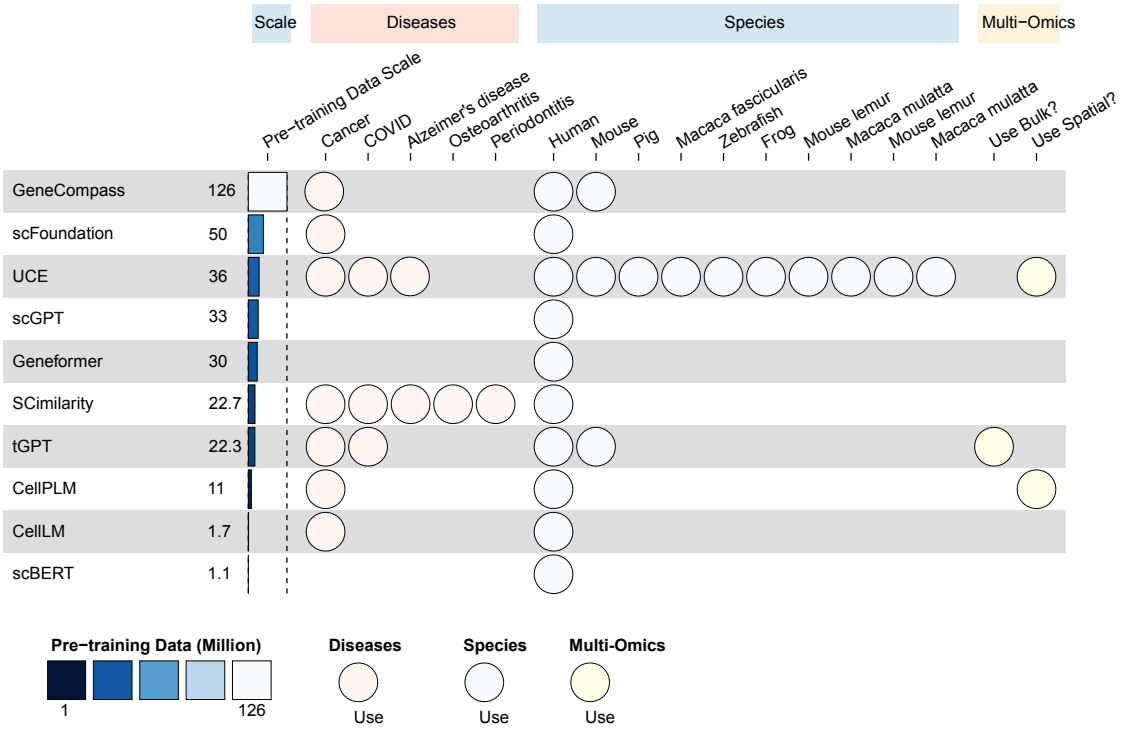
data: Figshare (https://figshare.com/articles/dataset/Benchmarking_atlas-level_data_integration_in_single-cell_genomics_-_integration_task_datasets_Immune_and_pancreas_/12420968); Immune atlas data: Web Resource (https://cellxgene.cziscience.com/collections/ddfad306-714d-4cc0-9985-d9072820c530); Liver cross data: Gene Expression Omnibus (https://www.ncbi.nlm.nih.gov/geo/query/acc.cgi?acc=GSE115469; https://www.ncbi.nlm.nih.gov/geo/query/acc.cgi?acc=GSE124395); Lung atlas data: Web Resource (https://cellxgene.cziscience.com/collections/6f6d381a-7701-4781-935c-db10d30de293); MB Spatial data: Python Package (squidpy.datasets.slideseqv2()); MCA data: GitHub (https://github.com/JinmiaoChenLab/Batch-effect-removal-benchmarking/tree/master); MHSP data: GitHub (https://github.com/JinmiaoChenLab/Batch-effect-removal-benchmark tree/master); Mouse Spatial data: GitHub (https://github.com/broadinstitute/Tangram/blob/master/tutorial_tangram_with_squidpy.ipynb); Mouse scRNA-seq data: GitHub (https://github.com/broadinstitute/Tangram/blob/master/tutorial_tangram_with_squidpy.ipynb); Norman data: Web Resource (pert_data.load(data_name='norman')); PanglaoDB: Web Resource (https://panglaodb.se/view_data.php?sra=SRA553822&srs=SRS2119548). Pancrm: GitHub (https://github.com/JinmiaoChenLab/Batch-effect-removal-benchmarking/tree/master); PBMC 368K: GitHub (https://github.com/AprilYuge/ResPAN); PBMC Multi-omics: Web Resource (https://stuartlab.org/signac/articles/pbmc_multiomic); Kidney: Gene Expression Omnibus (https://www.ncbi.nlm.nih.gov/geo/query/acc.cgi?acc=GSE117498); HSPC: Gene Expression Omnibus (https://www.ncbi.nlm.nih.gov/geo/query/acc.cgi?acc=GSE151302); Endo: Web Resource (https://github.com/qinzhu/VisCello.eht); Pancreas cross: Web Resource (https://www.nature.com/articles/s41467-023-35923-4#data-availability); Liver cross: Web Resource (https://github.com/SydneyBioX/scClassify).

The codes of **scEval** can be found in https://github.com/HelloWorldLTY/scEval with MIT license.

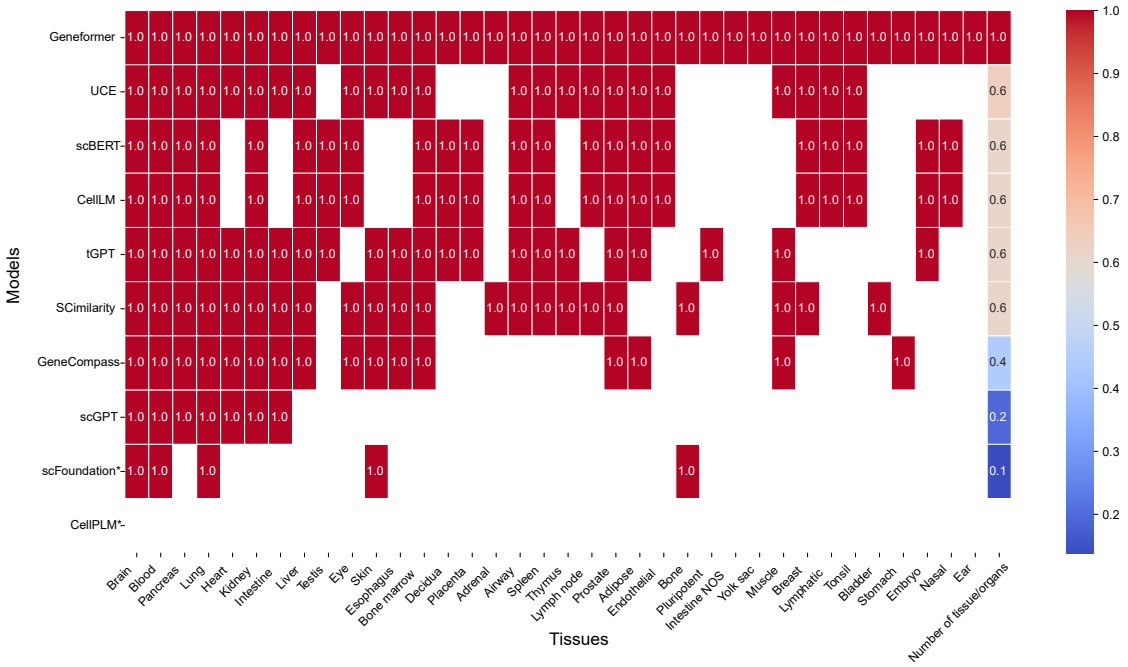
1.6 Supplementary Figures



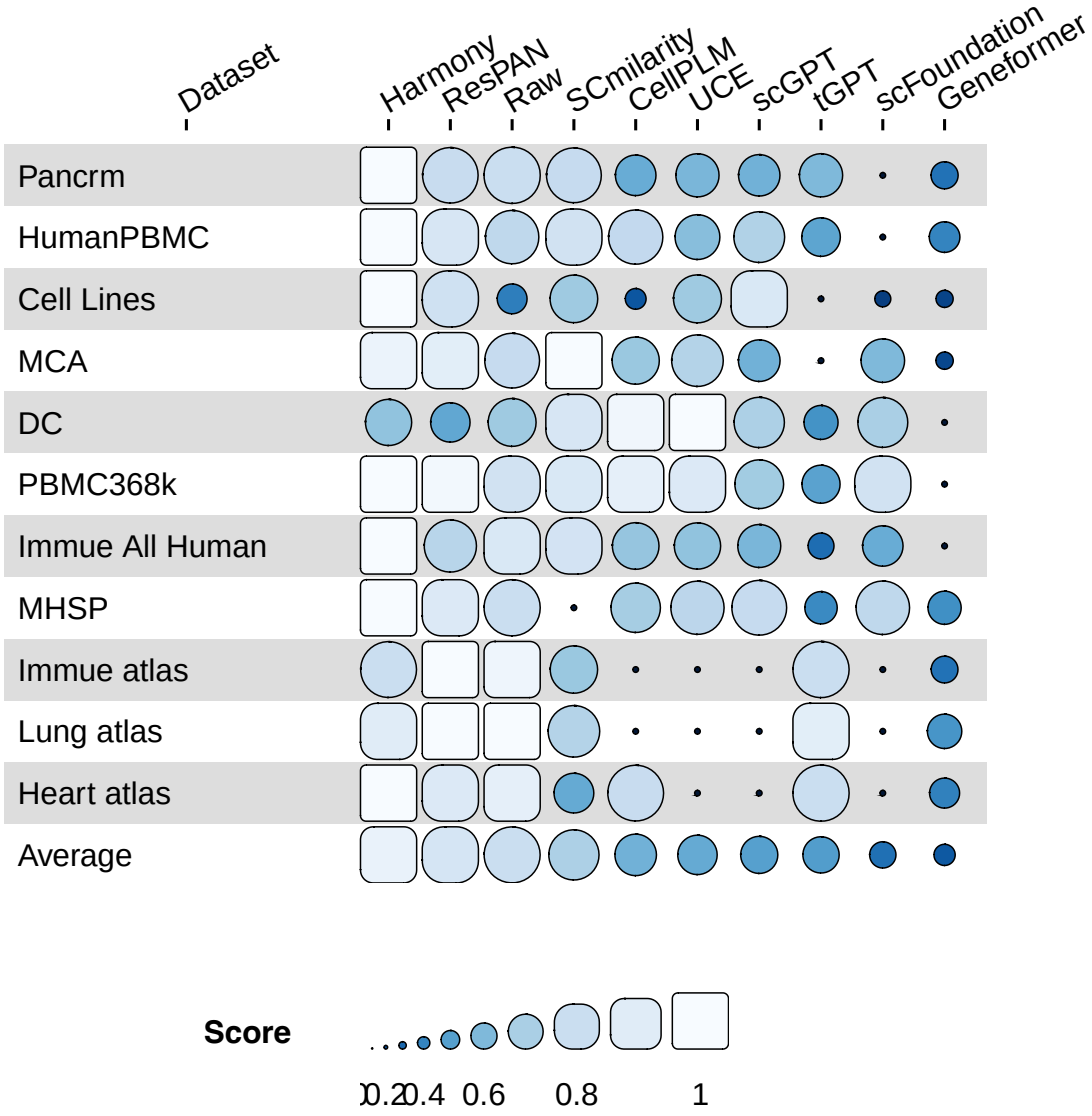
Appendix Figure S 1.6.1: Workflow of our evaluation. Here we considered ten single-cell FMs and generated their outputs based on their default usage settings. We also consider the evaluation of user accessibility for both open-source and closed-source tools. We then evaluated the outputs with task-specific metrics. Finally, we summarized our discoveries in this manuscript.



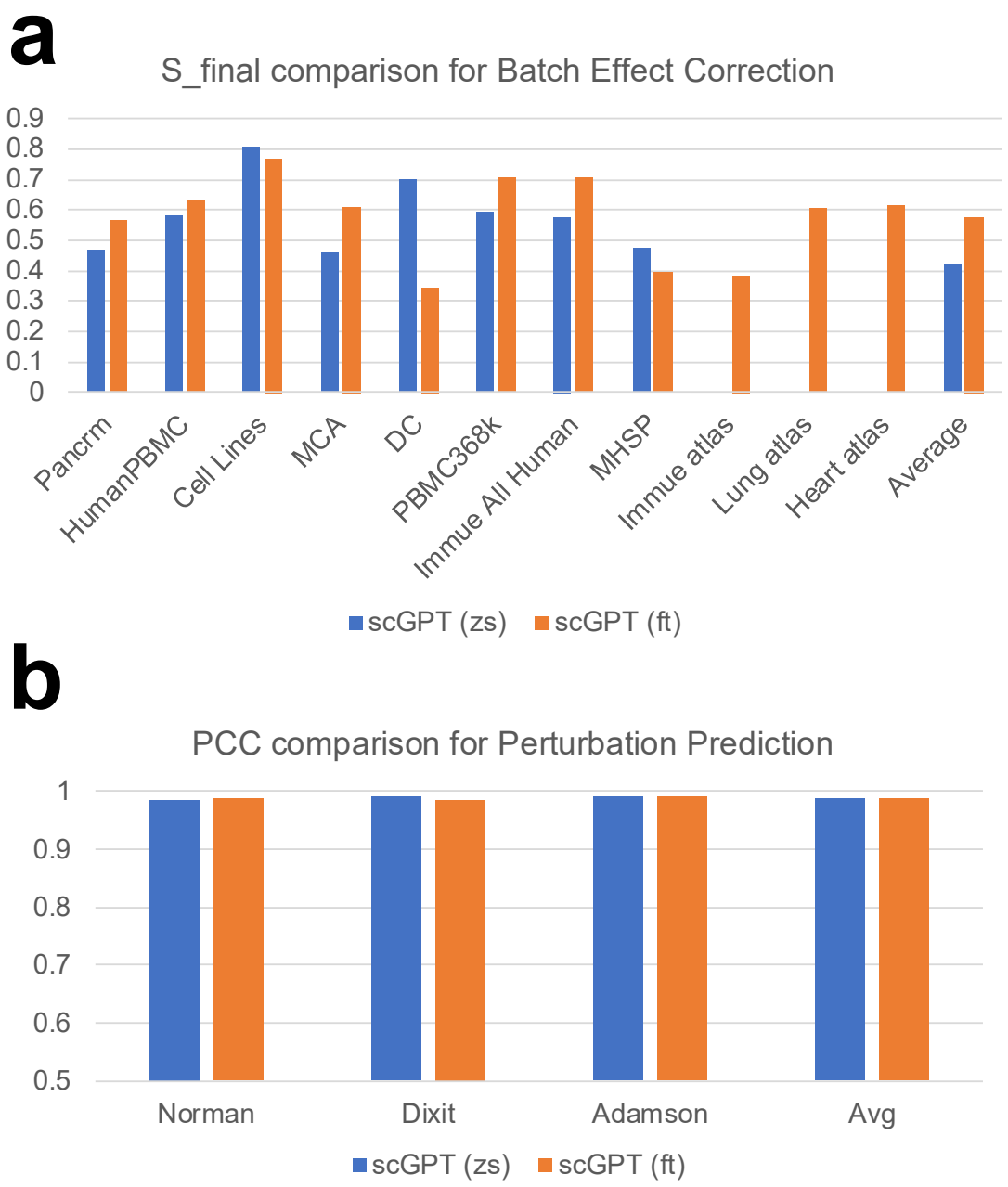
Appendix Figure S 1.6.2: Statistics of pre-training datasets for different single-cell FMs. Here we consider summarizing the scale of pre-training datasets, the overlap for major diseases in the pre-training datasets, the overlap for major species in the pre-training datasets, and the overlap of multi-omics in the pre-training datasets, across all the methods. The scale of pre-training dataset is a continuous variable, whether other statistics are recorded based on a binary variable.



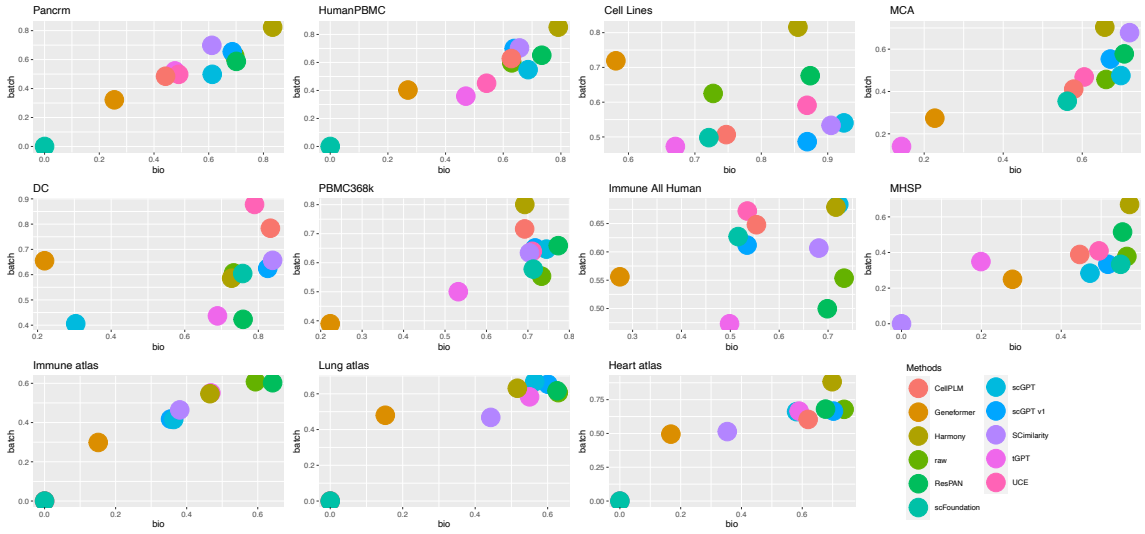
Appendix Figure S 1.6.3: The overlap of major human tissues or organs across different single-cell FMs' pre-training datasets. Here we choose tissues or organs included by Geneformer as a baseline because it is published and it includes most of the major information for the cells from human. *: For scFoundation, we only record the major tissues or organs based on its manuscript since we did not find the data sources description from its manuscript and supplementary files. For CellPLM, we did not record the major tissues or organs since we did not find the data source description from its manuscript and supplementary files.



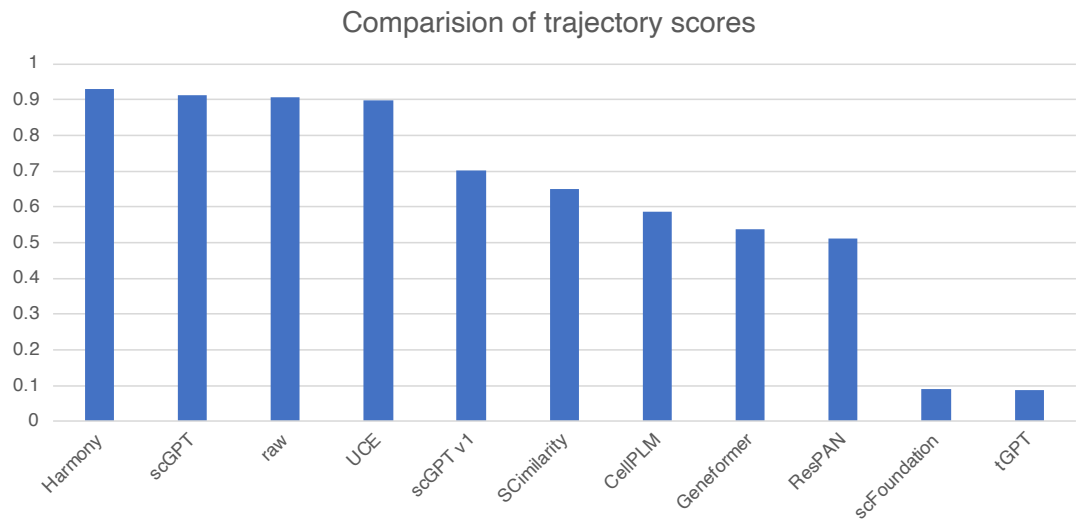
Appendix Figure S 1.6.4: S_{final} across different scFMs based on the zero-shot setting.



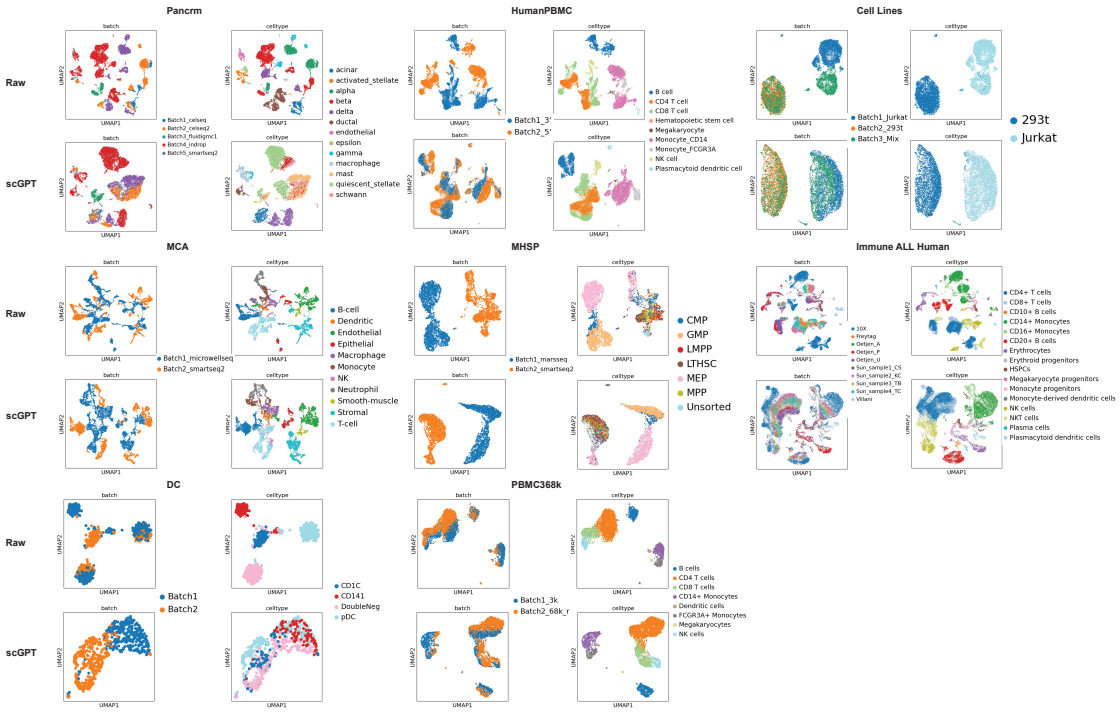
Appendix Figure S 1.6.5: Compassion of zero-shot setting and fine-tuning setting of scGPT. (a) Comparison based on S_{final} for batch effect correction. (b) Comparison based on PCC for perturbation prediction.



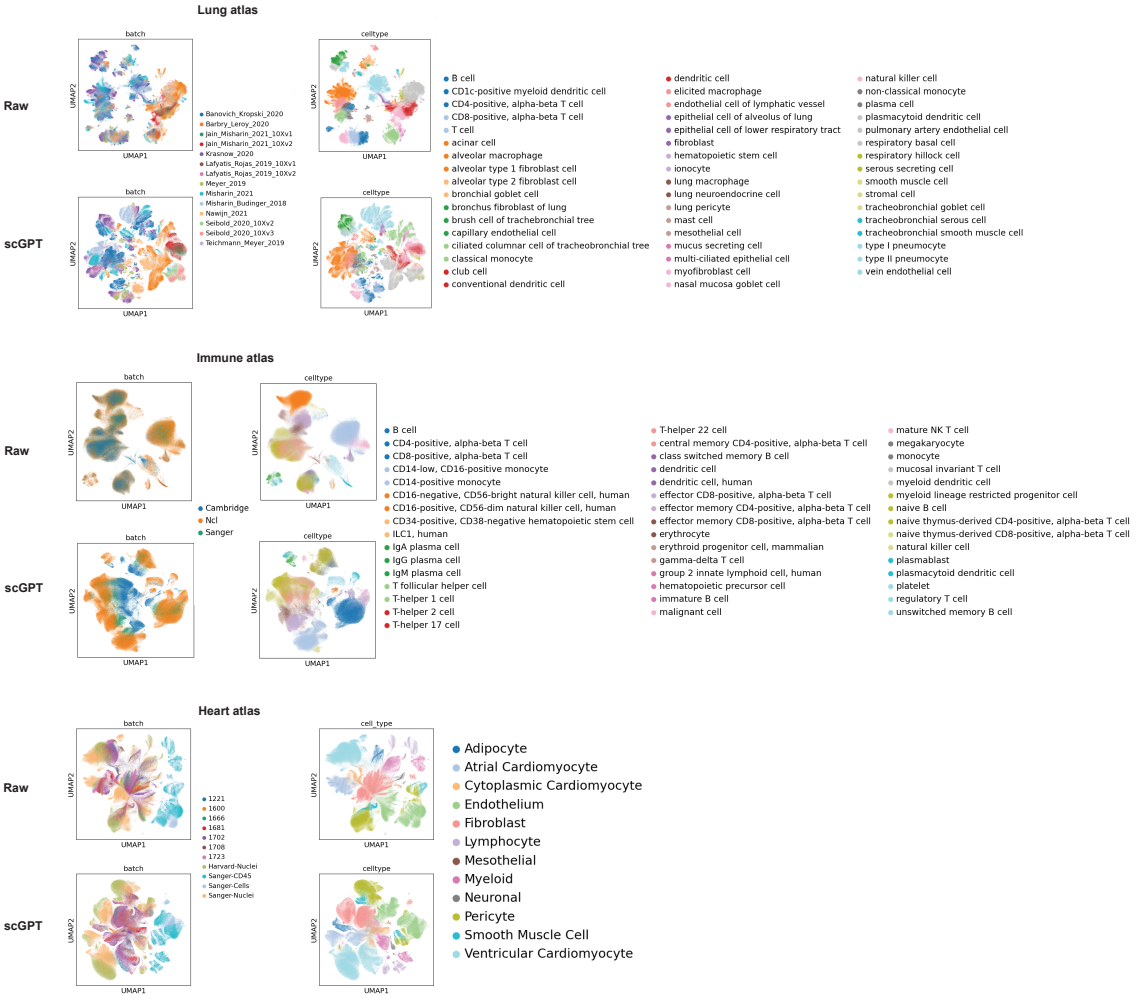
Appendix Figure S 1.6.6: Visualization of batch effect removal score and biology variation conservation score in the evaluation of batch effect correction task for each dataset.



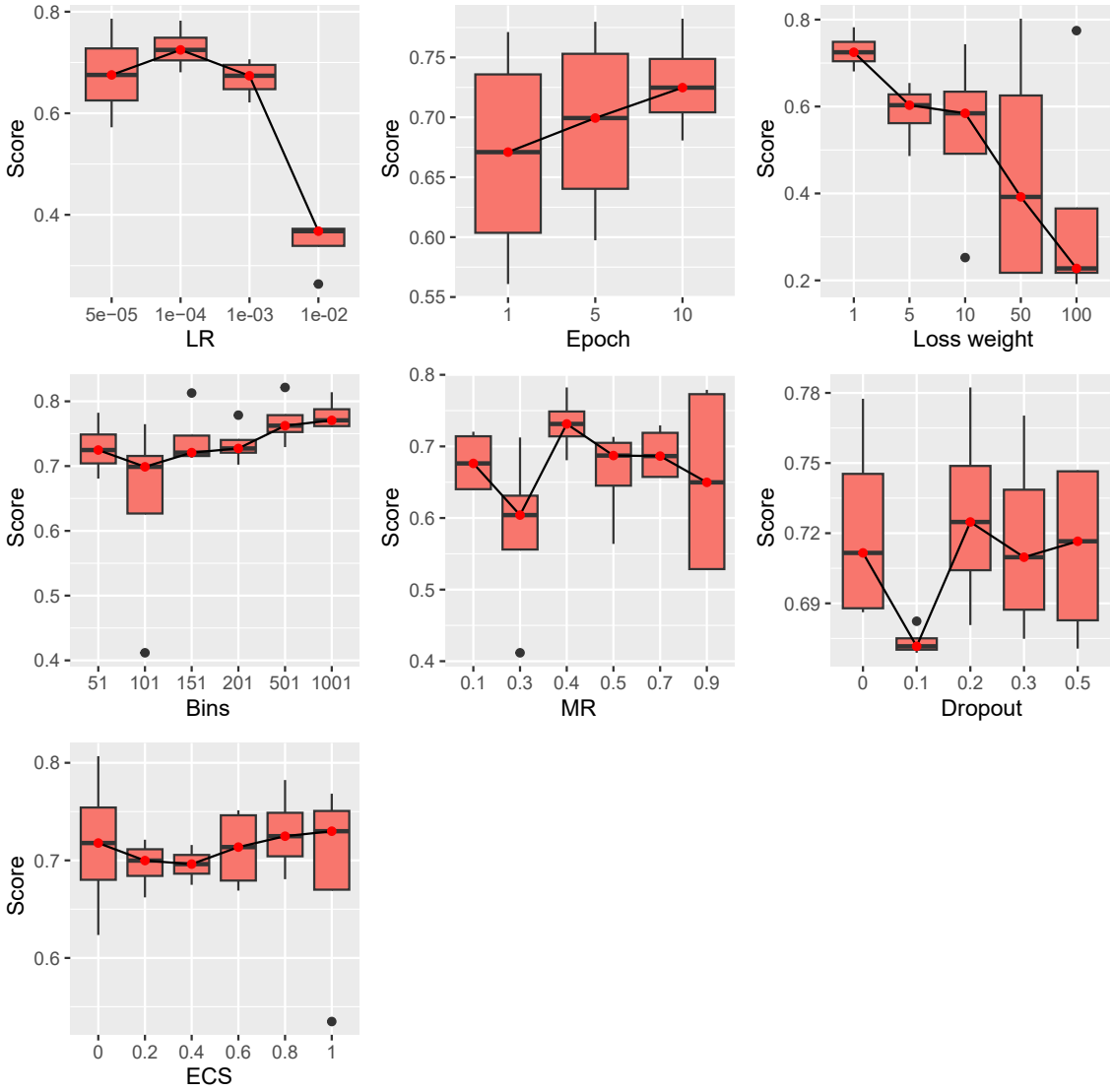
Appendix Figure S 1.6.7: Comparisons for different methods on preserving the trajectory information based on the trajectory score.



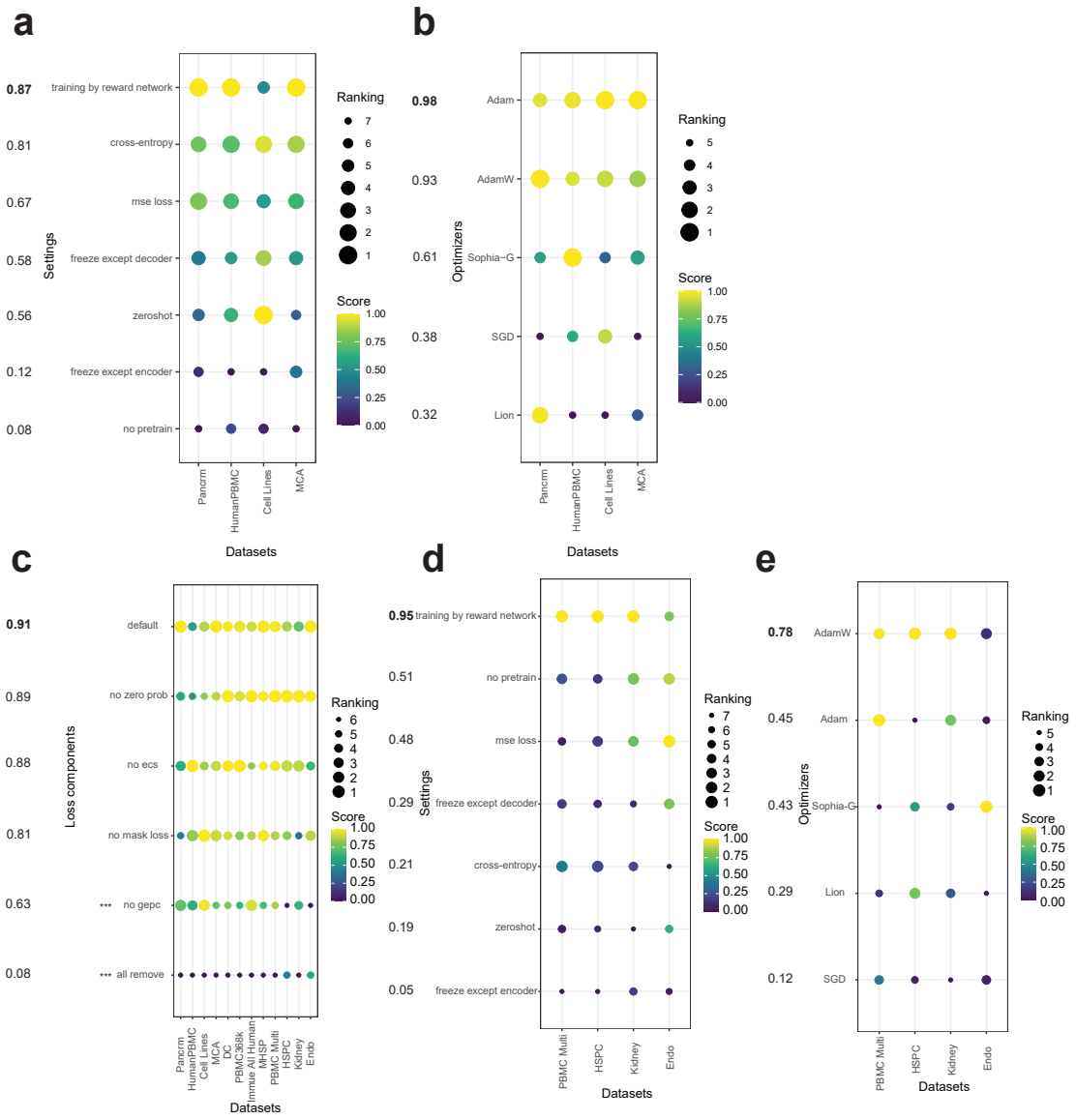
Appendix Figure S 1.6.8: UMAPs for raw data and embeddings of scGPT after batch effect correction.



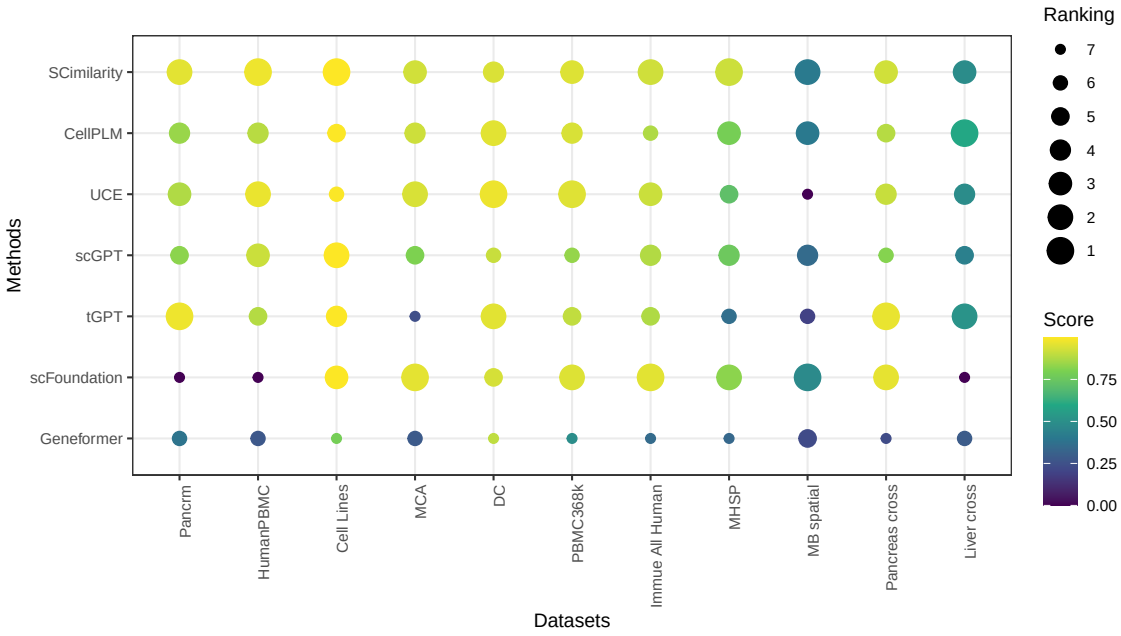
Appendix Figure S 1.6.9: UMAPs for raw data and embeddings of scGPT after batch effect correction.



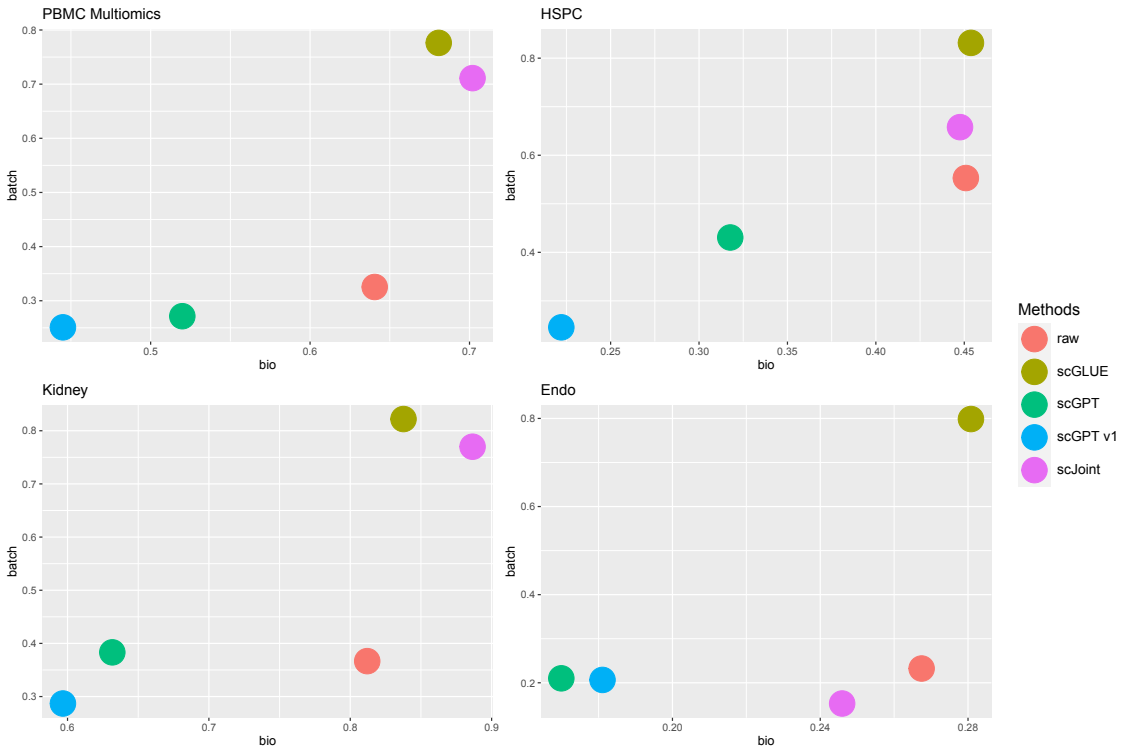
Appendix Figure S 1.6.10: Benchmarking results for different hyper-parameters for batch effect correction (one-sided Wilcoxon Rank-sum Test, $p - value = 0.03, n = 11$).



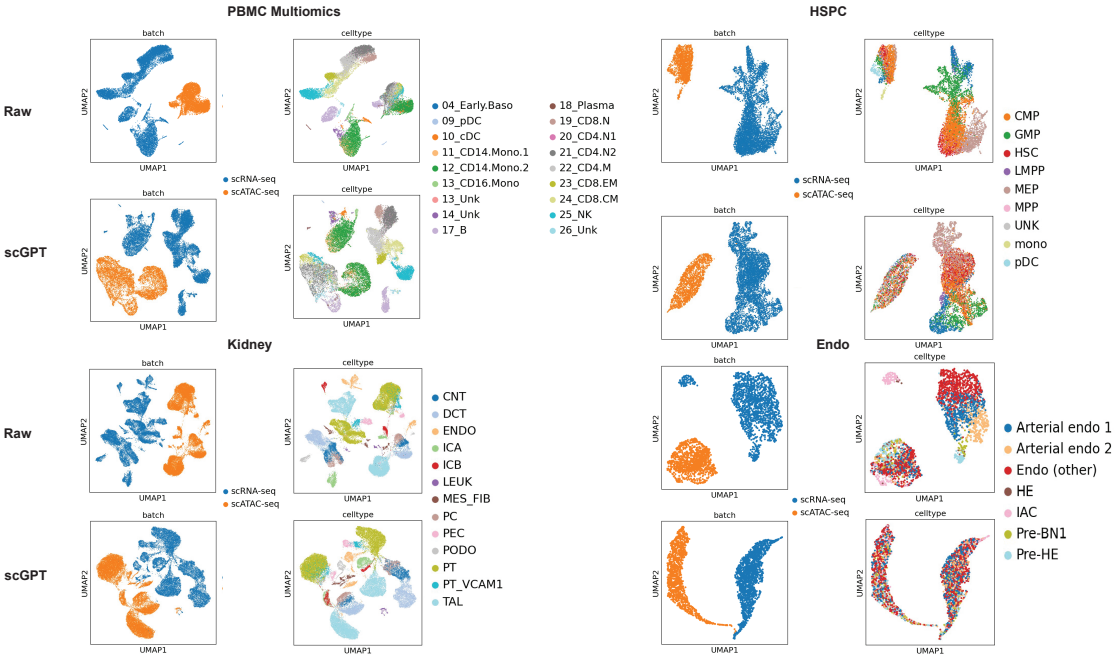
Appendix Figure S 1.6.11: Overall evaluation of different components of scGPT. (a): The final score of different settings across different datasets for batch effect correction. (b): The final score of different optimizers across different datasets for batch effect correction. (c): The final score of including different loss function terms across different datasets. The number of stars represents the significance level (one sided Wilcoxon Rank-Sum test, sample size $n = 12$, $***: p - value < 0.005, **: p - value < 0.05, *: p - value < 0.1$). The numbers on the left side of each sub-figure represent the average score across different datasets for one condition. We combine the datasets for batch effect correction and multi-omic data integration for testing. (d): The final score of different settings across different datasets for multi-omic data integration. (e): The final score of different optimizers across different datasets for multi-omic data integration.



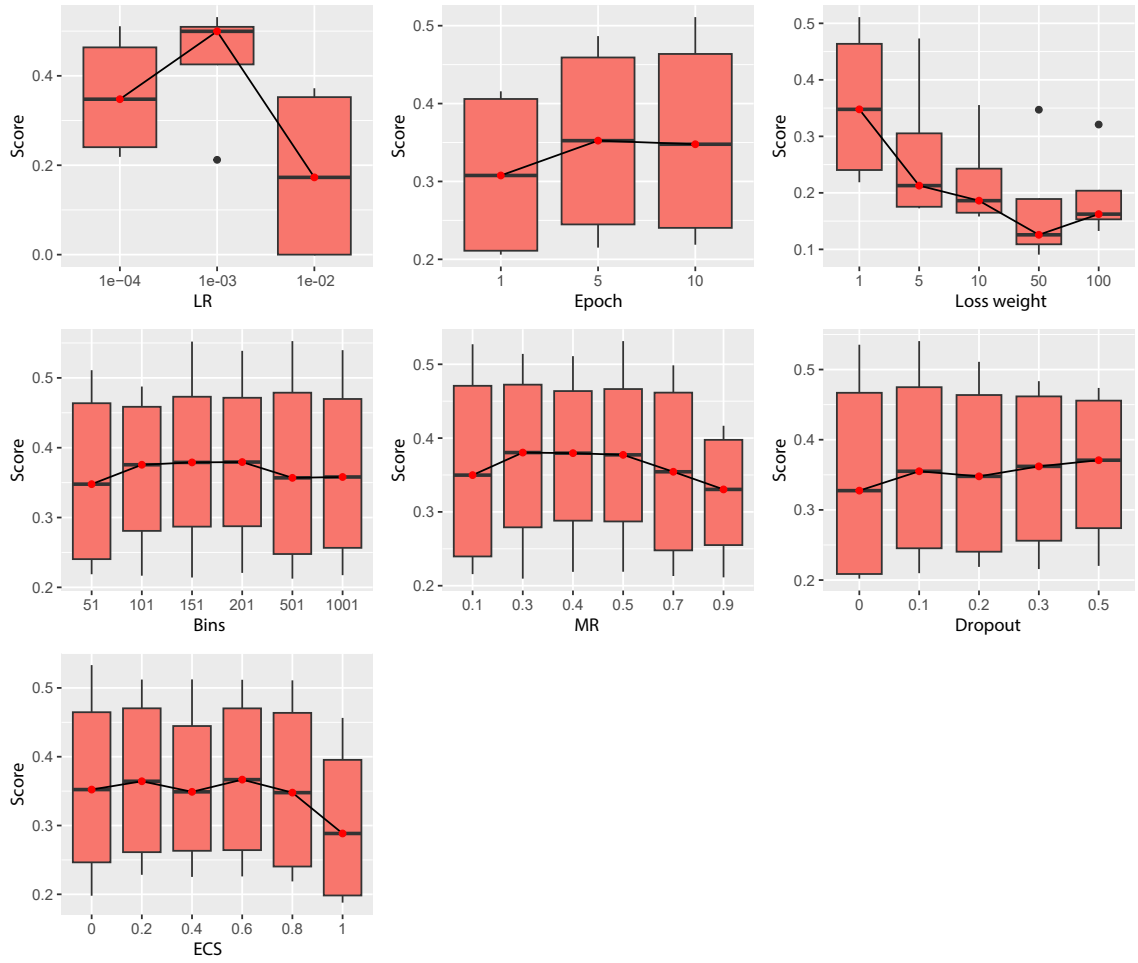
Appendix Figure S 1.6.12: Accuracy of different scFMs (zero-shot performances) for cell-type annotation.



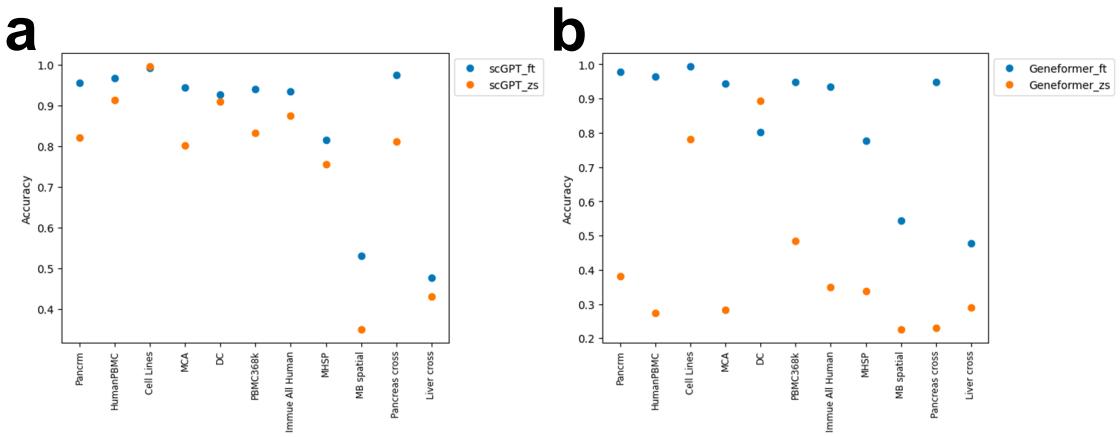
Appendix Figure S 1.6.13: Visualization of batch effect removal score and biology variation conservation score in the evaluation of multi-omic data integration task for each dataset.



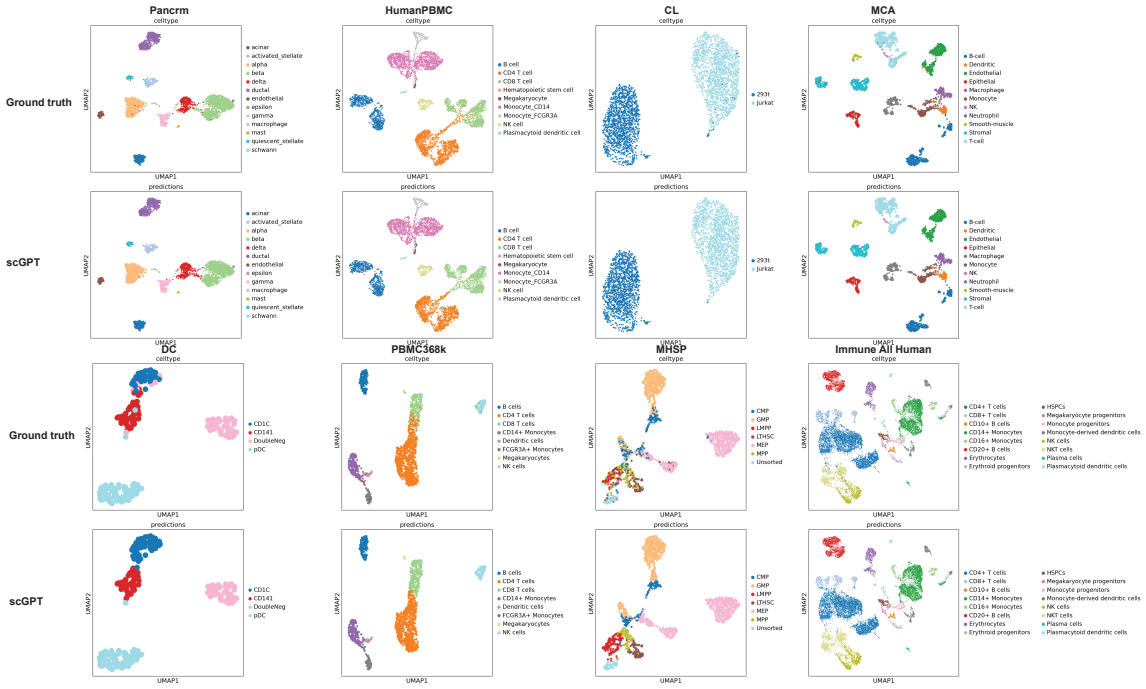
Appendix Figure S 1.6.14: UMAPs for raw data and embeddings of scGPT after multi-omic data integration.



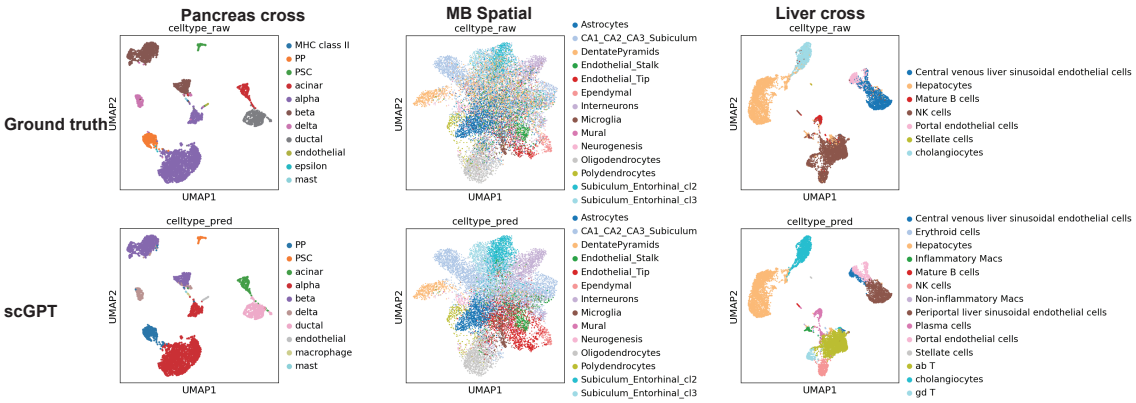
Appendix Figure S 1.6.15: Tuning parameters for multi-omic data integration. Subfigures represent the score of scGPT under different hyper-parameters after training (sample size $n = 4$).



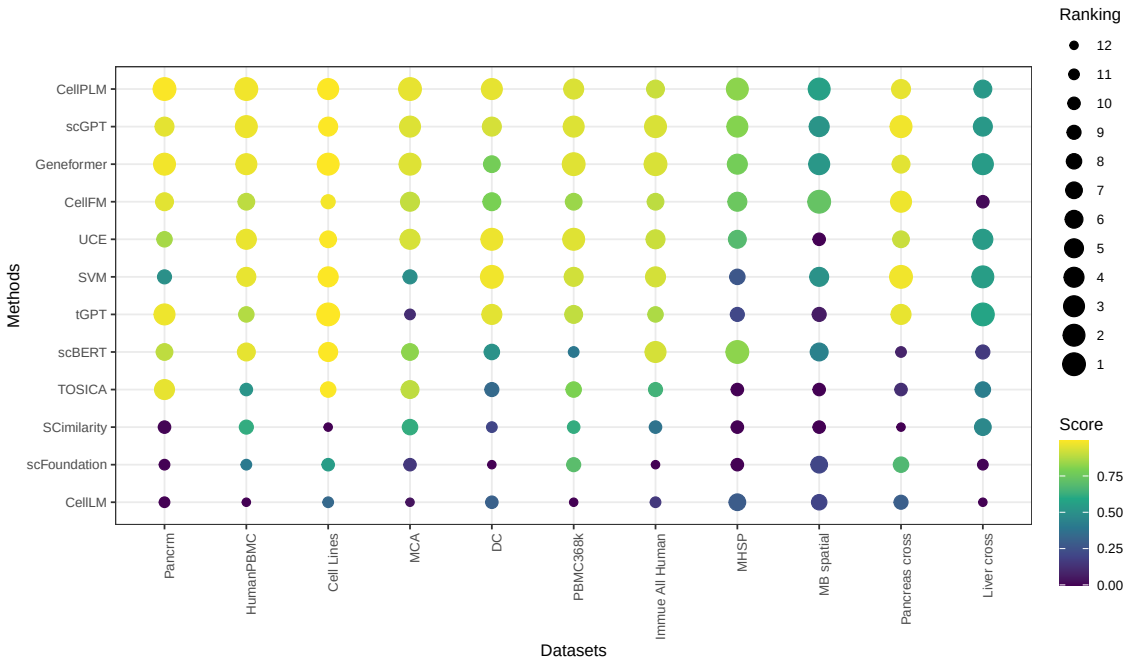
Appendix Figure S 1.6.16: Results of cell-type annotation with the different modes of scGPT and Geneformer. (a) The performance of cell-type annotation of scGPT based on both fine-tuning mode and zero-shot learning mode. (b) The performance of cell-type annotation of Geneformer based on both fine-tuning mode and zero-shot learning mode.



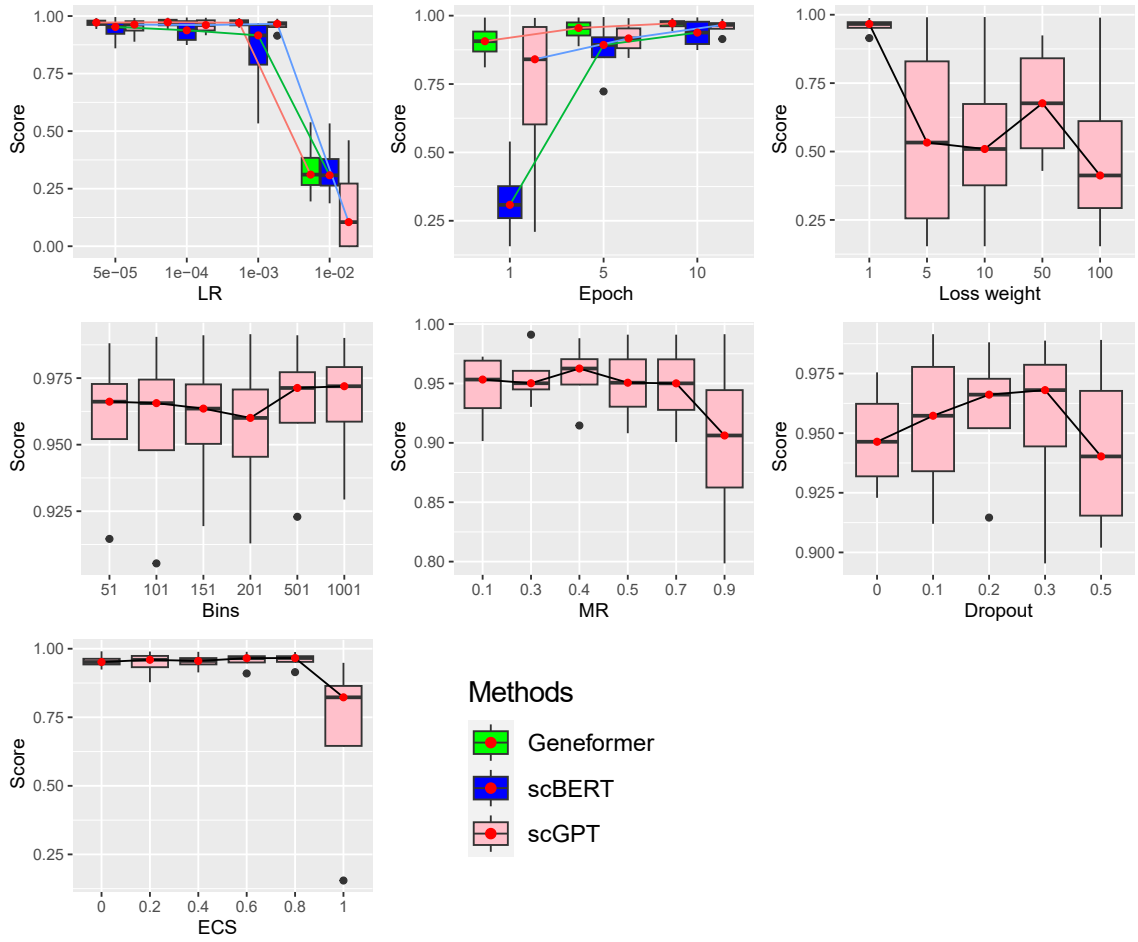
Appendix Figure S 1.6.17: UMAPs for ground truth cell types and prediction results based on scGPT.



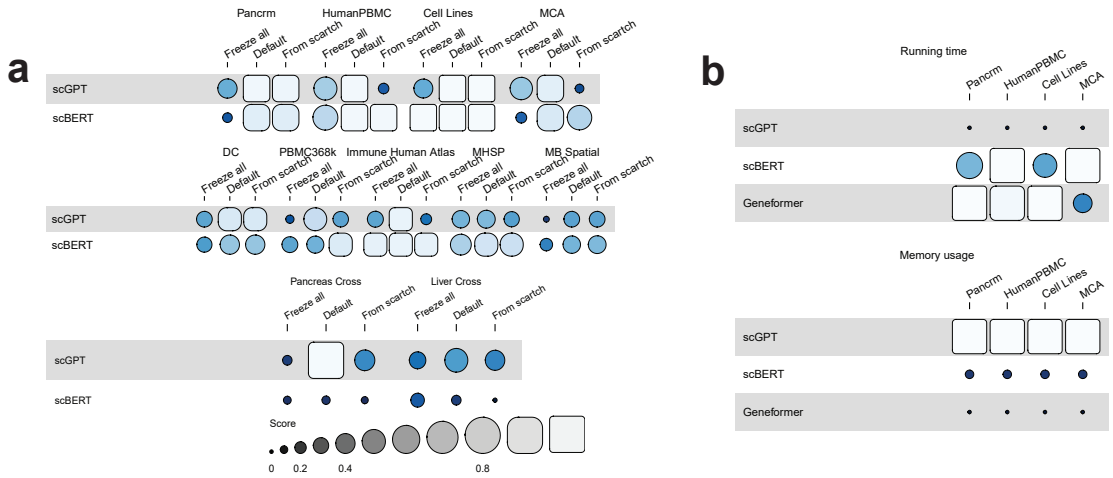
Appendix Figure S 1.6.18: UMAPs for ground truth cell types and prediction results based on scGPT.



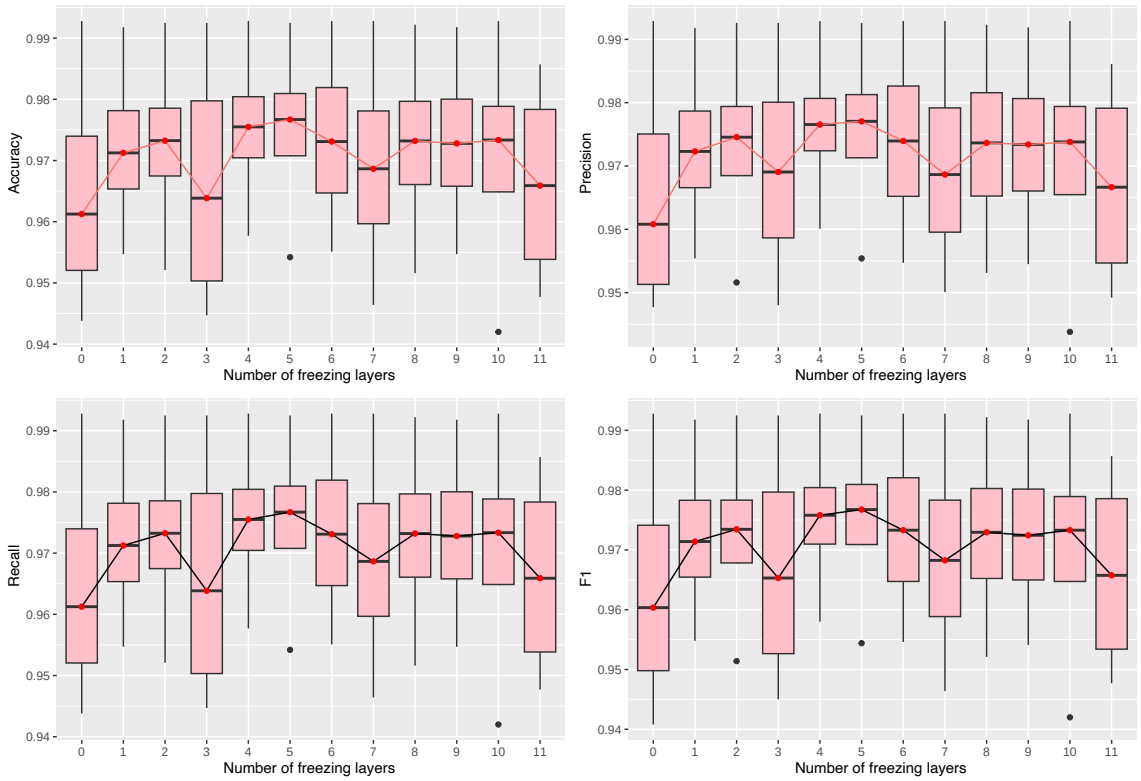
Appendix Figure S 1.6.19: Weighted F1 score across different methods for cell-type annotation.



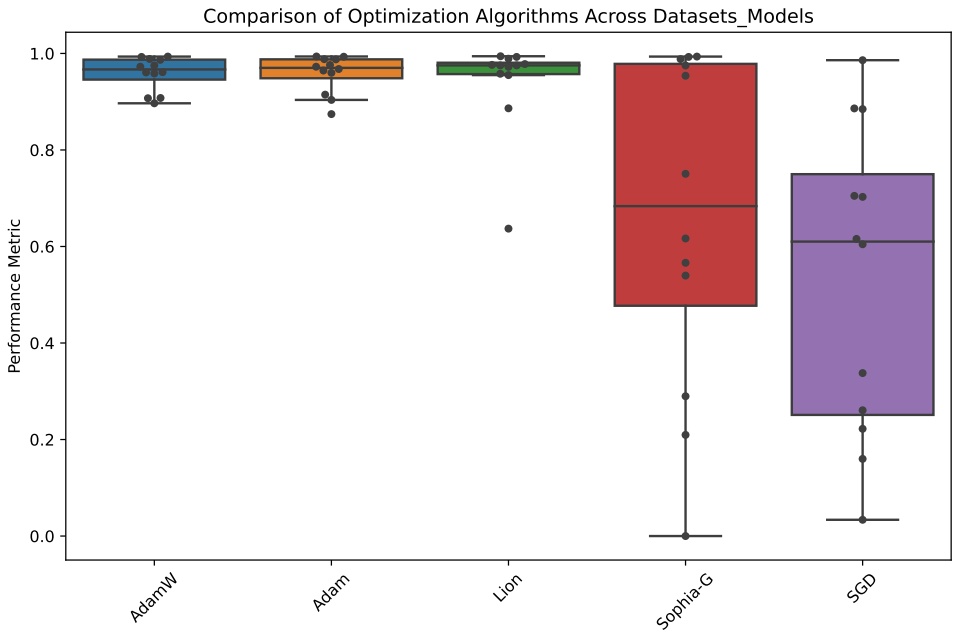
Appendix Figure S 1.6.20: Tuning parameters for cell-type annotation. Sub-figures represent the score of scGPT under different hyper-parameters after training (sample size $n = 4$).



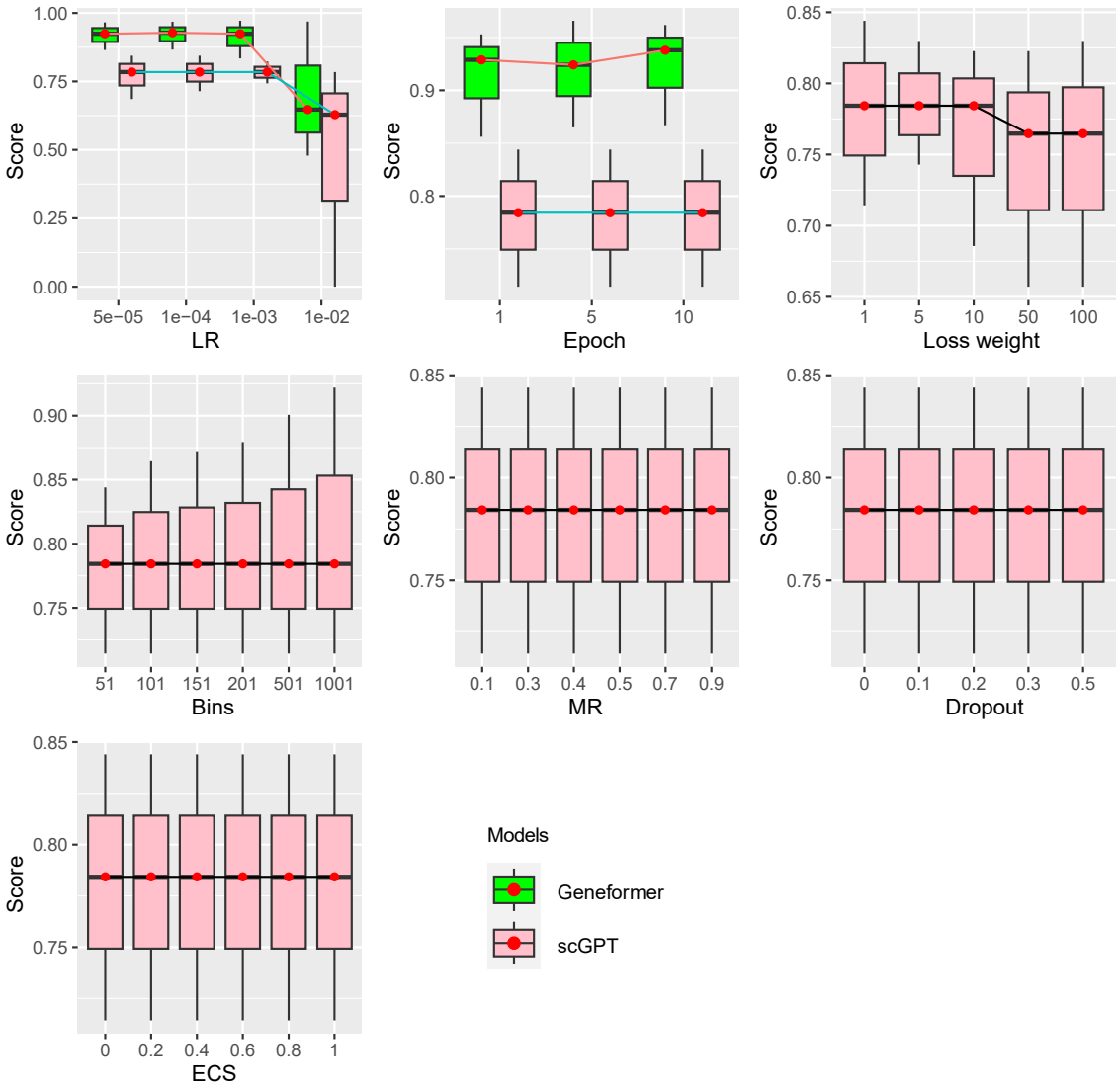
Appendix Figure S 1.6.21: Results of different settings, running time, and memory usage for cell-type annotation task. (a): Accuracy of scGPT and scBERT for the Cell-type Annotation task across different datasets. (b): Scaled running time (up) and scaled memory usage (down) statistics for all three single-cell FMs.



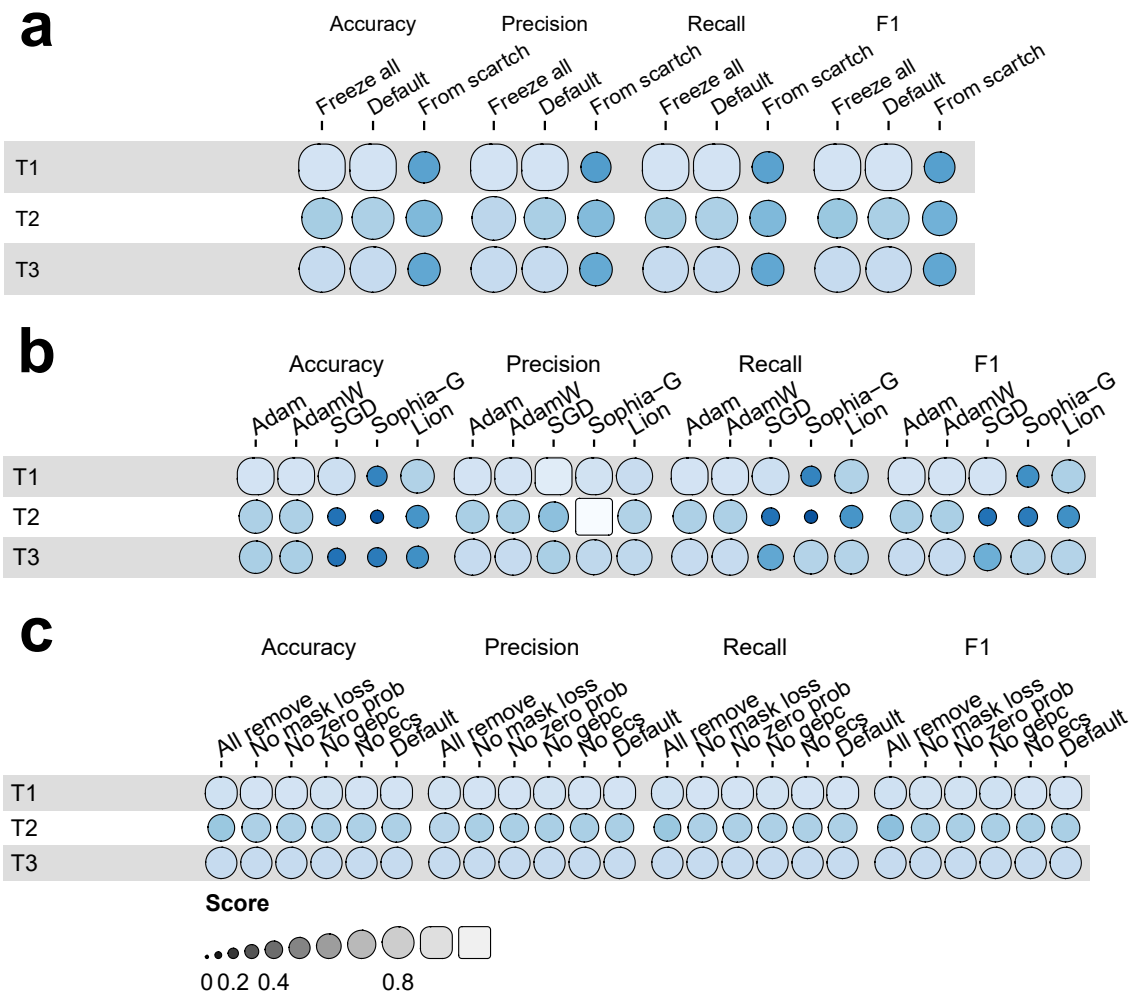
Appendix Figure S 1.6.22: Results of cell-type annotation with the different number of freezing layers based on scGPT. The x-axis represents the number of freezing layers and the y-axis represents the value of specific metrics (sample size $n = 4$).



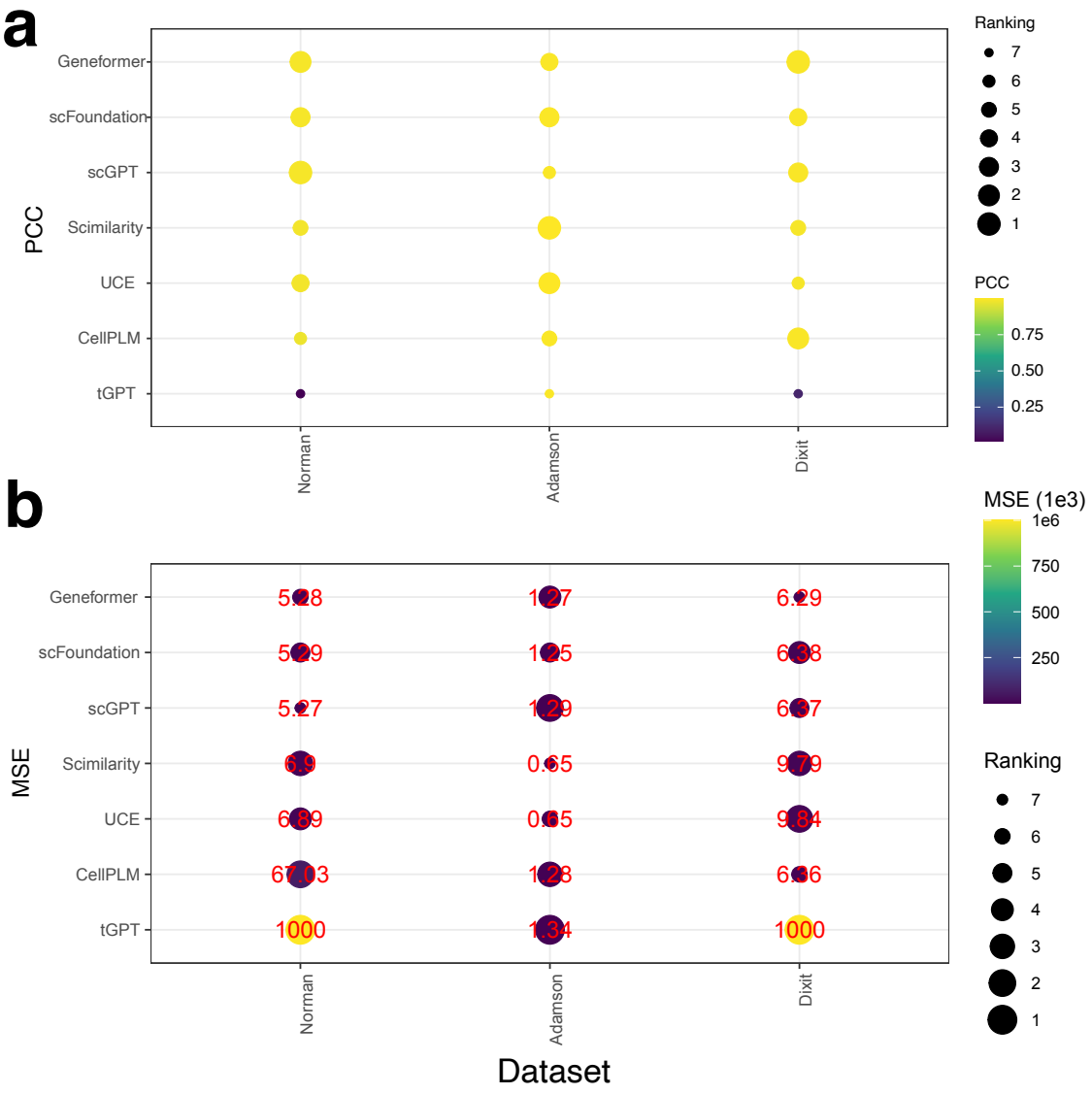
Appendix Figure S 1.6.23: Benchmarking results for different optimizers for cell-type annotation. Here each box contains the scores of fine-tuned scFMs across different datasets (sample size $n = 10$).



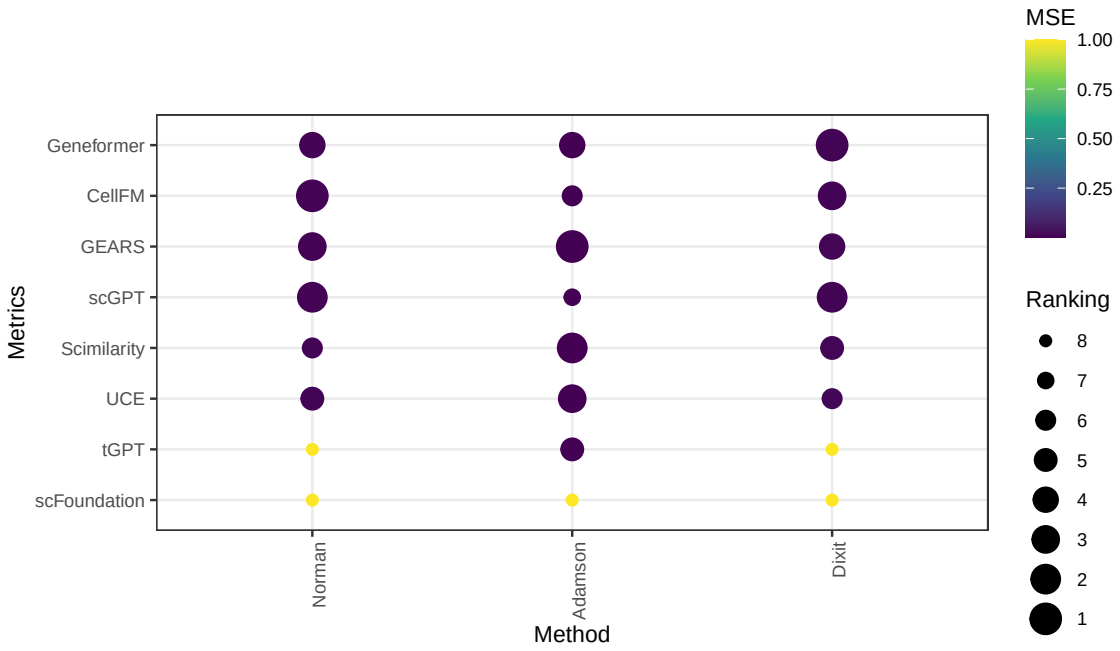
Appendix Figure S 1.6.24: Tuning hyper-parameters for gene function prediction. Sub-figures represent the score of scGPT under different hyper-parameters after training (sample size $n = 3$).



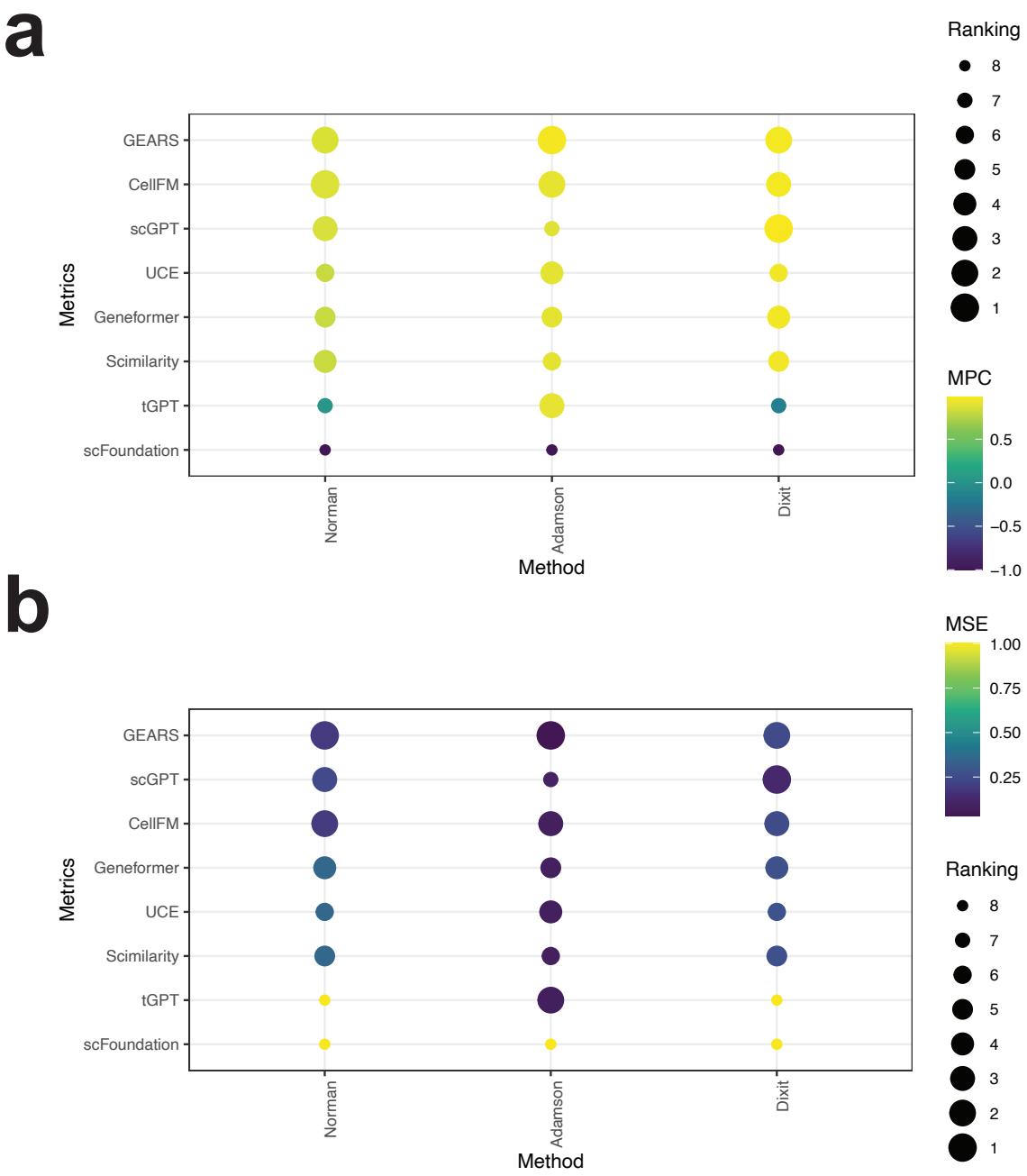
Appendix Figure S 1.6.25: Benchmarking results of different initial settings, optimizers, and loss components for gene function prediction. T1-T3 represent different gene prediction cases. (a): Results of adjusting initial settings. (b): Results of adjusting optimizers. (c): Ablation tests based on different loss components.



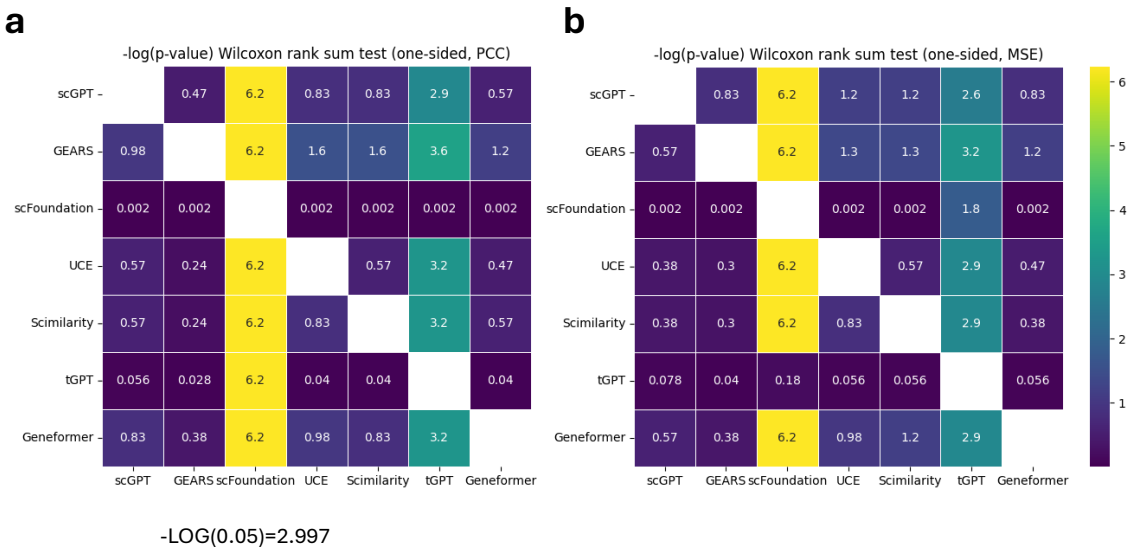
Appendix Figure S 1.6.26: Comparison based on the zero-shot settings of different scFMs. (a) PCC based on the results from different scFMs across three datasets. (b) MSE based on the results from different scFMs across three datasets.



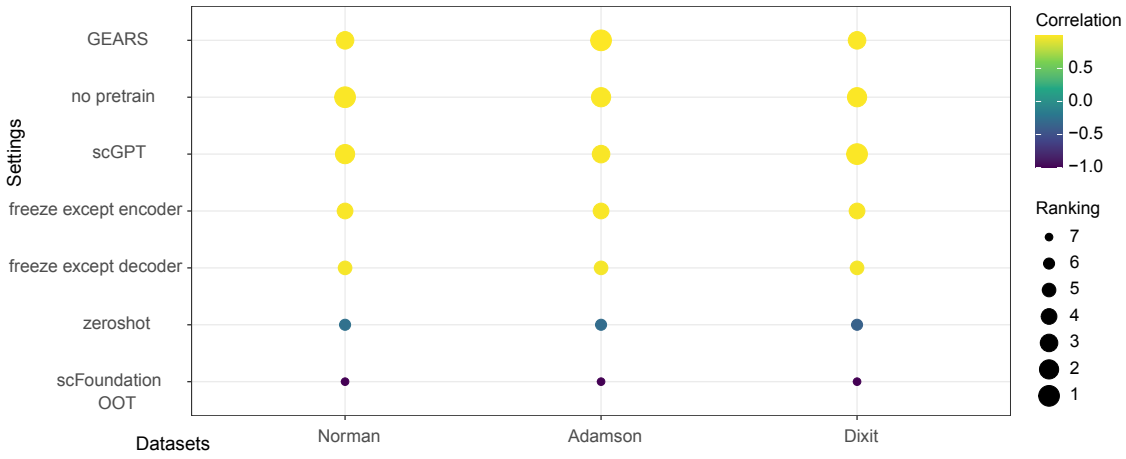
Appendix Figure S 1.6.27: MSE across different models to evaluate the performances for perturbation prediction. The maximal value for display is 1 for better visualization, while the true MSE of tGPT is larger than 1e5 for both Norman dataset and Dixit dataset. scFoundation meets the OOT error.



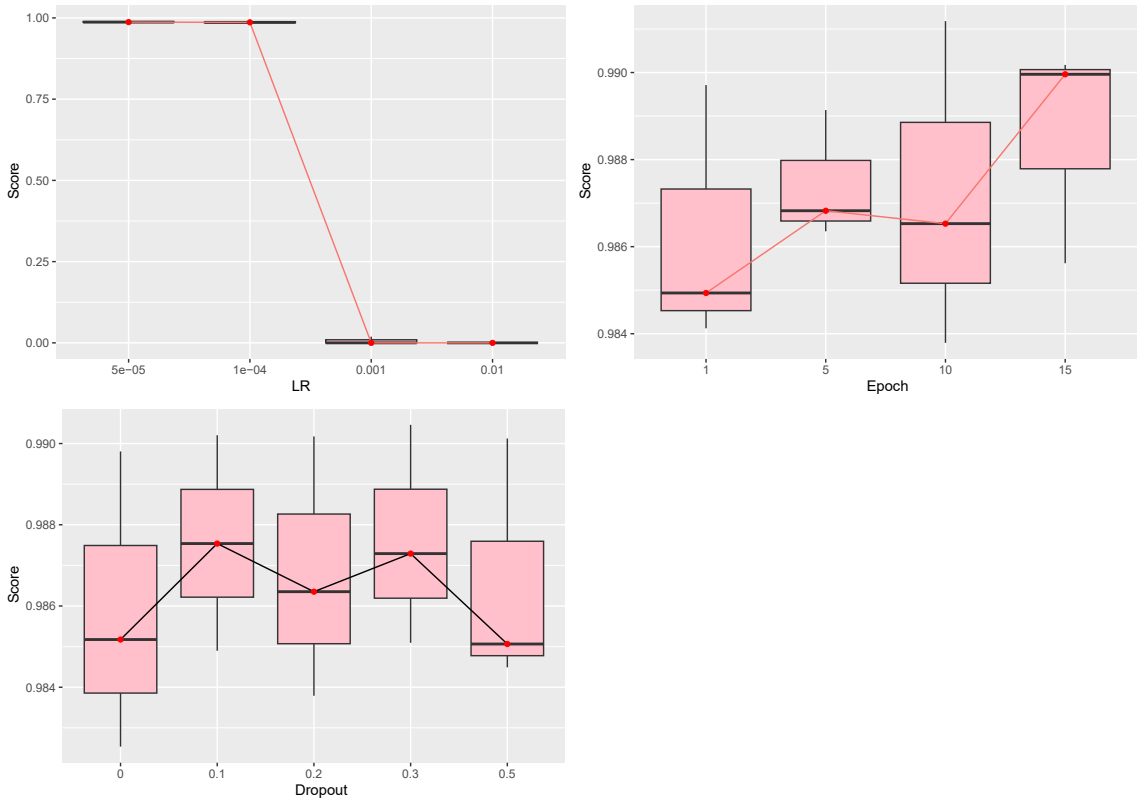
Appendix Figure S 1.6.28: Analysis of models for perturbation prediction based on DEGs. (a) MPC across different models to evaluate the performances for perturbation prediction. The averaged scores are reported in the figure. (b) MSE across different models to evaluate the performances for perturbation prediction.



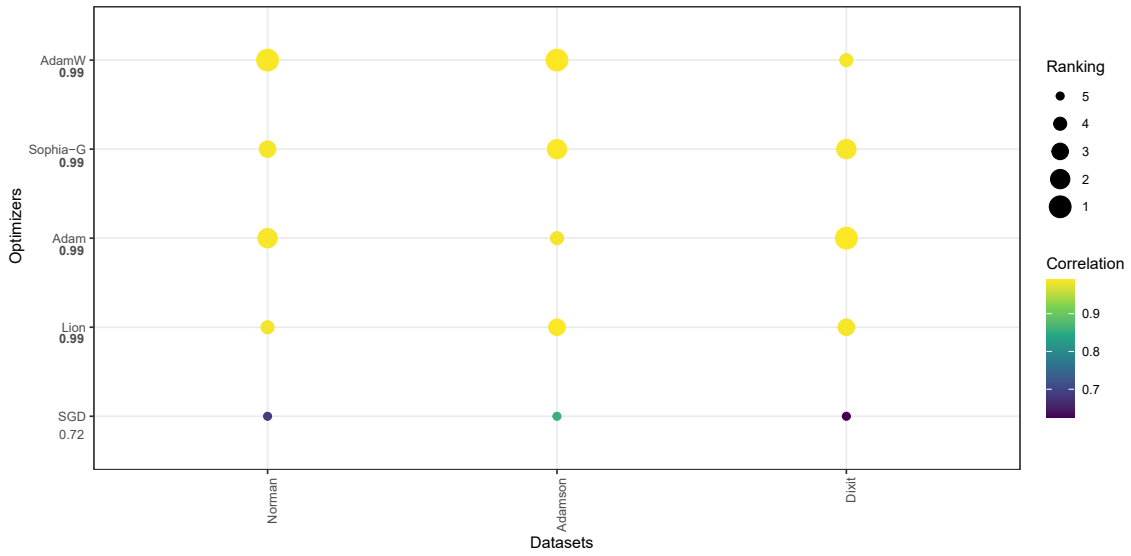
Appendix Figure S 1.6.29: Heatmap of $-log(p-value)$ to support the statistical analyses results of the perturbation prediction task. We conducted one-sided Wilcoxon Rank sums test based on different metrics, including (a) PCC (including both all genes and DEGs) and (b) MSE (including both all genes and DEGs), to examine the comparisons of different models. We mark the significance threshold 0.05 and corresponding $-log(p-value)$.



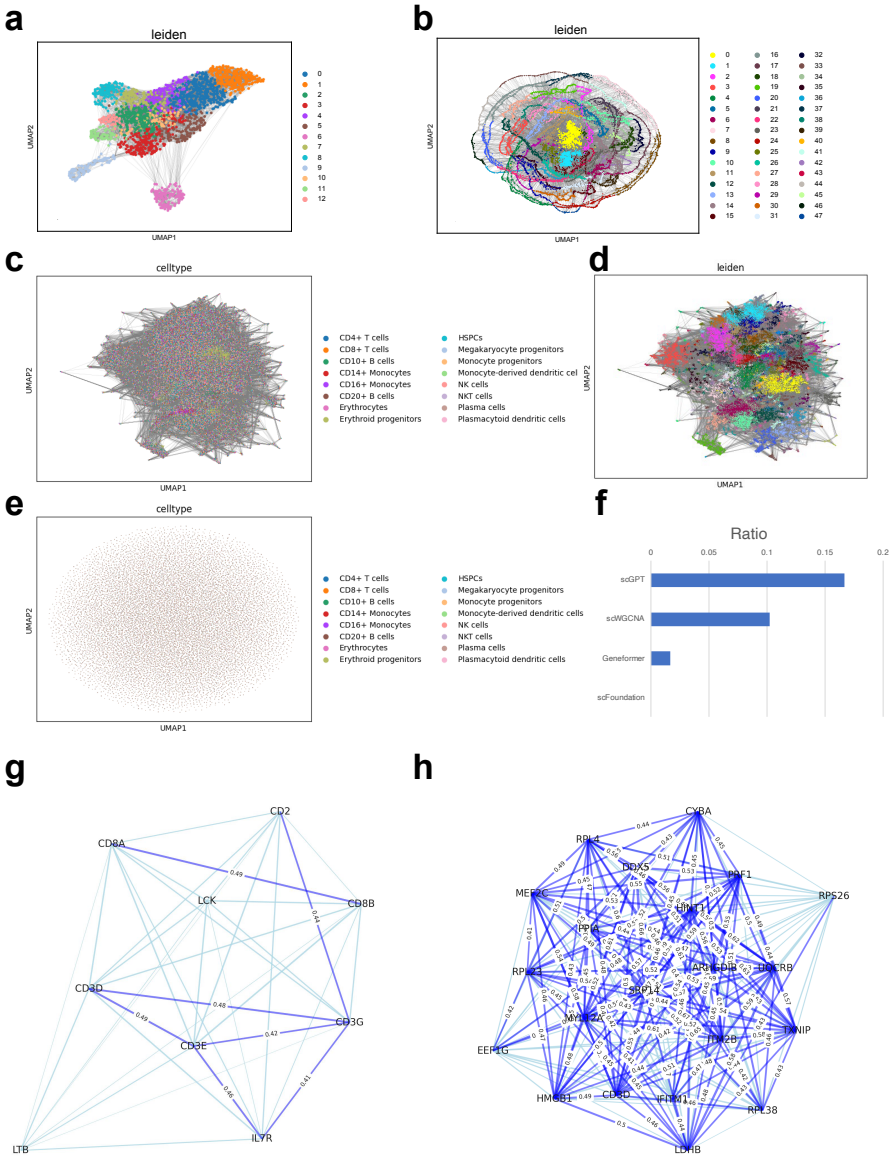
Appendix Figure S 1.6.30: MPC across different fine-tuned-based models and settings of scGPT to evaluate the performances for perturbation prediction.



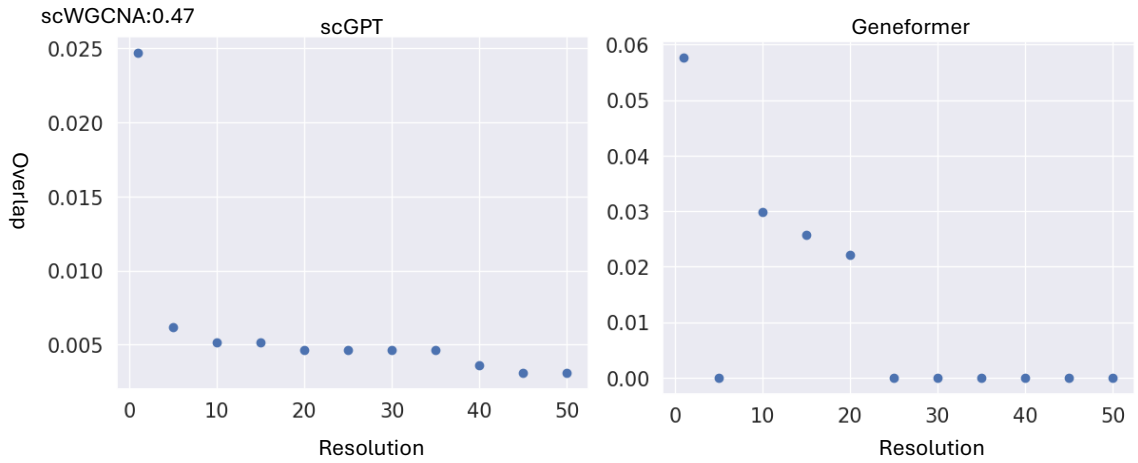
Appendix Figure S 1.6.31: Tuning hyper-parameters for perturbation prediction. Sub-figures represent the score of scGPT under different hyper-parameters after training (sample size $n = 3$).



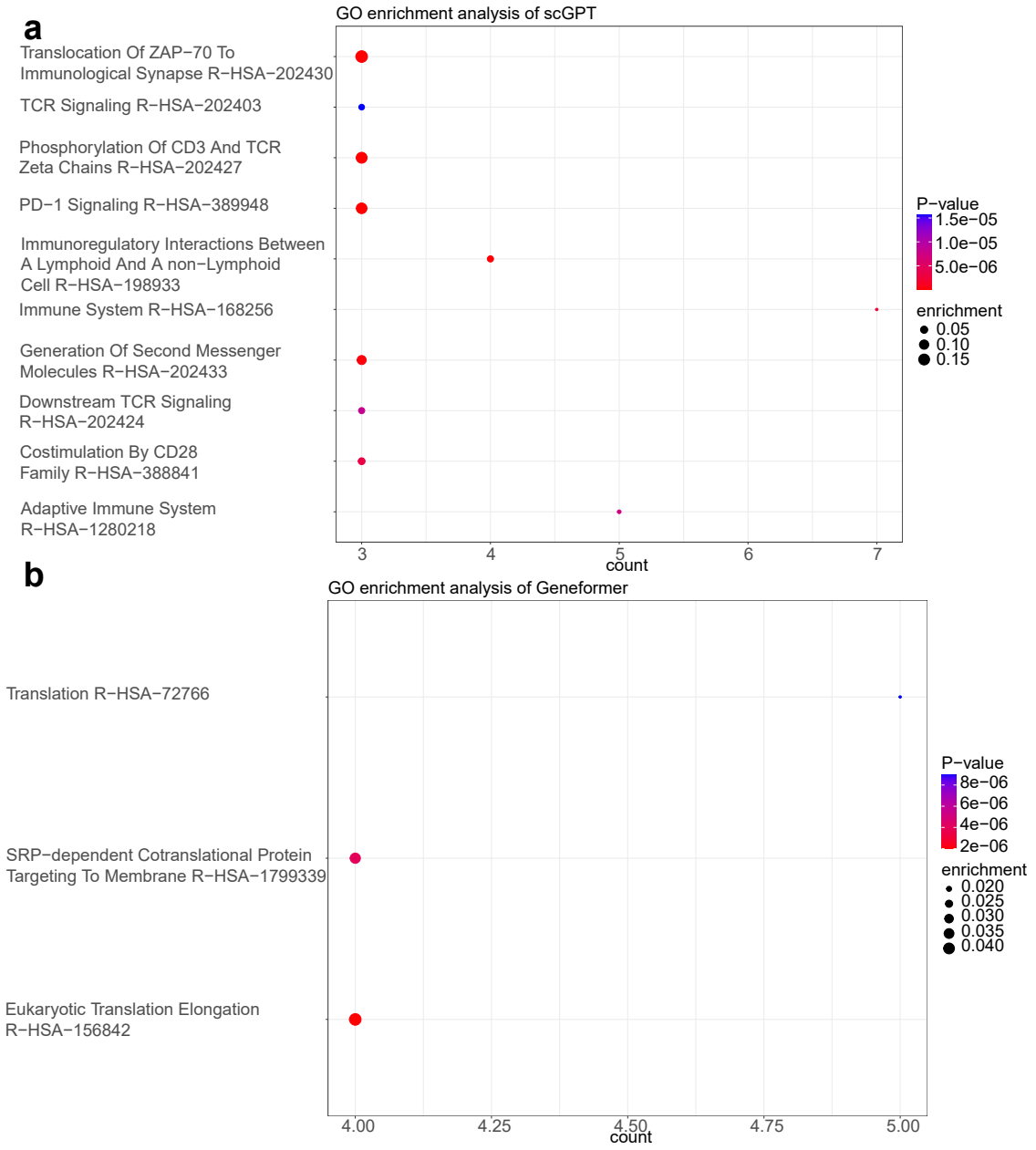
Appendix Figure S 1.6.32: Benchmarking results of different optimizers for perturbation prediction.



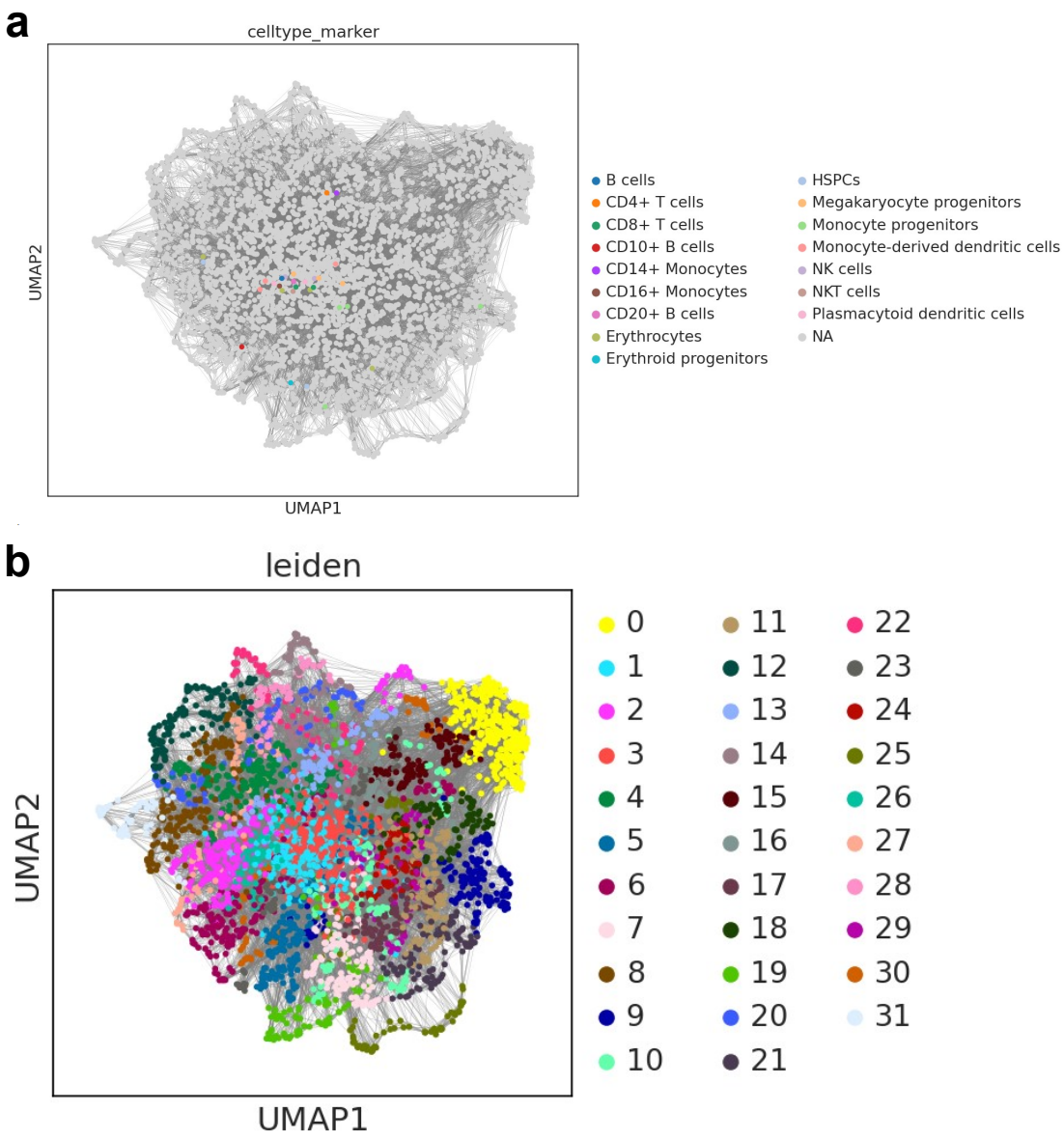
Appendix Figure S 1.6.33: Examples of GCN inference results for the Immune Human Atlas (IHA) dataset. (a): Dataset-level gene embeddings from scGPT colored by Leiden clusters. (b): Dataset-level gene embeddings from Geneformer colored by Leiden clusters. (c): Cell-type-level gene embeddings from scGPT colored by the cell types. (d): Leiden cluster results based on the cell-type-level gene embeddings from scGPT. (e): Cell-type-level gene embeddings from Geneformer colored by the cell types. We omitted plotting the Leiden cluster results of Geneformer because of the plotting size limit. (f): Comparison of significant pathways ratio between scGPT and Geneformer for CD3-related gene sets. (g): An example of GCN for IHA based on scGPT. It is a network with CD3-related genes as major nodes. (h): An example of GCN for the IHA dataset based on Geneformer.



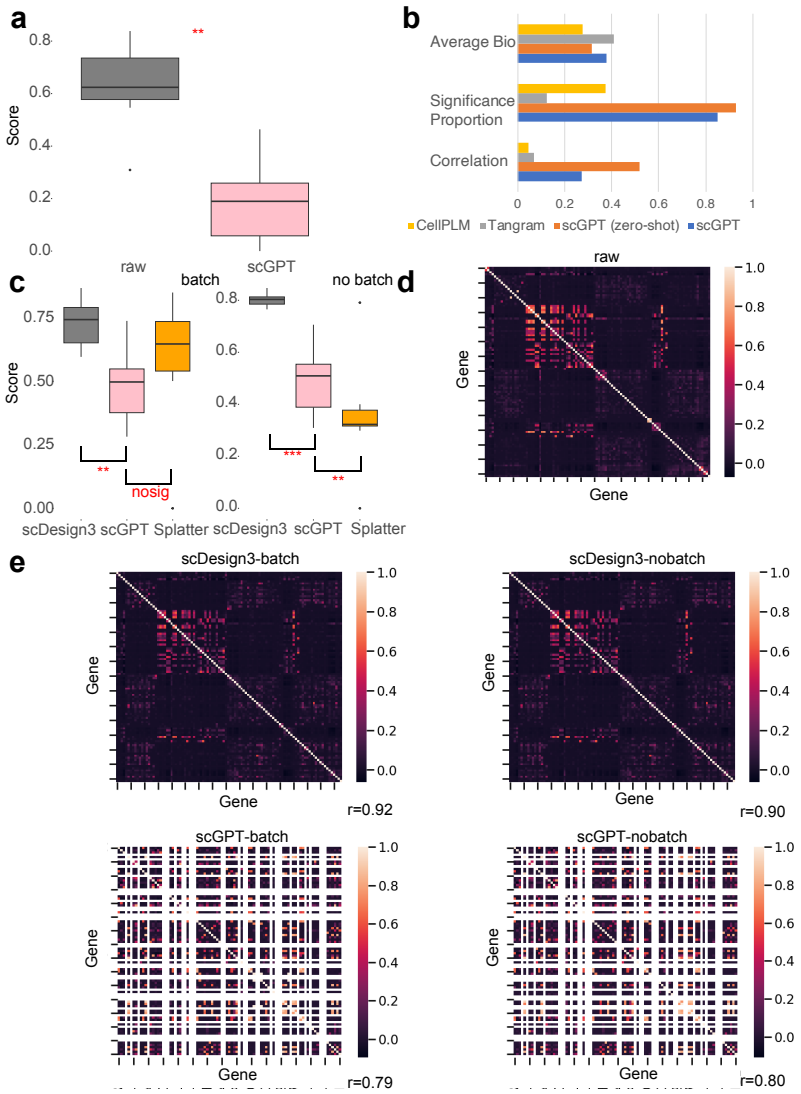
Appendix Figure S 1.6.34: Results of overlap ratio by adjusting resolution based on the gene embeddings from (a) scGPT and (b) Geneformer.



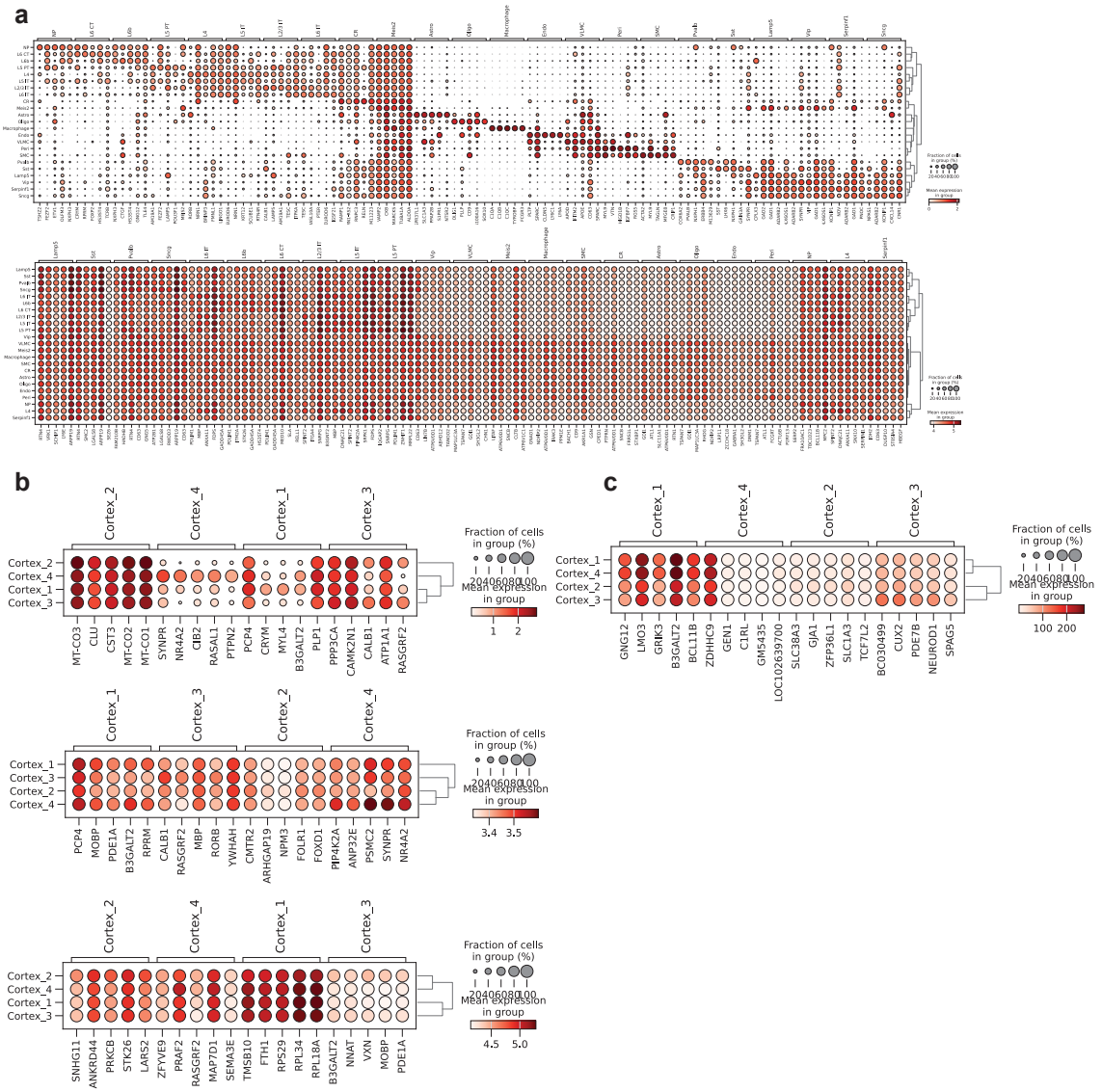
Appendix Figure S 1.6.35: Detailed pathway information. (a) The pathway enrichment information from scGPT. (b) The pathway enrichment information from Geneformer.



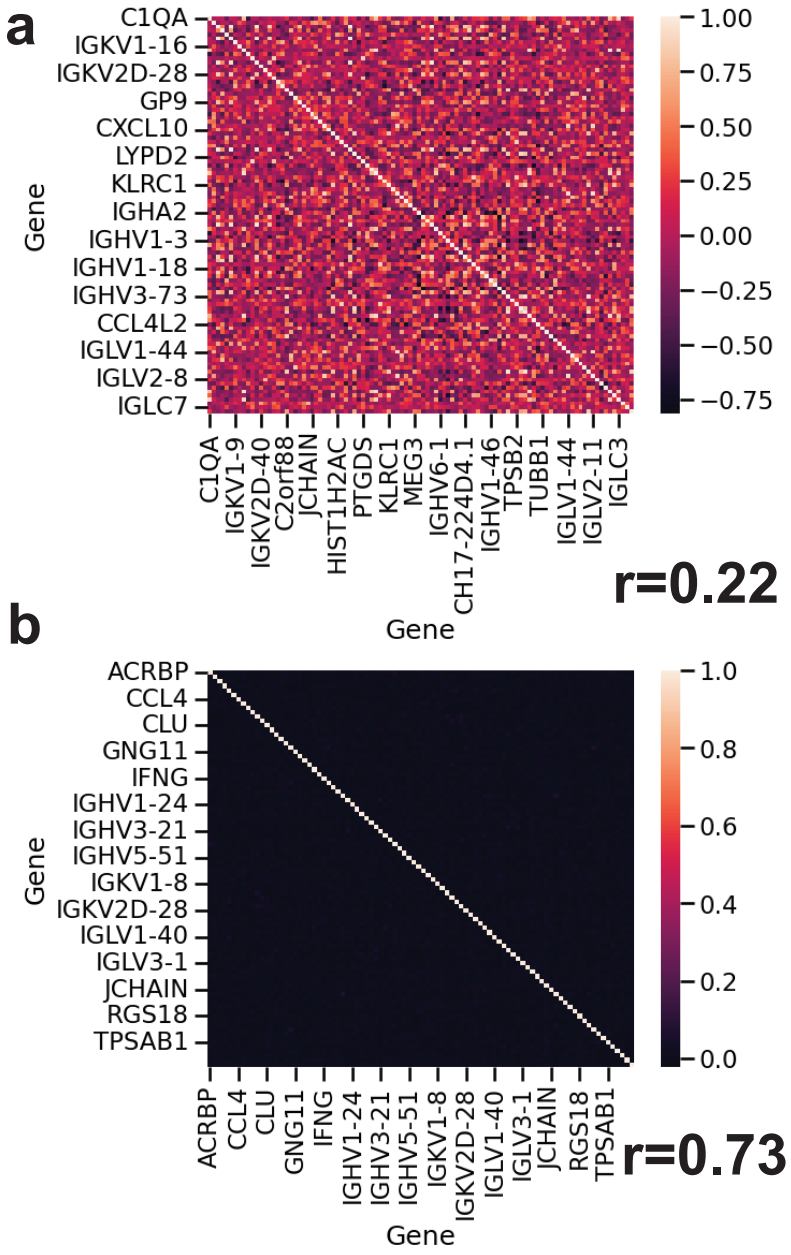
Appendix Figure S 1.6.36: Examples of dataset-level GCNs from Geneformer based on HVGs. (a) Dataset-level gene embeddings from Geneformer colored by sources of marker genes. (b) Dataset-level gene embeddings from Geneformer colored by Leiden clusters.



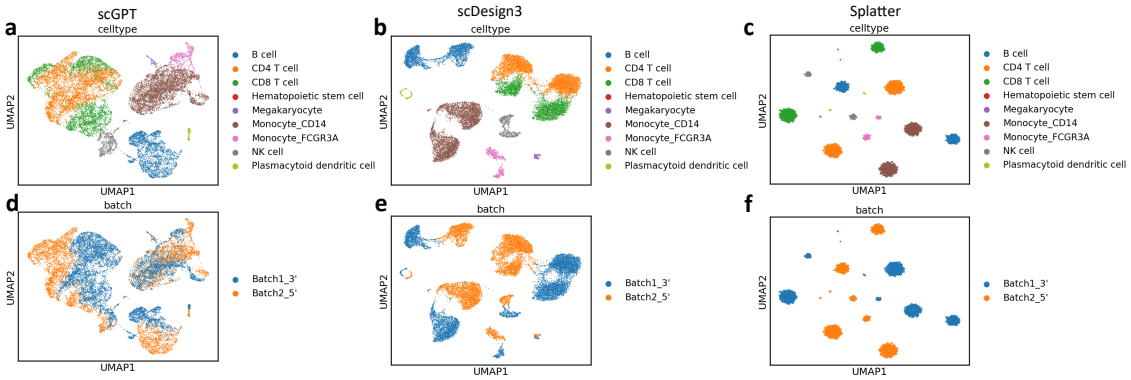
Appendix Figure S 1.6.37: Experimental results of the Imputation task and the Simulation task. The number of stars represents the significance level (one-sided Wilcoxon Rank-sum test, sample size $n = 8$, $***: p - value < 0.005, **: p - value < 0.05$). (a): Comparison of the average bio score between the raw data and imputed data by scGPT in the scRNA-seq imputation task. (b): Comparison of the average bio score, average correlation score, and average significance level score among Tangram, CellPLM, scGPT, and scGPT (zero-shots) in spatial transcriptomics imputation task. (c): Comparison of the average bio score among scDesign3, Splatter, and scGPT for simulation. (d): Gene-gene correlation heatmap from the raw HumanPBMC dataset. We select the subset of the top 100 highly variable genes. (e): Comparison of different simulation methods by correlation. The heatmap represents the top 100 highly variable genes (for raw and scDesign3) or the subset of the top 100 highly variable genes (for scGPT) based on the HumanPBMC dataset. The correlation "r" represents the Pearson correlation between the gene correlation of raw data and the gene correlation of simulation data. The white section represents NaN values caused by the problematic model outputs.



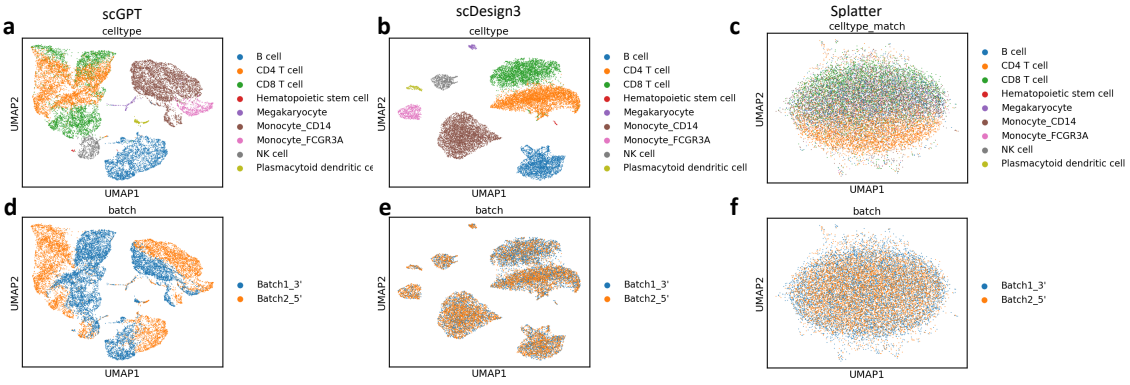
Appendix Figure S 1.6.38: Deferentially expressed gene discovery based on results before imputation and after imputation. We used the Mouse scRNA-seq and the Mouse spatial transcriptomic datasets as examples. (a): Deferentially expressed genes by cell types for scRNA-seq data based on pre-imputation data (top) and post-imputation data (bottom). (b): Deferentially expressed genes by cluster types for spatial transcriptomic data based on pre-imputation data (top), post-imputation data based on zero-shot learning (middle), and post-imputation data based on fine-tuning (bottom). (c): Deferentially expressed genes by cluster types for spatial transcriptomic data based on Tangram.



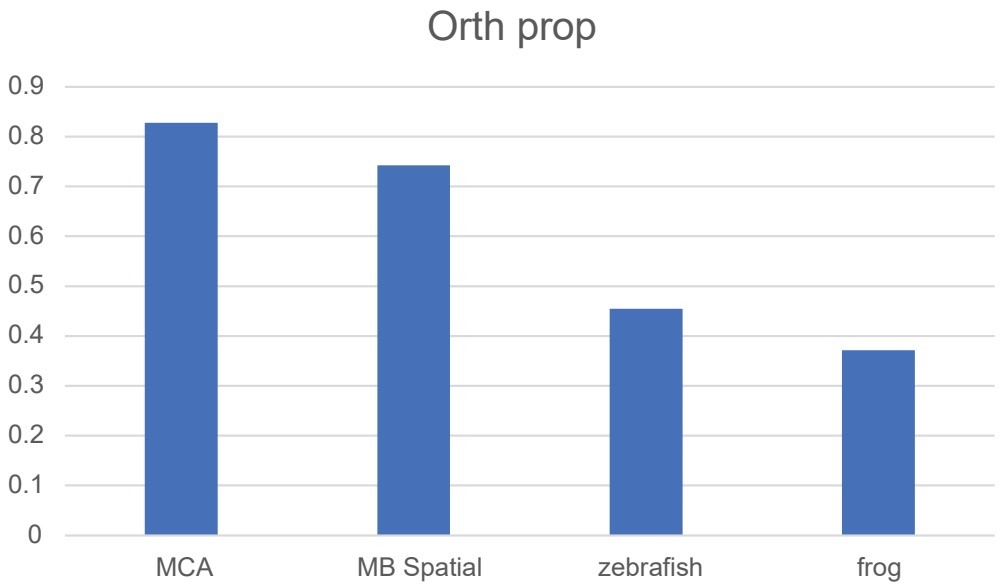
Appendix Figure S 1.6.39: Heatmaps for the gene-gene correlation of datasets simulated by Splatter, using the HumanPBMC dataset as an example. The correlation "r" represents the Pearson correlation between the gene correlation of raw data and the gene correlation of simulation data. (a): The heatmap for the simulation dataset with batch effect. (b): The heatmap for the simulation dataset without batch effect.



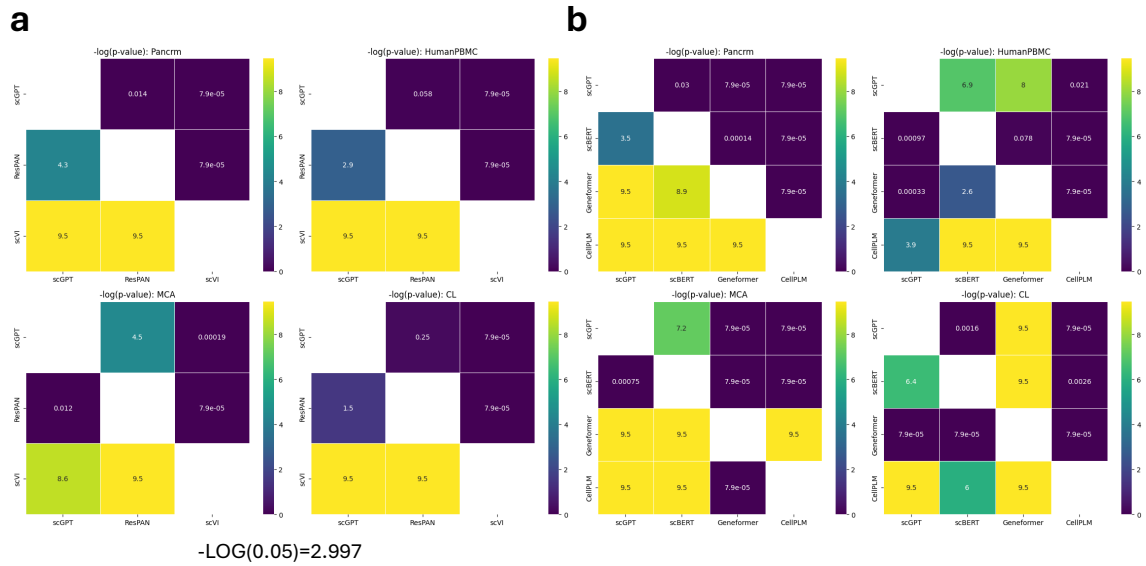
Appendix Figure S 1.6.40: UMAPs for the simulation results with batch effect, using the HumanPBMC dataset as an example.



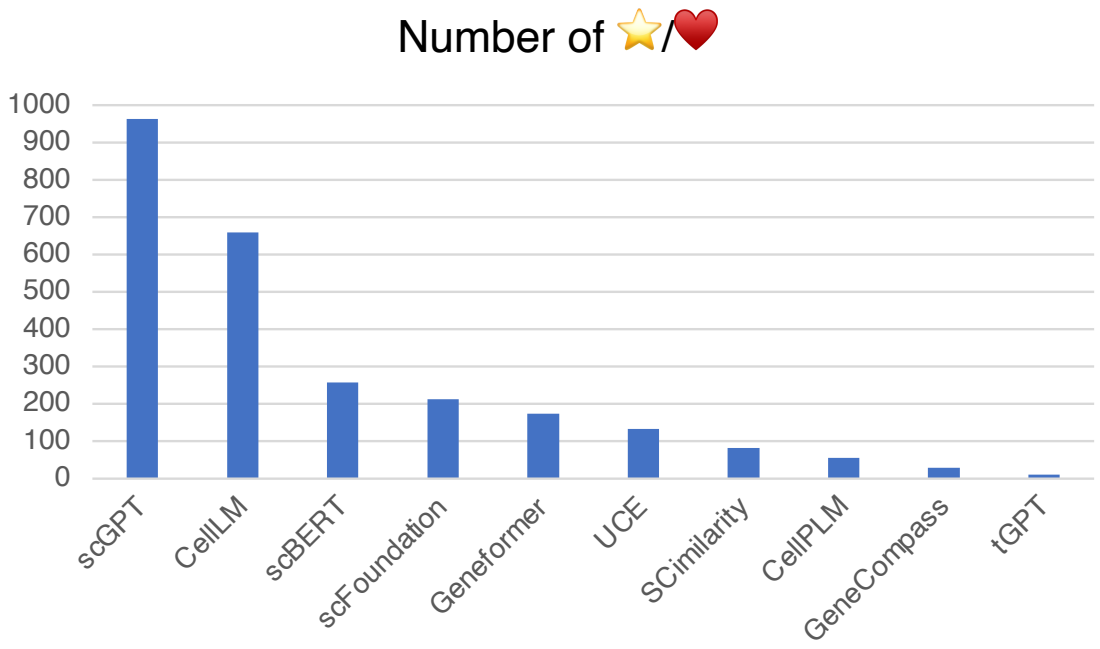
Appendix Figure S 1.6.41: UMAPs for the simulation results without batch effect, using the HumanPBMC dataset as an example.



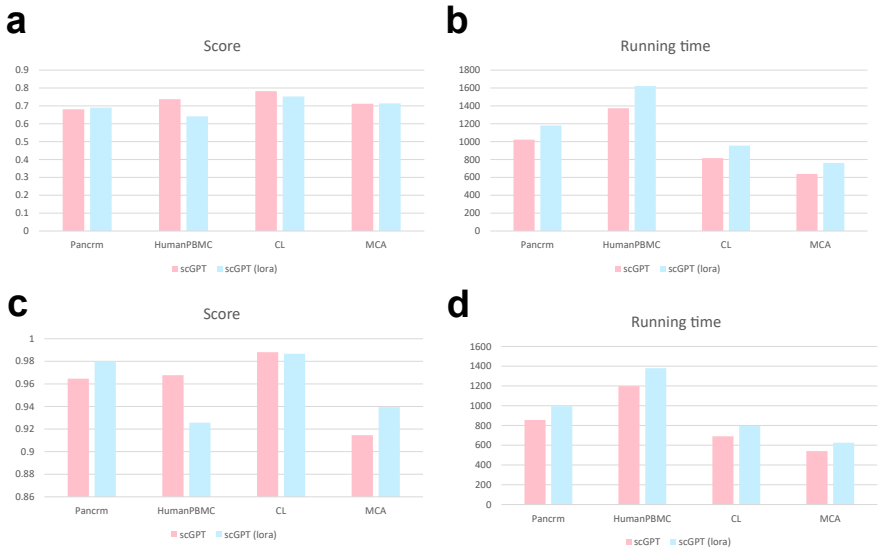
Appendix Figure S 1.6.42: Proportion of orthologous genes between specified species and human.



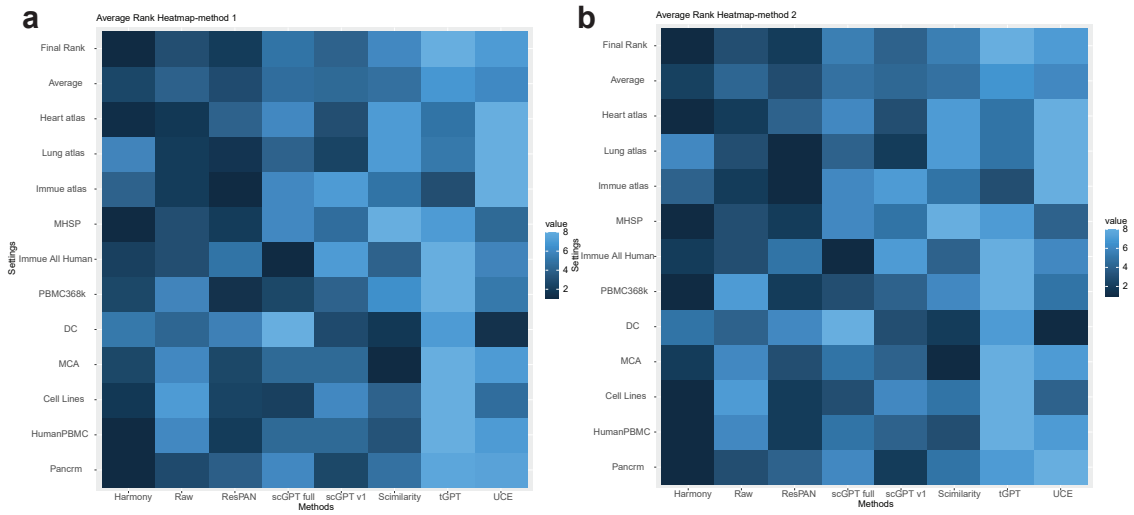
Appendix Figure S 1.6.43: Heatmap of $-log(p-value)$ to support the statistical analyses results of the stability examination. We conducted one-sided Wilcoxon Rank sums test based on different tasks, including (a) batch effect correction and (b) cell-type annotation, to examine the comparisons of different models. We mark the significance threshold 0.05 and corresponding $-log(p-value)$.



Appendix Figure S 1.6.44: The number of stars in GitHub or likes in Huggingface for different single-cell FMs.



Appendix Figure S 1.6.45: Preliminary results of parameter-efficient fine-tuning based on LoRA for Batch Effect Correction (the upper two figures) and Cell-type Annotation (the bottom two figures). (a): The comparison of scores for scGPT with/without LoRA. (b): The running time for scGPT with/without LoRA. (c): The comparison of scores for scGPT with/without LoRA. (d): The running time for scGPT with/without LoRA.



Appendix Figure S 1.6.46: Heatmaps for the ranks of different models across different datasets. (a): The heatmap of ranks based on the first method, known as adjusting the weights and computing the ranks. (b): The heatmap of ranks based on the second method, known as using the default weights for S_{bio} and S_{batch}.

Chapter 2

Leveraging Multi-Modal Foundation Models for Analyzing Spatial Multi-Omic and Histopathology Data

This paper has been published by Nature Biomedical Engineering.

Citation to original (if applicable)

Abstract

Recent advances in pathology foundation models (PFMs), pretrained on large-scale histopathology images, have greatly advanced disease-focused applications. At the same time, spatial multi-omic technologies now measure gene and protein expression with high spatial resolution, offering valuable insights into tissue context. Yet, existing models struggle to integrate these complementary data types. To address this challenge, we present spEMO, a computational framework that unifies embeddings from PFMs and large language models (LLMs) for spatial multi-omic analysis. By leveraging multimodal representations, spEMO surpasses single-modality models across diverse downstream tasks, including spatial domain identification, spot-type classification, whole-slide disease prediction and interpretation, multicellular interaction inference, and automated medical reporting. These results highlight spEMO's strength in both biological discovery and clinical applications. Furthermore, we introduce a new benchmark task—multi-modal alignment—to evaluate how effectively PFMs retrieve complementary information. Together, spEMO establishes a powerful step toward holistic, interpretable, and generalizable AI for spatial biology and pathology.

2.1 Introduction

Pathologists need to make accurate diagnoses based on limited and complex data. [192]. Recently, pathologists and biomedical scientists have started using machine-learning-based methods to assist pathology analysis, but models trained with limited data and knowledge might not provide enough support, especially for diagnosing rare cancer types or predicting survival of patients from different cohorts [255]. Therefore, efforts have been made to collect large-scale pathology data, demonstrated by several pathology foundation models (PFM) pre-trained with pathology image data [31]. In addition, biomedical scientists also collect multimodal information, for example, gene expression profiles from spatial transcriptomics [149] and biological feature text descriptions [226], to make a better decision and more accurate discovery together with images. Furthermore, the image feature from PFMs might also help the analysis of other modality data, presenting another potential use of these foundation models.

A foundation model (FM) is defined as a model pre-trained with large-scale datasets and is capable of handling various downstream tasks. There exist both general FMs such as ChatGPT and Claude, and domain-specific FMs such as UNI [31] and scGPT [38], defined by the knowledge scope used for model training. Specifically, in pathology, researchers can pre-train a PFM based on various pathology images and generate robust representations of different images, and perform various experiments to investigate the generalization ability of such representation. Moreover, spatial transcriptomics is closely linked with these pathology images, as the process of sequencing gene expression in the spatial context always involves the screening of the background tissue slide. By collecting large-scale spatial transcriptomic data, people can also train an FM for analyzing the gene expression patterns across different spatial positions of spots [181]. Recently, researchers have developed multi-omic spatial sequencing to simultaneously measure different omic data (including genes, proteins, chromatin accessibility, etc.) from one slide, which can further enrich the modality for describing the cell activity at the spatial level. Considering the possibility of joint spatial multi-omic features and pathological images, we believe that it is feasible to design a system to achieve this goal.

Although it has been shown that training a PFM can enhance the image classification accuracy as well as gene expression prediction performance [31], the necessity of having an FM trained based on transcriptomics or other omic data is not resolved yet. Research papers [124] discussed the limitations of FMs pre-trained with single-cell transcriptomics, e.g., their inferior performance to simple linear regression in certain tasks. Since the noise level of sequencing data is high and difficult to eliminate, pre-training another FM with such data is extremely challenging. Fortunately, recently developed methods such as scELMo [121] open a new direction by utilizing embeddings from Large Language Models (LLMs) trained with other modalities to

analyze single-cell multi-omic data, and their experiments demonstrate that embeddings generated based on external biology knowledge can help analyze such data and gain biological insights. Since we have not seen such methods designed for analyzing spatial data, especially spatial transcriptomic data, we explore the capacity of such design for understanding the spatial context of biological features.

We present spEMO, a novel framework for Spatial multi-modal data analysis leveraging Embeddings from diverse foundation models, designed to advance biological discovery and translational research. spEMO integrates biological prior knowledge from multiple modalities by incorporating embeddings from large language models (LLMs) and pathology foundation models (PFMs), particularly those derived from whole slide images (WSIs). By unifying these embeddings with spatially resolved transcriptomic and multi-omic data, spEMO operates as a training-flexible framework and enhances downstream analyses through task-specific expert modules. Critically, spEMO enables robust biological insights across a spectrum of spatial tasks, including spatial domain identification, spot-type classification, and spatial feature extraction, without requiring additional experimental cost. Beyond methodological improvements, we demonstrate the utility of spEMO in biologically and clinically meaningful applications: predicting disease states from multi-modal data, inferring intercellular communication networks, aligning morphological image patches with transcriptomic signals, and generating comprehensive, pathology-informed medical reports. These tasks not only validate spEMO's effectiveness but also highlight its potential to uncover novel cellular and molecular mechanisms in situ. Moreover, our findings suggest that integrating multi-modal embeddings enhances the completeness and interpretability of model-generated clinical summaries, often surpassing the consistency of human pathologist reports. Finally, we discuss emerging challenges and future directions in aligning transcriptomic and histopathological information, underscoring the biological significance and untapped potential of spatial multi-modal integration.

The core innovation of spEMO lies in the following points: 1. spEMO is the first AI system to integrate histopathology image and text description embeddings for spatial multi omic analysis. 2. spEMO offers two modes, including (i) a zero shot pipeline that directly projects spatial spots into the joint embedding space, and (ii) an expert model pipeline that plugs the fused embeddings into task specific neural or graph models. Therefore, we also propose several new models and applications under the mode 2 in tasks such as patch-spot retrieval and spatial omic clustering by fine-tuning PFMs. 3. spEMO achieves broad performance gain across various application scenarios, and its power in generating medical reports is validated with feedback from pathologists and have better quality compared with human experts. 4. spEMO also introduces a new evaluation setting for pathology foundation models, known as multi-modal spot alignment, for assessing Pathology Foundation Models in

the representations of spatial transcriptomic data. 5. spEMO provides empirical but solid insights into when and why external embeddings (such as text embeddings) help downstream tasks, which can explain the observed performance change and guide future model design and development. In summary, its contribution lies in the principled design of a cross-modal integration and understanding system that enables pathology foundation models and large language models to effectively inform spatial-omic analysis. This is distinct from a simple aggregation and provides a blueprint for integrating future pathology and omic foundation models in a reproducible and generalizable manner.

2.2 Results

Overview of spEMO. spEMO is an artificial intelligence system (AIS) for spatial multi-omic data analysis by integrating external knowledge as embeddings extracted from FMs. We support two different settings of spEMO, known as zero-shot embedding design and fine-tuning design. The former framework allows users to directly combine the embeddings from PFMs and LLMs for projecting spatial data into a new space, and the latter framework allows users to integrate embeddings with known methods to enhance their performances. The spot-level input images are computed by splitting the whole slide image into several patches with a pre-selected radius. We then embedded the patches with PFMs as spot-level embeddings. Furthermore, we define and perform several new tasks to demonstrate the power of multi-modal pathology analysis. The overall pipeline of spEMO is summarized in Figures 2.6.1 (a) and (b).

spEMO improves spatial domain identification by introducing features with strong spatial correlation. A fundamental challenge of spatial multi-omic data analysis is the identification of spatial domains or niches. Such specific patterns are always treated as a specific group of cells with co-localization and thus similar biological functions. Similar to the analysis of single-cell multi-omic data, the naive approach to annotate the spatial domain information is based on principal components (PCs) [2] of expression profiles. Meanwhile, there are methods to improve the accuracy of domain identification with the help of deep learning design but with little reference to the use of PFMs to address this task. In this work, we hypothesize that the embeddings from PFMs may contribute to accurate domain identification by either combining the spot embeddings with corresponding PCs or integrating the embeddings into the input data of task-specific methods, or both. We considered a dataset consisting of spatial transcriptomic samples from the human brain, known as SpatialLIBD [143]. This dataset contains annotated spatial domain information and is collected through the 10X Genomics Visium platform. We tested the abil-

ity of spEMO to identify spatial domains under different settings, shown in Figure 2.6.2 (a). The metrics of this task include Normalized Mutual Information (NMI), Adjusted Rand Index (ARI), and Averaged Silhouette Width (ASW). These metrics are widely used in evaluating clustering performance [121], and we averaged them in our comparisons of different methods. Here the default clustering algorithm is Leiden, and all methods except the series of SC3 [99] and SpaGCN [79] are based on such algorithm. SpaGCN and SC3 are task-specific methods for spatial domain identification and spot clustering. Novae [15] represents using the embeddings from a spatial transcriptomics foundation model for clustering. iSTAR [253] and Pix2Path [59] are two baseline methods by jointly modeling pathology images and spatial transcriptomics using a predictor. PCA represents using the PCs for clustering, and spatial represents using spatial locations. LLMemb represents using the embeddings from LLM for clustering, referred from the *wa* mode of scELMo. To select the most suitable PFM for this task, we combined PCs with embeddings from several PFMs, including GPFM [137], UNI(2) [31], GigaPath [234], Virchow(2) [262], CHIEF [220], MUSK [231], and CONCH(1.5) [46, 131]. Loki [32] is trained with the paired image patches and transcriptomic profiles, so we also include it as a baseline. These methods are either novel or reported with good performance in benchmarking analysis [90]. According to Supplementary Figure 1, GPFM and CHIEF are the best settings to generate embeddings for this task in the zero-shot embedding mode, and thus we have PCA+GPFM to represent the concatenation of PCs and GPFM embedding, and spEMO represents the combination of GPFM embeddings with SpaGCN. Furthermore, alternating embeddings from different PFMs in the combination does not affect the model performance obviously, shown in Supplementary Figure 2. We also test the best radius for generating image patches for each spot, shown in Supplementary Figure 3. Since they do not show obvious differences, we select the radius of 112 to match the default setting of a public pathology dataset [90]. We also considered fine-tuning PFMs with spatial transcriptomics by predicting the expression profiles based on the image embeddings from these modes, annotated with "(ft)" at the end of each model. Our results show that incorporating the embeddings from GPFM can significantly improve the performance of SpaGCN (p-value=3e-2 based on the Wilcoxon Rank-sum test for NMI scores, p-value=1e-1 based on the Wilcoxon Rank-sum test for Avg scores), and it also surpasses other settings including finetuned models. It seems that the zero-shot embedding setting, known as PCA+GPFM, is not the optimal choice for this task, as its overall performance is worse than others with the exception of PCs. Moreover, Figure 2.6.2 (b) shows that fine-tuning different PFMs cannot surpass our combination design with SpaGCN. However, fine-tuning PFMs can surpass the zero-shot embedding combination design based on most of the PFMs (such as PCA+GPFM), shown in Supplementary Figure 4. Therefore, although fine-tuning PFMs can generate a better embedding space for integrating

histopathology and transcriptomic information, it is still not the optimal choice in this task. Instead, modifying expert models such as SpaGCN and combining them with PFM embeddings implies a more promising solution. Finally, we visualize the best clustering result in Figure 2.6.2 (c), where SpaGCN with embeddings from PFMs identifies regions including WM, L1, L3, and L6.

To understand the reason for such improvement, we further assessed the auto-correlation score between the features of embeddings from GPFM and the spot location. The Moran's I score and Geary's C score are used here for evaluation. These two scores are common metrics for evaluating the auto-correlation level and a higher Moran's I score represents a stronger correlation. The trend in Geary's C score is inversely proportional to the trend in Moran's I score due to differences in the way it is calculated. According to Figure 2.6.2 (c), the embeddings from GPFM have stronger features correlating with spatial information than PCs (p-value=2e-3 based on Wilcoxon Rank-sum test for Moran's I scores), and thus they can better describe the spatial pattern in a given sample. We also compared the embeddings from PFMs and traditional image features for this clustering task based on two new Visium datasets (Visium H&E and Visium Fluorescence (Fluo)) from Squidpy [157], and these embeddings also have better performances shown in Supplementary Figures 5 (a) and (b), which further supports our conclusions. Therefore, these results suggest that incorporating image embeddings from PFMs can improve the accuracy of spatial domain identification.

spEMO improves domain identification of multi-omic spatial data. We explored the possibility of using spEMO to improve spatial domain identification for multi-omic datasets. Such data have recently become available through technological advances with few having expert labels. Here we focus on one spatial dataset with measurements of both gene expression profiles and protein information from human lymph node [127] as well as expert annotations [127]. Since this dataset does not contain image information, we consider including gene and protein description embeddings from LLMs for clustering. Meanwhile, we also consider the performance of using single modality information for clustering, denoted as RNA only and ADT only. As shown in Supplementary Figure 6 (a), incorporating LLM embeddings can improve the accuracy of spatial domain identification, and the results are even comparable with domain-specific method SpatialGLUE [127], supported by the insignificant difference between the performances of these two methods (p-value=0.25 based on the Wilcoxon Rank-sum test with the two-tailed mode). We also found that the concatenation (LLMemb RNA||ADT) and addition (LLMemb RNA+ADT) of embeddings do not show obvious differences for this task. We further visualize the best clustering result in Supplementary Figure 6 (b), which shows that spEMO clearly identifies cortex, capsule, and pericapsular adipose tissue.

spEMO can annotate spot types more accurately. Another important

task in spatial transcriptomic data analysis is the annotation of spots, either for niche information or cell-type information. Similar to the cell-type annotation task in single-cell transcriptomics analysis, annotation of spot types can also be treated as a multi-label classification task based on training a classifier from the reference spatial datasets and then performing inference based on the testing dataset. Here we utilized both the Visium data from SpatialLIBD and that from Squidpy to test the contribution of image embeddings for this task. First, we tested the effect of different classifiers for spot annotation based on the Visium data from SpatialLIBD under different settings, shown in Supplementary Figure 7 (a). We found that XGB and LR are the top two classifiers for annotating spot types. However, since XGB meets OOM errors when using the combination of expression features and image embeddings as inputs, it is hard for us to test the contribution of including mode modality data, and thus we finally choose LR as our final choice. The results for the same set of datasets under different input features are shown in Supplementary Figure 7 (b). This figure shows that using the combination of gene expression profiles from HVGs and image embeddings from UNI has the best performance, which outperformed the choice of only using HVGs (p-value=3e-2 based on Wilcoxon Rank-sum test) as well as only using PCs (p-value=5e-8 based on Wilcoxon Rank-sum test) significantly. Moreover, incorporating image embeddings from different PFMs all can improve the annotation performance. Furthermore, increasing the patch radius can also improve the annotation performance, and thus the annotation accuracy may be further improved through the adjustment of patch size.

Our results based on the data from Squidpy are summarized in Supplementary Figures 8 (a) and (b). We found that combining expression profiles from HVGs and image embeddings can improve the prediction performance when the testing dataset is selected as the Visium fluo dataset, which could be explained as the strong relationship between image quality and prediction performance improvement.

spEMO integrates image embeddings for whole-slide disease-associated prediction. Disease diagnosis is a key task for pathology research with success of machine learning for disease prediction based on pathology information. In real data application, we collect training datasets with known disease information and predict the unknown disease state in a new dataset, as a multi-label classification problem. By investigating the disease-state annotation of the HEST dataset [90], we are also interested in the capacity to utilize gene expression information as well as pathology information for disease-state prediction. We first cleaned the disease-state labels and performed qualify control for the HEST dataset, and selected human tissues for experiments. Here we selected GigaPath, GPFM, CHIEF, UNI2, Virchow2, MUSK, and CONCH1.5 for generating image embeddings, and also concatenated gene expression profiles with image embeddings from GPFM to represent the joint embeddings. We first illustrated the sample-level representation of gene expression levels and im-

age embeddings by performing dimension reduction with UMAP [145], and colored them by disease states and tissues. Based on Figures 2.6.3 (a), (b) and Supplementary Figures 9 (a) and (b), gene expression profiles and image embeddings convey different information. For example, pseudobulk gene expression profiles captured the heterogeneity of tissue-specific signals in healthy samples, while image embeddings from PFMs such as GigaPath, CHIEF, UNI2, and CONCH1.5 tended to co-embed healthy samples. Moreover, image embeddings might also help us distinguish more disease states such as COAD, EPM, and SCCRCC. These are three subtypes of cancer [105]. Given the need for building accurate disease-state predictors, we intended to use the embeddings to cluster the same disease from different tissues in a similar group, so we averaged NMI and ARI scores to measure the quality of the embeddings. According to Figure 2.6.3 (c), UNI2 attains the highest disease-clustering score but also the nearly worst (highest) tissue-clustering score, suggesting that its better disease performance may be confounded by tissue information. As shown in Figure 2.6.3 (d), CHIEF and UNI2 have the highest average rank based on the clustering scores from these two conditions, implying their capacity to balance the discrimination of disease states and tissues. The pseudobulk expression profiles have the lowest clustering scores based on disease states, but integrating the image embeddings with gene expression profiles has a better clustering score compared with both ablations, which highlights the complementarity between molecular and morphological signals.

According to Figures 2.6.3 (c) and (d), introducing the text embeddings of gene information cannot help in clustering disease signals as well as tissue signals. Furthermore, we tested if the introduction of text embeddings could affect clustering performances of different PFMs by performing joint embedding clustering analysis, shown in Supplementary Figures 10 (a) and (b). These figures show that introducing text embeddings also cannot help in clustering disease signals or reducing batch effect. Therefore, text embeddings might not directly contribute to this task.

Moreover, according to Supplementary Figures 11 (a) and (b), embeddings from pseudobulk gene expression are also less affected by batch effect, demonstrated by the comparison based on batch_Average Silhouette Width (ASW) score (0.46 for expression embeddings, and 0.42 for image embeddings). Therefore, the learned embeddings are not driven by batch effect and their distribution is mostly determined by disease-associated signals.

Considering the similarity of samples from the same study, we had two task settings with two levels of difficulty for predicting disease states from related embeddings. We denote the two different cross-validation settings as sample-level cross validation (SampleCV) and study-level cross validation (StudyCV). Based on our assumption, StudyCV is more difficult but closer to the real application scenarios. For both settings, we chose three-fold cross validation. We illustrate the comparisons of different classifiers with gene expression profiles for disease-state prediction

in Figure 2.6.4, where LR has obviously strong performance for this task. We then compared different input settings based on the results shown in Figure 2.6.4 (b) for the SampleCV mode, and it is clear that only using the image embeddings with LR could also have good performances. If we combine gene expression profiles and image embeddings as input, the LR model will be further boosted, as the option of using pseudobulk gene expression profiles and Virchow2 has the highest averaged score. Moreover, we can also improve the combination setting with the Oversampling algorithm (RandomOverSample) [111] shown in the given figure, and the combination of expression profiles and GPFM embeddings outperforms the pseudobulk setting significantly (p-value=0.0025 based on Wilcoxon Rank-sum test). However, we note that the combination with embeddings from CHIEF performed worse than other PFMs, which could be explained as the embeddings from CHIEF might not learn the distance of diseases in the sample level, as it tended to group samples with different disease states or tissues together, shown in Figures 2.6.3 (a) and (b). Furthermore, the introduction of text embeddings could not improve the disease-state prediction performance, which aligns well with our previous investigation. The selected method MultiMIL based on multi-instance learning did not perform well in this task, which could be caused by overfiting or the noise level in the training dataset. Therefore, image embeddings can help build a classifier with higher accuracy in identifying the disease states of spatial transcriptomic samples. To further interpret the prediction performances after introducing the image embeddings with PFMs, we show the confusion matrix of different validation sets based on the best setting in Figure 2.6.4 (c). According to this figure, our method can successfully capture the correct disease-state information with a relatively large sample size. For some diseases with a small number of samples (for example, GBM and PPAD), our method was more error-prone, which was limited by the diversity of training data. Therefore, we believe that it is important to sequence more spatial transcriptomic samples with such diseases to build a more accurate classifier. We also visualize the results for StudyCV in Supplementary Figure 12 (a), which did not show significant improvement by integrating the image information with expression profiles or fine-tuning with PFMs and spatial transcriptomics. To explore the possibility of enhancing model performance with different sampling designs, we considered the oversampling and undersampling approaches [111] to address the label imbalance problem in the training dataset, and the corresponding results are shown in Supplementary Figures 13 (a) and (b) for the SampleCV and StudyCV settings. However, the undersampling approach cannot contribute to performance improvement, while both approaches cannot significantly improve the model performance under the StudyCV setting. Therefore, there is still room for developing a better classifier. We finally test if we can use a voting ensemble to perform prediction rather than constructing the inputs by concatenating gene expression profiles with image embeddings, and the results are shown in Supplemen-

tary Figures 14 (a) and (b). We did not find obvious difference between the default setting and the voting setting, and thus our selection for model construction may be optimal.

Furthermore, we explored the capacity of using spEMO to infer drug response at the individual level based on different omics. Previous methods for drug response prediction all start from expression profiles only [122], while in this section we utilized spEMO to predict the drug responses for each patient based on the WSI information. We utilized the dataset from a breast cancer cohort with trastuzumab as the treatment. In Supplementary Figure 15 (a), we performed clustering based on both gene expression information (GEX) as well as WSI embeddings encoded by GPFM (spEMO), and we found that the latter option has better clustering performance. The advantages of using image embeddings are also reflected in the results of prediction evaluation, shown in Supplementary Figure 15 (b). Details of our method and sample split are summarized in the Methods section. Therefore, we demonstrated that spEMO can also provide novel insights for understanding the heterogeneity of drug responses.

Finally, we also found that introducing novel features such as image embeddings and text embeddings can help predict patient survival, demonstrated through the analysis of The Cancer Genome Atlas (TCGA) data [224]. Here we collected $\sim$ 900 samples from the cancer-type COAD as well as the paired gene expression profiles at the whole-slide level. To demonstrate that using gene expression profiles and embeddings from GPFM can improve model performance in survival prediction, we selected DeepSurv [97] as a baseline model and considered three different input types: only using gene expression (gene), only using image embeddings (image), and using both gene expression and image embeddings (gene+image). The C-index [GitHub] from the testing dataset with the 10-fold cross-validation is used to evaluate the prediction performance. According to Supplementary Figure 12 (b), the mode gene+image reached the highest C-index score, and the improvement is significant compared with the mode based only on image embeddings (p-value=0.03 based on the Wilcoxon Rank-sum test). We tested different combinations by alternating the options of PFMs to generated embeddings, and Figure 2.6.4 (d) demonstrates that GPFM is also the best option in predicting survival for this dataset, while the combination of text embeddings and GPFM embeddings cannot boost model performance. Furthermore, the mode gene+image also had faster convergence speed and reduced the overfiting effect compared with the second-place mode, demonstrated in Figure 2.6.4 (e). Therefore, spEMO can also improve survival analysis, which further strengthens the clinical utility of our proposed method.

spEMO helps in inferring novel multi-cellular interactions from cancer slides. It has been shown that the patch information can be utilized as features for predicting the gene expression levels in the selected region, which can be treated

as a regression problem [109]. However, as far as we know, there are few methods have examined its utility in biological discoveries after predicting gene expression based on patch information. Here, we investigated the possibility of leveraging the predicted gene expression information from images to detect novel cell-cell (or spot-spot) communication and interaction from the COAD samples. Here we collected four COAD samples with WSI information from the HEST dataset, denoted as TENX111, TENX149, TENX148, and TENX147. We then utilized STFlow [83], a state-of-the-art image-to-expression predictor, to generate 50 genes with corresponding expression profiles of the selected samples. The location of each patch is determined by the default setting of the HEST dataset. We first used Moran's I score to detect spatially variable genes and visualize the top-ranked gene by the scores of different samples in Figure 2.6.5 (a). According to this figure, STFlow can generate genes with clear spatially expressed patterns, which also correspond to the functions of these genes. Therefore, we could analyze the predicted spatial transcriptomic data for information communication. Next, we chose COMMOT [22] as the tool for analyzing multi-cellular interactions. We followed the tutorial of this tool and computed the Leiden cluster results based on expression profiles and visualized it in Figure 2.6.5 (b). According to this figure, some clusters show clear spatially-enriched gene expression levels, such as clusters 2 and 11. Further analysis focused on a newly detected CXCL13-CCR7 interaction. While both genes are known to play key roles in immune responses, their combined activity has not previously been reported in COAD. By mapping signal direction onto the spatial transcriptomic data of sample TENX111, we observed strong interaction signals in clusters 2, 9, and 11 (Figure 2.6.5 (c)). These findings suggest that CXCL13-CCR7-mediated communication may be particularly active in specific tumor regions and could represent a novel target for immunological investigation. Future experiments focusing on these regions may unveil new opportunities for understanding COAD pathogenesis and for developing targeted therapeutic strategies.

spEMO can generate medical reports with a multi-modal instruction setting. Accurate medical reports are important for doctors to provide effective diagnosis and treatment for patients, and recent methods [46] have demonstrated the capacity of using pathology images to generate corresponding medical reports with pre-trained model based on paired image-text data. However, access to such functions for these methods is restricted by hospital policy or specific requests, and these methods do not consider gene expression levels as an extra modality in generating medical reports. Therefore, we consider a new task for generating medical reports with foundation models by using pathology images and gene expression information as prompts. Here we computed the top 10 highly variable genes (HVGs) based on HEST data for different diseases and paired the gene information with pathology images as well as disease and tissue information to generate the medical

report. Besides HEST, we also considered the PatchGastricADC22 [212] dataset with 991 pathology images and corresponding human-generated medical reports for evaluation. Our multi-modal agents used in this task are GPT-4o and o1. These models are strong foundation models for interpreting and reasoning with multi-modal inputs. To evaluate the quality of generated reports without real medical reports, we considered two aspects, shown in Figure 2.6.6 (a). We not only considered evaluating the conceptual similarity between sample annotation and generated reports in both text and embedding spaces (Embeddings similarity (Similarity) ranges from 0 to 1, while BERT score [Zhang et al.] and MEDCON score [191] range from 0 to 100.), but also recruited experienced pathologists ($n = 5$) to perform human evaluation. The pathologists wrote medical report based on whole-slide images and graded the medical reports with different resources and inputs from three aspects, including Completeness (Which report more completely captures important information?), Correctness (Which report includes less false information?), and Conciseness (Which report contains less non-important information?). Each metric ranges from 0 to 1. We also invited them to write the advantages and disadvantages of different reports, which could help us understand the contributions of spEMO more comprehensively. We have provided grading rubrics and instructions in Supplementary Data 1.

First, we analyzed the similarity between spEMO-generated reports and human-generated reports with images and texts from PatchGastricADC22 to compare GPT-4o's and o1's capacity. According to Figures 2.6.6 (b)-(d), the embeddings of reports generated by both GPT-4o and o1 have high cosine similarity (higher than 0.6 for both cases) compared with human-generated reports. Furthermore, reports generated by o1 generally have significantly higher scores under the evaluation with three different metrics, tested by the two-sided Wilcoxon rank-sum test. Therefore, improving the capacity of a Large Multimodal Model (LMM) can also improve its ability to summarize medical reports with multi-modal data, which demonstrates the potential of using advanced LMMs for generating medical reports. The reports used for human evaluation in the next paragraph are generated with o1.

By collecting the human evaluation results for human-generated reports and spEMO-generated reports with 10 sampled image-text pairs from PatchGastricADC22, we summarize the scores in Figure 2.6.6 (e), reports generated by spEMO have significantly higher completeness and correctness scores compared with human-generated reports. Since the spEMO-generated reports generally have longer lengths, it is reasonable to have a lower conciseness score than human-generated reports. Considering the advantages provided by pathologists shown in Figure 2.6.6 (g), spEMO-generated reports contain more throughvery information for disease diagnosis and interpretation. Therefore, the reports generated by the AI model are able to summarize the patient's pathology more comprehensively than human reports and are highly accurate. Furthermore, we also compared spEMO-generated reports with different inputs,

including only gene information (Gene), only image information (Image), and both gene and image information (Gene+Image) for testing the contribution of our integrative design. According to Figure 2.6.6 (f), the reports generated by the Gene+Image mode have significantly higher scores evaluated by completeness and correctness, while the conciseness scores are similar. Moreover, reports from the Gene+Image mode can also surpass the reports generated with image information only in two out of three evaluation approaches. Considering the advantages provided by pathologists shown in Figure 2.6.6 (h), spEMO-generated reports with the Gene+Image mode contain integrated information with both pathology and genetic descriptions. As a result, combining transcriptomic information with pathology whole-slide image information can also further enrich the content that medical reports can convey.

To evaluate the quality of generated reports without real medical reports and perform ablation studies, we consider two perspectives, including the completeness and consistency of model outputs. We also compared the multi-modal information (denoted as gene+image) with other ablation designs, including only using the whole slide image as input (denoted as image) or HVG information (denoted as gene). Details of our experiment design are discussed in the Methods section. A good model's output should keep a good balance between report completeness and report consistency, as an extremely long document might lose consistency in report format and disease similarity, while an extremely short document might not offer enough information for doctors or patients. We evaluated the output completeness based on the report length and the output consistency based on the Pearson Correlation Coefficients (PCCs) across text embeddings from the medical reports of different diseases. As the results shown in Supplementary Figures 16 (a) and (b), the gene+image mode reaches a good balance between the length of generated reports and the consistency of generated reports, and also demonstrated by the significance of the statistical tests annotated in the figure legend. Moreover, the generated information for 18 diseases also matches known biomedical documents, with minimal hallucination effect. Furthermore, we found that incorporating multi-modal information can also inspire foundation models to unify the contributions of different modalities, for example, only the Gene+Image mode links the histopathological findings (highlighted in red) with genetic information (highlighted in purple) for suggesting a medical report, shown in Supplementary Figures 16 (c)-(e). Moreover, the generated report also did not contain the privacy information of the patients who contributed the samples, which is also an advantage of using foundation models for generating medical reports. We investigated the stability of the gene+image mode by changing the random seeds with the same multi-modal prompts, with very high correlation of generated reports from different seeds, shown in Supplementary Figure 17. We also tested Prism [185] but its outputs contain incorrect disease labels for all kinds of testing data. We invited one pathologist to write medical report based on the same inputs, but the

molecular information was not correctly included in the report. By evaluating the similarity, BERT score, and MEDCON score between the expert-generated reports and spEMO-generated reports based on WSI, shown in Supplementary Figures 18 (a)-(c), the similarity between two reports are high under both context and medical information. Therefore, spEMO works as a more reliable and professional approach to write medical report. Finally, we tried o1 to explore the possibility of model enhancement with stronger reasoning ability, and we found that o1 could generate more comprehensive information for histopathological image analysis. The generated reports can be found in Supplementary Data 2.

spEMO defines a new metric for evaluating PFMs with extra modality. Training and evaluation are two important steps to developing a good PFM. The traditional approach for evaluating the quality of embeddings from a PFM relies on the reconstruction loss [31], which is indirect and lacks interpretation for its linkage with downstream applications. Recently, HEST [90] considered evaluating the performances of using embeddings from different PFMs to predict the corresponding gene expression at the spot level. However, the predicted results may not be reliable due to the noise level of transcriptomic data, and the evaluation conclusions are also confounded by the choices of classifiers, as demonstrated by our previous analysis for annotation and disease-state prediction. Therefore, we considered a new evaluation task for the embeddings of PFMs. For a single spot, we consider matching the embeddings of its background image and embeddings from its gene expression vector, as a retrieval task. There are several advantages of considering the setting of multi-modal alignment over predicting gene expression or other downstream tasks used for comparison. First, the encoder can embed the gene expression profiles in a space with lower dimensions and thus reduce the noise level. Therefore, our assessment of the qualify of image embeddings will be more reliable. Secondly, finding the corresponding spot based on image embeddings can yield key information beyond the gene expression, such as the disease status of the sample, gender, age, and other important metadata. Finally, it has been shown that precise multi-modal alignment for single-cell multi-omic data is difficult, and thus we assume that this task will also be challenging for PFMs without pre-training with transcriptomic data to achieve. A more challenging task may also stimulate further PFM research.

Here we utilize the same datasets that are used by HEST for evaluating the performances of gene expression prediction, including a series of spatial transcriptomic samples across different diseases. In total, we have datasets from 10 different diseases. The separation of training and testing datasets is pre-defined for different patients. We trained an encoder with the same architecture for the gene expression profiles to learn gene expression representation, and minimize the distance between the spot-wise similarity based on such representation and the spot-wise similarity based on image embeddings from different PFMs, including GPFM [137], UNI2 [31], GigaPath

[234], Ciga [218], and CTransPath [219]. We included three metrics for evaluating the alignment performance based on the testing dataset, including cross-entropy loss (CEL), top 1 precision (Precision@1), and top 10 precision (Precision@10). Details of methods and metrics are included in the Methods section. Our results are summarized in Table 2.5, which shows that the best model for this task is UNI2, followed by GigaPath and GPFM. Furthermore, none of the methods have a high Precision@1 score, suggesting the task's difficulty. One possible reason is that the spot with similar biological functions might have similar gene expression profiles, which might confuse the model for making the correct decision. However, the low score of Precision@10 further supports the necessity of developing a PFM with a strong ability to identify the linkage between gene expression and image features, which can further enhance the function of similar spot retrieval. Meanwhile, considering the gene expression embeddings from single-cell foundation models [38, 205] or spatial transcriptomic foundation models in the pre-training stage might also contribute to the improvement of model performance in this task.

2.3 Discussion

The contributions of pathology foundation models (PFMs), as well as large language models (LLMs), have been demonstrated by various applications in their specific domains, e.g., pathology image analysis or text generation. However, we lack effective approaches to transferring such success in extended biological areas, for example, the analysis of spatial transcriptomic datasets or multi-omic spatial datasets. Here we introduce spEMO as an AI system that integrates the embeddings from PFMs and (or) LLMs and the expression profiles from multi-omic spatial datasets to perform joint analysis. In our comprehensive experiments, we demonstrated that spEMO can help with various important tasks in the analysis of spatial domains including spatial domain identification, spot-type annotation, and disease-state prediction as a new task. Furthermore, we also defined a new task for evaluating different PFMs and motivating further exploration of multi-modal data integration for data-driven discovery.

spEMO has a strong ability to identify the spatial domains for both spatial transcriptomic data and multi-omic spatial data by computing a new spot-level embedding as a combination of expression profiles and embeddings from other FMs. Moreover, we can also utilize spEMO to transfer the spot annotation of known spatial datasets to annotate new datasets, which further supports the importance of using embeddings with prior as a bridge to link multi-modal information with specific biomedical datasets. We also explored the potential of using joint embeddings to predict the disease states from various whole-slide samples and discovered the

strong ability of linear classifiers with such embedding to perform prediction in this task. Therefore, spEMO provides reliable features to identify and understand different types of diseases. Furthermore, spEMO can also inspire new discoveries based on uncovering novel cellular interactions, predicting survival information and generating medical reports. Finally, we considered the flawed nature of existing evaluation methods for PFM pre-training and thus proposed a new task for screening out better FMs. For this multi-modal alignment task, although UNI has the best alignment performance, there still exists a large space for the improvement of different PFMs to identify the correct match. Therefore, this challenging task may motivate future developments of PFMs for multi-modal pathology analysis.

As a system, spEMO summarizes the current progress of different FMs and can provide optimal choices for each task. To cluster spatial transcriptomics with HE images or predict survival, the best solution is the GPFM-enhanced models. To annotate spatial domains, GPFM/UNI2/GigaPath-enhanced XGB models show leading performances. Considering the Mixture-of-Expert (MOE) design and the leading performance in predicting gene mutation, GPFM is a good PFM for handling bimolecular-driven tasks. To represent diseased samples, CHIEF shows a more promising performance as it balances information from diseases and tissues well. To predict disease states, embeddings from Virchow2 with an over-sampling strategy are highlighted. CHIEF and Virchow2 consider various data augmentation strategies, which could be the reason for their performance in disease-driven multi-omic tasks. To generate medical reports, expression profiles from highly variable genes and reasoning-based LMM, such as o1 are recommended. To retrieve the image patch with expression profiles, the UNI2-enhanced model is highlighted. Finally, the introduction of text embeddings might only improve model performance in clustering spatial multi-omic data especially in the case of a small feature sets, and thus the exploration of knowledge transfer should be further investigated. Our findings also demonstrate that a single PFM is not capable of performing all multi-omic analyzing tasks at the state-of-the-art (SOTA) level, which is consistent with the conclusions of benchmark analyses in the clinical [21] and spatial omics [246] fields, further highlighting the significance of our work.

There are limitations of the current implementation of spEMO. First, we found that the improvement of introducing information from PFMs might not always obviously improve the model performance, especially in the disease-state prediction task with StudyCV partition. The reason might be the heterogeneity from patients' biomarkers in both pathology and molecular levels. We need a better embedding model for encoding features from different omics. Second, the model performance is limited by the size of the training dataset, and thus our approach might not fully uncover the ability of embeddings from multi-modal information.

In summary, we believe that spEMO offers a new direction for analyzing multi-

omic data with the help of various FMs. In the future, this system can be further improved by incorporating more powerful FMs as well as considering more meaningful tasks to improve clinical utility and gain. As an example, we still expect to see models focusing on the joint pre-training with pathology image information and gene expression information, which has been explored in recent work [237].

2.4 Methods

Problem definition. For a typical spatial dataset $(X^{n\times m}, I^{p\times q\times 3})$ after normalization [229] with n spots and m features for the expression profile (X) and $p \times q$ as image (I) dimensions under the RGB setting, our target is to utilize the text description and its embeddings from a large language model as two mapping functions ($\mathcal{LLM}()$, $\mathcal{LLM}_{\mathrm{e}}()$) for processing feature-level metadata information $f^{m\times 1}$, and a pathology function model as another mapping function ($\mathcal{PFM}_{\mathrm{e}}()$) for processing image information, to formalize a set of spot embeddings for spatial data analysis.

Now we define the embedding generation layer of $\mathcal{LLM}()$ as $\mathcal{LLM}_{\mathrm{e}}()$, and the set of patches from a whole-slide image as I'_r, where r represents the radius of patches. Our spot embeddings can be represented as:

$$\begin{aligned} e_F &= \mathcal{LLM}_{\mathrm{e}}(\mathcal{LLM}(Prompt(F))), \\ e_I &= \mathcal{PFM}_{\mathrm{e}}(I'_r), \\ e_{spots} &= Agg(e_F, e_I), \end{aligned}$$

where $Prompt$ is a mapping function that can transfer the name of input data to the prompt space F. The prompts can be used as the input of language models and the outputs will be used to generated embeddings. The function $Agg()$ represents the method we used to aggregate the embeddings of all features for each spot. This setting is different for different downstream tasks, to achieve the optimal performance of spEMO. We will discuss the details of different downstream applications.

Furthermore, we only consider analyzing two modes of scELMo [121] to generate gene embeddings and spot embeddings in this manuscript. That is, we divide each entry in a row of X by the sum of its row. The function $AVG()$ represents the method we used to average the embeddings of all genes for each cell. If the mode is aa, we divide X by m. If the mode is wa, we divide each row of X by the sum of this row. Considering the spot with index i, we can define these two processes as:

$$\begin{aligned} AVG_{aa}(X_i) &= \frac{X_i}{m}, \\ AVG_{wa}(X_i) &= \frac{X_i}{sum(X_i)}. \end{aligned}$$

Moreover, scELMo shows that the wa is better for generating representations for cell embeddings, especially clustering. Considering the spot with index i and the corresponding function for averaging $AVG_{wa}()$, we can define this process as:

$$e_{spots} = AVG_{wa}(X)e_F||e_I.$$

Furthermore, e_I represents the set of image embeddings from a PFM, and $*||*$ represents the concatenation function. The final generated embeddings e_{spots} can be treated as a new representation of each spot and further improve the downstream applications related to spatial multi-omic data processing. To cluster data with different modalities, we consider a spatial multi-omic dataset $(X_1^{n_1 \times m_1}, X_2^{n_2 \times m_2})$ with two modalities X_1 and X_2. The feature embeddings for these modalities are based on prompting the LLMs and generating LLM embeddings using the outputs. Therefore, the integrated spot embeddings can be represented as:

$$e_{spots}^{multi} = AVG_{wa}(X_1)e_F^1||AVG_{wa}(X_2)e_F^2.$$

Here e_{spots}^{multi} represents the spot representations generated by multi-omic data, and e_F^1 and e_F^2 correspond the feature embeddings generated from modalities 1 and 2. In our current experiment, the image information is not provided. However, it is flexible to incorporate the image embeddings to update e_{spots}^{multi} by following the previous design. In our application, zero-shot embedding is the opposite of fine-tuning. Zero-shot embedding means we extract the embeddings directly from trained FMs to address downstream tasks (such as linear-probing-based application), while fine-tuning allows us to change model weights to generate domain-specific embeddings.

As a system, spEMO also support the integration of histopathology representations and transcriptomic representations with other tool sets, and these tools can be either fine-tuned or prompted with the provided representations to perform domain-specific applications to enhance performances or gain new insights, which serve as the second mode of spEMO. Moreover, spEMO can also determine the optimal solution for each task as a router and help human experts as a clinical assistant, based on the experimental results.

Spatial domain identification. For this task, our target is to perform clustering based on the spatial transcriptomic dataset and multi-omic spatial dataset. For the transcriptomic dataset, we considered the e_{spots} with different sources, including PCs (PCA), image embeddings only (GPFM[137]), the concatenation of PCs and image embeddings (PCA+GPFM), spatial location only (*Spatial*) and embeddings from LLMs only (LLMemb). We also consider the enhanced version of SpaGCN [79] with image embeddings (SpaGCN+GPFM), which is generated by changing the adjacency matrix computation process of SpaGCN based on the new averaged im-

age embeddings. Based on our evaluation, the best setting is SpaGCN+GPFM. To understand the contribution of image embeddings for this task, we computed the auto-correlation scores between spatial location and image embeddings and demonstrated the advantages of capturing more spatial-aware features with the help of image embeddings.

For the multi-omic dataset, since we cannot find the source of image information, we consider the e_{spots} from PCs computed by gene expression (RNA only), PCs computed by protein expression (ADT only), embeddings of LLMs with gene text description (LLMemb RNA only), embeddings of LLMs with protein text description (LLMemb ADT only), the addition of LLMemb RNA only mode and LLMemb ADT only mode (LLMemb RNA+ADT), and the concatenation of LLMemb RNA only mode and LLMemb ADT only mode (LLMemb RNA||ADT). Based on our evaluation, the concatenation of embeddings from LLMs has the best performance.

Spot-type annotation. For this task, our target is to predict the spot types of the testing dataset based on training a classifier. We performed cross validation based on different samples of two datasets to report the variance of metrics. Our classifiers [161] used in this section include LogisticRegression (LR), Support Vector Classifier (SVC), Random Forest Classifier (RF), and XGBClassifier (XGB). Our input data contain both expression level information and embedding level information, including PCs of gene expression profiles (PCs), highly-variable gene expression profiles (HVG), image embeddings (GPFM, default radius is 112), and the concatenation of HVG and image embeddings (HVG+GPFM (x), HVG+UNI (x), HVG+GigaPath (x); where x represents the radius).

Disease-associated prediction. For the disease-state prediction task, we intend to utilize the multi-modal information to predict the disease-state labels of different samples in the testing dataset. Our classifier used in this section includes LR for pseudobulk-level data and MultiMIL [119] for spot-level data. The settings of input data are very similar to the spot-type annotation task. The difference is for the sample-level prediction without the consideration of spot-level heterogeneity, we utilized the averaged gene expression and image embeddings across the spots in one sample to construct the input data, also known as the pseudobulk setting. For the multi-instance learning framework, our input data is defined at the spot level. We considered splitting the training and testing dataset under both sample ids and study ids, denoted as SampleCV and StudyCV, respectively. For the drug response prediction task, we perform dimension reduction based on PCA for both embeddings from PFMs and bulk-level gene expression profiles to obtain the latent representations, and then perform clustering and classification to validate the contributions of PFMs. The predictor used in this step is LR, and we control the ratio of training samples and testing samples as 7:3. For the survival prediction task, we combine expression profiles with image embeddings and utilize DeepSurv [97] for predicting

the survival function based on 10 fold cross-validation test and report the metrics under these testing datasets.

Multi-cellular interaction inference. In this task, we infer novel cell-cell interactions based on the predicted gene expression profiles from a whole slide image based on STFlow [83]. The image is divided into several patches and thus the expression profiles are measured for the centroid of each patch. We then utilized COMMOT [22] to compute the strength of the sender gene, receiver gene, and signal direction. The samples are from HEST 1k with tissue as Bowel and disease state as COAD. The training data are from Xenium.

Medical report generation. For this task, we extract the highly variable gene information of cancer samples and rank them by dispersion to obtain a gene list. We then input the gene list, metadata information as well as the whole-slide image into the multimodal foundation models (GPT4-o and o1) as prompts to generate medical reports. We select one sample for each disease in HEST 1k for evaluation.

Multi-modal information alignment. For this task, we define a new task as well as a new metric for evaluating the performance of different PFMs. A good set of image embeddings should also match the other modality from the same dataset. Therefore, we investigate the performance of PFM embeddings in matching the corresponding gene expression profiles from the same spot, across different datasets from HEST [90]. Therefore, we train a new encoder for gene expression profiles to learn the spot representation by minimizing the distance between paired image embeddings and gene expression embeddings in the training dataset. In the testing dataset, we tested the alignment between image embeddings and gene expression embeddings based on various metrics and thus offered a new task for embedding quality assessment.

Evaluation and metrics. In this section, we introduce the metrics we used for evaluating in different tasks and the performances of their corresponding methods.

For spatial domain identification, we included NMI, ARI, ASW and their averaged values (Avg) for the comparison. Details of these metrics, referred from [124, 132], are introduced below:

1. Normalized Mutual Information (NMI): NMI is a score to evaluate the performance of biological information conservation. We compute this score based on the mutual information between the optimal Leiden clusters and the known spatial domains and then take the normalization. NMI $\in (0, 1)$ and higher NMI means better performance.

2. Adjusted Rand Index (ARI): ARI is a score to evaluate the performance of biological information conservation. ARI is used to evaluate the agreement between optimal Leiden clusters and spatial domains. ARI $\in (0, 1)$ and higher ARI means better performance.

3. Average Silhouette Width (ASW): We only have spatial-main ASW (ASW)

for this metric. For one spot, ASW calculates the ratio between the inner cluster distance and the intra cluster distance for this spot. Therefore, higher ASW means better biological information conservation. To make all metrics consistent, for ASW, we take the normalization, that is:

$$ASW = \frac{ASW^{raw} + 1}{2}.$$

All metrics (including the averaged score) are in (0,1), and a higher score means better model performance.

For spot-type annotation, we included Accuracy and Weighted F1 score (Weighted F1) for the comparison. Details of these metrics [161] are introduced below:

1. Accuracy: The accuracy score is defined the proportion of correct matching between predicted labels y_p and observed labels y_t:

$$Accuracy = \frac{\sum_{i=1}^{n} I(y_p^i = y_t^i)}{n},$$

where $I()$ represents the indicator function. The weighted F1 score requires us to compute the F1 score ($F1$) of different classes, as well as the proportion of samples with corresponding label. Therefore, if we have N classes and for class j the proportion is defined as w_j, the Weighted F1 score is computed as:

$$Weighted\ F1 = \sum_{j=1}^{N} w_j F1_j.$$

Both of metrics are in (0,1), and a higher score means better model performance.

For disease-state prediction, our metrics are the same as we used in evaluating methods for spot-type annotation, as they are both multi-label classification problems.

For drug response prediction, we utilize the metrics the same as we used in evaluating methods for disease-state prediction, as it is also binary classification problem.

For survival prediction, we utilize C-index to evaluate the prediction performance of survival function. C-index [6] is a generalization of the area under the ROC curve (AUC) that can take into account censored data. It is similar to AUC evaluation criteria.

For multi-modal alignment, our metrics included Cross-Entropy Loss (CEL) and top k precision (Precision@k). Details of these metrics are introduced below:

1. Cross-Entropy Loss (CEL): In our training stage, we first pair the image and gene expression vector. We then minimize the CEL between the dot product of embeddings from images and embeddings from gene expression profiles and the

identity matrix. Considering the dot product matrix as $(l_{1,1}, l_{1,2}, ..., l_{n,n})$ and the identity matrix as $(y_{1,1}, y_{1,2}, ..., y_{n,n})$, and the CEL is computed by:

$$CEL_s = -\sum_{c=1}^{n} \log \frac{\exp(x_{s,c})}{\sum_{k=1}^{C} \exp(x_{s,k})} y_{s,c},$$
$$CEL = \frac{\sum_{s=1}^{S} CEL_s}{S}.$$

Here S represents the number of samples in the testing dataset.

2. Top k Precision (Precision@k): Here we still compare the dot product matrix with the identify matrix for each sample to compute the precision in top k candidates ranked by the dot product value. The choices of k include 1, 10, and 20.

The range of CEL is in (0, ∞), and the range of Precision@k is in (0,1). A lower score of CEL means better model performance, where a higher score of Precision@k means better model performance.

Baselines. In this section, we introduce the baseline models for different tasks. Since we have discussed the settings introduced in this manuscript in the previous section, we only discuss domain expert methods.

For spatial domain identification, we consider iSTAR [253], Pix2Path [59], SC3 [99], SpaGCN, Novae [15], and SpatialGLUE [127] as expert methods. iSTAR is a method developed for super-resolved spatial transcriptomic prediction. Pix2Path is designed to generate pathology images based on molecular information. SC3 is a clustering algorithm designed for single-cell RNA sequencing clustering by considering the averaged performances of k-means results under different resolutions. SpaGCN is a Graph Neural Network (GNN) designed for clustering of spatial transcriptomics. SpaGCN computes the adjacency matrix of spots based on both spatial locations and pixels of background images and then trains a GNN for self-supervised clustering. Novae is a foundation model trained with large-scale spatial transcriptomic data. SpatialGLUE is a method designed for spatial multi-omic clustering, which sets up GNNs for different modalities and learn their joint embeddings used for clustering.

For spot-type annotation, we consider various classifiers as expert methods. Logistic Regression is designed as a linear classifier by running linear regression after logit function transformation. Support Vector Classifier is a classifier based on support vectors from hyperplanes. Random Forest is used to perform classification based on aggregating different decision trees. XGBclassifier is a classifier based on the boosting algorithm.

For disease-state annotation, we consider MultiMIL as an expert method. MultiMIL is a Multi-Instance Learning (MIL) algorithm for multi-omic single-cell data analysis. It first learns cell embeddings and then aggregates cell embeddings into sample embeddings by attention mechanism. The loss function combines both re-

construction loss for expression profiles and classification loss for disease states.

For medical report generation, we consider embedding similarity, BERT score [Zhang et al.] and MEDCON score [191] as metrics. Embedding similarity is computed with cosine similarity between two embeddings from OpenAI API. BERT score is the cosine similarity between two embeddings from BERT, and MEDCON score examines the proportion of tokens related to medicine in the evaluated text.

For multi-modal alignment, the expert methods contain different pathology foundation models. Their motivation is to pre-train an image encoder with large-scale pathology image datasets.

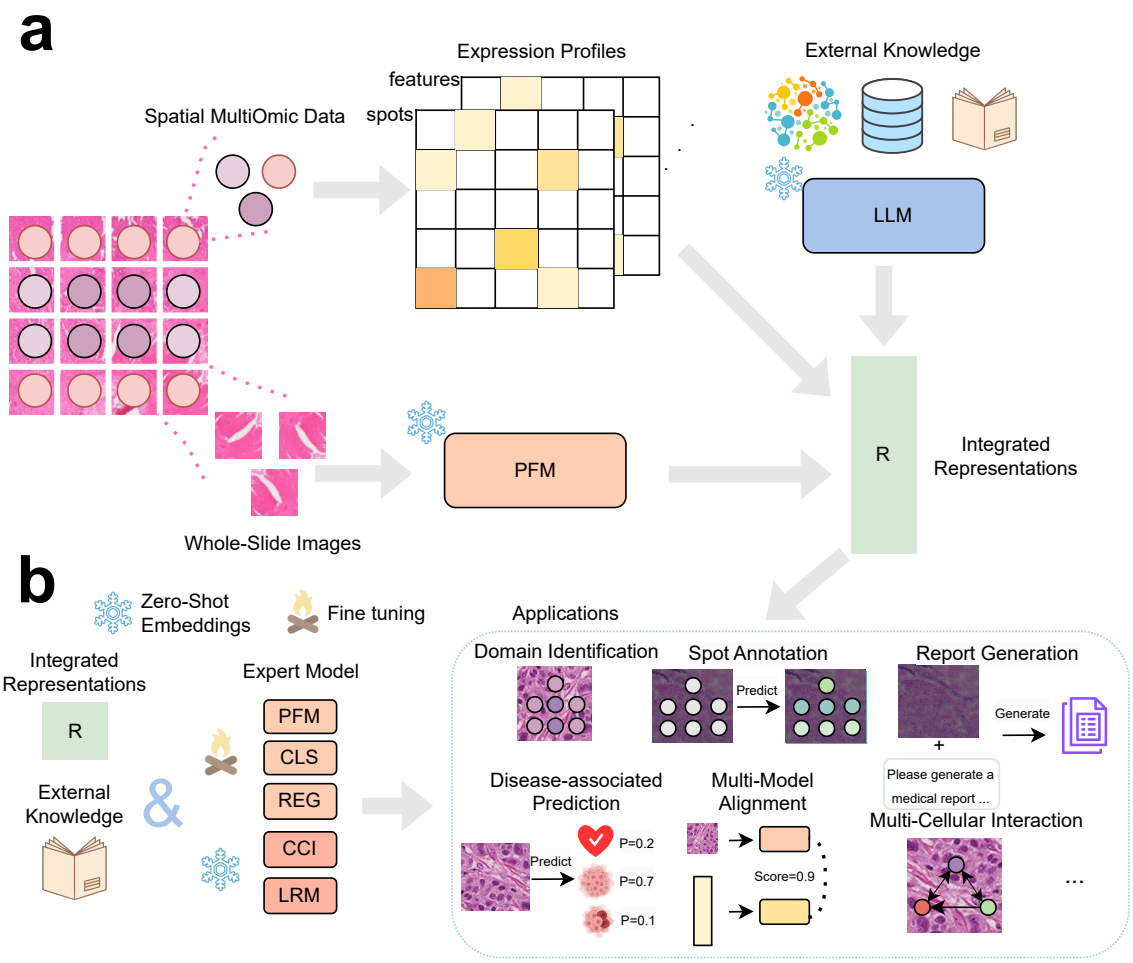
For all domain-expert methods, we used their parameters reported by the original authors for experiments, as they are already optimized for the given task.

Data Availability. We do not generate new data in this research and all data used in this manuscript are publicly available without restricted access. SpatialLIBD data can be accessed under access code [`https://research.libd.org/spatialLIBD/`]. Spatial multi-omic data can be accessed under access code [`https://drive.google.com/drive/folders/1RlU3JmHg_LZM1d-o6QORvykYPoulWWMI`] and [`https://drive.google.com/drive/folders/1RlU3JmHg_LZM1d-o6QORvykYPoulWWMI`]. Two 10X Visium data can be accessed under code [`https://support.10xgenomics.com/spatial-gene-expression/datasets/1.1.0/V1_Adult_Mouse_Brain`] and [`https://support.10xgenomics.com/spatial-gene-expression/datasets/1.1.0/V1_Adult_Mouse_Brain_Coronal_Section_2`]. TCGA data can be accessed under code [`https://portal.gdc.cancer.gov/`]. HEST data can be accessed under code [`https://huggingface.co/datasets/MahmoodLab/hest`]. We summarize the dataset statistics and download information in Supplementary Table 3.

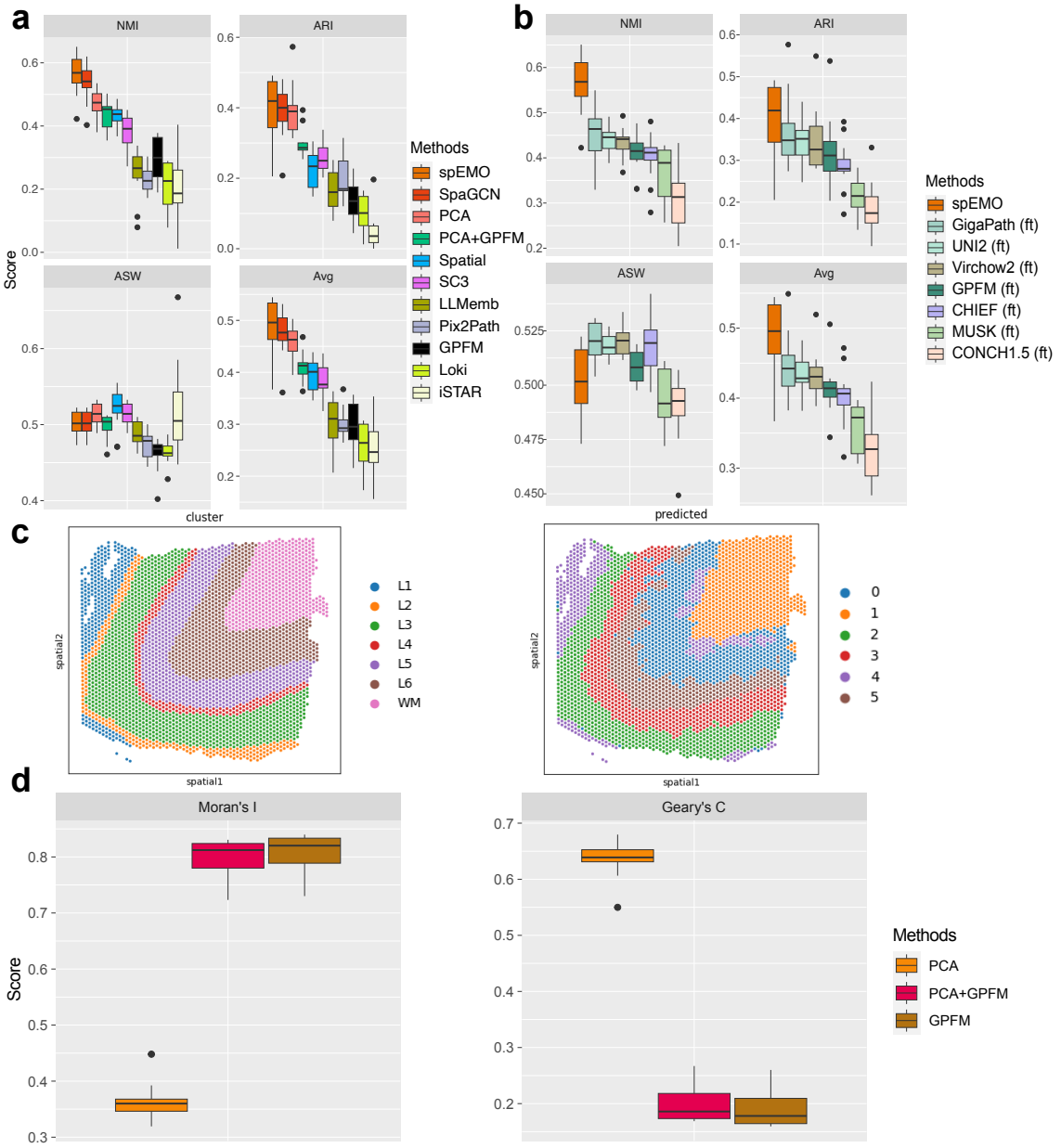
Code Availability To generate the text descriptions and corresponding embeddings, we rely on the API of OpenAI. To generate the embeddings from pathology foundation models as well as fine-tune them, we reply on Yale High-performance Computing Center (YCRC) and utilize one NVIDIA A100 GPU with up to 50 GB RAM. To run spEMO and compare it with other methods, we utilize one NVIDIA A100 GPU with up to 150 GB RAM. Information of running time and memory usage can be found in Supplementary Table 4. The codes of spEMO can be found in `https://github.com/HelloWorldLTY/spEMO`. The licence is MIT license.

2.5 Tables

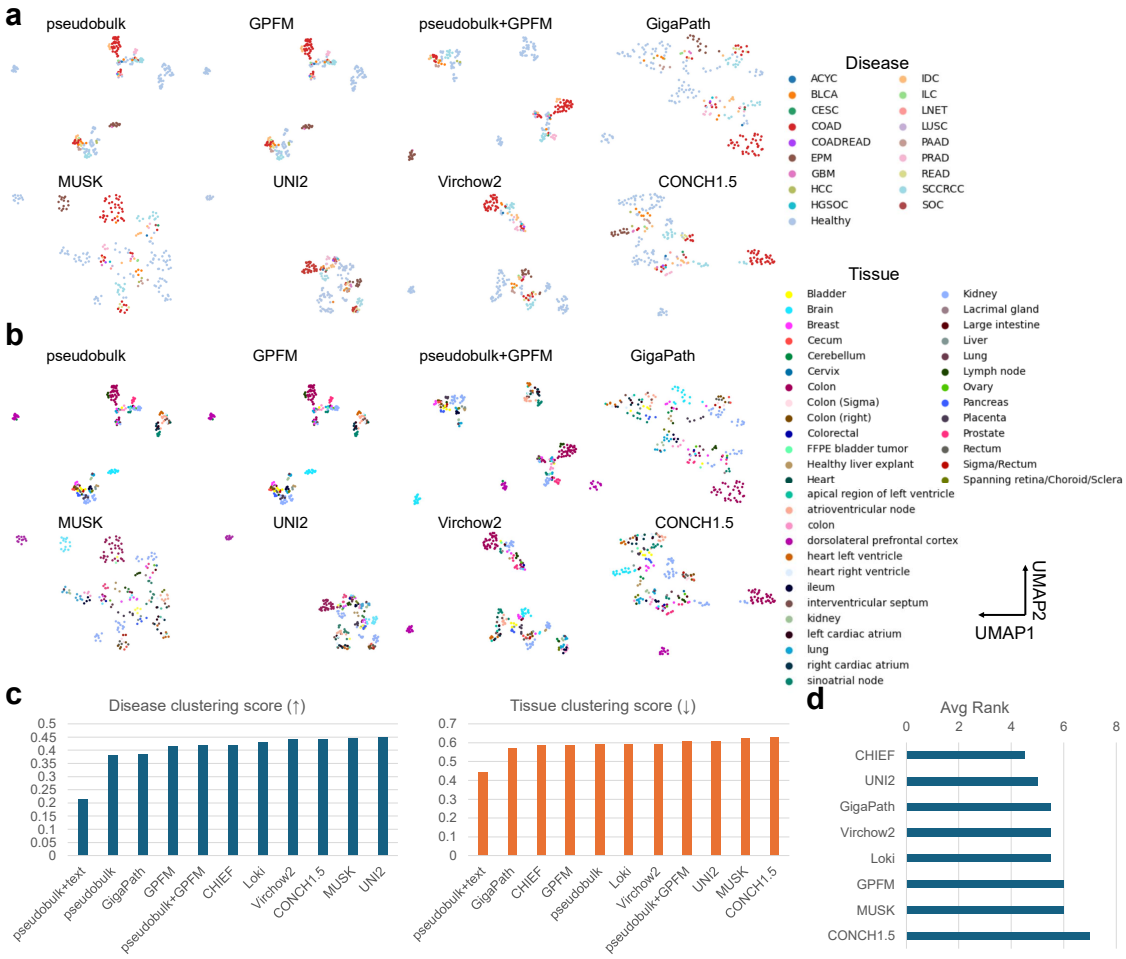
2.6 Figure Legends



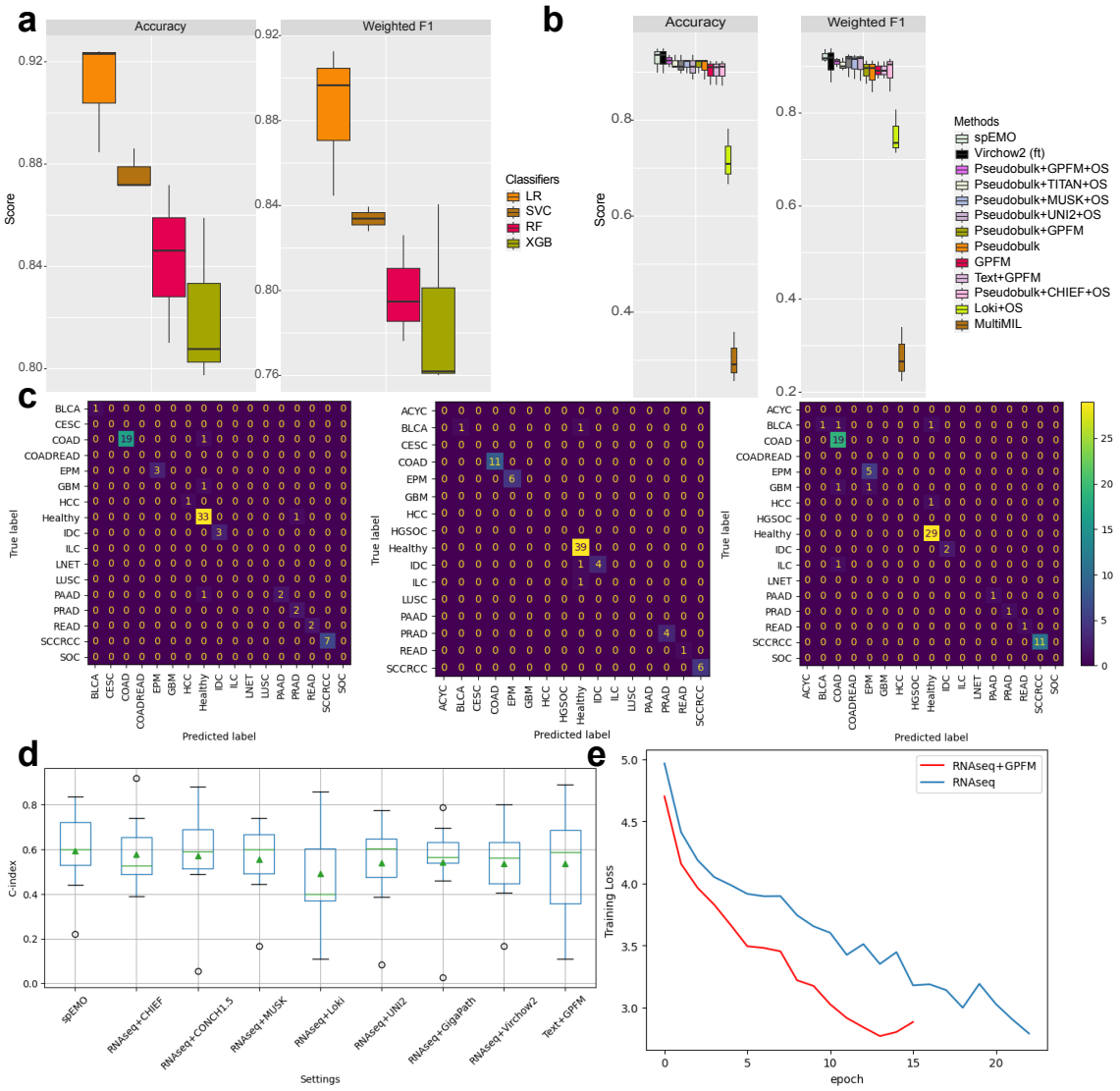
Appendix Figure S 2.6.1: The landscape of spEMO. (a) The zero-shot embedding framework of spEMO. Here we introduce embeddings from PFMs and LLMs to help the analysis of spatial multi-omic datasets, by transferring the selected dataset from the expression space to the embedding space. (b) The renewed framework with an expert-model-involved setting of spEMO. By integrating the image representations and expression profiles to domain-expert methods, spEMO is capable of performing more downstream applications.



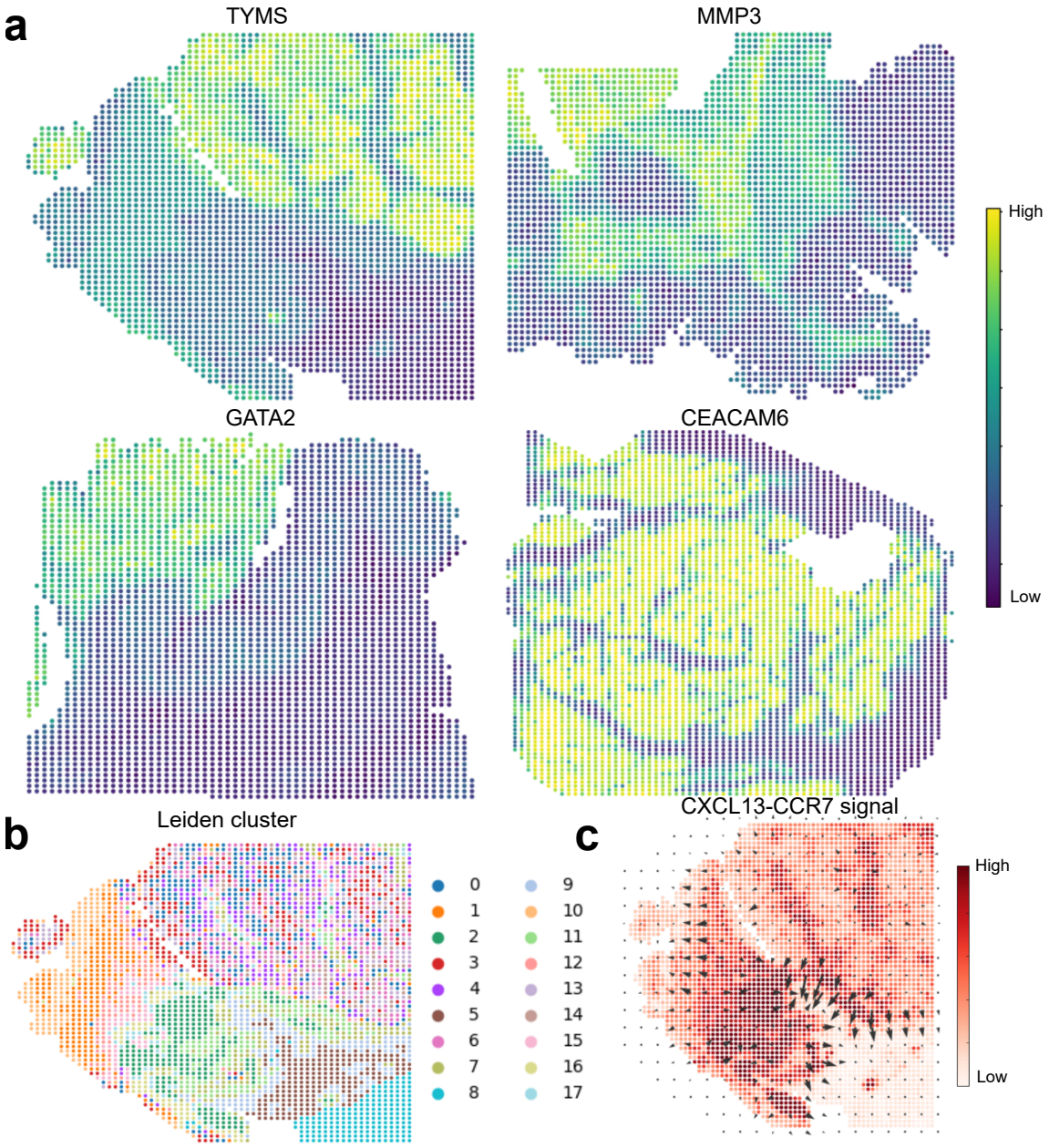
Appendix Figure S 2.6.2: Understanding the contribution of spEMO for spatial domain identification. (a) The evaluation results of different methods for domain identification of spatial transcriptomic datasets. Here spEMO represents the combination: SpaGCN+GPFM (n=12 per group; triangle, mean; center line, median; box limits, upper and lower quartiles; whiskers, up to 1.5×interquartile range; points, outliers). (b) The evaluation results of different PFMs as base models for domain identification of spatial transcriptomic datasets (n=12 per group; triangle, mean; center line, median; box limits, upper and lower quartiles; whiskers, up to 1.5×interquartile range; points, outliers). (c) The visualization of clustering performance based on the selected slide. The upper panel is colored by cortical layers, and the lower panel is colored by clustering labels. (d) The evaluation results of different settings for capturing spatial auto-correlated features (n=12 per group; triangle, mean; center line, median; box limits, upper and lower quartiles; whiskers, up to 1.5×interquartile range; points, outliers).



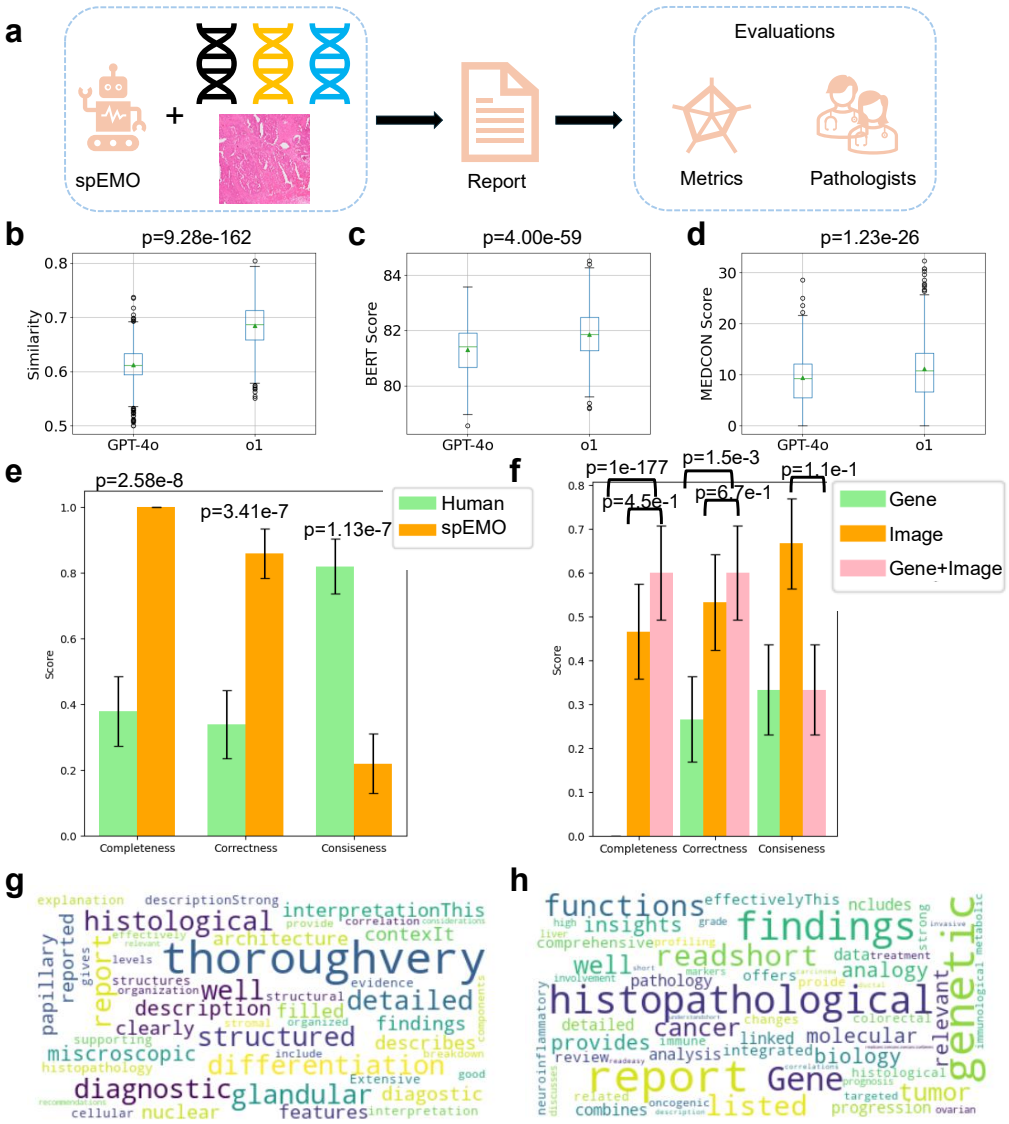
Appendix Figure S 2.6.3: Clustering scores and visualization of embeddings colored by different metadata. (a) UMAP visualization based on the different embeddings colored by disease states. (b) UMAP visualization based on different embeddings colored by tissues. (c) Averaged clustering scores computed based on disease states (left panel) and tissues (right panel) across different embeddings. The direction of arrows indicates embeddings with a better performance. (d) Average rank of different PFMs by balancing the discrimination of diseases and tissues.



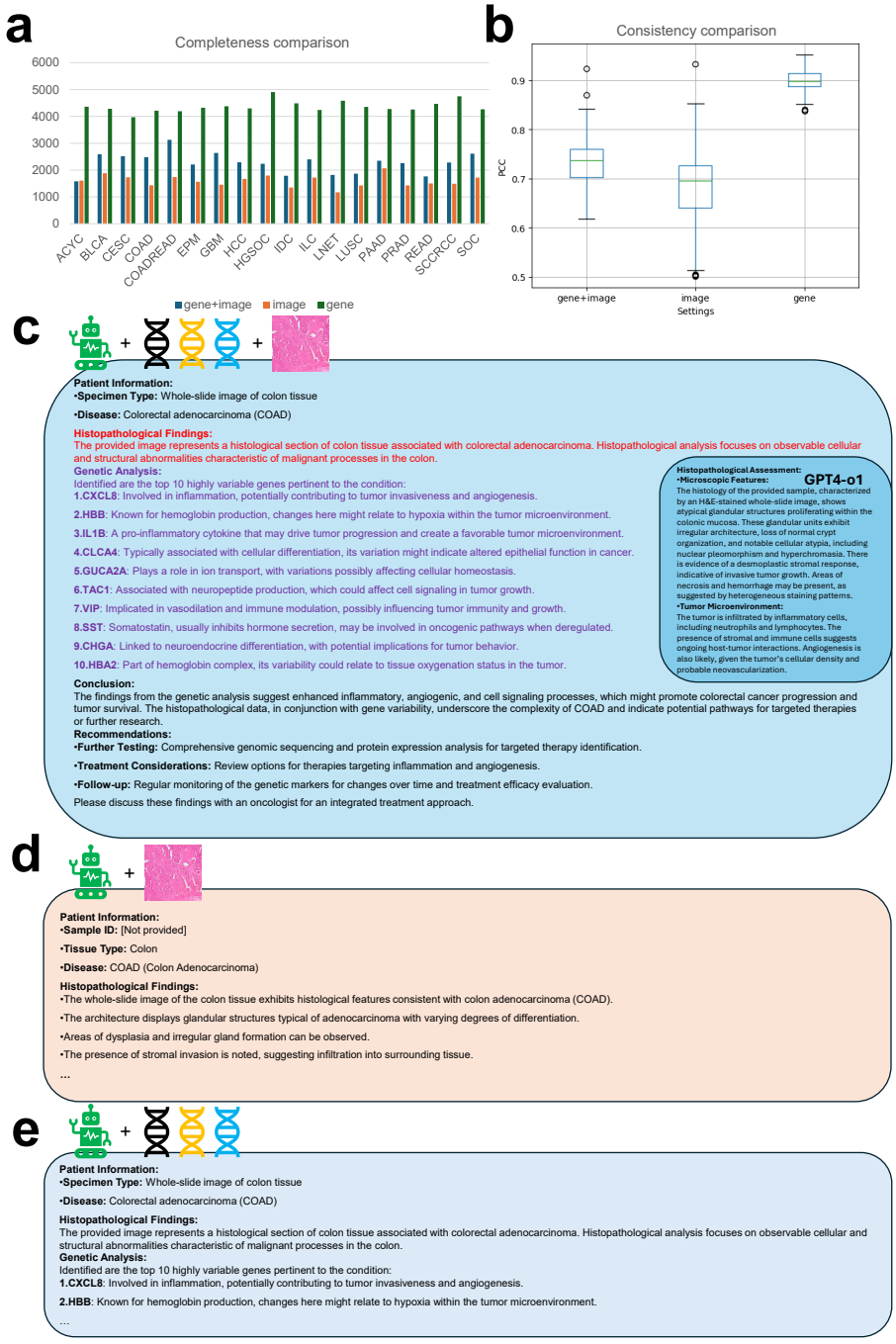
Appendix Figure S 2.6.4: Results of utilizing image embeddings for possibly improving disease-state prediction. (a) Comparisons of different classifiers under the SampleCV mode using selected metrics (n=3 per group; triangle, mean; center line, median; box limits, upper and lower quartiles; whiskers, up to 1.5×interquartile range; points, outliers). (b) Comparisons of different input settings under the SampleCV mode using selected metrics. Here spEMO represents the combination: Pseudobulk+Virchow2+Oversample (OS) (n=3 per group; triangle, mean; center line, median; box limits, upper and lower quartiles; whiskers, up to 1.5×interquartile range; points, outliers). (c) The confusion matrices of all validation sets under the SampleCV mode. (d) The C-index comparison across different input settings for survival prediction (n=5 per group; triangle, mean; center line, median; box limits, upper and lower quartiles; whiskers, up to 1.5×interquartile range; points, outliers). (e) The training loss curves of survival prediction between two modes with or without image embeddings.



Appendix Figure S 2.6.5: The analysis of multicellular interaction based on predicted gene expression profiles. (a) The expression levels of spatially variable genes across the four samples. (b) The Leiden cluster labels computed based on gene expression profiles for sample TENX111. (c) The signal direction of cell-cell interaction CXCL13-CCR7 computed based on the sample TENX111.

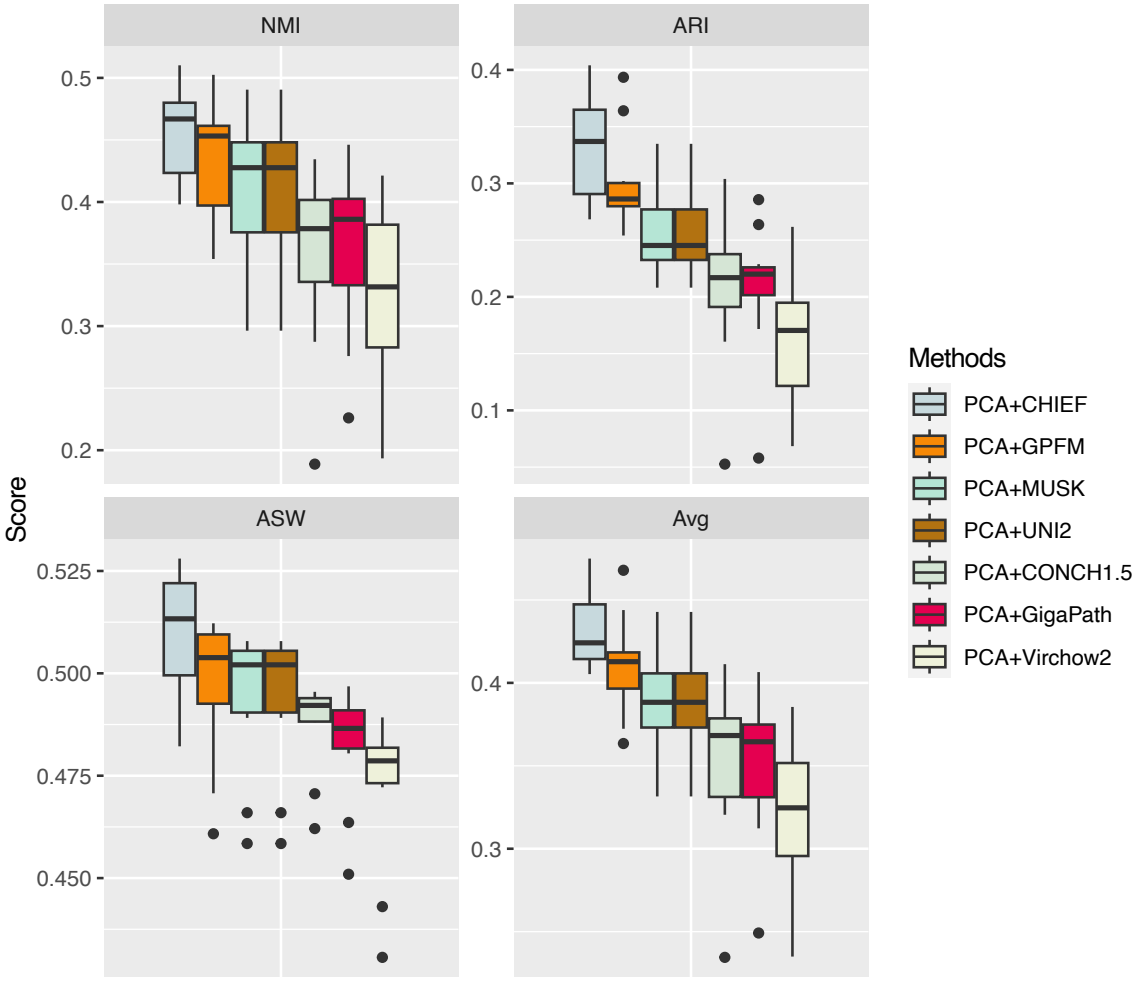


Appendix Figure S 2.6.6: Pipeline and results of generating medical reports with a multi-modal AI agent. (a) Workflow of using spEMO to generate medical reports and evaluate them from different perspectives. (b) Cosine similarity between the embeddings from generated reports with different base models and ground truth reports (n=991 per group; triangle, mean; center line, median; box limits, upper and lower quartiles; whiskers, up to 1.5×interquartile range; points, outliers). (c) BERT score between the generated reports with different base models and ground truth reports (n=991 per group; triangle, mean; center line, median; box limits, upper and lower quartiles; whiskers, up to 1.5×interquartile range; points, outliers). (d) MEDCON score between the generated reports and ground truth reports (n=991 per group; triangle, mean; center line, median; box limits, upper and lower quartiles; whiskers, up to 1.5×interquartile range; points, outliers). (e) Scores of human evaluation results under three different metrics between the human-generated reports and spEMO-generated reports ($n = 50$, standard deviation is annotated). (f) Scores of human evaluation results under three different metrics among generated reports with different input modality ($n = 50$, standard deviation is annotated). (g) Word cloud [151] of expert comments on the advantages of spEMO-generated reports in (e). (h) Word cloud of expert comments on the advantages of spEMO-generated reports with the Gene+Image mode in (f). The p-value is computed based on two-sided Wilcoxon rank-sum test.



Appendix Figure S 2.6.7: Evaluations and examples for the medical report generation task. The statistical test used here is two-side Wilcoxon Rank-sum test. (a) Evaluation results of completeness across different diseases. The statistics are: p-value=1.52e-5 for gene+image mode vs. image mode; p-value=7.63e-6 for gene+image mode vs. gene mode; p-value=7.63e-6 for gene mode vs. image mode. (b) Evaluation results of consistency for different modes. Each boxplot contains the PCCs from different disease (n=153 per group; triangle, mean; center line, median; box limits, upper and lower quartiles; whiskers, up to 1.5×interquartile range; points, outliers). The statistics are: p-value=1.21e-23 for gene+image mode vs. image mode; p-value=7.8e-27 for gene+image mode vs. gene mode; p-value=7.84e-27 for gene mode vs. image mode. (c) Medical report example from a COAD sample with gene+image mode. (d) Medical report example from a COAD sample with image mode. (e) Medical report example from a COAD sample with gene mode.

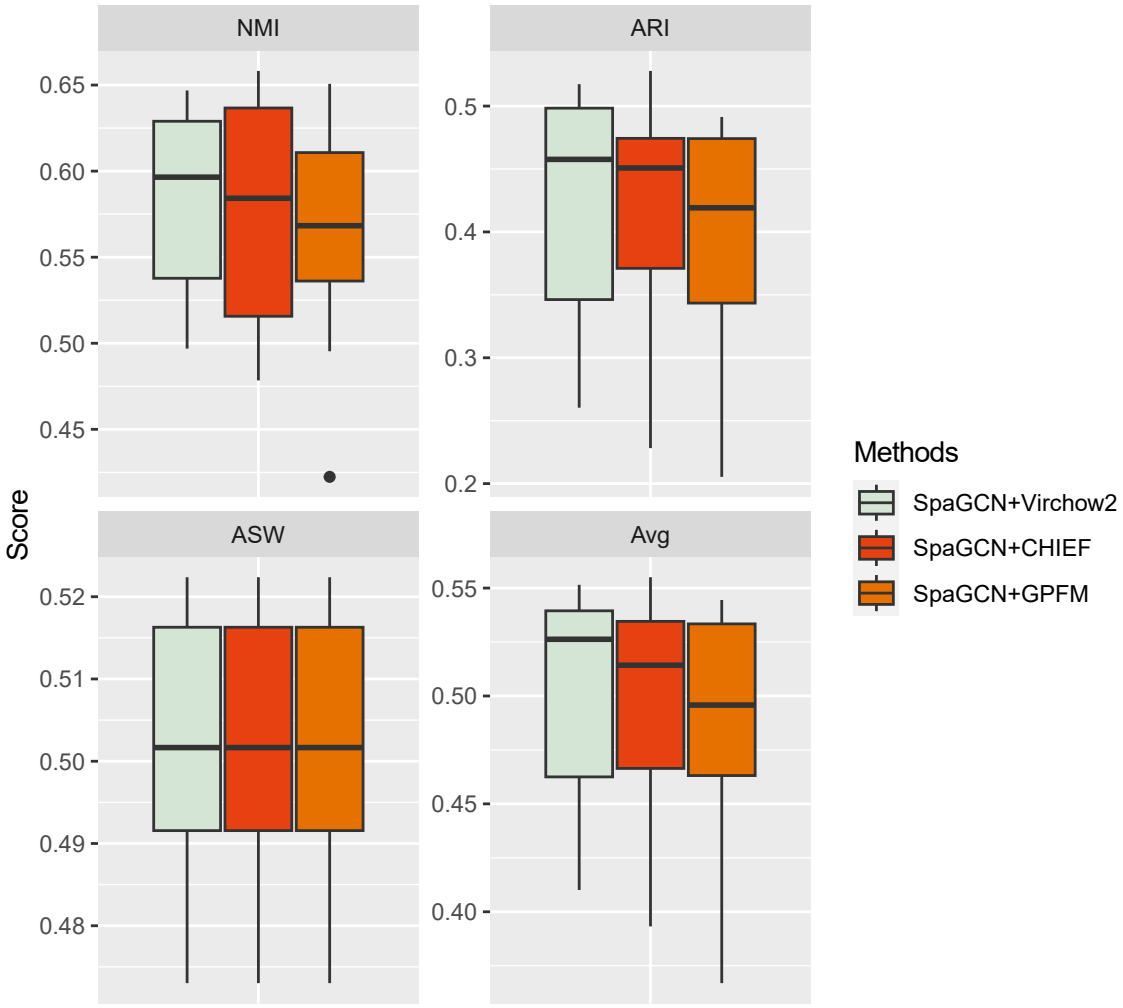
2.7 Supplementary Figures



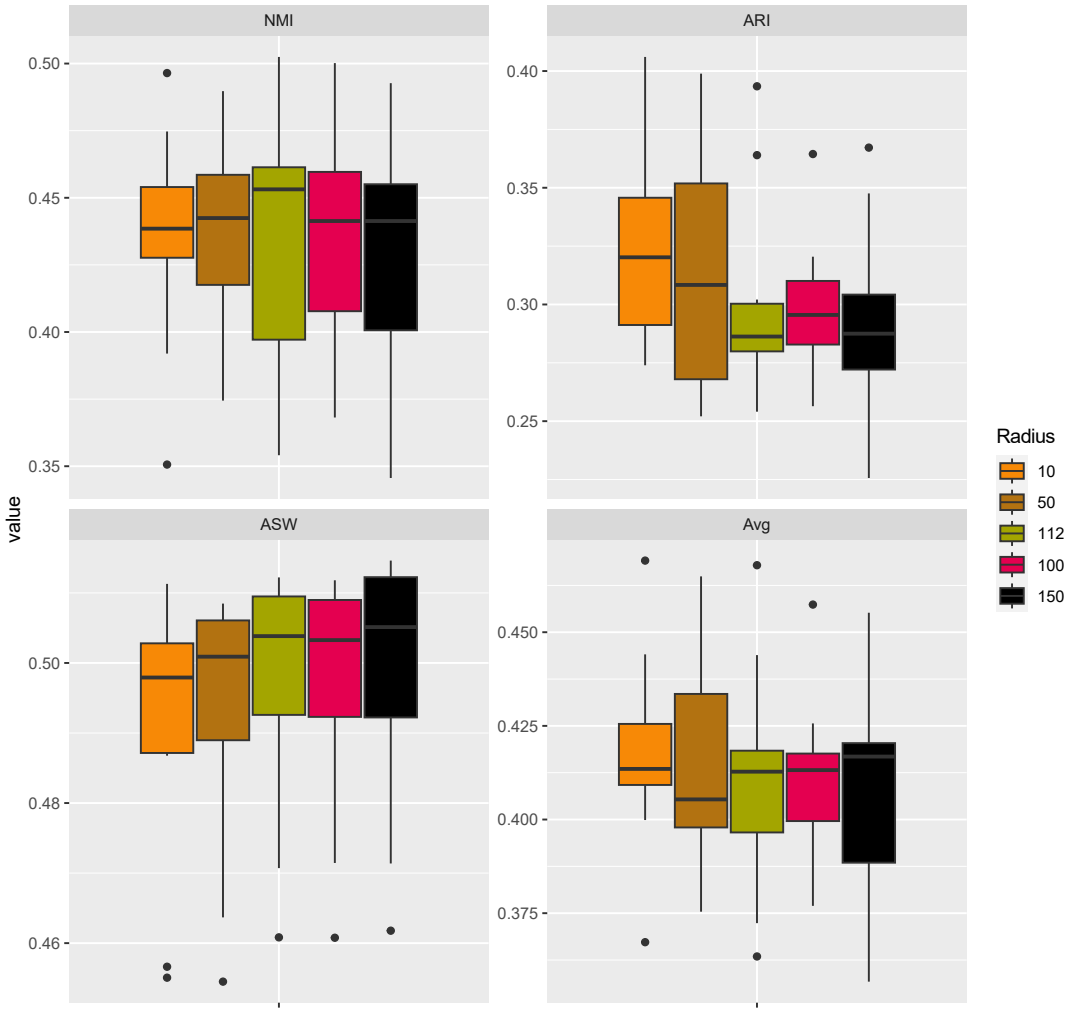
Supplementary Fig. 1: Testing the contributions of image embeddings from different PFMs for spatial domain identification with the zero-shot inference mode (n=12 per group; triangle, mean; center line, median; box limits, upper and lower quartiles; whiskers, up to 1.5×interquartile range; points, outliers). Our metrics are NMI, ARI, ASW and their average value (Avg).

CEL (↓)					
	Ciga	ctranspath	GPFM	UNI2	Gigapath
IDC	6.74(.427)	5.99(.457)	4.92(.410)	4.57(.471)	4.68(.440)
PRAD	8.45(.182)	8.38(.162)	7.70(.247)	7.61(.233)	7.43(.323)
PAAD	5.59(.035)	4.95(.034)	4.22(.075)	3.63(.086)	3.77(.140)
SKCM	5.70(.096)	4.46(.089)	3.95(.128)	3.65(.038)	5.47(.296)
COAD	6.11(.049)	5.59(.010)	4.86(.033)	4.21(.149)	4.25(.101)
READ	6.85(.311)	6.57(.291)	5.95(.357)	5.66(.297)	5.47(.296)
CCRCC	8.20(.108)	8.07(.095)	7.38(.091)	7.21(.089)	7.16(.100)
HCC	6.56(.028)	6.59(.015)	6.16(.049)	5.66(.081)	5.72(.042)
LUNG	5.24(.079)	4.35(.123)	3.48(.057)	2.89(.150)	2.93(.182)
LYMPH_IDC	7.42(.047)	7.18(.047)	6.55(.052)	6.20(.046)	6.14(.062)
Average	6.686	6.213	5.517	**5.129**	5.302
Precision@1(%) (↑)					
	Ciga	ctranspath	GPFM	UNI2	Gigapath
IDC	1.24(.005)	2.89(.010)	7.55(.023)	9.57(.037)	8.38(.030)
PRAD	0.24(.000)	0.28(.000)	0.67(.001)	0.89(.001)	1.22(.003)
PAAD	2.68(.003)	5.61(.000)	10.95(.009)	18.57(.008)	17.78(.014)
SKCM	3.75(.004)	7.92(.000)	13.30(.018)	19.09(.014)	8.57(.017)
COAD	1.83(.001)	2.96(.001)	6.56(.002)	12.05(.021)	12.11(.005)
READ	1.24(.003)	1.59(.004)	4.25(.011)	5.84(.008)	8.57(.017)
CCRCC	0.18(.000)	0.20(.000)	0.52(.000)	0.70(.001)	0.77(.000)
HCC	1.84(.001)	1.39(.001)	3.29(.003)	7.14(.002)	6.95(.002)
LUNG	3.74(.001)	9.67(.008)	18.35(.009)	29.62(.024)	28.34(.021)
LYMPH_IDC	0.38(.001)	0.61(.000)	1.48(.000)	2.75(.001)	3.21(.001)
Average	1.712	3.312	6.692	**10.622**	9.59
Precision@10(%) (↑)					
	Ciga	ctranspath	GPFM	UNI2	Gigapath
IDC	8.93(.031)	17.40(.057)	34.21(.077)	41.15(.099)	38.33(.089)
PRAD	2.01(.005)	2.20(.004)	5.04(.010)	6.12(.009)	7.29(.018)
PAAD	17.69(.003)	30.29(.001)	47.16(.023)	61.20(.019)	58.20(.025)
SKCM	23.86(.008)	38.53(.006)	52.01(.049)	60.39(.008)	30.02(.048)
COAD	12.67(.001)	19.53(.002)	33.34(.013)	48.29(.039)	47.27(.016)
READ	8.41(.027)	10.83(.027)	19.84(.050)	24.95(.040)	30.02(.048)
CCRCC	1.63(.001)	1.80(.001)	4.22(.002)	5.36(.004)	5.60(.004)
HCC	9.22(.002)	8.13(.006)	14.95(.001)	24.98(.007)	23.91(.005)
LUNG	23.99(.013)	42.72(.030)	63.50(.022)	76.71(.034)	75.80(.036)
LYMPH_IDC	3.56(.001)	4.79(.002)	10.17(.002)	15.27(.005)	16.60(.007)
Average	11.197	17.622	28.444	**36.442**	33.304

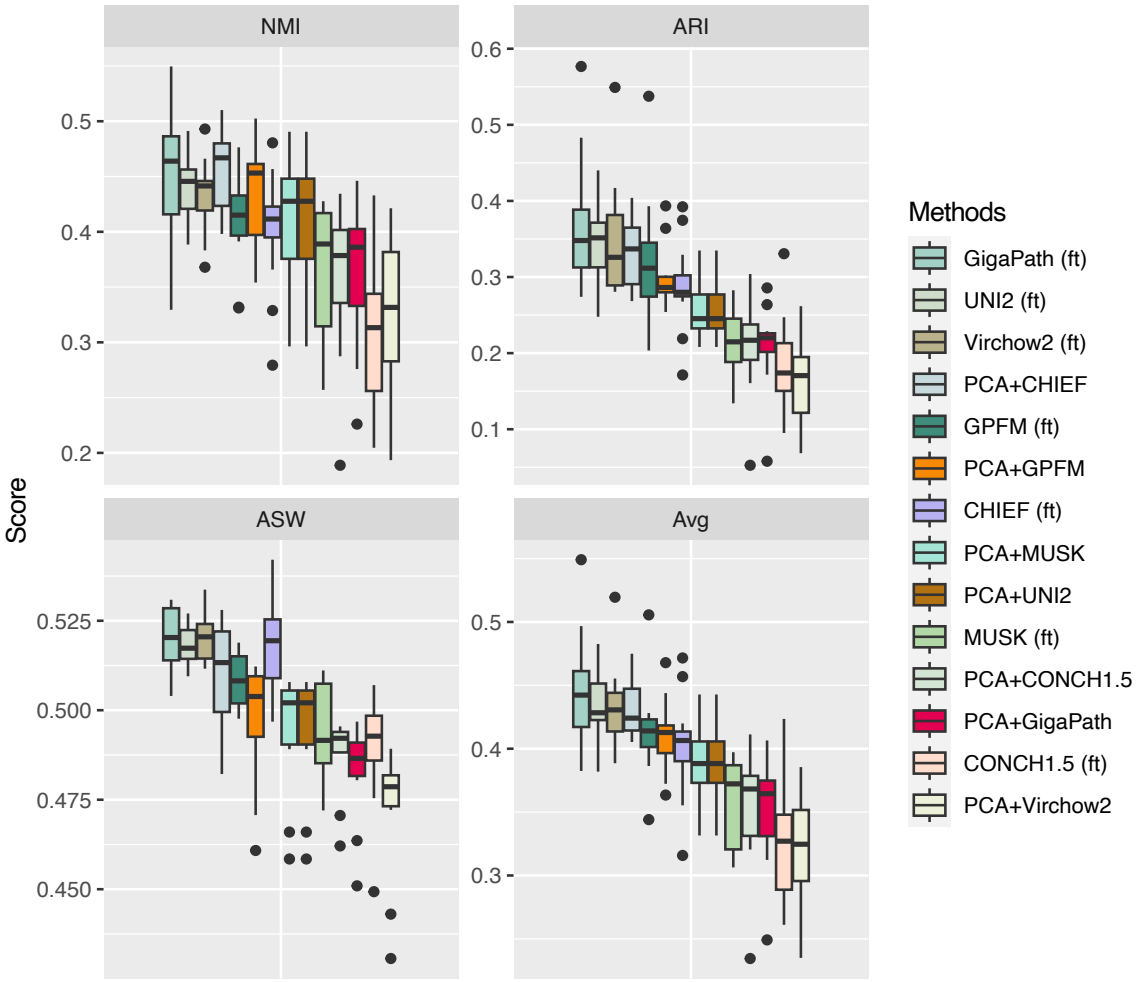
Table 2.1: Evaluations of different PFMs for the multi-modal alignment task. Here in the bracket, we report the averaged metric scores and their standard deviation for each dataset, as well as a final average score computed based on all the datasets. The direction of the arrow indicates the relationship between higher scores and better (or lower) performance. ↓ means that a higher score corresponds to worse model performance, and ↑ means a higher score corresponds to better model performance.



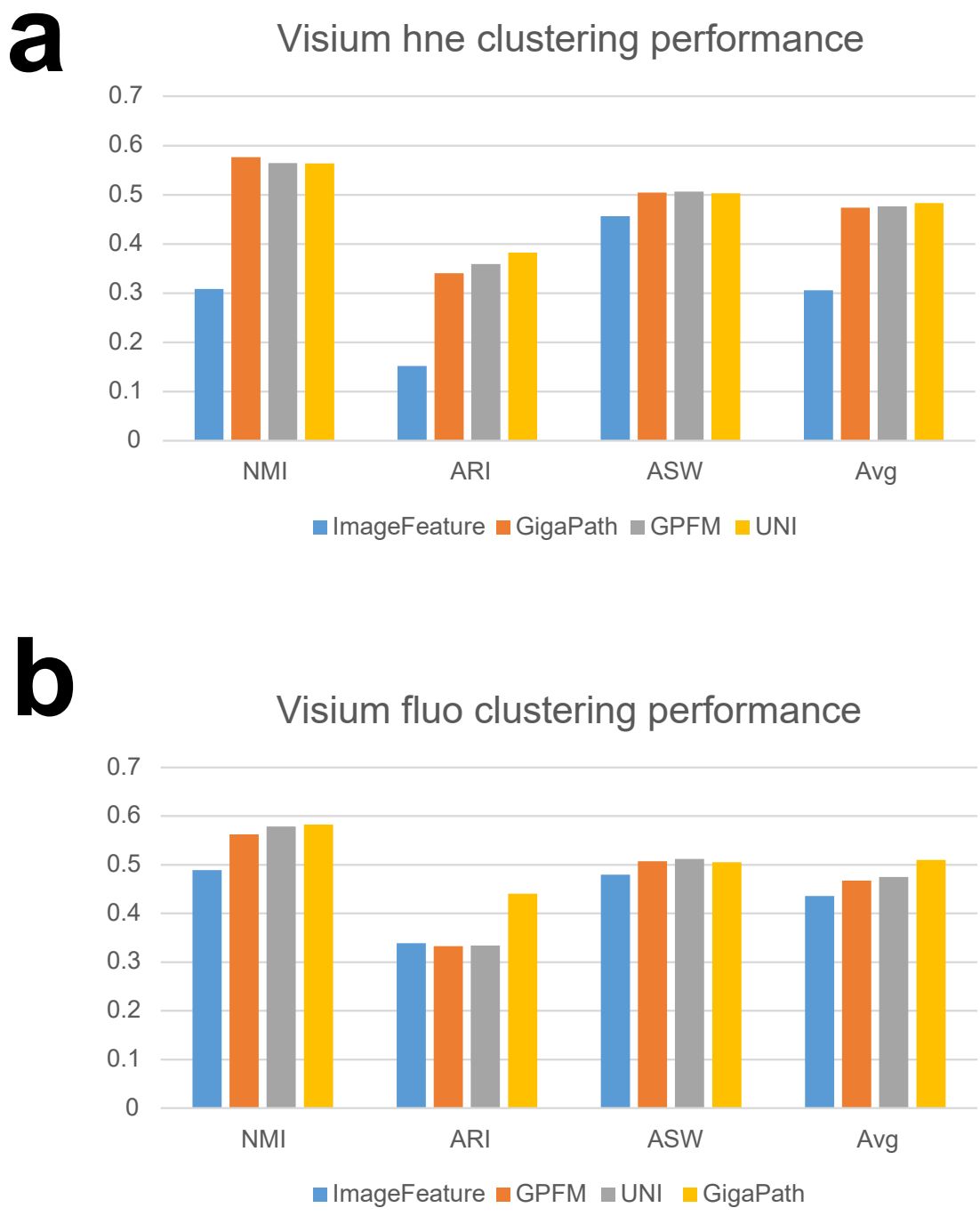
Supplementary Fig. 2: Testing the contributions of image embeddings from different PFMs with SpaGCN for spatial domain identification (n=12 per group; triangle, mean; center line, median; box limits, upper and lower quartiles; whiskers, up to 1.5×interquartile range; points, outliers). Our metrics are NMI, ARI, ASW and their average value (Avg).



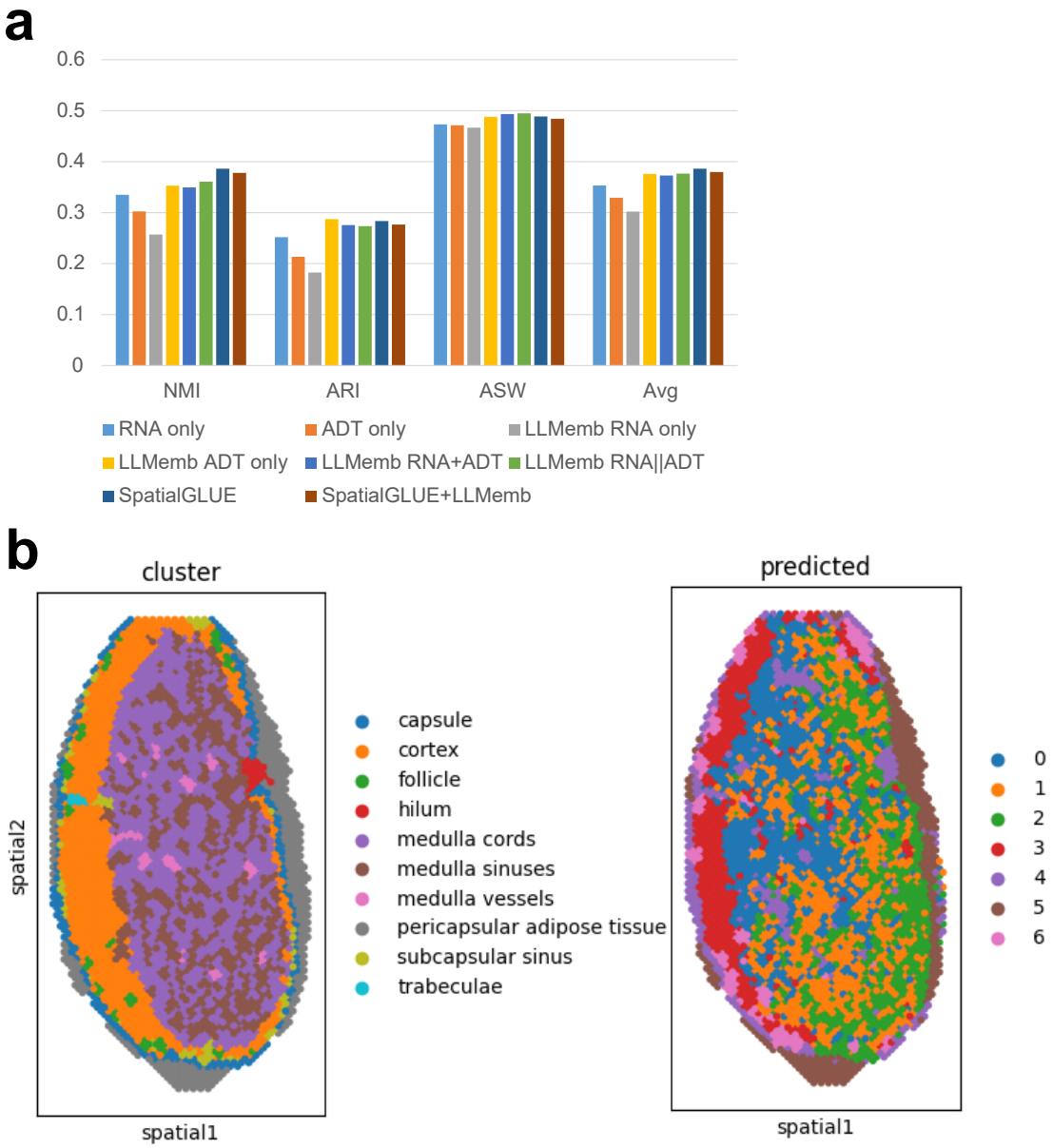
Supplementary Fig. 3: Testing the contributions of the patch size of image used for generating embeddings. Here the selected PFM is GPFM (n=12 per group; triangle, mean; center line, median; box limits, upper and lower quartiles; whiskers, up to 1.5×interquartile range; points, outliers). Our metrics are NMI, ARI, ASW and their average value (Avg).



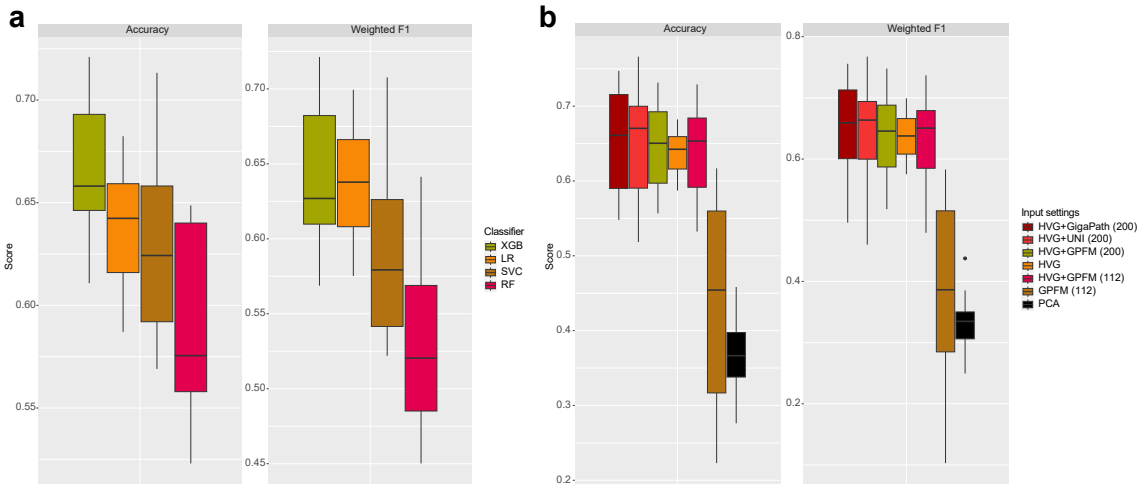
Supplementary Fig. 4: Testing the contributions of fine-tuning or zero-shot inference based on different PFMs for spatial domain identification (n=12 per group; triangle, mean; center line, median; box limits, upper and lower quartiles; whiskers, up to 1.5×interquartile range; points, outliers). Our metrics are NMI, ARI, ASW and their average value (Avg).



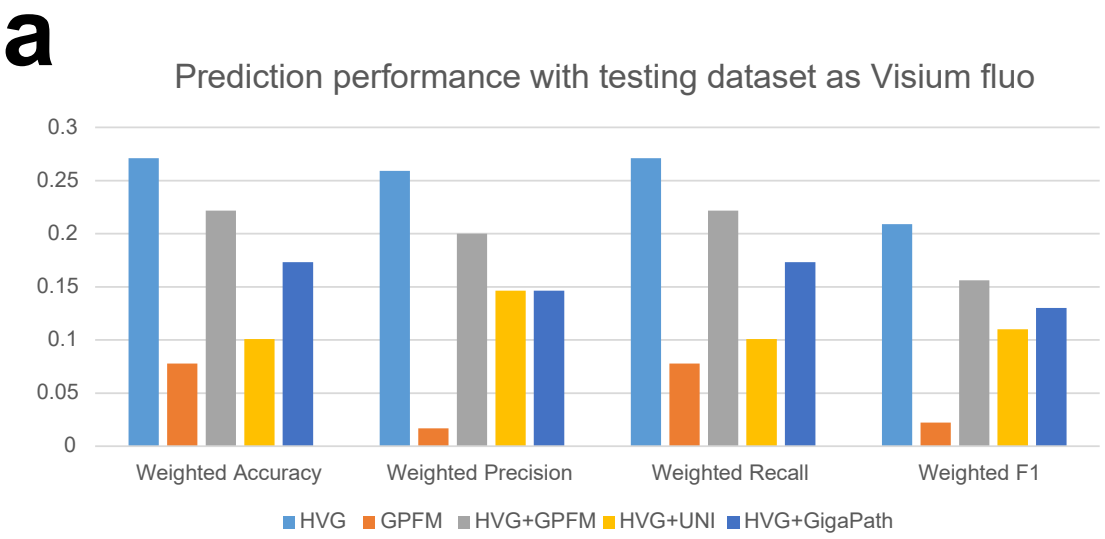
Supplementary Fig. 5: Spatial domain identification performances of the two Visium datasets from Squidpy. (a) Benchmarking results based on the Visium H&E dataset across different image features. (b) Benchmarking results based on the Visium fluo dataset across different image features.

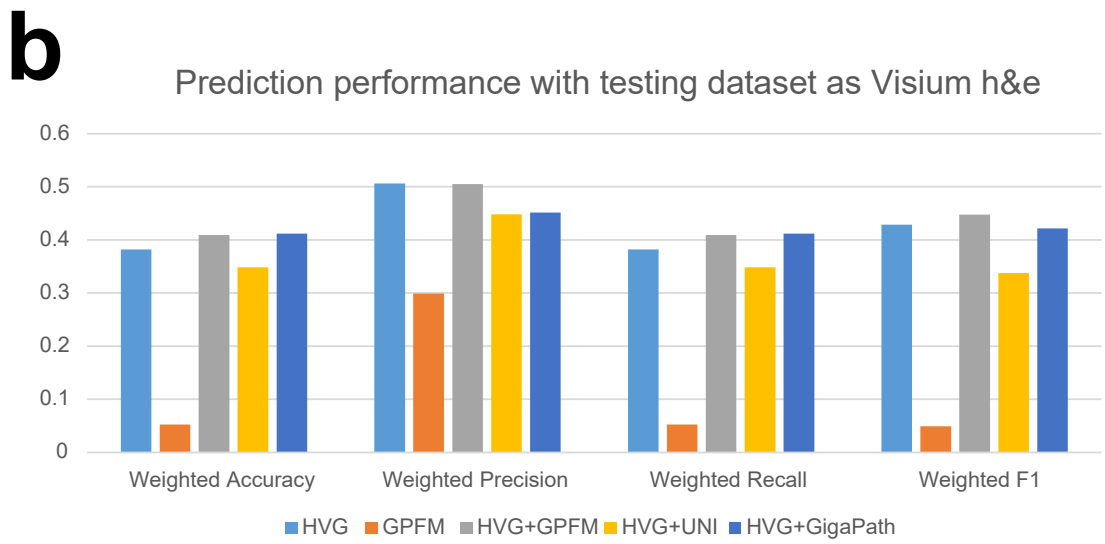


Supplementary Fig. 6: Results of spatial domain identification for spatial multi-omic data. (a) The evaluation results of different methods for domain identification of spatial multi-omic datasets. (b) The visualization of clustering performance based on the multi-omic slide. The left panel is colored by expert annotation, and the right panel is colored by clustering labels.

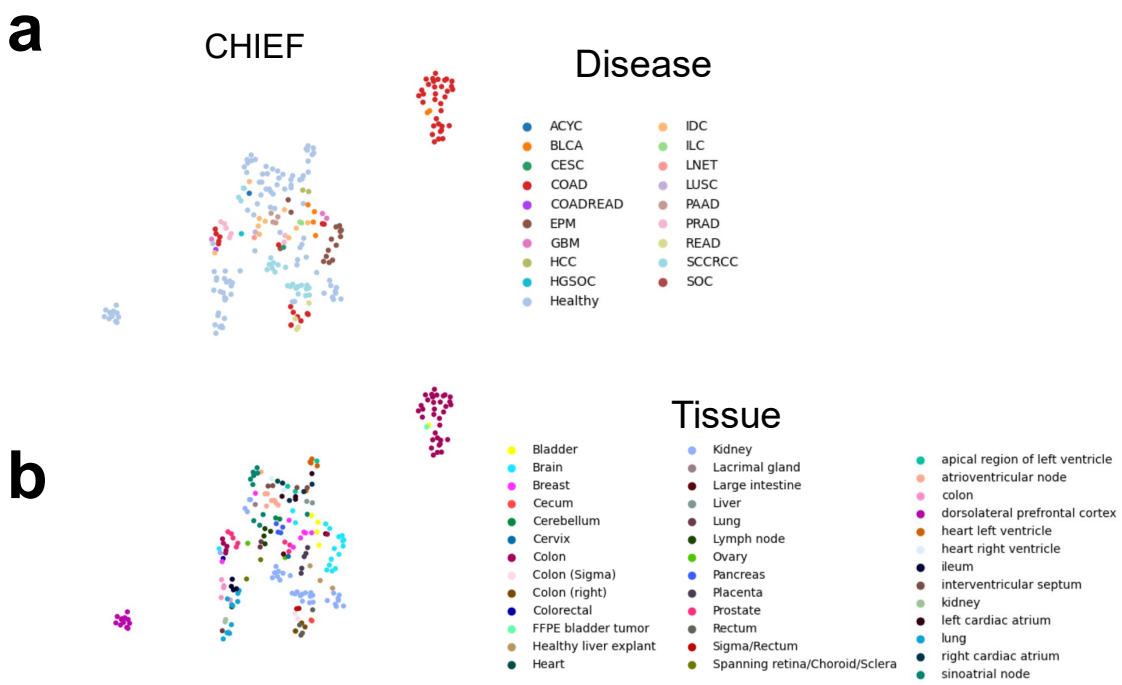


Supplementary Fig. 7: Comparisons of classifiers and settings for spot-type annotation (n=12 per group; triangle, mean; center line, median; box limits, upper and lower quartiles; whiskers, up to 1.5×interquartile range; points, outliers). (a) The evaluation of the performances of different classifiers. (b) The evaluation of the performances of different input settings based on LR.

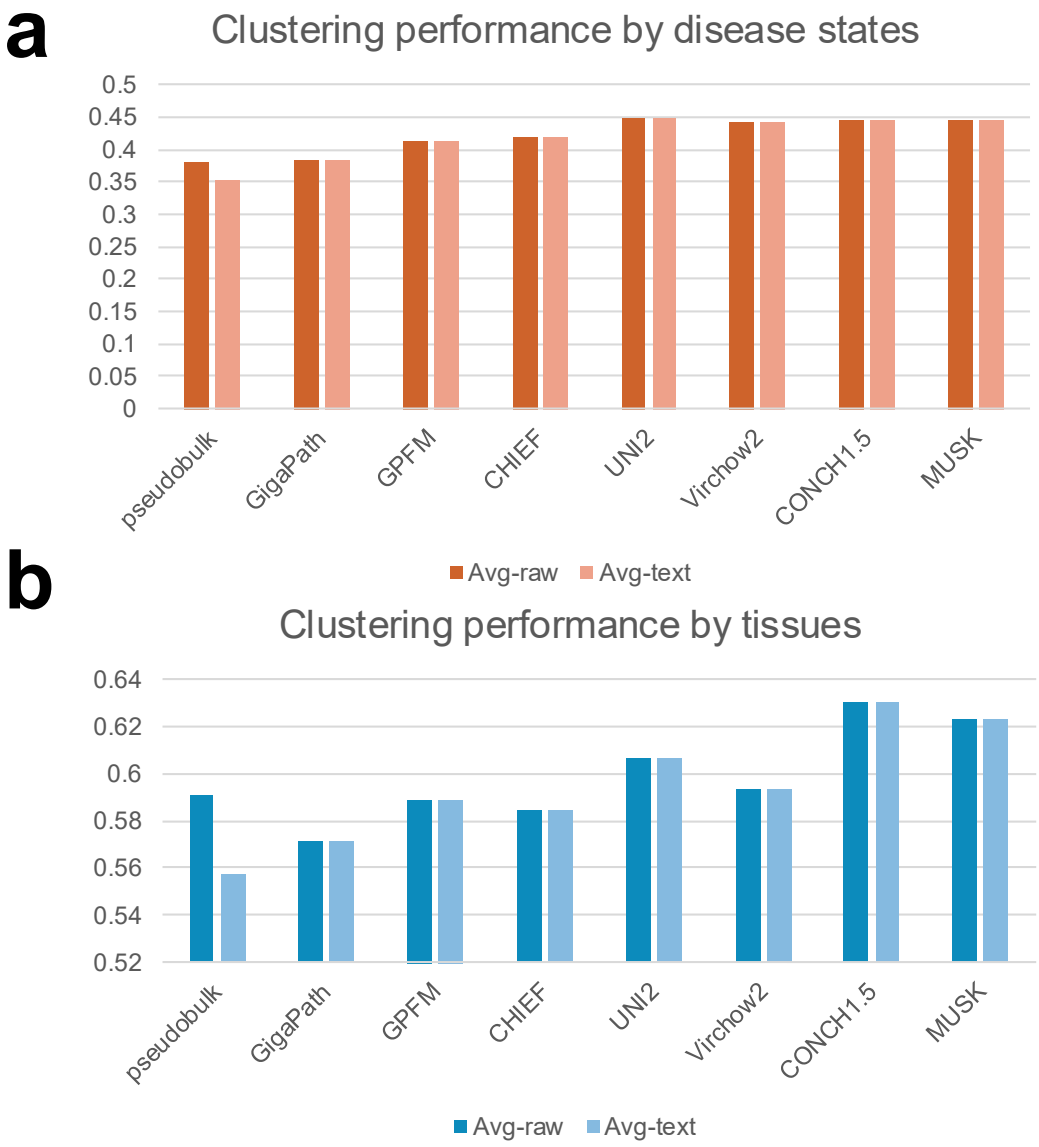




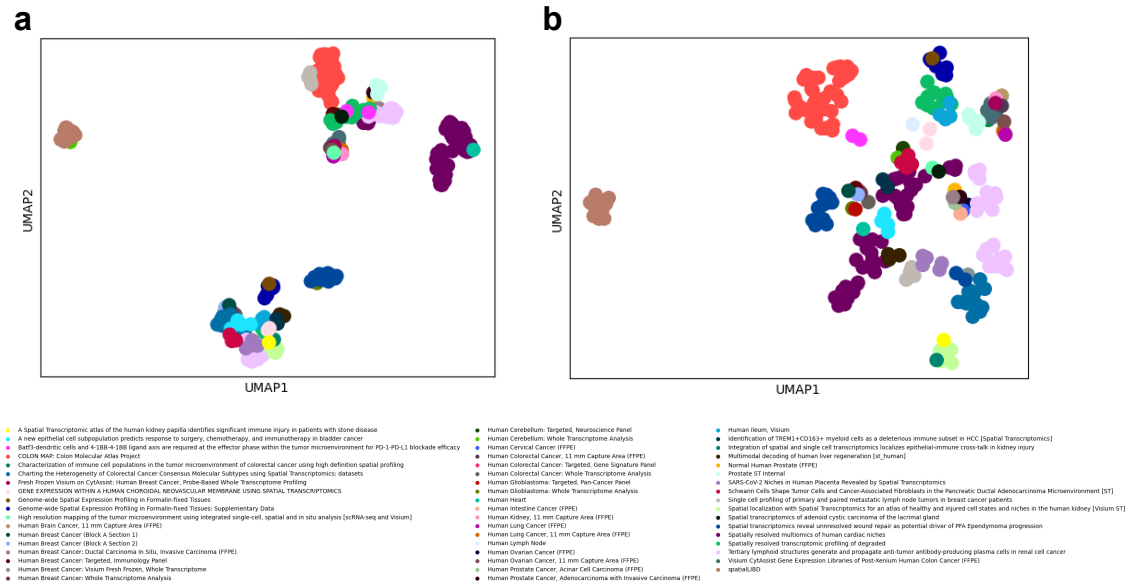
Supplementary Fig. 8: Spot-type prediction performances of the two Visium datasets from Squidpy. (a) Benchmarking results by using Visium fluo as testing dataset across different input settings. (b) Benchmarking results by using Visium H&E as testing dataset across different input settings.



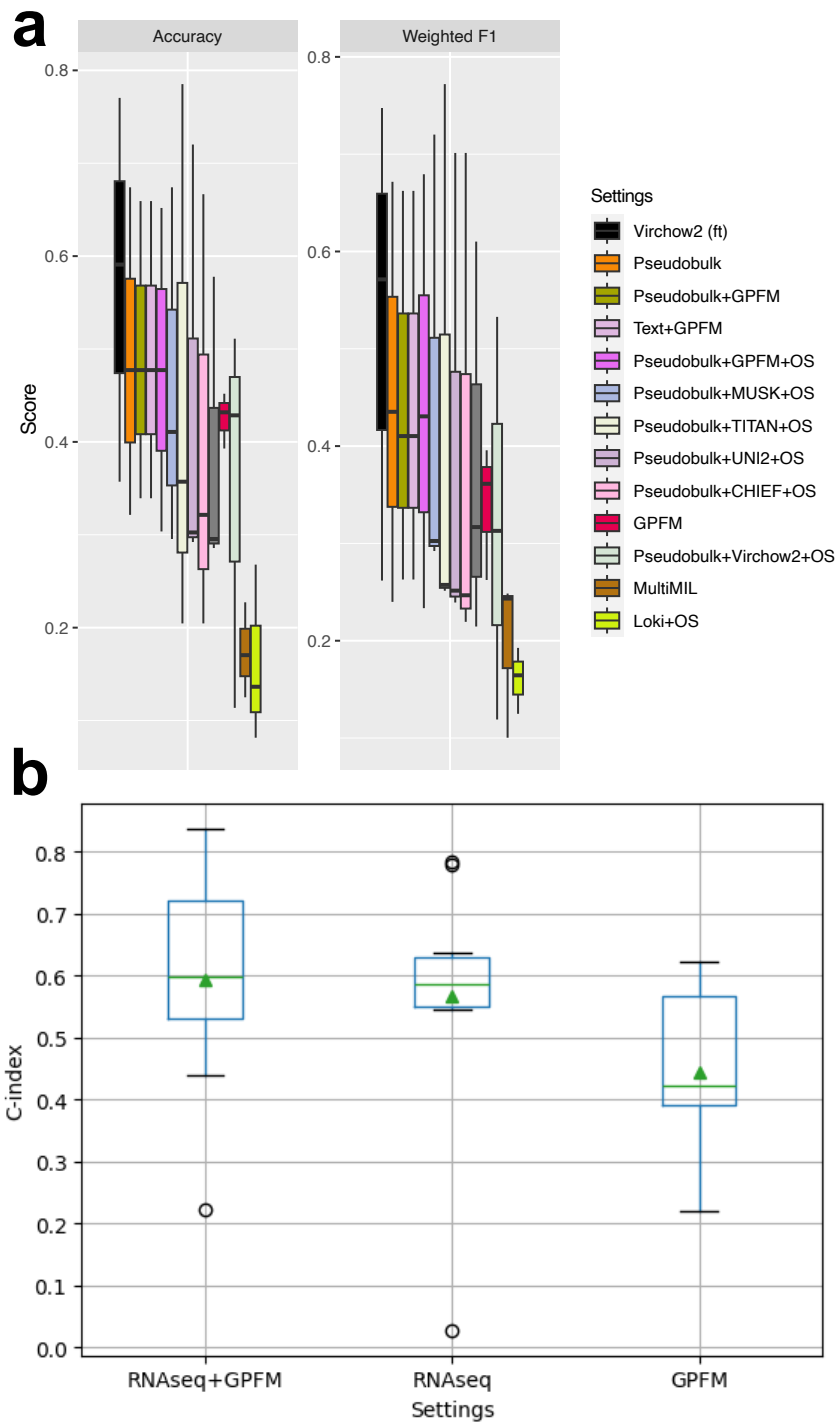
Supplementary Fig. 9: UMAP visualization of sample embeddings generated by CHIEF. (a) Embeddings colored by disease states. (b) Embeddings colored by tissues.



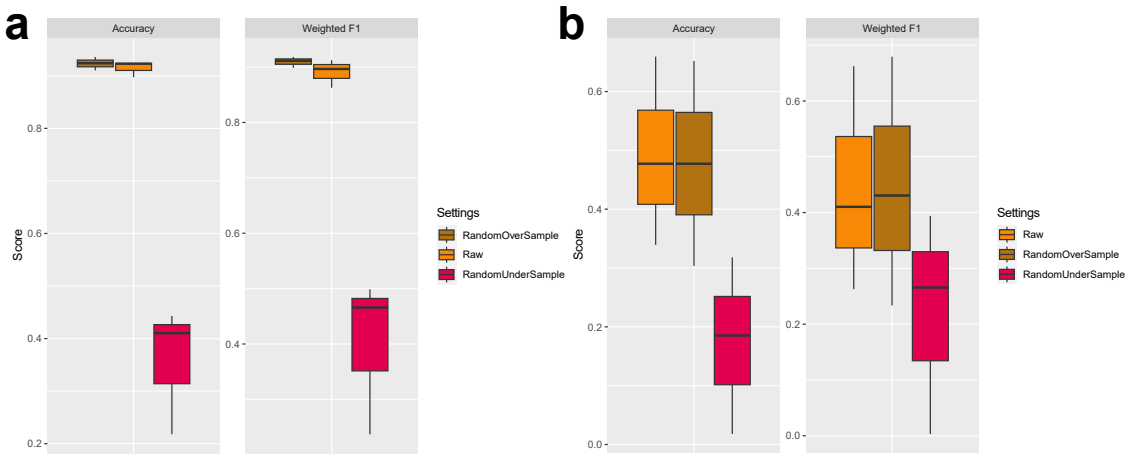
Supplementary Fig. 10: Clustering performances across different labels and combinations. (a) Clustering performances of different PFMs and text embeddings based on disease labels. (b) Clustering performances of different PFMs and text embeddings based on tissue labels.



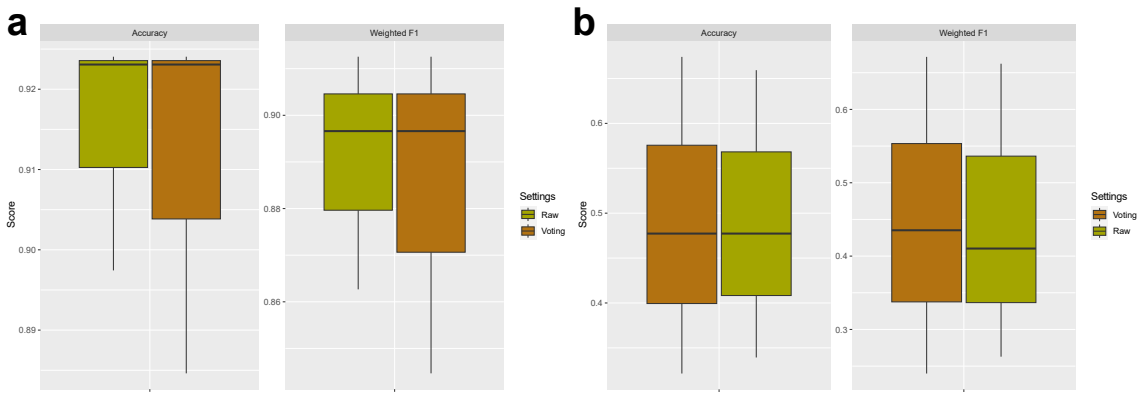
Supplementary Fig. 11: Embeddings from different resources colored by studies. (a) The UMAP based on embeddings from pseudo-bulk gene expression profiles. (b) The UMAP based on embeddings from PFMs. The study labels are shown below.



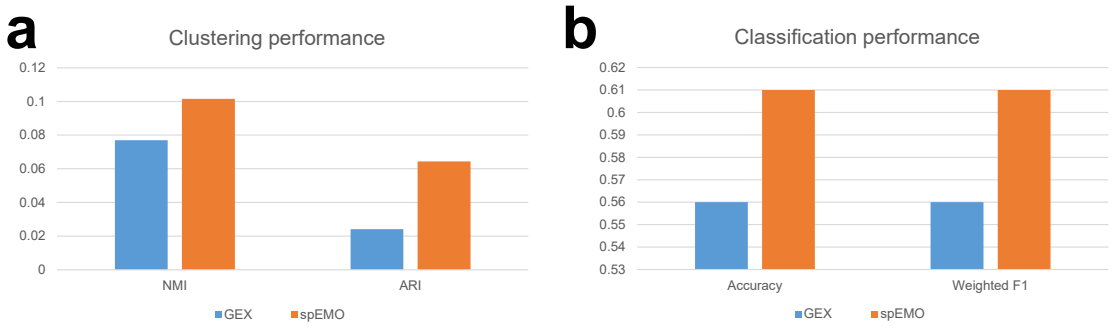
Supplementary Fig. 12: Boxplots with study-level cross-validation results for disease-state prediction and ablation studies for survival prediction. (a) Disease-state prediction performances by setting the cross-validation labels as studies (n=3 per group; triangle, mean; center line, median; box limits, upper and lower quartiles; whiskers, up to 1.5×interquartile range; points, outliers). (b) C-index for survival prediction based on different combinations (n=5 per group; triangle, mean; center line, median; box limits, upper and lower quartiles; whiskers, up to 1.5×interquartile range; points, outliers).



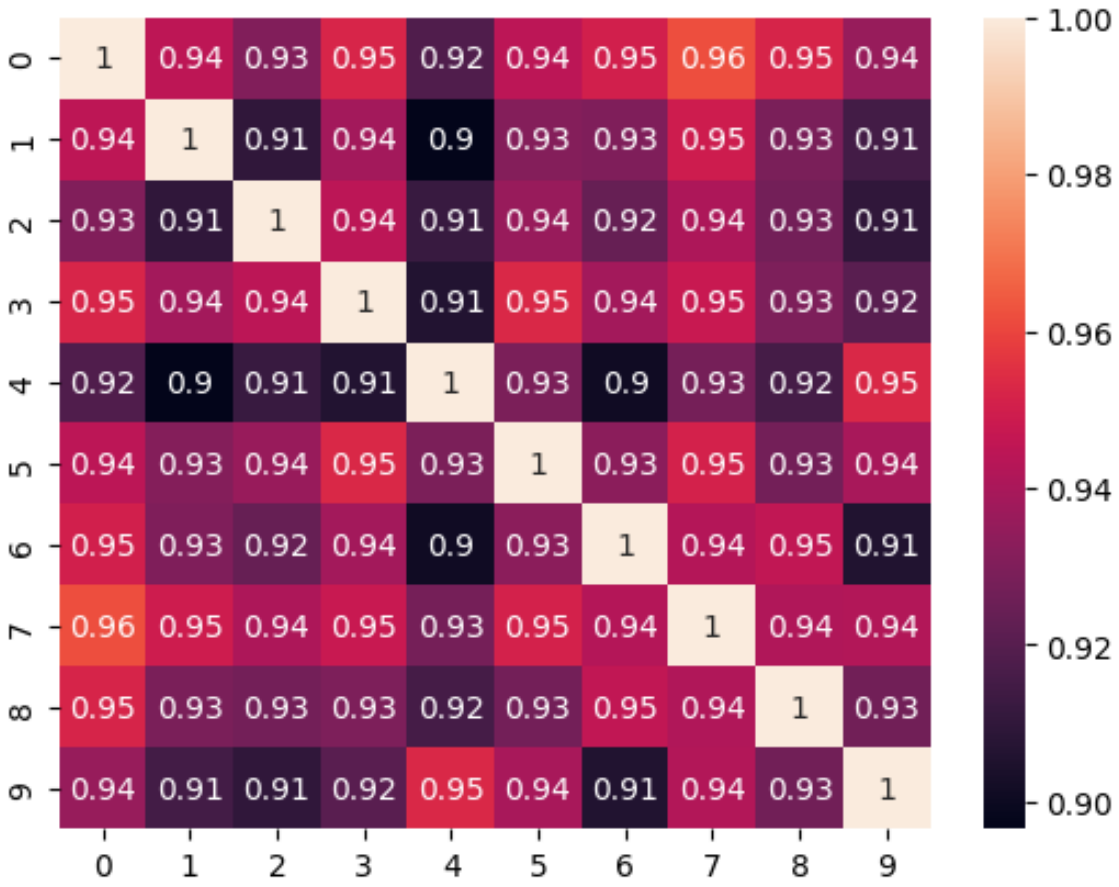
Supplementary Fig. 13: Comparisons between imbalance learning methods and raw methods. Here **raw** represents the default setting. We also tested clustering-based oversampling approach but met running errors due to small neighbor size (n=12 per group; triangle, mean; center line, median; box limits, upper and lower quartiles; whiskers, up to 1.5×interquartile range; points, outliers). (a) Classification results based on the SampleCV setting across different sampling methods. (b) Classification results based on the StudyCV setting across different sampling methods.



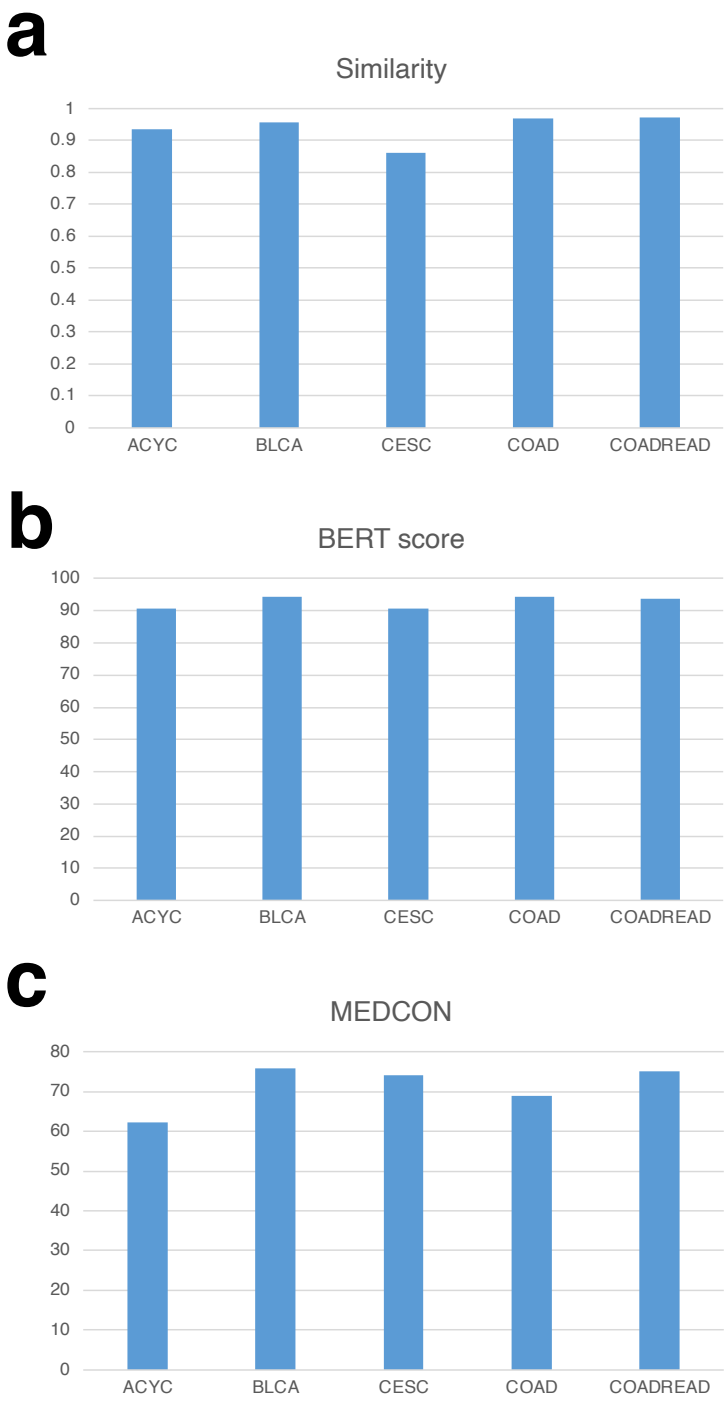
Supplementary Fig. 14: Comparisons between major voting methods and raw methods. Here **raw** represents the default setting (n=3 per group; triangle, mean; center line, median; box limits, upper and lower quartiles; whiskers, up to 1.5×interquartile range; points, outliers). (a) Classification results based on the SampleCV setting across different sampling methods. (b) Classification results based on the StudyCV setting across different sampling methods.



Supplementary Fig. 15: Comparisons between gene expression profiles and spEMO for understanding drug response for individuals. (a) Clustering results based on different input settings. (b) Prediction results based on different input settings.



Supplementary Fig. 16: Correlations of text embeddings based on the descriptions generated with different random seeds under the gene+image mode. We consider 10 different seeds for COAD report generation.



Supplementary Fig. 17: Comparison between expert-generated reports and spEMO-generated reports across five samples from different diseases based on (a) Cosine similarity in the embedding space, (b) BERT score, and (c) MEDCON score.

Chapter 3

Accelerating Scientific Discovery with Autonomous Goal-evolving Agents

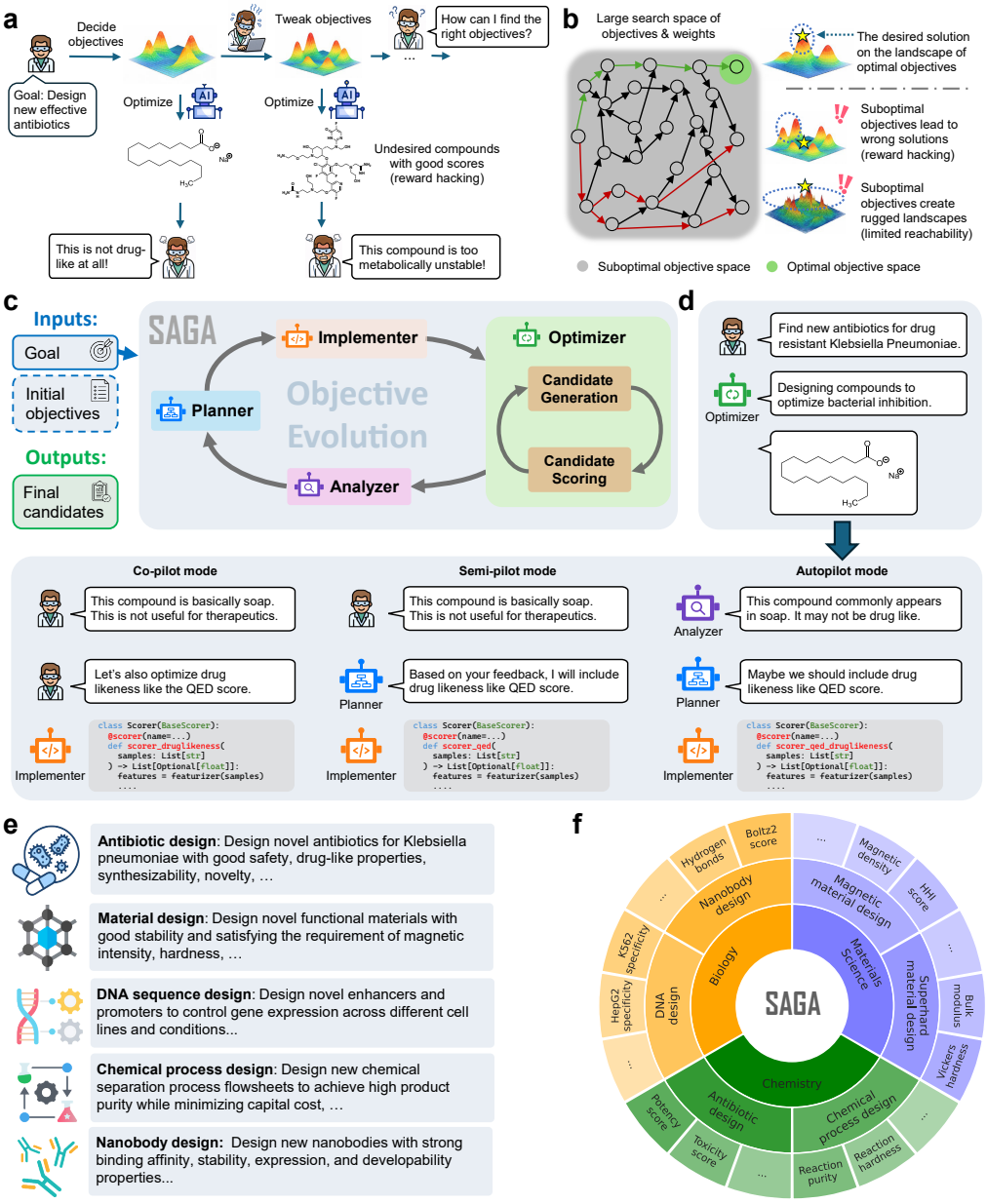
This paper is in review at Nature

Du, Yuanqi, Yu, Botao, Liu, Tianyu et al. "Accelerating Scientific Discovery with Autonomous Goal-evolving Agents." arXiv preprint arXiv:2512.21782 (2025).[51]

Abstract

There has been unprecedented interest in developing agents that expand the boundary of scientific discovery, primarily by optimizing quantitative objective functions specified by scientists. However, for grand challenges in science, these objectives may only be imperfect proxies. We argue that automating objective function design is a central, yet unmet need for scientific discovery agents. In this work, we introduce the Scientific Autonomous Goal-evolving Agent (SAGA) to address this challenge.

SAGA employs a bi-level architecture in which an outer loop of LLM agents analyzes optimization outcomes, proposes new objectives, and converts them into computable scoring functions, while an inner loop performs solution optimization under the current objectives. This bi-level design enables systematic exploration of the space of objectives and their trade-offs, rather than treating them as fixed inputs. We demonstrate the framework through a broad spectrum of design applications, including antibiotic drug discovery, novel inorganic materials, functional DNA sequences, and chemical separation processes, showing that automating objective formulation can substantially improve the effectiveness of scientific discovery agents.



Supplementary Fig. 1: *The SAGA framework and the examples of scientific applications.* (a) Scientists constantly suffer from reward hacking issues, where optimization agents exploit the approximation error of objective functions and propose undesirable solutions with good scores. (b) Finding optimal objectives that bypass reward hacking issues is difficult due to the large search space of objectives and their relative weights. (c) We propose the SAGA framework to automatically discover optimal objectives and candidate solutions through a bi-level procedure. (d) SAGA operates at three different levels of automation, allowing scientists to steer the objective discovery process in various ways. (e) We apply SAGA to scientific design tasks related to chemistry, biology, and materials. (f) The SAGA framework is capable of implementing different objective functions across disciplines.

3.1 Introduction

Scientific discovery has been driven by human ingenuity through iterations of hypothesis, experimentation, and observation, but is increasingly bottlenecked by the vast space of potential solutions to explore and the high cost of experimental validation. Recent advances in artificial intelligence (AI) agents based on large language models (LLMs) offer promising approaches to address these bottlenecks and accelerate scientific discovery. Leveraging massive pretrained knowledge and general capabilities for information collection and reasoning, these AI agents can efficiently navigate large solution spaces and reduce experimental costs by automating key aspects of the research process. For example, pipeline automation agents [81, 136] streamline specialized data analysis workflows, reducing the manual effort required for routine experimental processes. AI Scientist agents [67, 130, 148, 203, 239, 261] tackle the exploration challenge by autonomously generating and evaluating novel hypotheses (e.g., the relationship between a certain mutation and a certain disease) through integrated literature search, data analysis, and academic writing capabilities.

Our work embarks on a different and more ambitious goal in scientific discovery: building agents to discover new solutions to complex scientific challenges, such as proofs for conjectures, faster algorithms, better therapeutic molecules, and new functional materials. This problem is uniquely challenging due to the "creativity" and "novelty" required and the infinite combinatorial search space for potential solutions. Previous work has sought to address these challenges by developing optimization models that automatically find solutions maximizing a manually defined set of quantitative objectives, such as drug efficacy, protein expression, and material stability. These approaches, ranging from traditional generative models to more recent LLM-based methods, have demonstrated the ability to efficiently optimize against fixed objectives in domains including drug design and synthesis [25, 126, 200], algorithm discovery [154], and materials design [125, 251].

However, these optimization models operate under a critical assumption: that the right set of objective functions is known upfront. In practice, this assumption is seldom known completely *a priori*. Just as scientific discovery requires iterations of hypothesis, experimentation, and observation, determining the appropriate objectives for a discovery task is itself an iterative search process. Scientists must constantly tweak objectives based on intermediate results, domain knowledge, and practical constraints that emerge during exploration (Figure 1(a)). This iterative refinement is particularly crucial in experimental disciplines such as drug discovery, materials design, and protein engineering, where many critical properties can only be approximated through predictive models. Without this evolving process, the discovery suffers from *reward hacking issues* [214]: they exploit gaps between models and reality, producing solutions that maximize predicted scores while missing important

practical considerations that experts would recognize. The search space for objectives and their relative weights is itself combinatorially large (Figure 1(b)), making it extremely difficult to specify the right objectives from the outset. As a result, while existing optimization models can solve the low-level optimization problem efficiently, scientific discovery remains bottlenecked by the high-level objective search process that relies on manual trial-and-error.

In this work, we introduce SAGA as our first concrete step toward automating this iterative objective evolving process. SAGA is designed to navigate the combinatorial search space of objectives by integrating high-level objective planning in the outer loop with low-level optimization in the inner loop (Figure 1(c)). The outer loop comprises four agentic modules: a planner that proposes new objectives based on the task goal and current progress, an implementer that converts proposed objectives into executable scoring functions, an optimizer that searches for candidate solutions maximizing the specified objectives, and an analyzer that examines the optimization results and identifies areas for improvement. Within the optimizer module, an inner loop employs any optimization methods (e.g., genetic algorithms or reinforcement learning) to iteratively evolve candidate solutions toward the current objectives. Importantly, SAGA is a flexible framework supporting different levels of human involvement. It offers three modes (Figure 1(d)): (1) co-pilot mode, where scientists collaborate with both the planner and analyzer to reflect on results and determine new objectives; (2) semi-pilot mode, where scientists provide feedback only to the analyzer; and (3) autopilot mode, where both analysis and planning are fully automated. This design allows scientists to interact with SAGA in ways that best suit their expertise and preferences.

SAGA is a generalist scientific discovery agentic framework with demonstrated success across multiple scientific domains, from chemistry and biology to materials science (Figure 1(e)-(f)). In antibiotic design, SAGA successfully discovered new antibiotics with high predicted potency against Klebsiella pneumoniae while satisfying complex physicochemical constraints. In inorganic materials design, SAGA designed permanent magnets with low supply chain risk and superhard materials for precision cutting, with properties validated by Density Functional Theory (DFT) calculations. In functional DNA sequence design, SAGA proposed high-quality cell-type-specific enhancers for the HepG2 cell line, with nearly 50% improvement over the best baseline. Lastly, SAGA demonstrated success in automating the design of chemical process flowsheets from scratch. In summary, these results highlight the broad applicability of SAGA in many disciplines and the value of adaptive objective function design in scientific discovery agents.

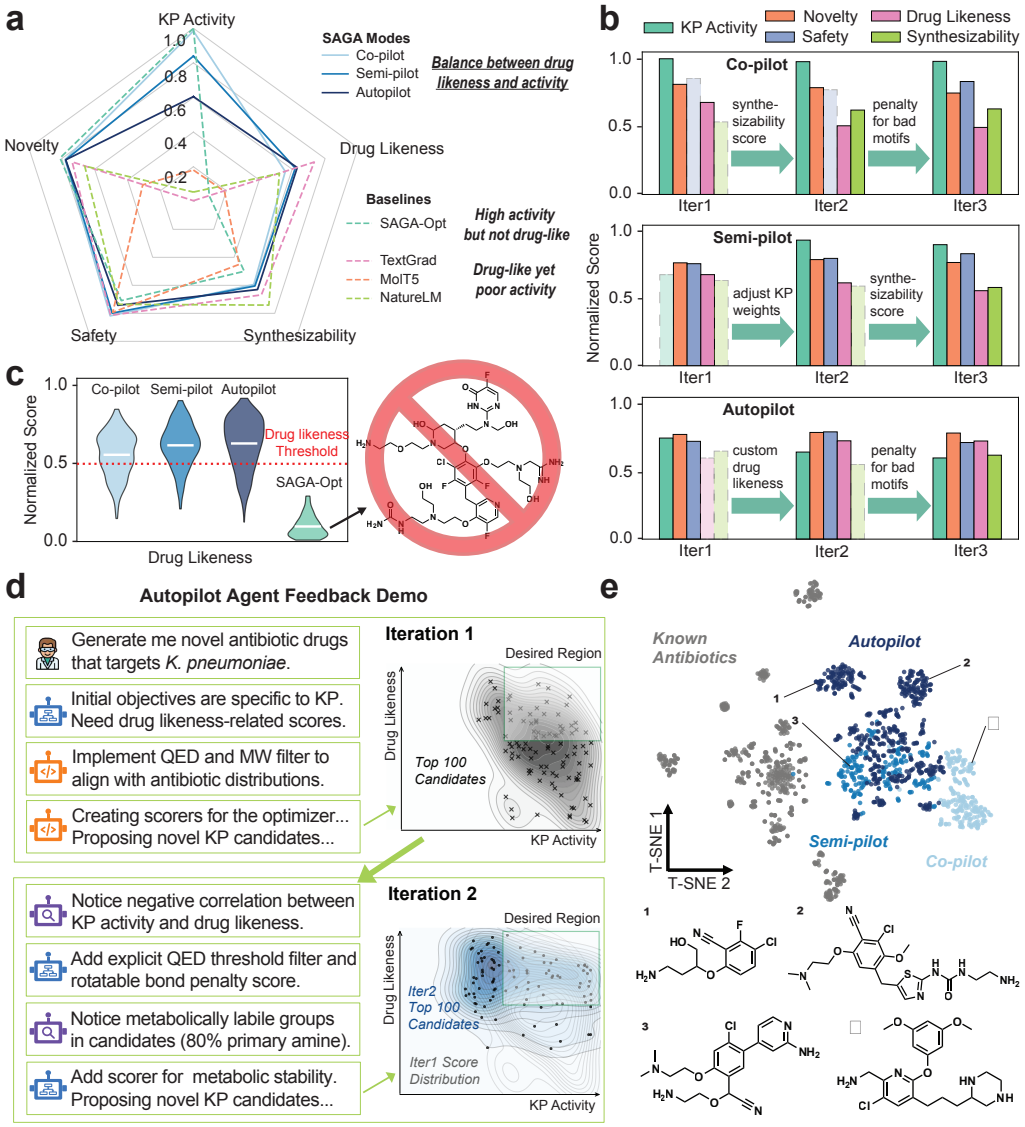
3.2 Results

3.2.1 SAGA for Antibiotic Design

Antimicrobial resistance (AMR) is rapidly eroding our ability to treat common Gram-negative infections, one of which is Klebsiella pneumoniae (*K. pneumoniae*), ranked as a critical priority pathogen by the World Health Organization (WHO) [18, 179]. However, designing novel inhibitors for Gram-negative bacteria is notoriously difficult, as optimization agents suffer from generating chemically unreasonable compounds that lack the necessary given objectives [214]. To address this challenge, we use SAGA to design novel *K. pneumoniae* inhibitors. Rather than relying on a static scoring function that attempts to encode every rule upfront, SAGA begins with only primary biological objectives for maximizing potency and minimizing toxicity, along with a constraint to avoid existing scaffolds. From this foundation, SAGA dynamically constructs a suite of auxiliary objectives that steer the generative process toward realistic chemical space at all three levels of automation. This strategy allows SAGA to learn the specific constraints of the Gram-negative landscape in an interpretable, iterative manner. Ultimately, SAGA produces more valid candidates that satisfy rigorous external evaluations and align with scientists' intuition.

SAGA discovers computationally selective and chemically reasonable candidates. We run SAGA at all three levels of automation with the same prompt and primary biological objectives. SAGA then iterates at different levels of automation until optimization is complete. To evaluate the quality of proposed candidates, we selected three biological evaluations: *K. pneumoniae* (KP) Activity defined by the Antibiotic activity score, a Novelty score, and Safety defined by (1 - the Toxicity score), as well as two chemical evaluations: Drug likeness defined by the Quantitative estimate of drug likeness (QED) score, and Synthesizability defined by the Synthetical Accessibility (SA) score. These scores are further elaborated in Figure S6.2.

As illustrated in Figure 1(a), SAGA achieves a significantly higher percentage of candidates passing all evaluations (detailed in Figure S6.2) compared to state-of-the-art language model baselines. In contrast to SAGA , these baselines exhibit distinct failure modes. Several language models struggle to overcome the optimization difficulty of the KP activity score alone, resulting in chemically valid but inactive molecules. Conversely, the Optimizer agent (SAGA -Opt), which does not have the capacity to dynamically evolve objectives, achieves high KP activity but suffers a catastrophic drop in medicinal chemistry quality. Furthermore, SAGA successfully balances the scores of both biological objectives and standard medicinal chemistry filters, discovering drug-like molecules with high predicted activity across all 3 modes of operation as shown in Figure S6.1(b). As seen in Figure 1(c), while SAGA candi-



Supplementary Fig. 1: *Results for antibiotic design.* (a) Comparisons between SAGA and language model baselines. Candidates from all SAGA modes achieve the "drug likeness and activity sweet spot", whereas baselines struggle to balance biological objectives, especially KP activity, with chemical assessments like Drug likeness and Synthesizability. (b) Comparisons across SAGA iterations. Text annotations highlight specific agent feedback on objective evolution that drives the improvement in metric scores across iterations. The solid line means objectives address the evaluation metrics, and the dash line means the metric has not been addressed. (c) Distribution of drug likeness score. Most molecules from SAGA surpass the drug likeness threshold (red dashed line), while AlphaEvolve falls below it, demonstrating its critical misalignment with final objectives. (d) An example of the autopilot feedback loop. The analyzer identifies issues and the planner dynamically evolves objectives, shifting the distribution of the Top 100 Candidate more to the Desired Region of high activity and drug likeness. (e) T-SNE plot of SAGA molecules against known antibiotics. Numbered structures (1-4) are examples of molecules passing all evaluation metrics (Figure S6.2) that are likely function well in experiments. They contain novel scaffolds distinct from known antibiotic clusters.

dates consistently score above the Drug likeness Threshold, SAGA -Opt's population distribution falls almost entirely below it, designing unrealistically large and undrug-like molecules. This observation confirms that, without dynamic auxiliary objectives, language models either fail to optimize the primary biological goal or, like SAGA -Opt, exploit the scoring function to propose active but chemically invalid structures. SAGA , on the other hand, successfully aligns with the desired distribution of realistic drug candidates, producing the most promising candidates.

SAGA can effectively analyze, propose, and implement informative and necessary objectives across different levels of automation. In addition to raw performance, SAGA enables explainable and robust optimization by dynamically evolving its scoring functions, reorienting the trajectory to align with both the global *K. pneumoniae* optimization goals and chemical intuition. As illustrated in Figure 1(b) and communication history, the components from SAGA demonstrate context-awareness regarding the optimization landscape. While the co-pilot mode incorporates nuanced human feedback to address low synthesizability, the semi-pilot agent intelligently defers adding strict chemical constraints in early iterations to instead adjust weights and prioritize the optimization of the KP activity objective. In the autopilot mode, the analyzer agent provides chemical insights that anticipate expert concerns. As shown in Figure 1(d) and communication history, SAGA goes beyond individual molecular analysis and identifies population-level trends, such as "negative correlation between KP activity and drug likeness". Furthermore, it performs granular structural analysis to pinpoint specific over-represented metabolically labile groups, insights that typically require systematic review to uncover. In response, SAGA autonomously constructs filters and scorers that steer the generated population to the "Desired Region" of the physicochemical space, leading to a higher passing rate for external chemical motif alerts. Collectively, these examples demonstrate the practical utility of dynamic objective evolution in solving the hard, multi-objective optimization problem, generating final candidates that show more promises.

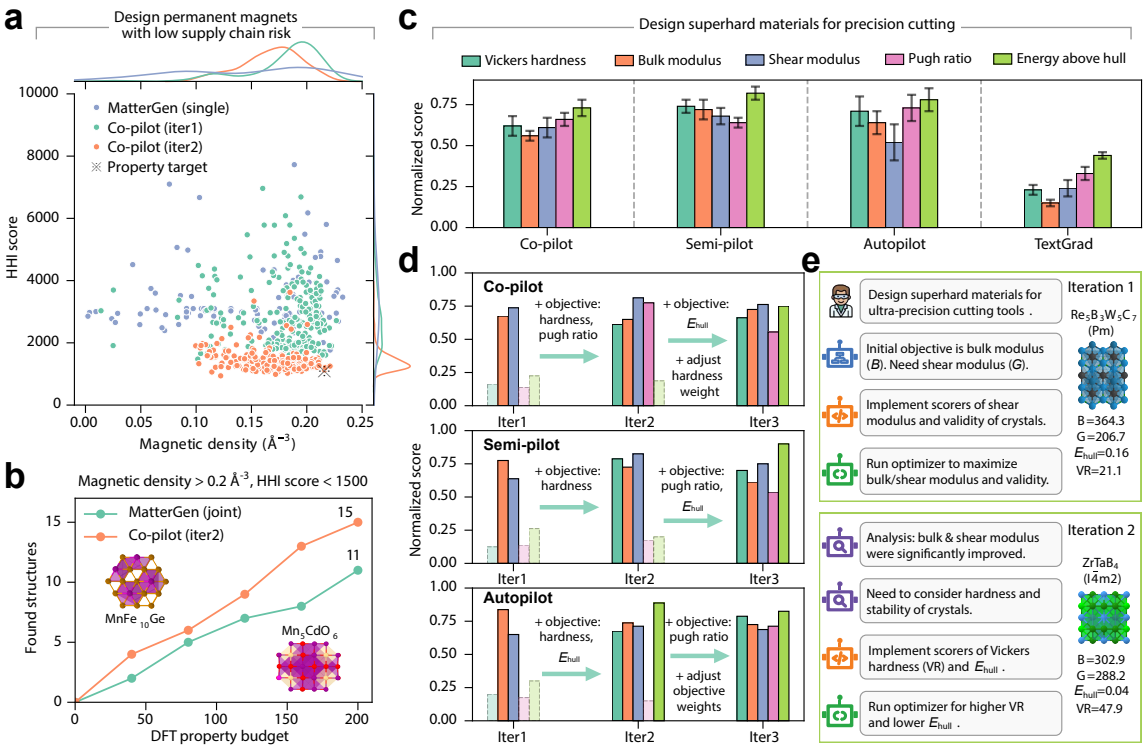
SAGA discovers computationally performant molecules with novel, synthetically accessible scaffolds distinct from existing antibiotics. A primary goal of *de novo* design is to discover potent, drug-like candidates that diverge from the existing antibiotics space. Therefore, after SAGA finishes optimization, we first filter all proposed candidates by applying a set of stringent evaluation cutoffs to select molecules with high probability of experimental success and then assess how similar they are to existing antibiotics. As shown in Figure 1(e), the selected molecules occupy diverse regions distinct from the tight clusters of over 500 known antibiotics. Specific examples (Structures 1–4) further illustrate that, rather than only optimizing around one fixed scaffold, SAGA generalizes the rules of bacterial inhibition to assemble novel backbone architectures and halobenzene cores. Crucially,

despite this high degree of novelty, these antibiotic candidates are either directly purchasable, or have commercial analogs with high similarity, suggesting that they could serve as promising candidates for experimental validation in the next step.

3.2.2 SAGA for Inorganic Materials Design

The discovery of novel materials is critical for driving technological innovation across diverse fields, including catalysis, energy, electronics, and advanced manufacturing [27, 28, 125, 146, 190, 243, 251]. Most material design tasks involve multiple objectives encompassing electronic, mechanical and physicochemical properties, as well as production costs [27, 251]. These design objectives are often intricately interrelated and may exhibit competitive or even conflicting trade-offs [27, 159, 232]. Optimization with fixed objectives may overlook other important material properties or fail to refine optimization objectives based on deficiencies identified in proposed candidates. To address this challenge, we apply SAGA to design the desired novel materials for specific applications through iterative optimization with dynamic objectives. SAGA can guide LLMs to search materials with desired properties, iteratively analyzing and adjusting optimization objectives, while automatically programming scoring functions to evaluate the new objectives and provide feedback. We propose two design tasks to assess the SAGA 's effectiveness.

SAGA enables efficient magnet materials design. First, we evaluate SAGA on the task of designing permanent magnets with low supply chain risk, and compare against MatterGen [251], one of the state-of-the-art generative models for inorganic materials design. In this task, two objectives are specified: magnetic density higher than 0.2 $Å^{-3}$ and Herfindahl–Hirschman index (HHI) score less than 1500, where a lower HHI score indicates lower supply chain risk and the absence of rare earth elements [62, 251]. The SAGA Co-pilot mode is deployed with iteratively refined objectives: maximizing magnetic density in the first iteration, followed by the addition of HHI score minimization in the second. Performance was compared against the MatterGen model that targets only high magnetic density (single) or both properties (joint) [251]. As shown in Figure 2(a), Co-pilot proposes crystal structures with high magnetic density after the first iteration, exhibiting a higher distribution density near the target value of 0.2 $Å^{-3}$ compared to MatterGen (single). However, these structures display a broad range of HHI scores, with over 80 % exceeding 2000. After the second iteration, Co-pilot successfully discovers crystal structures with both high magnetic density and low HHI scores. The majority of proposed structures exhibit magnetic density above 0.15 $Å^{-3}$ and HHI scores below 1500. This demonstrates that SAGA 's Co-pilot mode can continuously and iteratively optimize material properties with human feedback to accomplish multi-objective tasks. Moreover, within a computational budget of 200 DFT property evaluations (Figure 2b), Co-pilot mode



Supplementary Fig. 2: *Results for inorganic materials design.* (a) Property distributions of generated structures from co-pilot across different iterations and from MatterGen (single) targeting only high magnetic density. (b) Number of stable and novel structures satisfying property requirements found by co-pilot and MatterGen (joint) within 200 DFT property calculations, for targets with magnetic density above 0.2 $Å^{-3}$ and HHI score below 1500. It also displays 3D visualizations of two crystal structures proposed by the co-pilot mode that satisfy the design goal. (c) Comparisons between different levels of SAGA and selected baselines on the design task of superhard materials for precision cutting. All evaluation metrics are normalized, with higher scores representing better performance. (d) Comparison of different SAGA modes over three iterations with the same held-out metrics. In each iteration, SAGA analyze the optimized crystal structures, propose new objectives, run property optimization, and select the best candidates across all current iterations. Text annotations highlight specific agent feedback on objective evolution that drives the improvement in metric scores across iterations. The solid line means objectives address the evaluation metrics, and the dash line means the metric has not been addressed. (e) An example of the autopilot feedback loop. SAGA identifies issues and dynamically evolves objectives, successfully proposed novel structures exhibiting high hardness, high elastic modulus, and thermodynamic stability.

identify 15 novel and stable structures satisfying the desired properties, outperforming MatterGen (11 structures). These results demonstrate that SAGA can continuously optimize dynamic objectives, potentially outperforming specialized generative models that are constrained to fixed objectives.

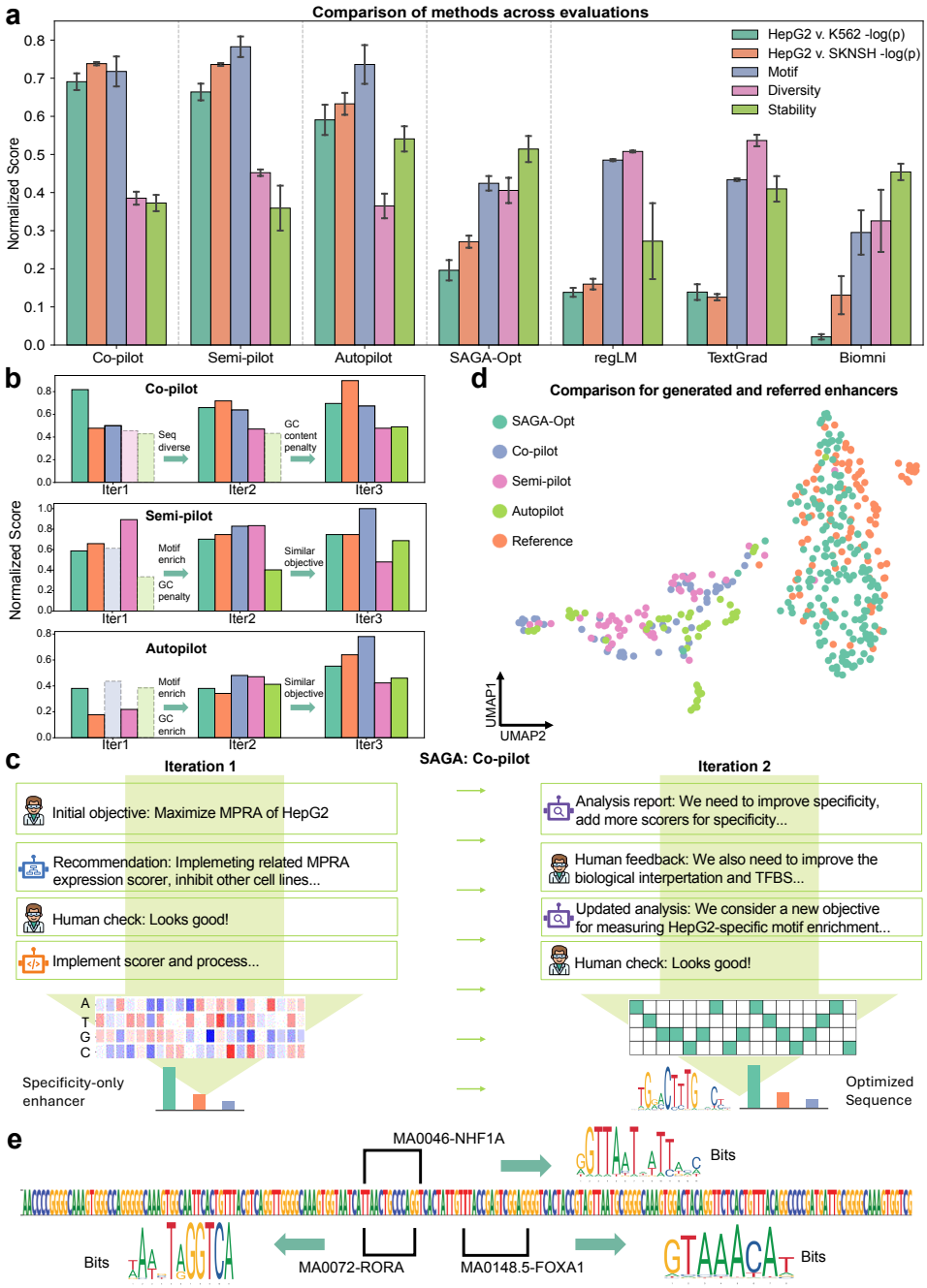
SAGA enables efficient superhard materials design. Subsequently, we evaluate SAGA on the task of designing superhard materials for precision cutting and compare with an LLM-based optimization algorithm, TextGrad [247], as MatterGen [251] requires fine-tuning on large amounts of DFT-labeled data when switching tasks. This task involves more than three target material properties, whereas conventional methods that optimize with fixed targets may only achieve high scores on certain metrics but ignore other important properties of the designed materials. As shown in Figure 2(c), the crystal structures designed by three modes (co-pilot, semi-pilot, autopilot) achieve high scores on all metrics. Benefiting from iterative optimization and dynamic objective refinement, all SAGA modes successfully propose novel structures exhibiting high hardness, high elastic modulus, appropriate brittleness, and thermodynamic stability. In contrast, the TextGrad approach, which employs fixed optimization objectives, demonstrated moderate performance for energy above hull and Pugh ratio but achieved much lower scores for hardness and elastic modulus. These results demonstrate that SAGA 's iterative optimization and dynamic objective strategy are effective for complex multi-objective tasks. Furthermore, we analyze the crystal structures proposed by SAGA in the final iteration and found that the underlying patterns correlate with key factors for superhard material formation reported in experimental studies. More than 90 % of the proposed crystals contain light elements such as boron, carbon, nitrogen, and oxygen, aligning with experimental findings that light elements are essential for superhard materials because their small atomic radii enable short, directional covalent bonds with high electron density [91, 158]. In addition, over 75 % of the proposed crystals are transition metal carbides, nitrides, and borides. Correspondingly, experimental studies have demonstrated that the combination of light elements (boron, carbon, nitrogen) with electron-rich transition metals can form dense covalent networks and enhance material hardness [91, 158].

SAGA proposes reasonable and important objectives aligning with materials scientists. The co-pilot and semi-pilot modes incorporate human input into the agent's decision-making process to review and refine candidate analyses and proposed objectives for subsequent iterations. As shown in Figure 2(d), integration of human feedback enabled SAGA to consider additional relevant objectives across multiple iterations, resulting in comprehensive performance improvements of the designed materials. For instance, explicit prioritization of Vickers hardness, elastic modulus and Pugh ratio [139, 245], guided by expert input, led to substantial enhancement of the mechanical properties in proposed crystalline materials. The

results demonstrate that SAGA can effectively integrate human feedback through adaptive objective formulation, improving the overall performance of proposed materials. Moreover, SAGA 's autopilot mode can analyze results and set objectives autonomously without human intervention. By analyzing materials proposed in the current iteration, autopilot mode can identify their weaknesses and adaptively refines objectives for iterative improvement. As illustrated in Figure 2(d), autopilot mode can propose important optimization objectives for the design goal, similar to expert guidance. Autopilot achieves excellent overall performance comparable to co-pilot and semi-pilot across all five metrics, underscoring its remarkable intelligence and automation capabilities. Figure 2e demonstrates that autopilot can correctly understand the design goal and analyze properties of proposed materials, subsequently proposing appropriate and highly relevant new objectives (e.g., Vickers hardness, Pugh ratio, and energy above hull) targeting mechanical performance and stability. For newly proposed objectives, SAGA implements property evaluators through web search and automated programming, leveraging publicly available pretrained models or empirical methods. Upon analyzing designed structures and determining that a particular objective has been sufficiently optimized, SAGA automatically adjusts the optimization weight for that objective. Specifically, SAGA employs scaling or truncation of material property values to prevent over-optimization of individual objectives while neglecting others. Overall, these results demonstrate that SAGA enables automated materials design with different levels of human intervention through dynamic iterative optimization.

3.2.3 SAGA for Functional DNA Sequence Design

Programmed, highly precise, and cell-type-specific enhancers and promoters are fundamental to the development of reporter constructs, genetic therapeutics, and gene replacement strategies [65]. Such regulatory control is particularly important in HepG2, a human hepatocellular carcinoma cell line that retains key hepatic functions within a single cell type, including plasma protein synthesis and xenobiotic drug metabolism [150]. Although enhancers play a central role in establishing cell-type-specific gene expression programs [5], their rational design remains challenging due to the vast combinatorial space of possible functional DNA sequences. This task can be naturally formulated as an optimization problem with predefined oracle functions, such as DNA expression level predictor [123]. However, optimizing solely against expression-based oracles often results in sequences that generalize poorly with respect to biologically relevant constraints, including transcription factor motif enrichment, sequence diversity, and DNA stability. To address these limitations, we apply SAGA to discover novel cell-type-specific enhancers while iteratively refining the optimization objectives. Here, the SAGA framework is initialized using cell-type-



Supplementary Fig. 3: *Results for functional sequence design.* (a) Comparisons between different levels of SAGA and selected baselines with evaluation metrics (both average score (higher is better) and scaled standard error are reported (lower is better)). Our selected task is to design HepG2 (a cell line of epithelial-like cells from liver) specific enhancers. (b) The comparisons of different iterations of two different levels with the same held-out metrics. Each iteration will create new objectives. The solid line means objectives address the evaluation metrics, and the dash line means the metric has not been addressed. (c) An example of SAGA to correct the issues in previous iterations. (d) UMAP visualization of enhancers designed by SAGA , SAGA-Opt, and from references. (e) HepG2-specific motif visualization.

specific expression measurements obtained from Massively Parallel Reporter Assays (MPRA) [3] and subsequently performs optimization with respect to an initial set of objectives. Crucially, SAGA closes the design loop by systematically analyzing deficiencies in the designed sequences and adaptively modifying the objective functions to guide subsequent exploration. Through this iterative refinement process, SAGA converges toward a more comprehensive and biologically grounded objective set, yielding optimized enhancer candidates that better satisfy multifaceted design requirements.

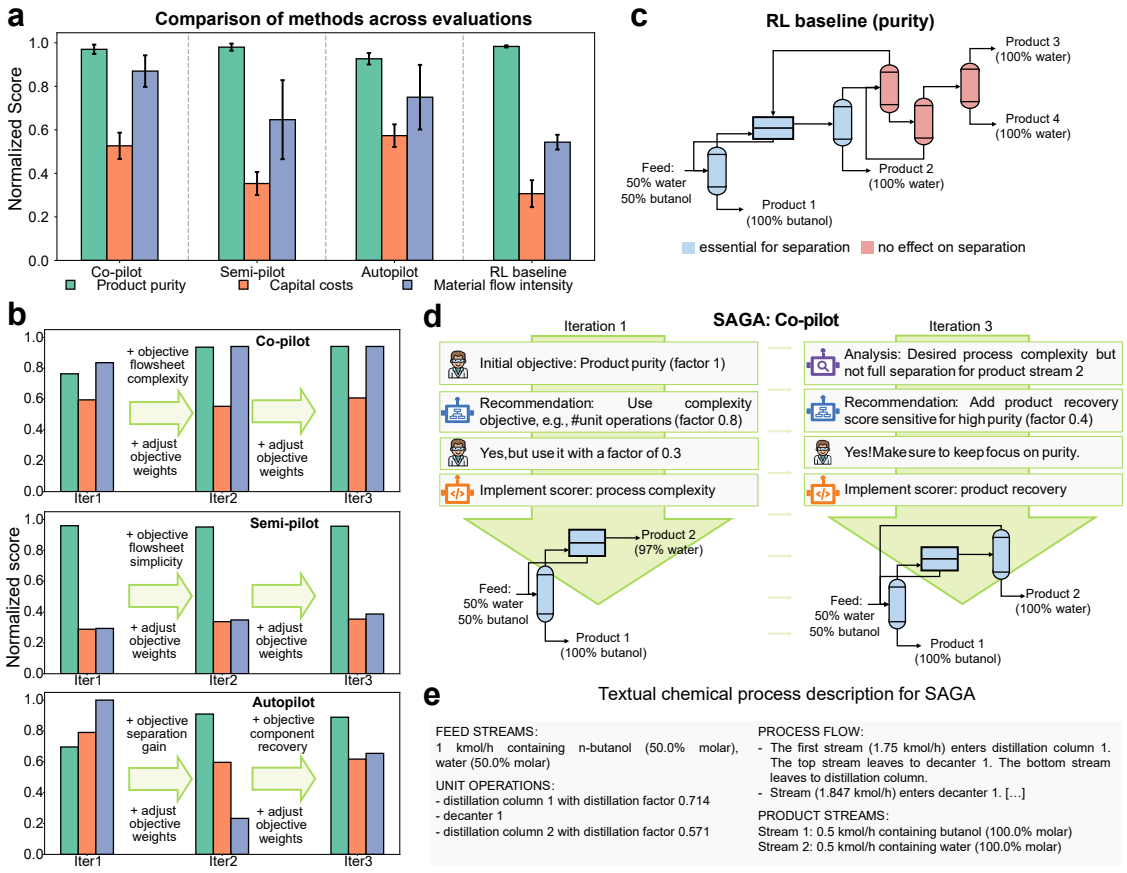
SAGA effectively discovers biologically plausible functional DNA sequences. We compare SAGA 's discovery capabilities by benchmarking it against established domain-specific models and AI agents [81, 106, 247]. Figure 3(a) reveals that our agents in different modes surpass selected baselines on metrics probing both statistical validity and biological function by 176.2% at most and 19.2% at least, based on an average comparison. Under controlled conditions where all baselines targeted the same objectives, our system exhibits marked improvements in MPRA specificity (by at least 48.0%), motif enrichment (by at least 47.9%), and sequence stability (by at least 1.7%). To further demonstrate the superiority of the multi-objective optimization method proposed by SAGA , we utilize the analyzer to examine the differences between enhancers produced by the Optimizer of SAGA with initial objectives only (SAGA -Opt) and SAGA . Figure S8.1 indicates that our designed enhancers exhibit obviously higher specificity and possess richer biological features compared to the former. These results suggest that SAGA effectively captures the complex interplay between statistical likelihood and biological constraints.

SAGA proposes reasonable and helpful objectives to assist human scientists for enhancer design. As already shown for drug discovery and materials design, the inclusion of human feedback via Co-pilot and Semi-pilot leads to marked improvements in biologically meaningful outcomes for DNA enhancer design as illustrated in Figure 3(b). For example, explicitly prioritizing transcription factor motif enrichment and sequence stability can be guided by expert input (Figure 3(c) and communication history), which results in enhanced biological validity of the designed sequences. SAGA 's Autopilot mode can also fully and automatically design enhancers, achieving overall performance comparable to that obtained with human intervention, particularly with respect to HepG2 specificity and improvements in sequence diversity and stability, and consistently outperforms other fully automated AI-agent baselines across all evaluated metrics (Figures 3(a) and (b)). As illustrated in Supplementary Figure S8.2, the objectives proposed by the agent are highly consistent across independent replicates and optimization iterations, with the majority (88.8%) corresponding to statistically driven objectives. Therefore, it also has more stable properties compared with other baseline agents, and indicates an advantage of using autonomous systems for scientific invention.

SAGA uncovers both novel enhancer candidates and known biological patterns. As shown in Figure 3(d), we compare the distributions from SAGA and SAGA -Opt with sampled HepG2-specific enhancers from a known experimental pool [65]. The enhancers discovered by SAGA exhibit distinctly different distributions, and given their outstanding performance in held-out metric evaluations, we can leverage SAGA from different modes to design more enhancers with high quality. Moreover, SAGA also recapitulates key biological principles, recovering multiple liver-specific transcription factor motifs [178] (shown in Figure 3(e)), supporting the biological plausibility of the designed sequences. When being evaluated on more biological-relevant multimodal regulatory readouts, including Cap Analysis Gene Expression sequencing (CAGE-seq) [187] and DNase I hypersensitive sites sequencing (DNase-seq) [16] predictions (shown in Figure S8.3 (a)), the designed enhancers again display strong HepG2 specificity, and they also show higher HepG2-specific expression levels compared with baseline methods (see Figure S8.3 (b)). These results highlight SAGA 's ability to leverage information encoded in pre-trained sequence-to-function models such as Enformer [8] to capture multimodal regulatory signals. In cell types where enhancers are active, lineage-defining and signal-responsive transcription factors bind to the enhancer sequence and recruit chromatin remodeling complexes, leading to localized chromatin opening and elevated DNase I hypersensitivity. This accessible chromatin state further facilitates the recruitment of the transcriptional machinery, giving rise to enhancer-associated bidirectional transcription that is captured by CAGE-seq [4, 41, 68]. Together, the coordinated elevation of DNase-seq and CAGE-seq signals provides complementary evidence of functional enhancer activity, reinforcing that SAGA successfully designs enhancers that recapitulate authentic, cell-type-specific regulatory programs rather than optimizing for a single assay in isolation.

3.2.4 SAGA for Chemical Process Design

Finally, we consider the use of SAGA for chemical process engineering applications, which is of high practical relevance within the chemical industry. While chemical process engineering has historically developed various heuristics and optimization-based approaches for the design of process flowsheets over the last decades, cf. [14, 213], more recently generative ML models combined with Reinforcement Learning (RL) have been investigated as a promising approach to automate chemical process design, with exemplary applications in reaction synthesis, separation, and extraction processes [61, 66, 196]. However, the focus on single design objectives predefined by human experts [61] can result in process flowsheets that lack characteristics of practical relevance and thus require iterative refinement in subsequent manual steps. It has also proven challenging to the LLM/agentic domain [50, 173], and only a



Supplementary Fig. 4: *Results for chemical separation process design.* (a) Comparisons between different levels of SAGA and baseline RL (trained to maximize product purity) with evaluation metrics (higher is better). Our selected task is to design process flowsheets for separation of an azeotropic butanol/water mixture with varying feed compositions (between 2%/98%). (b) The comparisons of different iterations of the three SAGA levels ran for three iterations with the same evaluation metrics. In each iteration, SAGA will analyze the processes, create new objectives, run the RL optimization, and select the best candidates across all current iterations. (c) Exemplary designed process by baseline RL agent for separating a 50%/50% butanol-water mixture, demonstrating that maximizing the product purity only leads to full separation but also in applying unit operations that do not have an effect on the separation quality (marked in red), as the RL agent is not penalized for using unnecessary operations. (d) Workflow for using SAGA co-pilot with agent and user actions for two iterations, resulting in optimal process design for separating a 50%/50% butanol-water mixture. (e) Text description of an exemplary chemical process that is used by SAGA .

few recent studies have utilized LLMs to optimize parameters for given chemical processes, e.g., in [249]. Here SAGA is used to advance automation of the chemical process design loop for separation of mixtures by proposing objectives that lead to more practical designs.

SAGA finds practically relevant processes by refining objectives in RL-based flowsheet design. As shown in Figure 4(a) and (c), using only the key objective of product purity for designing a separation process, i.e., the baseline, incentivizes the baseline RL agent to propose a flowsheet that results in optimal purity. However, without considering further objectives, such as capital costs, the RL agent might place unit operations that do not have an effect on the separation quality, or use a more complex flowsheet structure than needed. When using SAGA , we observe increased objective scores for capital costs and material flow intensity compared to the RL baseline, while the purity is at a high level that is close to ideal separation (Figure 4(a)). SAGA effectively assists human experts (at the co- and semi-pilot levels) in the iterative refinement and addition of objectives which includes balancing multiple objective weighting factors, illustrated exemplary with the human-agent interaction in Figure 4(d) and quantified along the iterations in Figure 4(b). Also at the autopilot level, we observe significantly increased process performance compared to the RL-based chemical process design, such that SAGA enables automation of practically improved chemical processes.

SAGA identifies and implements objectives that align with early-stage chemical process design. Starting with maximizing the product purity as an initial objective, SAGA proposes a diverse set of useful objectives, such as process complexity, component recovery, and material efficiency, to be considered in the design. In fact, we find that SAGA identifies and implements suitable process objectives and scoring functions at all levels (co-, semi- and autopilot), leading to higher overall scores on the evaluation metrics, cf. Figure 4(a). Adding additional objectives to the optimization also requires setting appropriate objective weights, see, e.g., Figure 4 (b) and (d), since we combine multiple objectives into one reward for the RL design agent. As the design is sensitive to these objective weights, we see larger variation in individual objectives with less human intervention, particularly, for material flow intensity at semi- and autopilot level, see Figure 4(a). Notably, the product purity across all levels also shows some slight variations, which can be explained by partly conflicting objectives, as SAGA achieves high gains in capital costs and material flow intensity compared to the baseline. Therefore, SAGA is able to enrich the chemical process design by relevant early-stage objectives.

SAGA effectively analyzes chemical processes based on text representations. To analyze the flowsheet designs and propose new objectives, SAGA requires a text representation of the chemical processes. As indicated in Figure 4(e), we represent the flowsheets as natural text description with the four categories: feed streams,

unit operations, process flow, and product streams. SAGA successfully utilizes this representation to analyze process design potentials, e.g., by highlighting suboptimal product purity, as shown in Figure 4(d), and identifying unit operations and flowsheet patterns that result in desired separation, cf. communication history. These examples highlight the capability of SAGA to capture complex process context and advance automated chemical process design.

3.3 Discussion

Scientific discovery is often limited not only by the vastness of the hypothesis space, but also by the "creativity" of defining objectives that ultimately leads to new discovery. Fixed proxy objectives can be incomplete, problem-specific, or vulnerable to misalignment and reward hacking, and are rarely sufficient to navigate open-ended discovery problems. SAGA , as a generalist agentic framework, address this challenge by iteratively evolving objectives and their realizations based on observed failure modes. By introducing an outer loop that proposes new objectives, implements executable scoring functions, analyzes outcomes, and selects final candidates, SAGA makes objective formulation a dynamic and autonomous discovery process.

Across four tasks spanning antibiotic drug discovery, inorganic materials design, functional DNA sequence creation, and chemical process optimization, we find that iterating objectives is the key driver of progress in achieving a novel discovery with practical viability. For antibiotic drugs, SAGA grounds the optimization of biological activity in chemical reality, ensuring that high predicted potency reflects genuine therapeutic potential rather than exploitation of the score of a predictive model. In inorganic materials design, SAGA proposes objectives targeting mechanical properties and thermodynamic stability for superhard materials, which lead to excellent quality in the designed materials via independent metrics. In the discovery of functional DNA sequences, SAGA develops biology-driven objectives for joint optimization with expression-associated objectives, and designs enhancers with both significant novelty and and strong cell-type-level specificity. For chemical process engineering, SAGA reveals excessively complex flowsheet structures in the designed processes and circumvents them by considering the process complexity and flow intensity objectives to realize more relevant flowsheets.

A clear advantage of SAGA comes from its alignment with scientific practice: discovery is typically an interactive loop in which scientists interpret intermediate results, revise what to optimize next, and decide which constraints matter the most at a given stage. SAGA balances the time and resources spent by human scientist effort and automated agents, and operationalizes the scientific workflow through an agentic system with multiple levels of automation. Furthermore, SAGA enables scientists

to interact and intervene with the discovery process when warranted while retaining the ability to run autonomously when objectives and evaluation pipelines are sufficiently mature. This interpretability and controllability offer significant efficiency and flexibility across tasks. In antibiotic design, SAGA allows chemists to identify problematic motifs such as uncommon rings or extended conjugated systems at intermediate interaction points. As a result, chemists could inject synthesis-oriented constraints early on, effectively steering the model away from synthetic liabilities and toward a more realizable chemical space. Similarly, in functional DNA sequence design, SAGA analyzes the properties of designed enhancers or promoters from the previous iteration and figures out the problems such as high off-target rates and lack of cell-type-specific motif, which may inspire biologists to modify the coming objectives and finally improve the quality of candidates to match the biological constraints.

Finally, SAGA instantiates a new path towards automated AI scientist, where most current AI scientists rely on scaling model capability and tool space. The autopilot mode effectively discovers important objectives aligning with the goal of the task and implements the scoring functions to guide the optimization process. By assessing the failure modes in the optimized candidates across iterations, SAGA effectively leverages the optimization algorithm in the discovery process. One key advantage of this structure is mimicking the two thought modes, "thinking, fast and slow" [94]. The inner loop optimization is thinking fast, exploring all reachable solutions given specific objectives and preferences, and the outer loop is thinking slow, evolving objectives and preferences given the full optimization results.

One limitation of SAGA is the reliance on computationally verifiable objectives. For scientific problems where results cannot be validated computationally at all, SAGA needs to be extended in two ways: (1) feedback or scoring provided by human or (2) autonomous lab-in-the-loop. Finally, the current high-level goals considered by SAGA often predefine the design space, e.g. all possible small organic molecules. To expand SAGA to handle more flexible tasks, SAGA needs to be extended to adjust the design space with high-level goal only being finding a drug for curing a certain disease, and automatically formulate the design space, such as small molecule binders, antibiotics, nanobodies or RNA sequences.

3.4 Methods

3.4.1 SAGA Framework

Overview

SAGA transforms open-ended scientific discovery into structured, iterative optimization by dynamically decomposing the high-level discovery goal into computable objectives and scoring functions. The framework comprises two nested loops: an *outer loop* that explores and evolve objectives for the optimization; and an *inner loop* that systematically optimize candidates against the scoring functions of the specified objectives.

The workflow proceeds as follows (Figure 1(c)): users provide a *high-level goal* in natural language, such as "design novel antibiotic small molecules that are highly effective against Klebsiella pneumoniae bacteria," and can optionally provide more context information such as task background or specific requirements, as well as initial objectives and initial candidates as the starting points. The system then iterates through four core agentic modules: (1) *Planner* formulates measurable objectives aligned with the overarching goal and informed by previous analysis; (2) *Implementer* realizes executable scoring functions for proposed objectives; (3) *Optimizer* optimizes candidates by iteratively generating and assessing candidates that maximize the objective scores, as the inner loop; and (4) *Analyzer* assesses progress and determines whether to continue optimization or terminate upon goal satisfaction.

Core Modules

Planner. This agent decomposes the scientific goal into concrete optimization objectives at each iteration. Given the goal and current candidate analysis, it identifies gaps between the present state and desired outcome, proposing computable objectives with associated names, descriptions, optimization directions (e.g., maximize or minimize), and (optional) objective weights.

Implementer. This agent instantiates callable scoring functions for proposed objectives. When the implementations are not provided with the initial objectives, it develops custom implementations by conducting web-based research on relevant computational methods and software packages, then implements and validates the scoring function within a standardized Docker environment to ensure executability and correctness.

Optimizer. This module constitutes the inner optimization loop. Given objectives and their scoring functions, it employs established optimization algorithms to generate improved candidates. The process alternates between batch evaluation using objective scoring functions and generation of new candidates designed to outperform

previous iterations. The architecture accommodates diverse optimization strategies, such as prompted language models, trained reinforcement learning agents, or any optimization algorithms, enabling flexible tuning. The default optimizer for SAGA is a simple LLM-based evolutionary algorithm with three essential steps: (1) candidate proposal: LLM proposes new candidates based on the current candidate pool, (2) candidate scoring: scores all proposed new candidates, and (3) candidate selection: constructs the updated pool from the previous pool and new candidates.

Analyzer. This agent evaluates optimization outcomes and recommends subsequent actions. It examines objective score trajectories, investigates candidate properties using computational tools, and synthesizes insights into actionable reports. The analyzer also determines whether candidates satisfy the goal and can trigger early termination when success criteria are met.

Autonomy Levels

SAGA aligns with human scientific discovery workflows and seamlessly supports human intervention at varying levels. We define three operational modes based on the degree of autonomy (Figure 1(d)):

- **Co-pilot**: Human scientists collaborate closely with both the planner and analyzer. At each iteration, these agents generate proposals (i.e., new objectives from the planner, and analysis from the Analyzer), which scientists can either accept directly or revise based on domain expertise. The implementer and optimizer operate autonomously within the outer loop, executing the human-approved objectives. This mode maximizes human control while automating implementation details.

- **Semi-pilot**: Human intervention is limited to the analyzer stage. Scientists review progress reports and optimization outcomes, providing feedback that guides the planner's subsequent objective proposal. The planner, implementer, and optimizer function autonomously, but strategic decisions about continuation, termination, or pivoting remain human-guided. This mode balances automation with critical oversight at decision points.

- **Autopilot**: All four modules operate fully autonomously without human intervention. The system independently plans objectives, implements scoring functions, optimizes candidates, and analyzes results. This mode enables large-scale automated exploration when domain constraints are well-specified and trust in the system is established.

This tiered design ensures scientists can interact with SAGA in ways that maximize productivity for their specific research context, from hands-on collaboration to fully autonomous discovery.

3.4.2 Task Configurations

Antibiotic discovery. We formulate this task to discover novel small-molecule antibiotics against *K. pneumoniae*. In practice, we set the high-level discovery objective as designing candidates with strong predicted antibacterial efficacy while maintaining structural novelty, favorable predicted mammalian-cell safety, avoidance of dominant known-antibiotic motifs, and practical feasibility aligned with purchasable-like chemical space for wet-lab validation (details in Figure S6.2). Both the high-level goal and the accompanying contextual information explicitly encode our design target and related constraints (detailed in Figure S6.2). For each SAGA instance, our initial objectives are always to maximize antibiotic activity, molecule novelty, and synthesizability, while minimizing toxicity to human and similarity to known antibiotic motifs in the designed molecules. During the loop of optimization, we use the default LLM-based evolutionary algorithm. The initial populations are selected from the Enamine REAL Database [188], which also serves as the first group of molecules in the parent node. We provide molecules from the parent node, individual score from each objective, and an aggregated score (by product individual scores) to the LLM, and generate new molecules after crossover operation. We then select the top molecules based on the list containing both generated molecules and molecules from the parent node. To encourage diversity, we consider a cluster-based selection strategy (Butina cluster-based selection [19] detailed in Figure S6.2). Finally, we combine all scoring functions into a single scalar value by product of expert to discourage ignoring any objective and select top molecules across all iterations. We use the standard implementation of the planner, implementer, optimizer, and analyzer modules.

Inorganic materials discovery. We consider two materials inverse design tasks. The first task aims to design permanent magnets with low supply chain risk, specified by two objectives: magnetic density higher than 0.2 $Å^{-3}$ and HHI score below 1500. The initial objective is set to maximize magnetic density. The SAGA Co-pilot mode is deployed with iteratively refined objectives: maximizing magnetic density in the first iteration, followed by the addition of HHI score minimization in the second. This task provides a direct comparison with MatterGen [251]. The second task is to design superhard materials for precision cutting, requiring high hardness, high elastic modulus, appropriate brittleness, and thermodynamic stability. The high-level goal and contextual information explicitly encode design requirements and constraints (Supplementary Section S7.1 and Figure S7.1). For each SAGA experiments of superhard materials design, initial objectives are set to maximize bulk modulus and shear modulus, which are important indicators for screening superhard materials [139]. The optimization loop employs a default LLM-based evolutionary algorithm. Initial populations are randomly sampled from the Materials

Project database [88], which also serves as the first group of crystals in the parent node. Based on LLM-proposed chemical formulas, pretrained diffusion models provide 3D crystal structures, with geometric optimization performed using universal ML force fields [241]. Evaluators assign objective scores based on the 3D structure of each crystal. Chemical formulas from the parent node and individual score of each objective were provided to the LLM, which generated new formulas through crossover operations. Optimal structures are then selected via Pareto front analysis from a combined pool of generated and parent crystals. The standard implementation of the planner, implementer, optimizer, and analyzer modules are used for all materials design tasks.

Functional DNA sequence design. Functional DNA sequences, also referred to as cis-regulatory elements (CREs), primarily include enhancers and promoters and play a central role in regulating gene expression levels [43, 228]. We focus on the *de novo* design of cell-type-specific enhancers and promoters across multiple cellular contexts, including HepG2 (enhancer and promoter), K562 (enhancer and promoter), SKNSH (enhancer and promoter), A549 (promoter only), and GM12878 (promoter only). The selection of these cell lines is constrained by the availability of high-quality, publicly accessible datasets. We formulate the discovery task using a high-level natural-language prompt that specifies the objective of generating functional DNA sequences with strong cell-type specificity. Both the high-level goal and the accompanying contextual information explicitly encode target and off-target cell-type constraints (see Figure S8.2). During optimization, the primary objectives are to maximize predicted expression in the target cell line while suppressing activity in non-target cell lines. For optimization, we employ a default LLM-based evolutionary algorithm. The initial population is selected by sampling from a pool of random DNA sequences. During candidate selection, we keep all candidates that satisfy the expression selectivity. Moreover, we also keep top 50% diverse candidates measured by average pairwise Hamming distance. Finally, we use the standard implementation of the outer loop and the analyzer, planner, and implementer agents.

Chemical process design. We use SAGA for the design of chemical process, more specifically separation process flowsheets, which is a central task in chemical engineering [14, 61, 213]. The high-level goal is formulated as a natural language prompt targeting the design process flowsheets for the steady-state separation of an azeotropic binary mixture of water and ethanol at different feed compositions into high purity streams, cf. Figure S9.2. For the optimizer designing process flowsheets in the inner loop, we use an RL agent based on the separation process design framework by Göttl et al. [66], see details in Figure S9.2. The the action space of the RL agent comprises the (1) selection of suitable unit operations, such as decanters, distillation columns, and mixers with their specifications, and (2) determination of the material flow structures (including recycles) that connect the unit operations. We translate

the RL-internal matrix representation of process flowsheets to a text description, see Figure S9.1 for details. We use the standard implementation of the analyzer, planner and implementer, and the text description is provided to the agents in the outer loop. The proposed new objectives – with corresponding weighting factors to aggregate the objective values into one reward value – are automatically added to the RL framework and used for the next iteration of process design, which always starts from scratch without an initial population, whereby the initial objective for the first iteration is the product purity. We thus focus on the iterative addition and refinement of suitable chemical process design objectives and their weighting factors.

3.4.3 Task Evaluations

We validate the performance of SAGA on each individual task by setting up a set of evaluation metrics. The evaluation metrics are unseen during the online running procedure of SAGA . Below, we briefly discuss the evaluation procedures for each task.

Antibiotic discovery. To mimic real-world lab experiment, we consider evaluating the candidates from the perspectives of biological objectives, synthesizability, and drug likeness. These three areas can be covered with 11 different computational metrics. To evaluate generated molecules with biological objectives, we consider antibiotic activity score, novelty score, toxicity score, and known motif filter score as metrics. For synthesizability, we consider a synthetic accessibility score as the metric. Last but not least, to evaluate drug likeness, we consider QED score, DeepDL prediction score, molecular weight score, PAINS filter score, BRENK filter score, and RING score as metrics. Detailed implementation and evaluation protocols are provided in Figure S6.2. When evaluating candidates proposed by baselines and SAGA , we compute both the absolute score and pass rate of the top 100 molecules selected using each model's optimization objectives for a fair comparison.

Inorganic materials design. To evaluate material properties, density functional theory (DFT) calculations were employed to determine the electronic, magnetic, and mechanical properties of generated materials, as well as energy above hull [27, 251]. HHI scores were computed using the pymatgen package [155]. In the task of designing permanent magnets with low supply chain risk, two objectives were specified: magnetic density higher than 0.2 Å^{-3} and HHI score less than 1500. In the task of designing superhard materials for precision cutting, the evaluation metrics include Vickers hardness, bulk modulus, shear modulus, Pugh ratio, and energy above hull. More details of the evaluation protocols are described in Figure S7.2.

Functional DNA sequence design. To emulate real-world experimental evaluation, we adopt a blind computational assessment framework based on five established computational oracles drawn from prior studies [34, 41, 65, 106]. As a

representative task, we focus on the design of HepG2-specific enhancer sequences. The evaluation metrics include statistical comparisons of MPRA-measured expression between the target cell line and non-target cell lines (e.g., HepG2 vs. K562 and HepG2 vs. SKNSH), together with knowledge-driven criteria such as transcription factor motif enrichment, sequence diversity, and sequence stability. Detailed implementation and evaluation protocols are provided in Figure S8.2.

Chemical process design. To cover early-stage process design goals, we utilize the short-cut simulations models developed in [66] and calculate three process performance indicators. These are used as the evaluation metrics and include the product purity, capital costs, and material flow intensity. The product purity corresponds to the average purity of the product streams received from the simulation. The capital costs represent the sum of individual unit operation costs estimated on a simple heuristic, similar to [66]. For the material flow intensity, we calculate the recycle ratios and introduce penalty terms for excessive ratios and very small streams ($< 1\%$ of the feed stream). We refer to the Figure S9.2 for further implementation details.

S5 Implementation Details

S5.1 Framework Design

The SAGA framework translates high-level scientific goals into iterative optimization procedures targeting dynamically evolving sets of specific objectives. The system accepts the following inputs:

- **Goal**: A high-level design goal specified in natural language that defines the scientific task.
- **Context information** (optional): Supplementary descriptions of task background or specific requirements regarding objectives, enabling the framework to better understand the task domain and propose objectives aligned with domain-specific scientific needs.
- **Initial objectives** (optional): Predefined objectives accompanied by their corresponding scoring functions, which are incorporated into the first iteration as a foundation for optimization.
- **Initial candidates** (optional): Randomly initialized candidate solutions defining the initial search space.

Upon completion, SAGA outputs design solutions that satisfy the specified goal.

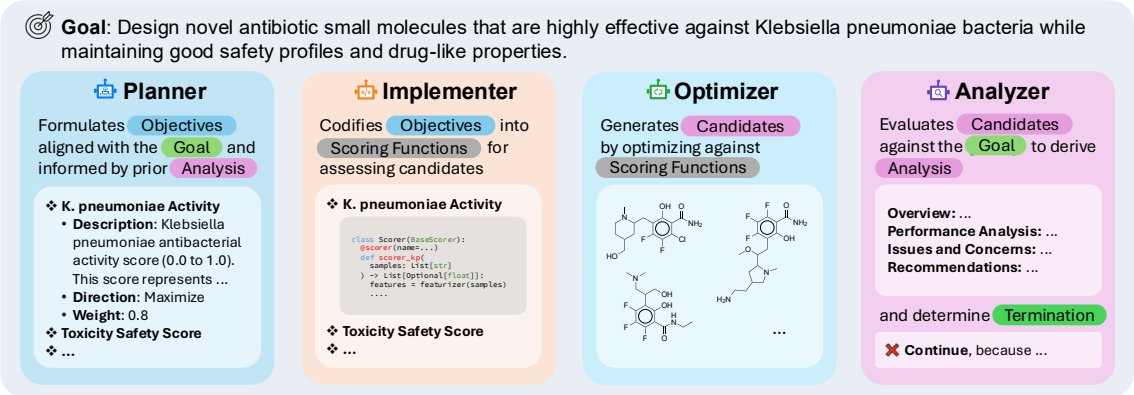
Key Concepts

The framework defines four key concepts that structure information flow throughout the system:

Candidate represents an individual solution in the optimization space. Each candidate encapsulates a domain-specific representation (e.g., SMILES strings for molecular structures, multi-domain dictionary object for materials) and objective scores from multiple objectives. Candidates are uniquely identified and tracked across iterations, enabling provenance tracking and historical analysis.

Population manages collections of candidates as a cohesive unit. Beyond storing candidate lists, each population provides methods for batch objective scoring and statistics over stored candidates. Populations serve as the primary data structure passed between modules during optimization.

Objective specifies what should be optimized using a natural language description. Each objective can be one of the three types: candidate-wise objectives that evaluate individuals independently, population-wise objectives that assess collective



Supplementary Fig. S5.1: The functional illustration of the four core modules.

properties, and filter objectives that enforce binary constraints. Each objective includes an optimization direction (maximize or minimize, not applicable for filter objectives) and an optional weight for multi-objective aggregation.

Scoring Function implements the computational logic for evaluating candidates against objectives. Scoring functions accept candidate representations and return numerical scores (for candidate-wise or population-wise objectives) or boolean values (for filter objectives). Each scoring function is implemented as an Model Context Protocol module, enabling isolated execution in Docker containers with standardized input/output interfaces. This design provides dependency isolation, reproducibility across environments, and safe execution of potentially unsafe code.

Modules

The framework comprises four **core modules** (Figure S5.1) that implement the main optimization workflow:

Planner systematically decomposes the high-level scientific goal into concrete, computationally tractable objectives at each iteration. The module receives as input the optimization goal, optional context information that introduces the task background or specifies specific requirements, and analysis report from the last iteration. Through LLM-powered reasoning, it identifies gaps between the current state and desired outcomes, formulating objectives that specify what should be measured (description), how it should be optimized (maximize or minimize), and its relative importance (weight). Each objective must be specific enough to guide implementation and broad enough to capture meaningful scientific properties. The module outputs a structured list of objectives with complete specifications that guide subsequent implementation and optimization phases.

Implementer codifies abstract objectives into executable scoring functions. Given a list of proposed objectives, the module first attempts to match each objective

with existing scoring functions that may be provided with the initial objectives or proposed in previous iterations through semantic similarity analysis, comparing objective descriptions using LLM-based pairwise evaluation. When an exact match cannot be found, the implementer autonomously implements new scoring functions through a multi-step process: (1) conducting web-based research to identify relevant computational methods, software packages, and scientific models; (2) synthesizing this information into executable Python code following standardized interfaces; (3) testing the implementation with specified dependencies and sample candidates. All scoring functions, whether retrieved or newly created, are deployed in isolated Docker containers with specified dependencies, ensuring reproducibility and preventing conflicts. The module outputs objectives with attached, validated scoring functions ready for optimization.

Optimizer executes the inner optimization loop, systematically generating and evaluating candidate solutions to optimize objective scores. The architecture supports diverse optimization algorithms, from evolutionary strategies to reinforcement learning agents, through a unified interface. The default implementation uses an LLM-based evolutionary algorithm that operates in three phases per generation: (1) *candidate proposal*, where the LLM analyzes high-performing candidates from the current population and generates novel candidates designed to improve upon observed patterns; (2) *batch evaluation*, where all candidates (existing and newly generated) are scored against each objective using their attached scoring functions, with parallel execution for efficiency; and (3) *candidate selection*, where the next generation's population is assembled by ranking candidates according to weighted objective scores and maintaining fixed population size. Convergence detection monitors score improvement across consecutive generations, triggering early termination when progress plateaus. The optimizer receives the current population and objectives with scoring functions as input, and outputs an improved population.

Analyzer performs comprehensive evaluation of optimization progress and synthesizes actionable insights to guide subsequent planning. The module receives the current optimized population, active objectives, and historical summaries from previous iterations. It conducts multi-faceted analysis: computing statistical distributions of objective scores (mean, standard deviation, extrema) and comparing trends across iterations; identifying top-performing candidates and examining their properties to understand success patterns; assessing population diversity and detecting potential issues such as premature convergence or objective conflicts; and optionally employing domain-specific computational tools for deeper candidate investigation. The analysis synthesizes these quantitative and qualitative findings into a structured natural language report containing four sections: (1) *overview* summarizing current state and key characteristics; (2) *performance analysis* highlighting improvements, regressions, and trends; (3) *issues and concerns* identifying problems like stagnation or conflict-

ing objectives; and (4) *strategic recommendations* proposing actionable adjustments for the next iteration. Beyond reporting, the analyzer makes a termination decision by evaluating whether optimization goals have been satisfied, whether scores have converged, and whether continued iteration would yield meaningful improvements. The module outputs an analysis report that informs the planner's next objective formulation and a termination decision (continue or stop).

In addition to the above core modules, upon the end of the iterations, a **selector** agent is used to systematically evaluates all candidates generated throughout the optimization process, selecting solutions that best satisfy the discovery goal. It writes code and optionally call scientific tools to comprehensively assess and rank all candidates and returns a specified number of top candidates.

S5.2 Execution Workflow

The SAGA framework executes through a structured outer loop coordinated by the orchestrator, with each iteration progressing through five sequential phases:

Phase 0: Initialization. Before the first iteration begins, the system instantiates all modules and processes user-provided inputs. If users provide an initial population, it is stored and marked as iteration 0. If initial objectives with scoring functions are provided, the planner will be suggested to use them, and the implementer would omit implementing them on its own.

Phase 1: Planning. At the start of each iteration, the planner receives the optimization goal, optional context information that introduces task background or specifies specific requirements, and the analysis report from the previous iteration (iterations 2 onward). The Planner formulates objectives for the current iteration, considering optimization progress, remaining gaps, and strategic priorities.

Phase 2: Implementation. The Implementer processes each objective in parallel, attempting to match it with existing scoring functions through LLM-based semantic comparison or creating new implementations when matches are not found. If some objectives cannot be matched or implemented, the system records the unmatched objectives and invokes the planner with information about which objectives failed and why. The planner then revises its objective proposals to better align with available computational capabilities. This planning-implementation retry loop continues for up to a configurable maximum number of attempts until all objectives have attached scoring functions. Successfully matched objectives proceed to optimization.

Phase 3: Optimization. The optimizer receives the current population (from the previous iteration) and objectives with scoring functions. If configured, a specified ratio of the population is randomly replaced with newly generated random candidates to maintain diversity. The optimizer then executes its algorithm-specific optimization process, which for the default LLM-based evolutionary optimizer involves multiple

generations of candidate proposal, batch evaluation, and selection until convergence criteria are met or generation limits are reached.

Phase 4: Analysis. The analyzer receives the optimized population, current objectives, and historical summaries. It evaluates all candidates against objectives, computes score statistics and trends, and conducts detailed candidate investigations using coding and domain-specific tools. The analysis results are synthesized into a structured report. The analyzer also evaluates termination criteria, including goal satisfaction, score convergence, and resource constraints, and recommends whether to continue to the next iteration or terminate optimization. If termination is not recommended and the maximum iteration count has not been reached, the workflow returns to phase 1 for the next iteration.

Phase 5: Finalization. Upon termination (either by analyzer decision or maximum iterations reached), the system invokes the Selecto to perform retrospective candidate evaluation. It retrieves all candidates generated across all iterations, evaluates them comprehensively against the original discovery goal using computational tools and custom analysis code, and selects the top candidates holistically with all objectives considered rather than solely by final-iteration scores. This retrospective selection ensures that high-quality solutions from early iterations exploring different objective combinations are retained. The system then collects the results and process logs and saves them for reproducibility and provenance.

S5.3 Experimental Setups

We used the following LLM settings for each module:

- **Planner**: `gpt-5-2025-08-07`
- **Implementer**: `gpt-5-2025-08-07` for matching existing scoring functions to objectives, and the Claude Code agent with `claude-sonnet-4-5-2025-0929` for implementing new scoring functions.
- **Optimizer**: Task-specific. Different tasks may use different LLMs as backbones.
- **Analyzer**: the Claude Code agent with `claude-sonnet-4-5-2025-0929` for investigating specific candidates, `gpt-5-2025-08-07` for writing comprehensive analysis and making termination decisions.
- **Selector**: the Claude Code agent with `claude-sonnet-4-5-2025-0929`.

For each of the tasks, the input includes a high-level goal that briefly describes the design goal, context information that provides task background or specifies specific

requirements, a set of initial objectives with scoring functions and a population of randomly initialized candidates as the starting point. See task-specific sections for more details.

All code and configurations will be released online.

S5.4 Extensibility and Customization

The SAGA framework is designed for extensibility at multiple levels:

Custom Module Implementations. Each module defines an abstract base class with required methods. Users can implement custom modules by subclassing these bases and registering implementations in the module registry. For example, users can easily replace the default LLM-based optimizer with a genetic algorithm or Bayesian optimization, or implement new workflow and add advanced features for planner. This modular and extensible codebase enables SAGA to be continuously updated and customized to more tasks.

Custom Scoring Functions. Users can add manually implemented task-specific scoring functions by following a specific code template. This allows users to provide their existing objectives and corresponding computational methods to the system, for more accurate and faster optimization. The implementer agent will also refer to user-provided scoring functions when implementing new ones.

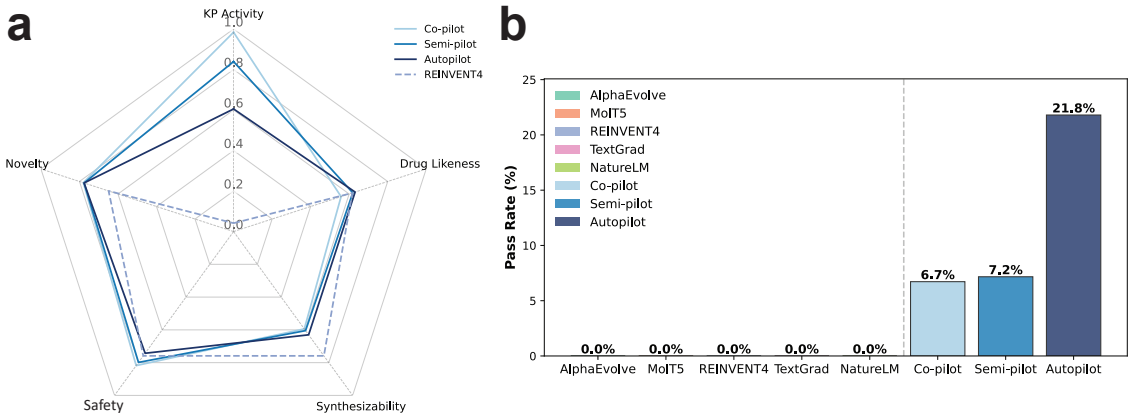
Configuration-Based Customization. The framework uses structured configuration files (JSON or YAML) that specify module selections and versions, LLM settings per module (model, temperature, max tokens), loop parameters (max iterations, convergence thresholds, random injection ratio), and logging verbosity and output directories. This configuration-driven design enables experimentation with different setups without code modification.

S5.5 SAGA Reduces to the Optimizer agent (SAGA -Opt)

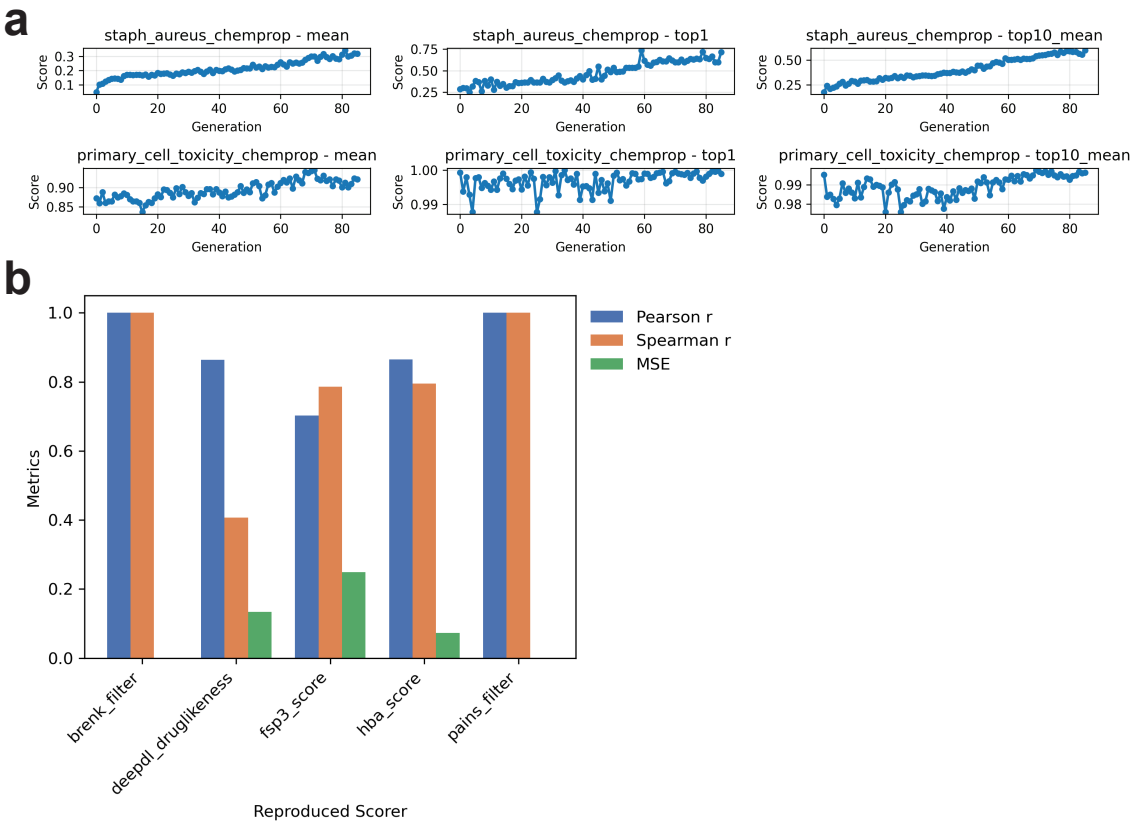
The default optimizer agent is a simple evolutionary optimization algorithm, thus SAGA reduces to SAGA -Opt when the outer loop is disabled. SAGA -Opt only optimizes with fixed objectives as input. For SAGA -Opt experiments, we always use this default version without any specialized design.

S6 Antibiotic Discovery

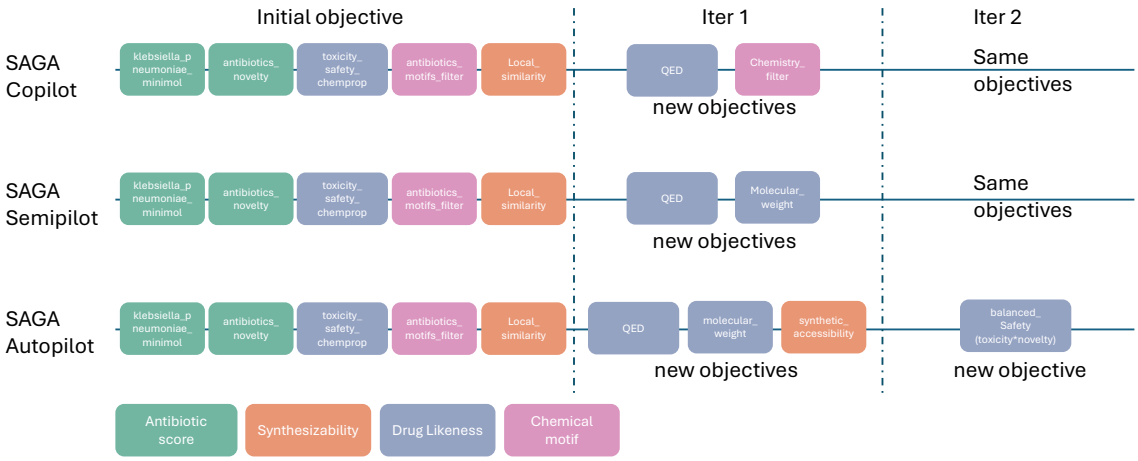
S6.1 Supplementary Figures



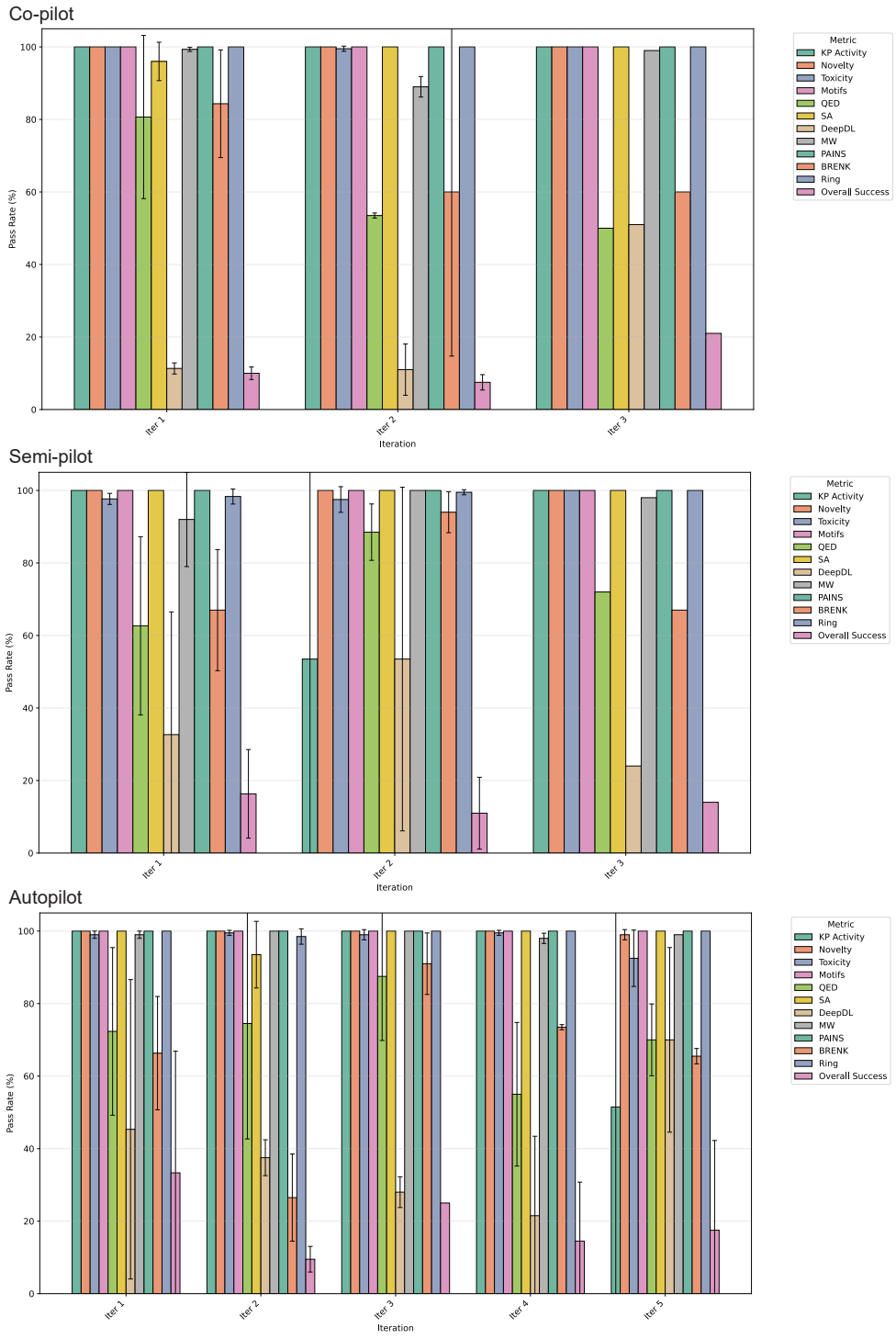
Supplementary Fig. S6.1: Additional benchmarks of antibiotic drug design. (a) Comparison of the property distributions of final candidates proposed by three SAGA levels and REINVENT4 [126]. (b) Pass rate for all final molecules proposed by SAGA and different baselines based on external evaluations. Definition of the property scores and pass rate can be found in Figure S6.2.



Supplementary Fig. S6.2: Ablation studies of antibiotic design. (a) The change of scores from different objectives across all epochs in the optimization process, based on the optimizer only. (b) Examining the designed scorers based on random molecules ($n = 1000$). The output score correlation between human-implemented and implementer-implemented scoring functions. We report Pearson correlation, Spearman correlation and mean squared errors.



Supplementary Fig. S6.3: Objectives proposed in early-stage iterations across three modes of SAGA .



Supplementary Fig. S6.4: Pass rate (proportion of molecules passing the metric-specific threshold) changes across iterations from different modes in SAGA .

S6.2 Experimental Setups

Objectives, metrics and baselines

Here we describe the experimental setup for antibiotic discovery targeting on Klebsiella pneumonia.

Initial objectives. Our initial objectives are:

- Maximize: **Antibiotic activity**, predicted by an ensemble of 10 Minimol [100] models trained on an internal high-throughput screening dataset for bacterial inhibition. We utilize 5-fold cross validation to select the best model.
- Maximize: **Novelty**, defined as $1 - s_{\text{tan}}$, where s_{tan} is the Tanimoto similarity between the selected molecule and the closest known compound from a predefined pool with antibiotic indication.
- Minimize: **Toxicity**, predicted by an ensemble of Chemprop [73] models trained on mammalian cell toxicity data [257].
- Minimize: **Known antibiotic motifs**, comprising six major motif classes (sulfonamides, aminoglycosides, tetracyclic_skeletons, beta_lactams, pyrimidine_derivatives, quinolone), implemented via 19 SMARTS patterns covering common variations.
- Maximize: **Synthesizability**, measured as the Tanimoto similarity to the closest compound from a subset of Enamine REAL database space [188] to ensure the existence of a purchasable analog.

Evaluation metrics. We evaluate generated molecules using the following predictive scores and rule-based filters, together with fixed acceptance thresholds. Unless otherwise noted, all metrics are normalized to $[0, 1]$, with higher values indicating more desirable properties.

- **Antibiotic activity score** is predicted by an ensemble of 10 Minimol [100] models trained on an internal high-throughput screening dataset for bacterial inhibition. The final prediction is rescaled to be from 0 to 1. We classify a candidate as computationally active if it has a score of 0.2 or higher.
- **Novelty score** is defined as $1 - s_{\text{tan}}$, where s_{tan} is the Tanimoto similarity to the closest known or investigated compound with antibiotic indication. We require novelty to be greater than or equal to 0.6.
- **Toxicity score** is predicted by an ensemble of Chemprop [73] models trained on mammalian cell toxicity data. Molecules are considered acceptable if toxicity to be greater than or equal to 0.5, indicating lower predicted cytotoxic risk.

- **Known antibiotic motifs filter** comprises six major motif classes (sulfonamides, aminoglycosides, tetracyclic_skeletons, beta_lactams, pyrimidine_derivatives, quinolone), implemented via 19 SMARTS patterns covering common variations [182]. A score of 1.0 indicates no known motifs are present.

- **Quantitative estimate of drug likeness (QED) score** is the quantitative estimation of drug likeness [13], combining physicochemical properties into a single score in $[0, 1]$. We require QED to be greater than or equal to 0.5.

- **Synthetic accessibility (SA) score** estimates how easily a molecule may be synthesized [73], normalized to $[0, 1]$, where higher values indicate easier synthesis. We require SA to be greater than or equal to 0.5.

- **DeepDL drug likeness** is an unsupervised deep-learning score trained on approved drugs and normalized from its original scale to $[0, 1]$. We require DeepDL to be greater than or equal to 0.3 [108].

- **Molecular weight score** is a pass indicator that equals 1.0 if molecular weight lies between 150 and 500 Da and 0.0 otherwise; we require MW to be equal to 1.0.

- **PAINS filter** is a binary score returning 1.0 if no PAINS (A/B/C) alerts are present and 0.0 otherwise; we require PAINS to be equal to 1.0.

- **BRENK filter** assigns 1.0 for no structural alerts, 0.5 for exactly one alert, and 0.0 for two or more alerts; we require BRENK to be greater than or equal to 1.0.

- **Ring score** measures how common the molecule's ring systems are relative to ChEMBL statistics [63], where 1.0 indicates common (or no) rings and 0.0 indicates rare or unseen ring chemotypes. We require ring_score to be equal to 1.0.

Summarizing the above threshold, we define **pass rate** as the proportion of molecules whose selected properties are higher than a given threshold, over the whole population (Numerical setting determined by chemists: KP Activity$\geq$ 0.2, Novelty$\geq$ 0.6, Toxicity$\geq$ 0.5, Motifs$\geq$ 1.0, QED$\geq$ 0.5, SA$\geq$ 0.5, DeepDL$\geq$ 0.3, MW$\geq$ 0.5, PAINS$\geq$ 1.0, BRENK$\geq$ 1.0, RING$\geq$ 1.0. These sets of **evaluation thresholds** are also used to select final candidates for future experiments.

Baselines. We benchmark against four representative previous approaches spanning (1) general-purpose LLM-driven optimization, (2) generalist science language models, and (3) RL-based molecular generative model.

- **TextGrad** [247] is an LLM-based optimization framework that iteratively edits molecular SMILES strings using critique-style feedback from the scoring functions and LLMs.
- **NatureLM** [230] is a multi-domain, sequence-based science foundation model enabling instruction-driven generation/optimization across molecules, proteins, nucleic acids and materials.
- **REINVENT4** [126] is based on reinforcement-learning fine-tuning of a SMILES generator to maximize (possibly multi-objective) scoring functions, with diversity-aware goal-directed design. We utilize the latest version of this package.
- **MolT5** [53] is a T5-based molecule–language translation model supporting text-to-molecule and molecule-to-text generation.

Hyperparameters. *Klebsiella pneumoniae* (KP) small-molecule design: For each iteration, the LLM produces 70 offspring per generation via crossover and mutates 7 of the best molecules from the current population. Then 120 molecules are selected as the next population. The total oracle budget is capped at 10,000 evaluations. Parents are chosen via **size-3 tournament selection** (two parents per mating event), and survival selection uses a diversity-aware `diverse_top` strategy: we perform **top-k** selection by the aggregated score and then preserve chemical diversity by enforcing a **Tanimoto similarity** constraint, retaining a candidate only if its fingerprint similarity to all already-selected survivors is below 0.4 (i.e., $s_{\text{tan}} < 0.4$). Multi-objective optimization is performed by a simple product aggregator (`simple_product`) over all objective scores. We preserve **elitism** by retaining the top 5% candidates each generation, where elites are selected by the KP activity field (`klebsiella_pneumoniae_minimol`).

Workflows. Our three different levels (co-pilot, semi-pilot, and autopilot) follow the default setting, discussed in Figure 3.4.1. For all experiments, we have three replicates.

Moreover, to increase the diversity, we consider applying Butina cluster-based selection [19] at the end of each iteration. The clustering steps include 1. Compute pairwise similarities (Calculate all pairwise Tanimoto similarities between fingerprints); 2. Choose a similarity cutoff (our current setting is 0.4); and 3. Greedy clustering (Find the molecule with the largest number of neighbors above the cutoff, perform clustering, remove all clustered molecules from the pool, and repeat until no molecules remain). The selection process means we select the top samples by aggregated scores in each cluster as final molecules.

We consider two sets of molecules for wet-lab validation. For the molecules tested with Enamine set, the experimental molecules are generated from 10 seeds of SAGA

's Co-pilot mode. We find Enamine close neighbors (sim ¿ 0.6) for all generated molecules passing held-out metrics. And then revaluate the held-out objectives on these Enamine close neighbor. The Enamine close neighbour molecules that pass the held-out requirements are manually selected by human expert for wet-lab validation. For the molecules tested with Topone set, the experimental molecules are generated from three levels of SAGA . We find Topone close neighbors (sim $>$ 0.5, as the molecules from Topone are generally smaller) for all generated molecules passing held-out metrics. And then revaluate the held-out objectives on these Topone close neighbor. The Topone close neighbour molecules that pass the held-out requirements are manually selected by human expert for wet-lab validation.

High-level Goal

Design novel antibiotic small molecules that are highly effective against Klebsiella pneumoniae bacteria while maintaining good safety profiles and drug likeness-related properties.

Context Information

For this task, we want to design novel antibiotics targeting K. pneumoniae bacteria. The molecules should: 1. Show high predicted activity against K. pneumoniae. 2. Maintain low toxicity to human cells 3. Avoid problematic substructures for medicinal chemistry 4. Show structural novelty compared to existing antibiotics 5. Have good drug likeness-related properties and molecular weight for small molecule drug design. The optimizer will automatically enforce SMILES validity and length constraints, so do not propose objectives related to these. IMPORTANT SCORER REQUIREMENTS:

- For candidate-wise objectives: Scores must be normalized to [0, 1] range, where higher values are better (maximization direction).
- For filter objectives: Scores must return 1.0 for pass and 0.0 for fail. Filters do not need normalization or inversion when multiplied into aggregated scores.

S6.3 Additional Experimental Results

Analyses of optimization convergence in the experiment

SAGA different modes have similar performances. We observed an interesting phenomenon in this task: the performance of the three different modes was relatively similar. To explain this reason, we further investigated the types of objectives suggested by the analyzer for SAGA . According to Figure S6.3, the objectives

proposed by different modes in each iteration are convergent to a specific list of types (e.g., QED scores occur in the iteration 1 from all modes), which can explain the similarity of performances across different modes.

Important objectives have been proposed during the early iterations. We analyze all the proposed objectives from SAGA and find that the objectives coming out after iteration 1 cannot improve the quality of molecules obviously, shown in the pass rate comparison across different iterations (Figure S6.4). This can be explained by the relatively comprehensive information provided by the analyzer in the analysis report. Overall, this phenomenon demonstrates that SAGA can efficiently identify and enhance objectives that bind to outputs while reducing the cost of running agents.

Ablation studies

Evaluation of optimizer. To validate whether our optimizer can work as we expect, we include an ablation study to check the performance of this agent in improving the proposed objectives via iteration. As shown in Supplementary Figure Figure S6.2 (a), the optimizer can improve the corresponding objective as the iteration of optimization increases. Therefore, our LLM-based optimizer can successfully improve the quality of generated molecules.

Evaluation of implementer. We test the ability of implementer by validating whether it can propose similar objectives that have human implementation. Here, we focus on several important drug- and biology-related metrics, and compare the similarity between the results produced by methods from different sources with the same group of molecules as inputs (10,000 random sampled molecules from the Enamine REAL Database). According to Figure S6.2 (b), our implementer successfully implements five different objectives from different categories, and the comparison result shows low MSE and high correlation. Therefore, our implementer can successfully create scorers corresponding to the assigned objectives.

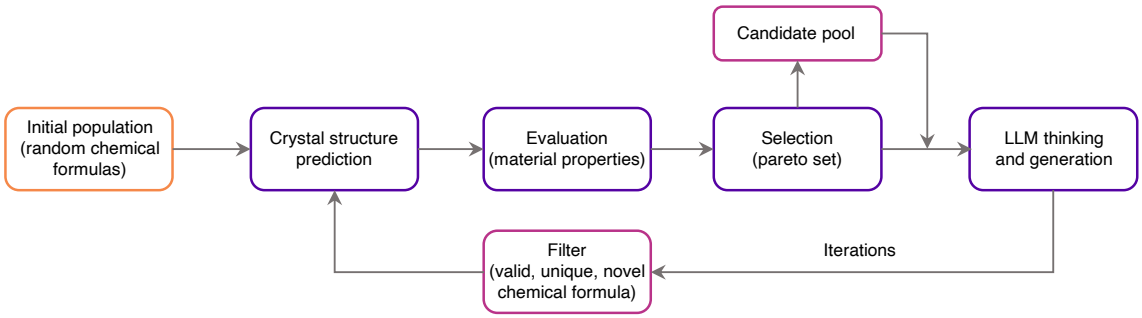
S7 Inorganic Materials Design

S7.1 Experimental Setups

LLM-based evolutionary algorithm for materials design

As shown in Figure S7.1, the material property optimization loop employed a LLM-based evolutionary algorithm. Initial populations (chemical formulas of crystals) were randomly sampled from the Materials Project database [88], which also serves as the first group of crystals in the parent node. The LLMs are used to generate

chemical formulas, and then DiffCSP diffusion model [93] is used to generate 3D crystal structures, which was pretrained on the MP-20 dataset [88]. Geometric optimization are performed for the generated structures using universal ML force fields (MatterSim [241]). Evaluators assigned objective scores based on the 3D structure of each crystal. Chemical formulas from the parent node and individual score of each objective were provided to the LLM, which generated new formulas through crossover operations. Optimal structures are then selected via Pareto front analysis from a combined pool of generated and parent crystals.



Supplementary Fig. S7.1: Schematic diagram of the optimizer using LLM-based evolutionary algorithm.

Objectives, metrics, and baselines

In the task of designing permanent magnets with low supply chain risk, two objectives were specified: magnetic density higher than 0.2 $Å^{-3}$ and HHI score less than 1500. The SAGA Co-pilot mode was deployed with iteratively refined objectives: maximizing magnetic density in the first iteration, followed by the addition of HHI score minimization in the second. During optimization, the ALIGNN model [37] pretrained on DFT data from the Materials Project database [88] is used as the scorer for magnetic density prediction. The HHI scores are calculated using the pymatgen package [155]. The thermodynamically Stability ($E_{hull} < 0.1$ eV/atom), Uniqueness, and Novelty (SUN) [251] of each generated structure was evaluated by MatterGen's method using their Alex-MP reference dataset and code [251], and only SUN structures were retained. During optimization, the generated structures were optimized using ML force field (MatterSim [241]) due to the high computational cost, and E_{hull} was obtained by MLIP energy. Finally, 200 crystal structures generated from each iteration were randomly selected and DFT verified.

MatterGen models [251] that target only high magnetic density (single) or both properties (joint) were used to generate 4000 structures by their conditional generation method. For MatterGen (single), the conditional generation's target is magnetic density of 0.2 $Å^{-3}$. And the conditional generation's target of MatterGen (joint) is

magnetic density of 0.2 $Å^{-3}$ with HHI score of 1500. Only SUN structures were retained, and 200 crystal structures were randomly selected for DFT evaluation.

In the task of designing superhard materials for precision cutting, the evaluation metrics include Vickers hardness, bulk modulus, shear modulus, Pugh ratio, and energy above hull. The initial objectives of SAGA experiments in different modes are to maximize the bulk modulus and the shear modulus. During optimization, the ALIGNN models [37] pre-trained on DFT data from the Materials Project database [88] are used as the scorers for prediction of bulk and shear modulus. For performance evaluation, the top 100 crystal structures from the final LLM-ranked candidates after convergence were selected for DFT calculations to obtain scores for each metric.

Baselines. We benchmark against both state-of-the-art domain-specific methods and general-purpose AI agents. The baselines include MatterGen [251] and TextGrad [247]. Details of each baseline are described below:

- **MatterGen** is a diffusion model for inorganic material generation. The unconditional MatterGen model was pretrained on the MP-Alex-20 dataset [251], which contains unlabeled crystal structures, enabling the generation of stable and novel structures. Furthermore, the adapter-equipped MatterGen model was fine-tuned on crystal structures with DFT-derived labels, thereby enabling controllable generation of crystals with desired properties.

- **TextGrad** is an LLM-based optimization framework. Initialized with the same objective functions, TextGrad is used to design chemical formula of inorganic materials. The 3D crystal structures corresponding to chemical formulas were generated using the DiffCSP diffusion model [93], which was pretrained on the MP-20 dataset [88].

Hyperparameters. Inorganic materials design follows a general outer-loop framework coupled with a domain-specific optimizer, in which LLMs act as optimizers to iteratively mutate sequences and improve the associated objective scores. We initialize the process with 1,000 random chemical formulas of crystals from Materials Project database [88] and use a batch size of 20. For agents from different modes, the optimization is run for up to ten iterations, with early stopping governed by a selection agent; all experiments support this early-stopping mechanism. When reporting both optimization scores and held-out evaluation metrics, we apply min–max normalization to place all scores on a comparable and interpretable scale. Each baseline method also has three replicates under different random seeds.

Workflows. Our three different levels (co-pilot, semi-pilot, and autopilot) follow the default setting, discussed in Figure 3.4.1. For all experiments, we have three replicates.

High-level Goal

In the task of designing permanent magnets with low supply chain risk, high-level goal for SAGA agent is "Generate a set of chemical formulas of crystals for permanent magnet materials with high magnetic density and low supply chain risk."

In the task of designing superhard materials for precision cutting, high-level goal for SAGA agent is "Generate a set of chemical formulas of superhard materials for Ultra-Precision Cutting Tools."

Context Information

In two tasks, the context information for SAGA agent is "Please ensure that the proposed objectives are common and well-defined material properties in materials science. Please try to propose material properties that have not been considered in previous iterations but are relevant to the design goal."

S7.2 Material Property Evaluation

We performed density functional theory (DFT) computations employing the Vienna Ab initio Simulation Package (VASP) [102, 103] in conjunction with the projector augmented wave (PAW) approach. The calculations were implemented through atomate2 [60] and pymatgen [155] software packages. The computational setup adhered to Materials Project [88] standards, incorporating the Perdew–Burke–Ernzerhof (PBE) functional under the generalized gradient approximation (GGA) framework [162, 163]. The computational procedures for various properties are described below:

(1) The total energy and energy above hull were computed through the `DoubleRelaxMaker` and `StaticMaker` modules in atomate2 [60] using default configurations. This protocol comprises two consecutive structural relaxations followed by a static energy calculation.

(2) Magnetic densities of the generated structures were evaluated using the `DoubleRelaxMaker` and `StaticMaker` modules in atomate2 [60] with standard configurations. This procedure consists of two sequential relaxations and a static calculation. We define magnetic density as the ratio of total magnetization (magnetic moment) of the unit cell to its volume.

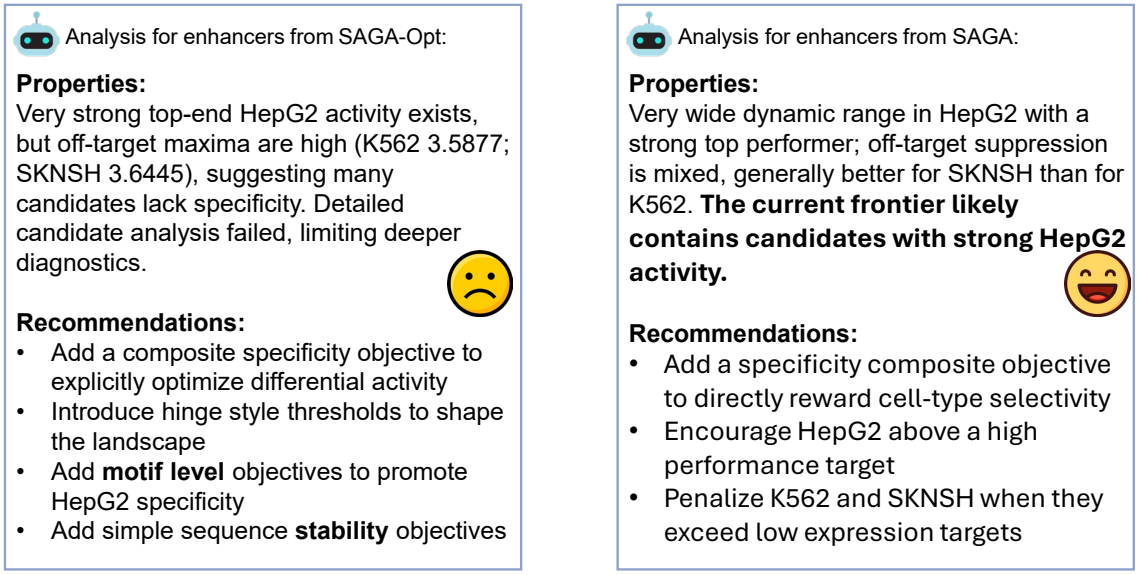
(3) Elastic modulus were determined via the `ElasticMaker` module in atomate2 [60] with standard configurations. First, the structure undergoes thorough structural optimization to achieve a nearly stress-free equilibrium configuration. Next, systematic deformations are introduced to the lattice parameters, and the corresponding stress tensors are computed using DFT calculations, with simultaneous optimization of atomic positions. The resulting stress-strain relationships are then fitted using linear elastic theory to determine the complete 6×6 elastic tensor. This tensor enables

the calculation of averaged mechanical properties, including the Voigt and Reuss estimates for bulk and shear moduli. Vickers hardness was calculated using Tian's empirical equation [208] based on DFT-computed bulk and shear moduli.

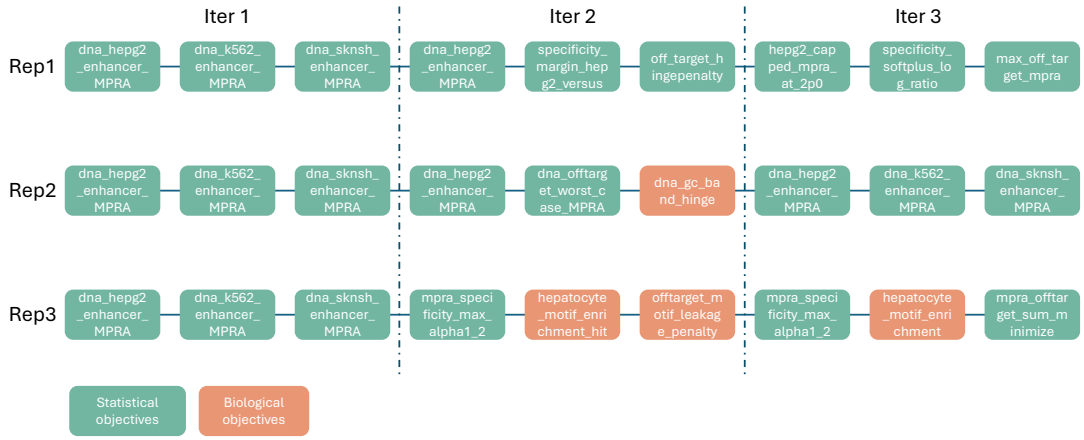
The Herfindahl-Hirschman index (HHI) scores based on geological reserves for crystals were calculated using the `HHIModel` class from the pymatgen package [155]. This compositionally-based metric, derived from geological and geopolitical data, quantifies resource-related economic factors and assesses the supply-demand risk of materials. Additionally, it measures the degree to which the constituent elements of a compound are geographically concentrated or dispersed. The HHI parameter is computed as the sum of squared market fractions (χ_i) for each country, based on either production ($\mathrm{HHI_P}$) or geological reserves ($\mathrm{HHI_R}$) of individual elements, using United States Geological Survey (USGS) commodity statistics [62]. For each composition, the weighted average $\mathrm{HHI_R}$ value was calculated using the weight fraction of each element in the chemical formula. According to the U.S. Department of Justice and Federal Trade Commission, markets are classified as unconcentrated (HHI $<$ 1500), moderately concentrated ($1500 \leq$ HHI ≤ 2500), or highly concentrated (HHI > 2500) for a given commodity. Lower HHI values are preferable, with materials having HHI scores below 1500 considered to exhibit low supply chain risk [62].

S8 Functional DNA Sequence Design

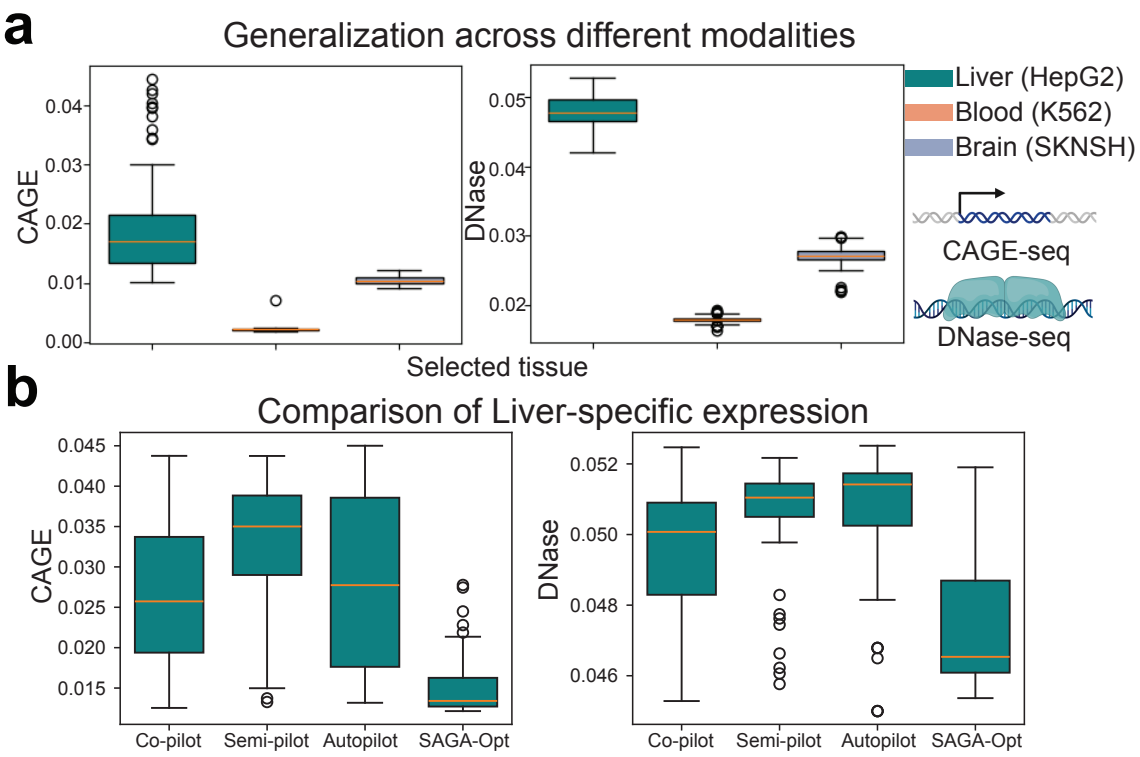
S8.1 Supplementary Figures



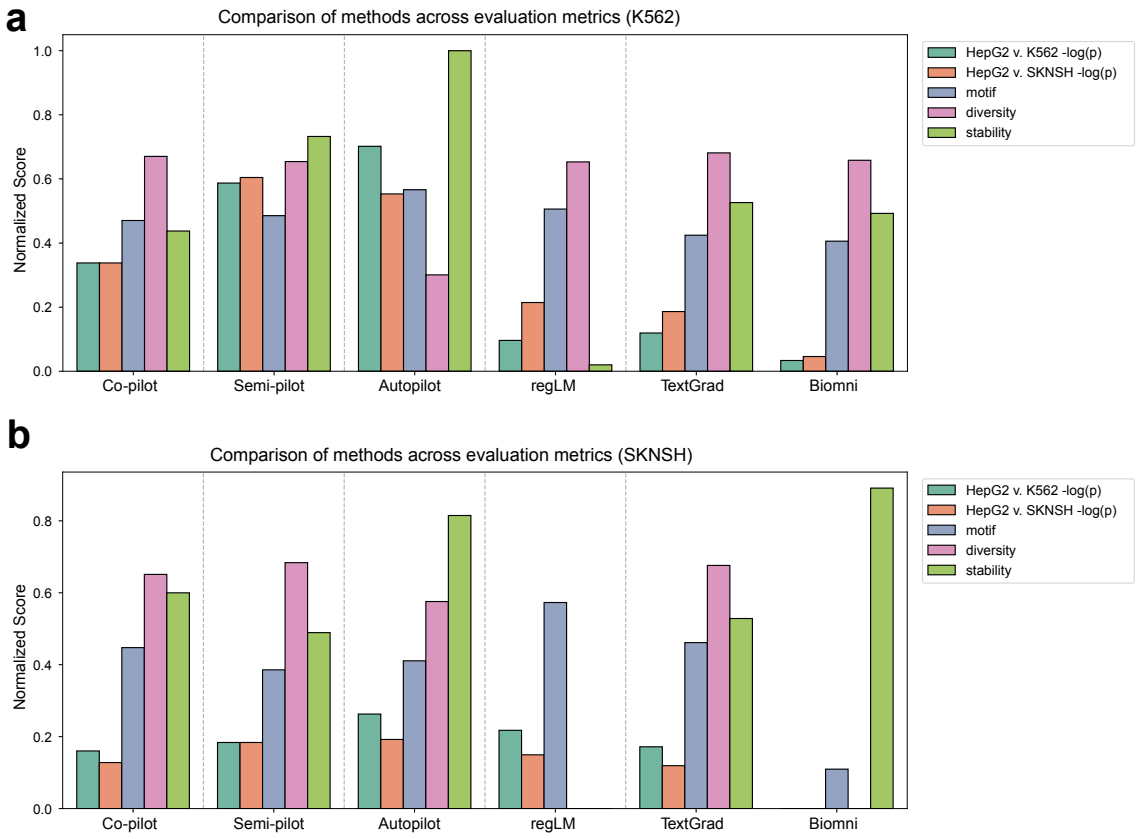
Supplementary Fig. S8.1: Illustration of analysis report generated by the analysis agent for two iterations.



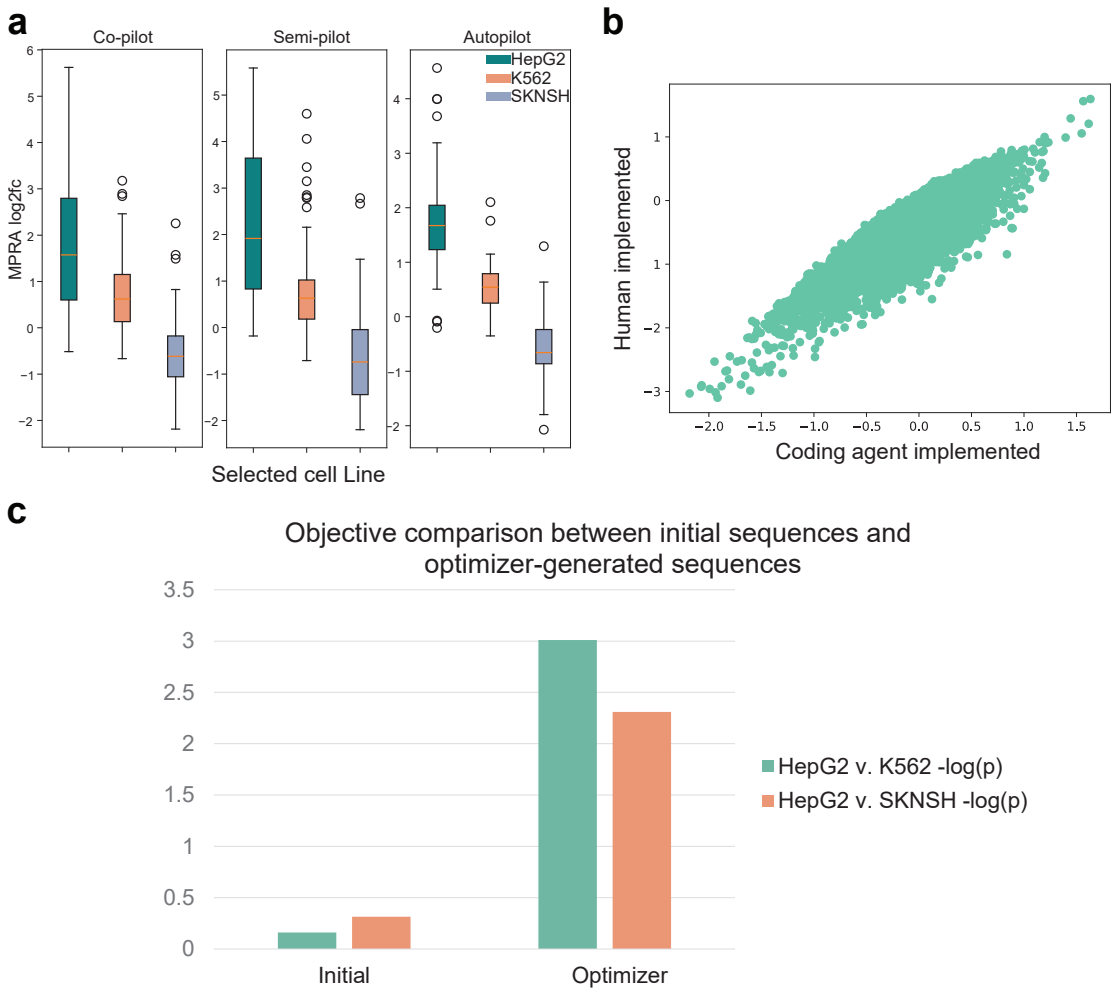
Supplementary Fig. S8.2: Objectives proposed in each iteration across three runs of the autopilot mode of SAGA .



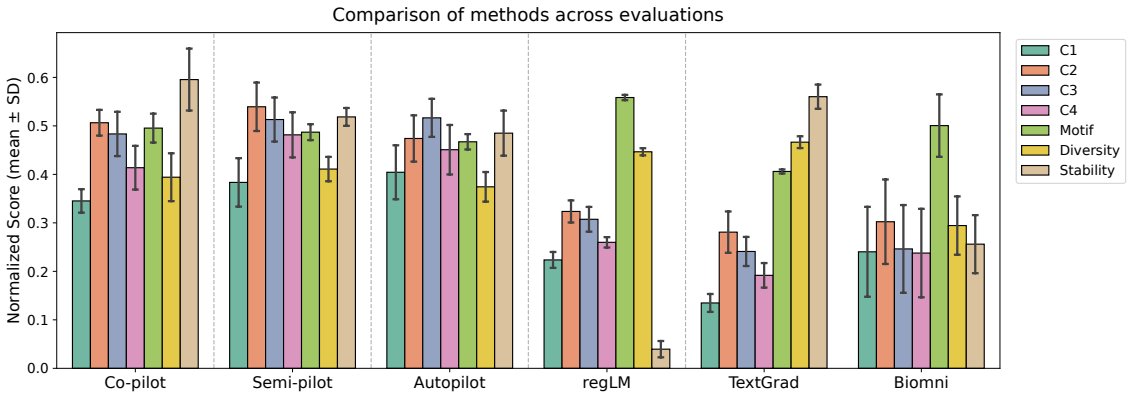
Supplementary Fig. S8.3: Cross-modality prediction analysis. (a) Consistency between the MPRA and other sequence expression measurement, including CAGE-seq and DNase-seq. The prediction is performed with Enformer. (b) Comparison of predicted CAGE-seq and DNase-seq expression levels from the enhancers generated by two different methods.



Supplementary Fig. S8.4: Comparison between SAGA and other baselines for (a) K562-specific enhancer design and (b) SKNSH-specific enhancer design.



Supplementary Fig. S8.5: Ablation studies for functional DNA sequence design. (a) MPRA expression of generated DNA sequences across different levels in SAGA (b) Correlation between human-proposed HepG2 scoring function and implementer-proposed HepG2 scoring function. (c) Validation of using LLMs and evolutionary algorithm as the optimizer for optimizing the cell-type-specific expression levels.



Supplementary Fig. S8.6: Comparison between SAGA and other baselines for promoter design across five different cell lines, and the variation is computed across five cell lines. The held-out metrics are consistent, while C1-C4 represents the -log(p-value) test result between the MPRA expression levels of the targeted cell line and the rest of cell lines.

S8.2 Experimental Setups

Objectives, metrics, and baselines

Here we describe the experimental setup for HepG2-specific enhancer design.

Initial objectives. Our initial objectives are:

- Maximize: HepG2-specific MPRA prediction score.
- Minimize: K562-specific MPRA prediction score.
- Minimize: SKNSH-specific MPRA prediction score.

To predict MPRA activity from generated DNA sequences, we fine-tune Enformer-based predictors using published MPRA datasets [8], with training, validation, and test splits performed at the chromosome level to prevent data leakage. The datasets used for enhancer design include MPRA measurements from three cell lines (HepG2, K562, and SKNSH) [65], while those for promoter design comprise five cell lines (K562, HepG2, SKNSH, GM12878, and A549) [168]. In both cases, the raw MPRA measurements are transformed into log-fold-change values prior to model training and evaluation.

Evaluation metrics. We adopt evaluation metrics that are standard in prior domain-specific studies on functional DNA sequence design and enhancer modeling [34, 41, 65, 106]. Importantly, all metrics are computed on held-out predictions and are not directly optimized during generation, ensuring a fair comparison across methods. Together, these metrics capture complementary aspects of statistical expression specificity, biological plausibility, and generative quality.

- **Specificity1 (HepG2 vs. K562 MPRA, $-\log p$)**. We quantify HepG2 specificity by applying a one-sided Wilcoxon rank-sum test between predicted HepG2 MPRA activities and K562 MPRA activities across the generated sequences. This choice mirrors experimental practice in MPRA studies, where statistical significance is used to assess whether a candidate enhancer shows cell-type-specific activity rather than global activation. Compared to simple score differences, a non-parametric test is robust to distributional shifts and outliers in model predictions, making it well suited for large, heterogeneous sequence sets. The metric ranges from 0 to ∞, with larger values indicating stronger and more statistically robust HepG2 specificity.

- **Specificity2 (HepG2 vs. SKNSH MPRA, $-\log p$)**. Analogous to Specificity1, we evaluate specificity against SKNSH, a neuronal cell line that is biologically distant from hepatocytes. Including a second, orthogonal off-target cell type ensures that designed enhancers are not merely suppressing hematopoietic programs, but instead exhibit broader hepatocyte-specific regulation. This dual-contrast design reduces the risk of overfitting to a single negative control and provides a more stringent assessment of cell-type specificity. As above, higher values indicate stronger specificity.

- **Motif enrichment score**. We assess motif enrichment by scanning designed sequences for known transcription factor binding sites (TFBSs) relevant to hepatocyte biology and computing the proportion of motif occurrences across the generated sequence set. The dataset of TFBSs is JASPAR18 [178]. This metric provides an explicit, interpretable link between sequence design and known regulatory mechanisms, complementing purely predictive MPRA-based scores. By grounding evaluation in curated TF motifs, motif enrichment serves as a biological plausibility check, ensuring that high-scoring sequences are consistent with established transcriptional programs rather than exploiting model artifacts. The metric ranges from 0 to #sequences $\times$ #motifs, with higher values indicating stronger enrichment.

- **Diversity score**. We compute diversity as the average pairwise Hamming distance across all generated sequences. This metric evaluates whether a method produces a diverse set of solutions rather than collapsing to a small number of high-scoring templates. Diversity is particularly important for regulatory sequence design, as multiple distinct sequence architectures can realize similar functional outputs in vivo. By explicitly measuring sequence-level variation, this metric discourages mode collapse and complements functional scores that alone could be optimized by near-duplicate sequences. The score ranges from 0 to ∞, with higher values indicating greater diversity.

- **Stability score (GC content).** We quantify sequence stability using the proportion of G/C nucleotides across the generated sequences. GC content is a well-established proxy for DNA thermodynamic stability due to increased hydrogen bonding and stacking interactions, and it has been widely used in prior sequence design work as a simple, interpretable constraint. While not a direct measure of enhancer activity, this metric helps ensure that generated sequences remain within a biologically reasonable compositional regime and avoids extreme or degenerate nucleotide distributions. The score ranges from 0 to 1, with higher values indicating higher GC content and increased stability.

Collectively, these metrics provide a balanced evaluation of functional specificity (Specificity1/2), mechanistic plausibility (motif enrichment), and generative quality (diversity and stability), reflecting both experimental and biological considerations in enhancer design.

Baselines. We benchmark against both state-of-the-art domain-specific methods and general-purpose AI agents. The baselines include regLM [106], TextGrad [247], and Biomni [81]. Details of each baseline are described below:

- **regLM** is a fine-tuned genomic language model (base model: HyenaDNA [153]). We finetune this model with same datasets used to train the predictor for MPRA prediction. We then generate 5,000 candidate HepG2-specific enhancers using regLM and randomly select 20 sequences for evaluation.

- **TextGrad** is an LLM-based optimization framework. Initialized with the same objective functions, TextGrad is used to design 20 HepG2-specific enhancer sequences.

- **Biomni** is an autonomous AI scientist designed for general biomedical tasks. Given the same objective specifications, Biomni is used to generate 20 HepG2-specific enhancer sequences.

Hyperparameters. Functional DNA sequence generation follows a general outer-loop framework coupled with a domain-specific optimizer, in which large language models act as optimizers to iteratively mutate sequences and improve the associated objective scores. We initialize the process with 5,000 random DNA sequences and use a batch size of 20. For agents from different modes, the optimization is run for up to three iterations, with early stopping governed by a selection agent; all experiments support this early-stopping mechanism. The initial objectives are defined by predicted MPRA expression levels from a trained predictor. When reporting both optimization scores and held-out evaluation metrics, we apply min–max normalization to place all scores on a comparable and interpretable scale. Each baseline method also has three replicates under different random seeds.

Workflows. Our three different levels (co-pilot, semi-pilot, and autopilot) follow the default setting, discussed in Figure 3.4.1. For all experiments, we have three replicates.

High-level Goal

Here we use HepG2 as an example: Generate a set of cell-type-specific enhancers for the HepG2 cell line, each with a length of 200 base pairs.

Context Information

Here we use HepG2 as an example: For this task, the enhancers should be specific to the HepG2 cell line, meaning they should drive high expression in HepG2 cells while minimizing expression in other cell lines (e.g., K562 and SKNSH). The sequences should also be diverse to cover a broad range of potential enhancer activities. You can consider including objectives related to known enhancer motifs and stability of DNA sequences. The optimizer will automatically enforce the length constraint, so do not propose any objectives related to enhancer length.

S8.3 Additional Experimental Results

Experiments for all cell lines

Examination of cell-type specificity in MPRA expression. According to Figure S8.5 (a), the designed HepG2-specific enhancers from SAGA all have obvious specificity, which is equivalent to the cell-type-specific expressions in predicted MPRA score.

Examination of evaluation metrics in enhancer design for different cell types. We further evaluate the performance of SAGA in designing K562-specific and SKNSH-specific enhancers, with results summarized in Figure S8.4(a) and (b). These results demonstrate that our system generalizes effectively across cell lines, consistently preserving both MPRA-based specificity and biology-driven metrics. In terms of average performance, SAGA outperforms all baselines by at least 16.3% in the K562 cell line and by at least 1.5% in the SKNSH cell line.

Examination of evaluation metrics in promoter design for different cell types. We also examine the performances of SAGA in designing cell-type-specific promoters across five different cell lines, including K562, HepG2, GM12878, SKNSH, and A549. The results (Co-pilot, Semi-pilot, and Autopliot versus other baselines) are summarized in Figure S8.6. SAGA can also generalize promoters with good quality and specificity, which further supports the capacity of our method in handling tasks from different modalities.

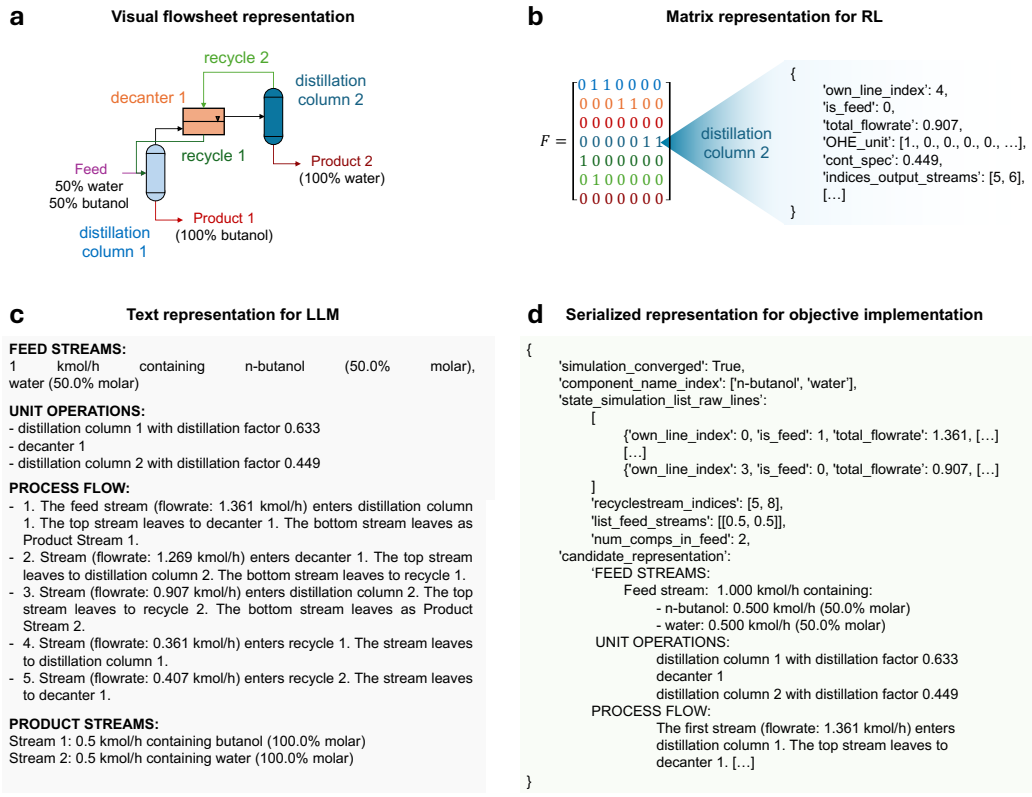
Ablation Studies

Evaluation of implementer. We test the ability of implementer by validating whether it can propose similar objectives that have human implementation. Here, we focus on one example of predicting the difference between HepG2's MPRA and other cell lines' MPRA. Figure S8.5 (b) also shows high and significant correlations, confirming that implementer can internalize the structure of the design landscape and construct objectives that reflect biological ground truth.

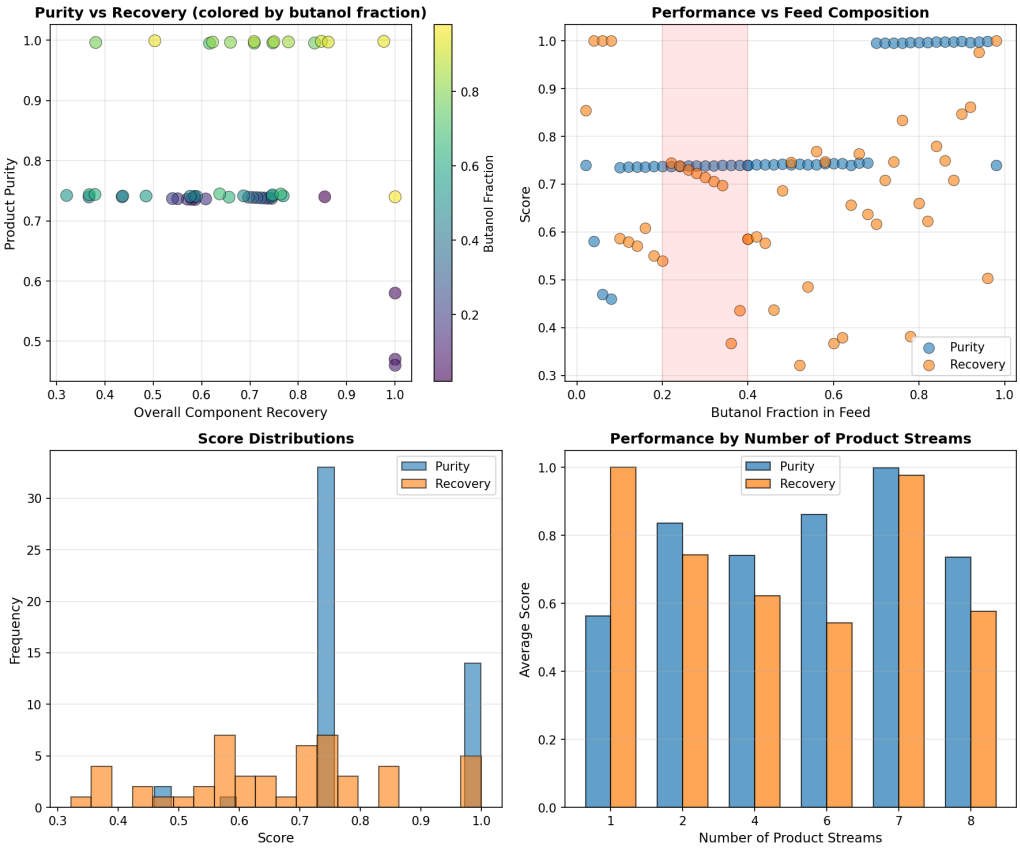
Evaluation of optimizer. To validate whether our optimizer can work as we expect, we include an ablation study to check the performance of this agent in improving the initial objectives versus the initial populations, which are random DNA sequences. As shown in Figure S8.5 (c), the optimizer can already identify HepG2-specific enhancer patterns when considering MPRA differences alone. However, because the optimizer cannot balance competing biological constraints, we need to integrate a planner, analyzer, implementer, and selector to propose more complicated objectives and produce final candidates. This motivates the development of more advanced agents.

S9 Chemical Process Design

S9.1 Supplementary Figures



Supplementary Fig. S9.1: Different representations of an exemplary chemical process: (a) Visual representation that is created manually for illustration purposes; (b) matrix representation used within the RL optimizer [66], whereas the entries correspond to connection between the unit operations and product streams (colors correspond to (a); (c) text representation that is automatically generated based on (b) for use within SAGA ; (d) serialized process representation that is automatically generated from (b) and (c), and used to implement objectives by SAGA .



Supplementary Fig. S9.2: Exemplary plot for process candidate analysis automatically generated by SAGA .

S9.2 Experimental Setups

Objectives, metrics, and baselines

Here we explain the experimental details of chemical process design.

Initial objectives. Our initial objective is maximize the product purity.

Evaluation metrics. We use three objectives as our evaluation metrics, which we implement based on the short-cut process simulation models in [66]:

- Product purity: average purity of the output/product streams, whereas streams are rewarded if they have a molar composition $x > 0.9$, with extra rewards for $x > \{0.95, 0.97, 0.99\}$. We choose this metric because achieving pure product streams is the main goal in designing separation processes. This metric is naturally normalized to the range from 0 to 1, and higher is better.

- Capital costs: negative sum of costs of process unit operations. Notably, the costs for the individual unit operations are based on simple heuristics, similar to the work [66]. We choose this metric because low capital costs are a key goal in chemical process design. This metric is normalized to the range from 0 to 1, and higher is better (as we negate the sum).

- Material flow intensity: negative penalty for process streams smaller than 1% of the feed stream and for (excessive) recycle streams greater than the feed stream. We choose this metric because excessive recycle ratios and very small streams lead to practical issues in process operation and equipment design. This metric is normalized to the range from 0 to 1, and higher is better (as we negate the penalty).

Evaluation set. The RL agent is evaluated on a set of processes covering the discretized composition range $x \in [0, 1]$ with a step size of $\Delta x = 0.02$. Thus, each evaluation covers a wide range of butanol/water mixtures, testing whether the RL agent can design chemical process flowsheets for varying feed compositions.

RL optimizer. For the optimization in the inner loop of SAGA , we use an RL agent based on the framework by Göttl et al. [66]. This RL agent operates on matrix representations of process flowsheets, whereas the rows and columns represent unit operations, recycles, and product streams; the entries in the matrix correspond to connections, i.e., material flows. The flowsheet design is then formulated as a sequential planning problem by selecting unit operations and corresponding material flows. Specifically, the design space comprises four unit operations, each with its own specifications: a distillation column, a decanter, a mixer, and a splitter. The design also determines the flow structure of the feed stream to these unit operations and classifies intermediate streams as inputs to additional unit operations or final

output streams. Each action is determined in a hierarchical manner including both discrete variables, e.g., selecting a distillation column as unit operation, and continuous variables, e.g., specifying the ratio of input to distillate flow rate within the distillation column. Material flows can also include recycles, which affect pervious process states, and thus require RL agents to plan ahead Göttl et al. [66]. The reward for the agent is based on evaluating the designed processes with short-cut process models and associated objectives.

Baselines. We consider the RL agent [66] trained only on product purity as baseline, as the purity is the main objective and typically further objectives would be added manually in an iterative fashion by human experts. We run the baseline three times under different random seeds. Furthermore, we considered adding TextGrad as a baseline. However, since the flowsheet design also requires determining continuous unit operations parameters, such as distillation factors and recycle ratios, to which the process (simulation) can be quite sensitive, we did not further consider purely LLM-guided process design. Notably, this topic is still highly underexplored due to the inherent complexity of process representations.

Hyperparameters. For the RL agent, we use the code base and hyperparameters from the original work in [66]. However, we only consider butanol/water mixtures without the possibility to add solvents. Since this decreases the problem complexity compared to training an RL agent to design flowsheets for different mixtures using solvents, and in order to save computational resources, we reduce the number of training batches from 10,000 to 1,000. For the outer loop, we use the settings for the analyzer, planner, and implementer as described in the main text, and run it for three iterations. We repeat each experiment three times under different random seeds.

Workflows. Our three different levels (co-pilot, semi-pilot, and autopilot) follow the default setting, discussed in SI 3.4.1. For all experiments, we have three replicates.

High-level goal

We provide the following goal as prompt to SAGA: "Design chemical process flowsheets for the separation of binary aezeotropic mixtures, i.e., separating a feed stream with two components into high purity streams."

Context

In addition, we provide the following context within the prompt: "For this task, the chemical processes will be generated by a reinforcement learning agent that will be trained to optimize your provided objective scores. Please note that the training starts from scratch in each iteration (each time new objectives are provided), so

the agent has to learn the process design from scratch in each iteration and should account for the main objective, i.e., the initial objective product purity should be kept as the focus. The agent is then tested to design separation processes for a set of different feed compositions, which form the population. Note that the designed processes are evaluated with short-cut models that only converge if a physically consistent process was designed, i.e., you do not need to consider convergence issues and physical consistency, e.g., mass balances, as part of your analysis or the objectives. The designed process flowsheets should fulfill early-stage process design goals relevant for practical application and further refined process design. The central goal for this is to get very high purity streams. Other relevant goals for efficient chemical processes and their implementation in practice should also be considered. For this, you can consider including objectives related to known process design goals and existing separation processes. Note that it is important to analyze all processes in the population as they have different feed compositions. Please also note that training the reinforcement learning agent is computationally expensive and not always stable, e.g., sensitive to the selected objective weights or the combination of certain objectives. This also includes that adding too many new objectives for a new iteration might lead to unstable training, so consider using less than the maximum number of objectives, especially in early iterations."

S9.3 Challenges in chemical process design

Using SAGA for chemical process design comes with several challenges including:

1. Text-based representation of chemical processes are complex, as they contain topological process information but also detailed specifications of process units, cf. S9.1 and [61, 66], making analysis and implementing design objectives highly difficult. Notably, Simplified Flowsheet Input-Line Entry-System (SFILES) – inspired by SMILES for molecules – were developed for representing chemical processes as text in a semantic, standardized way [40]. SFILES have also recently been adapted to work with transformer models [138, 216, 217]. Yet, they are rarely explored and usable for LLMs so far, which is why we use a simple, more intuitive text representation.

2. Process evaluations are based on simulation models that can be computationally expensive and sensitive to (continuous) process parameters [61].

3. Process design objectives can be conflicting, e.g., high purity versus capital costs.

4. The design relies on an RL agent which is sensitive to the objectives and their weights, thus can converge to non-practical process designs.

Therefore, we consider a simple separation process design case study, whereas our evaluation focuses on whether the SAGA framework is able to guide chemical process design by identifying process issues and suitable objectives. We stress that we expect variations with respect to the objectives on the designed processes along the iterations, due to the sensitivity and partly instability of the RL agent used for optimization.

References

[1] Abdelaal, T., Michielsen, L., Cats, D., Hoogduin, D., Mei, H., Reinders, M. J., and Mahfouz, A. (2019). A comparison of automatic cell identification methods for single-cell rna sequencing data. *Genome biology*, 20:1–19. 11

[2] Abdi, H. and Williams, L. J. (2010). Principal component analysis. *Wiley interdisciplinary reviews: computational statistics*, 2(4):433–459. 86

[3] Agarwal, V., Inoue, F., Schubach, M., Penzar, D., Martin, B. K., Dash, P. M., Keukeleire, P., Zhang, Z., Sohota, A., Zhao, J., et al. (2025). Massively parallel characterization of transcriptional regulatory elements. *Nature*, 639(8054):411–420. 143

[4] Ampuja, M., Rantapero, T., Rodriguez-Martinez, A., Palmroth, M., Alarmo, E., Nykter, M., and Kallioniemi, A. (2017). Integrated rna-seq and dnase-seq analyses identify phenotype-specific bmp4 signaling in breast cancer. *Bmc Genomics*, 18(1):68. 144

[5] Andersson, R. and Sandelin, A. (2020). Determinants of enhancer and promoter activities of regulatory elements. *Nature Reviews Genetics*, 21(2):71–87. 141

[6] Antolini, L., Boracchi, P., and Biganzoli, E. (2005). A time-dependent discrimination index for survival data. *Statistics in medicine*, 24(24):3927–3944. 103

[7] Armingol, E., Officer, A., Harismendy, O., and Lewis, N. E. (2021). Deciphering cell–cell interactions and communication from gene expression. *Nature Reviews Genetics*, 22(2):71–88. 2, 26

[8] Avsec, Ž., Agarwal, V., Visentin, D., Ledsam, J. R., Grabska-Barwinska, A., Taylor, K. R., Assael, Y., Jumper, J., Kohli, P., and Kelley, D. R. (2021). Effective gene expression prediction from sequence by integrating long-range interactions. *Nature methods*, 18(10):1196–1203. 2, 144, 178

[9] Badia-i Mompel, P., Wessels, L., Müller-Dott, S., Trimbour, R., Ramirez Flores, R. O., Argelaguet, R., and Saez-Rodriguez, J. (2023). Gene regulatory network inference in the era of single-cell multi-omics. *Nature Reviews Genetics*, pages 1–16. 17

[10] Bang, Y., Cahyawijaya, S., Lee, N., Dai, W., Su, D., Wilie, B., Lovenia, H., Ji, Z., Yu, T., Chung, W., Do, Q. V., Xu, Y., and Fung, P. (2023). A multitask, multilingual, multimodal evaluation of ChatGPT on reasoning, hallucination, and interactivity. In Park, J. C., Arase, Y., Hu, B., Lu, W., Wijaya, D., Purwarianti, A., and Krisnadhi, A. A., editors, *Proceedings of the 13th International Joint Conference on Natural Language Processing and the 3rd Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 675–718, Nusa Dua, Bali. Association for Computational Linguistics. 29

[11] Biancalani, T., Scalia, G., Buffoni, L., Avasthi, R., Lu, Z., Sanger, A., Tokcan, N., Vanderburg, C. R., Segerstolpe, Å., Zhang, M., et al. (2021). Deep learning and alignment of spatially resolved single-cell transcriptomes with tangram. *Nature methods*, 18(11):1352–1362. 19, 32

[12] Bibal, A., Cardon, R., Alfter, D., Wilkens, R., Wang, X., François, T., and Watrin, P. (2022). Is attention explanation? an introduction to the debate. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3889–3900, Dublin, Ireland. Association for Computational Linguistics. 27

[13] Bickerton, G. R., Paolini, G. V., Besnard, J., Muresan, S., and Hopkins, A. L. (2012). Quantifying the chemical beauty of drugs. *Nature chemistry*, 4(2):90–98. 166

[14] Biegler, L. T., Grossmann, I. E., and Westerberg, A. W. (1997). *Systematic methods for chemical process design*. Prentice Hall, Old Tappan, NJ (United States). 144, 152

[15] Blampey, Q., Benkirane, H., Bercovici, N., Andre, F., and Cournede, P.-H. (2024). Novae: a graph-based foundation model for spatial transcriptomics data. *bioRxiv*, pages 2024–09. 21, 87, 104

[16] Boyle, A. P., Davis, S., Shulha, H. P., Meltzer, P., Margulies, E. H., Weng, Z., Furey, T. S., and Crawford, G. E. (2008). High-resolution mapping and characterization of open chromatin across the genome. *Cell*, 132(2):311–322. 144

[17] Brandt, D. Y., Aguiar, V. R., Bitarello, B. D., Nunes, K., Goudet, J., and Meyer, D. (2015). Mapping bias overestimates reference allele frequencies at the hla genes in the 1000 genomes project phase i data. *G3: Genes, Genomes, Genetics*, 5(5):931–941. 18

[18] Brown, E. D. and Wright, G. D. (2016). Antibacterial drug discovery in the resistance era. *Nature*, 529(7586):336–343. 135

[19] Butina, D. (1999). Unsupervised data base clustering based on daylight's fingerprint and tanimoto similarity: A fast and automated way to cluster small and large

data sets. *Journal of Chemical Information and Computer Sciences*, 39(4):747–750. 151, 167

[20] Büttner, M., Miao, Z., Wolf, F. A., Teichmann, S. A., and Theis, F. J. (2019). A test metric for assessing single-cell rna-seq batch correction. *Nature methods*, 16(1):43–49. 30

[21] Campanella, G., Chen, S., Singh, M., Verma, R., Muehlstedt, S., Zeng, J., Stock, A., Croken, M., Veremis, B., Elmas, A., et al. (2025). A clinical benchmark of public self-supervised pathology foundation models. *Nature Communications*, 16(1):3640. 98

[22] Cang, Z., Zhao, Y., Almet, A. A., Stabell, A., Ramos, R., Plikus, M. V., Atwood, S. X., and Nie, Q. (2023). Screening cell–cell communication in spatial transcriptomics via collective optimal transport. *Nature methods*, 20(2):218–228. 93, 102

[23] Cao, Z.-J. and Gao, G. (2022). Multi-omics single-cell data integration and regulatory inference with graph-linked embedding. *Nature Biotechnology*, 40(10):1458–1466. 10

[24] Carta, T., Romac, C., Wolf, T., Lamprier, S., Sigaud, O., and Oudeyer, P.-Y. (2023). Grounding large language models in interactive environments with online reinforcement learning. In *International Conference on Machine Learning*, pages 3676–3713. PMLR. 26

[25] Cavanagh, J. M., Sun, K., Gritsevskiy, A., Bagni, D., Bannister, T. D., and Head-Gordon, T. (2024). Smileyllama: Modifying large language models for directed chemical space exploration. *ArXiv*, 2409.02231(in review). 133

[26] Chang, Y., Wang, X., Wang, J., Wu, Y., Zhu, K., Chen, H., Yang, L., Yi, X., Wang, C., Wang, Y., Ye, W., Zhang, Y., Chang, Y., Yu, P. S., Yang, Q., and Xie, X. (2023). A survey on evaluation of large language models. *arXiv preprint arXiv:2307.03109*. 29

[27] Chen, J., Guo, J., Fako, E., and Schwaller, P. (2025a). Accelerating inverse materials design using generative diffusion models with reinforcement learning. *arXiv preprint arXiv:2511.03112*. 138, 153

[28] Chen, J., Huang, X., Hua, C., He, Y., and Schwaller, P. (2025b). A multi-modal transformer for predicting global minimum adsorption energy. *Nature Communications*, 16(1):3232. 138

[29] Chen, J., Xu, H., Tao, W., Chen, Z., Zhao, Y., and Han, J.-D. J. (2023a). Transformer for one stop interpretable cell type annotation. *Nature Communications*, 14(1):223. 2, 11

[30] Chen, L., Zaharia, M., and Zou, J. (2023b). How is chatgpt's behavior changing over time? *arXiv preprint arXiv:2307.09009*. 26

[31] Chen, R. J., Ding, T., Lu, M. Y., Williamson, D. F., Jaume, G., Song, A. H., Chen, B., Zhang, A., Shao, D., Shaban, M., et al. (2024a). Towards a general-purpose foundation model for computational pathology. *Nature Medicine*, 30(3):850–862. 84, 87, 96

[32] Chen, W., Zhang, P., Tran, T. N., Xiao, Y., Li, S., Shah, V. V., Cheng, H., Brannan, K. W., Youker, K., Lai, L., et al. (2025c). A visual–omics foundation model to bridge histopathology with spatial transcriptomics. *Nature Methods*, pages 1–15. 87

[33] Chen, X., Liang, C., Huang, D., Real, E., Wang, K., Pham, H., Dong, X., Luong, T., Hsieh, C.-J., Lu, Y., et al. (2024b). Symbolic discovery of optimization algorithms. *Advances in Neural Information Processing Systems*, 36. 9

[34] Chen, X., Ma, S., Lin, R., Lin, J., and Wang, B. (2025d). Ctrl-dna: Controllable cell-type-specific regulatory dna design via constrained rl. *arXiv preprint arXiv:2505.20578*. 153, 178

[35] Chen, X., Miragaia, R. J., Natarajan, K. N., and Teichmann, S. A. (2018). A rapid and robust method for single cell chromatin accessibility profiling. *Nature communications*, 9(1):1–9. 2

[36] Chen, Y. and Zou, J. (2025). Simple and effective embedding model for single-cell biology built from chatgpt. *Nature biomedical engineering*, 9(4):483–493. 26

[37] Choudhary, K. and DeCost, B. (2021). Atomistic line graph neural network for improved materials property predictions. *npj Computational Materials*, 7(1):185. 170, 171

[38] Cui, H., Wang, C., Maan, H., Pang, K., Luo, F., Duan, N., and Wang, B. (2024). scgpt: toward building a foundation model for single-cell multi-omics using generative ai. *Nature Methods*, pages 1–11. 3, 17, 18, 84, 97

[39] Cusanovich, D. A., Daza, R., Adey, A., Pliner, H. A., Christiansen, L., Gunderson, K. L., Steemers, F. J., Trapnell, C., and Shendure, J. (2015). Multiplex single-cell profiling of chromatin accessibility by combinatorial cellular indexing. *Science*, 348(6237):910–914. 2

[40] d'Anterroches, L. (2006). *Process flow sheet generation & design through a group contribution approach*. PhD thesis, Technical University of Denmark. 187

[41] DaSilva, L. F., Senan, S., Patel, Z. M., Reddy, A. J., Gabbita, S., Nussbaum, Z., Córdova, C. M. V., Wenteler, A., Weber, N., Tunjic, T. M., et al. (2024). Dna-diffusion: leveraging generative models for controlling chromatin accessibility and gene expression via synthetic regulatory elements. *Biorxiv*. 144, 153, 178

[42] Davis, M. M., Tato, C. M., and Furman, D. (2017). Systems immunology: just getting started. *Nature immunology*, 18(7):725–732. 19

[43] de Boer, C. G. and Taipale, J. (2024). Hold out the genome: a roadmap to solving the cis-regulatory code. *Nature*, 625(7993):41–50. 152

[44] Deshpande, A., Chu, L.-F., Stewart, R., and Gitter, A. (2022). Network inference with granger causality ensembles on single-cell transcriptomics. *Cell reports*, 38(6):110333. 32

[45] Ding, N., Qin, Y., Yang, G., Wei, F., Yang, Z., Su, Y., Hu, S., Chen, Y., Chan, C.-M., Chen, W., et al. (2023). Parameter-efficient fine-tuning of large-scale pre-trained language models. *Nature Machine Intelligence*, 5(3):220–235. 26

[46] Ding, T., Wagner, S. J., Song, A. H., Chen, R. J., Lu, M. Y., Zhang, A., Vaidya, A. J., Jaume, G., Shaban, M., Kim, A., et al. (2024). Multimodal whole slide foundation model for pathology. *arXiv preprint arXiv:2411.19666*. 87, 93

[47] Dixit, A., Parnas, O., Li, B., Chen, J., Fulco, C. P., Jerby-Arnon, L., Marjanovic, N. D., Dionne, D., Burks, T., Raychowdhury, R., et al. (2016). Perturb-seq: dissecting molecular circuits with scalable single-cell rna profiling of pooled genetic screens. *cell*, 167(7):1853–1866. 32

[48] Driess, D., Xia, F., Sajjadi, M. S., Lynch, C., Chowdhery, A., Ichter, B., Wahid, A., Tompson, J., Vuong, Q., Yu, T., et al. (2023). Palm-e: An embodied multimodal language model. *arXiv preprint arXiv:2303.03378*. 2

[49] Du, J., Jia, P., Dai, Y., Tao, C., Zhao, Z., and Zhi, D. (2019). Gene2vec: distributed representation of genes based on co-expression. *BMC genomics*, 20:7–15. 13

[50] Du, W. and Yang, S. (2025). The potential and challenges of large language model agent systems in chemical process simulation: from automated modeling to intelligent design. *Frontiers of Chemical Science and Engineering*, 19(10):99. 144

[51] Du, Y., Yu, B., Liu, T., Shen, T., Chen, J., Rittig, J. G., Sun, K., Zhang, Y., Song, Z., Zhou, B., et al. (2025). Accelerating scientific discovery with autonomous goal-evolving agents. *arXiv preprint arXiv:2512.21782*. 131

[52] Dyer, S. C., Austine-Orimoloye, O., Azov, A. G., Barba, M., Barnes, I., Barrera-Enriquez, V. P., Becker, A., Bennett, R., Beracochea, M., Berry, A., et al. (2025). Ensembl 2025. *Nucleic Acids Research*, 53(D1):D948–D957. 21

[53] Edwards, C., Lai, T., Ros, K., Honke, G., Cho, K., and Ji, H. (2022). Translation between molecules and natural language. 167

[54] Evrony, G. D., Hinch, A. G., and Luo, C. (2021). Applications of single-cell dna sequencing. *Annual review of genomics and human genetics*, 22:171–197. 2

[55] Fabregat, A., Jupe, S., Matthews, L., Sidiropoulos, K., Gillespie, M., Garapati, P., Haw, R., Jassal, B., Korninger, F., May, B., et al. (2018). The reactome pathway knowledgebase. *Nucleic acids research*, 46(D1):D649–D655. 17, 18

[56] Feregrino, C. and Tschopp, P. (2022). Assessing evolutionary and developmental transcriptome dynamics in homologous cell types. *Developmental dynamics*, 251(9):1472–1489. 17

[57] Flynn, E., Almonte-Loya, A., and Fragiadakis, G. K. (2023). Single-cell multiomics. *Annual Review of Biomedical Data Science*, 6. 2

[58] Franzén, O., Gan, L.-M., and Björkegren, J. L. (2019). Panglaodb: a web server for exploration of mouse and human single-cell rna sequencing data. *Database*, 2019:baz046. 6

[59] Fu, X. and Chen, Y. (2024). Pix2path: Integrating spatial transcriptomics and digital pathology with deep learning to score pathological risk and link gene expression to disease mechanisms. *bioRxiv*, pages 2024–08. 87, 104

[60] Ganose, A. M., Sahasrabuddhe, H., Asta, M., Beck, K., Biswas, T., Bonkowski, A., Bustamante, J., Chen, X., Chiang, Y., Chrzan, D. C., Clary, J., Cohen, O. A., Ertural, C., Gallant, M. C., George, J., Gerits, S., Goodall, R. E. A., Guha, R. D., Hautier, G., Horton, M., Inizan, T. J., Kaplan, A. D., Kingsbury, R. S., Kuner, M. C., Li, B., Linn, X., McDermott, M. J., Mohanakrishnan, R. S., Naik, A. A., Neaton, J. B., Parmar, S. M., Persson, K. A., Petretto, G., Purcell, T. A. R., Ricci, F., Rich, B., Riebesell, J., Rignanese, G.-M., Rosen, A. S., Scheffler, M., Schmidt, J., Shen, J.-X., Sobolev, A., Sundararaman, R., Tezak, C., Trinquet, V., Varley, J. B., Vigil-Fowler, D., Wang, D., Waroquiers, D., Wen, M., Yang, H., Zheng, H., Zheng, J., Zhu, Z., and Jain, A. (2025). Atomate2: modular workflows for materials science. *Digital Discovery*. 172

[61] Gao, Q. and Schweidtmann, A. M. (2024). Deep reinforcement learning for process design: Review and perspective. *Current Opinion in Chemical Engineering*, 44:101012. 144, 152, 187

[62] Gaultois, M. W., Sparks, T. D., Borg, C. K., Seshadri, R., Bonificio, W. D., and Clarke, D. R. (2013). Data-driven review of thermoelectric materials: performance and resource considerations. *Chemistry of Materials*, 25(15):2911–2920. 138, 173

[63] Gaulton, A., Bellis, L. J., Bento, A. P., Chambers, J., Davies, M., Hersey, A., Light, Y., McGlinchey, S., Michalovich, D., Al-Lazikani, B., et al. (2012). Chembl: a large-scale bioactivity database for drug discovery. *Nucleic acids research*, 40(D1):D1100–D1107. 166

[GitHub] GitHub. Pycox. 92

[65] Gosai, S. J., Castro, R. I., Fuentes, N., Butts, J. C., Mouri, K., Alasoadura, M., Kales, S., Nguyen, T. T. L., Noche, R. R., Rao, A. S., et al. (2024). Machine-guided design of cell-type-targeting cis-regulatory elements. *Nature*, 634(8036):1211–1220. 141, 144, 153, 178

[66] Göttl, Q., Pirnay, J., Burger, J., and Grimm, D. G. (2025). Deep reinforcement learning enables conceptual design of processes for separating azeotropic mixtures without prior knowledge. *Computers & Chemical Engineering*, 194:108975. xxiii, 144, 152, 154, 183, 185, 186, 187

[67] Gottweis, J., Weng, W.-H., Daryin, A., Tu, T., Palepu, A., Sirkovic, P., Myaskovsky, A., Weissenberger, F., Rong, K., Tanno, R., et al. (2025). Towards an ai co-scientist. *arXiv preprint arXiv:2502.18864*. 133

[68] Guerrini, M. M., Oguchi, A., Suzuki, A., and Murakawa, Y. (2022). Cap analysis of gene expression (cage) and noncoding regulatory elements. In *Seminars in Immunopathology*, volume 44, pages 127–136. Springer. 144

[69] Han, X., Zhou, Z., Fei, L., Sun, H., Wang, R., Chen, Y., Chen, H., Wang, J., Tang, H., Ge, W., et al. (2020). Construction of a human cell landscape at single-cell level. *Nature*, 581(7808):303–309. 2

[70] Hao, M., Gong, J., Zeng, X., Liu, C., Guo, Y., Cheng, X., Wang, T., Ma, J., Zhang, X., and Song, L. (2024). Large-scale foundation model on single-cell transcriptomics. *Nature Methods*, pages 1–11. 3, 21, 27, 30

[71] Hao, Y., Hao, S., Andersen-Nissen, E., Mauck, W. M., Zheng, S., Butler, A., Lee, M. J., Wilk, A. J., Darby, C., Zager, M., et al. (2021). Integrated analysis of multimodal single-cell data. *Cell*, 184(13):3573–3587. 25

[72] Hecker, M., Lambeck, S., Toepfer, S., Van Someren, E., and Guthke, R. (2009). Gene regulatory network inference: data integration in dynamic models—a review. *Biosystems*, 96(1):86–103. 32

[73] Heid, E., Greenman, K. P., Chung, Y., Li, S.-C., Graff, D. E., Vermeire, F. H., Wu, H., Green, W. H., and McGill, C. J. (2024). Chemprop: A machine learning package for chemical property prediction. *Journal of Chemical Information and Modeling*, 64(1):9–17. PMID: 38147829. 165, 166

[74] Heimberg, G., Kuo, T., DePianto, D. J., Salem, O., Heigl, T., Diamant, N., Scalia, G., Biancalani, T., Turley, S. J., Rock, J. R., et al. (2024). A cell atlas foundation model for scalable search of similar human cells. *Nature*, pages 1–3. 3, 12

[75] Hernandez, D., Kaplan, J., Henighan, T., and McCandlish, S. (2021). Scaling laws for transfer. *arXiv preprint arXiv:2102.01293*. 21, 28, 34

[76] Heumos, L., Schaar, A. C., Lance, C., Litinetskaya, A., Drost, F., Zappia, L., Lücken, M. D., Strobl, D. C., Henao, J., Curion, F., et al. (2023). Best practices for single-cell analysis across modalities. *Nature Reviews Genetics*, pages 1–23. 2

[77] Hou, W., Ji, Z., Ji, H., and Hicks, S. C. (2020). A systematic evaluation of single-cell rna-sequencing imputation methods. *Genome biology*, 21:1–30. 32

[78] Hu, E. J., yelong shen, Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. (2022). LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*. 26

[79] Hu, J., Li, X., Coleman, K., Schroeder, A., Ma, N., Irwin, D. J., Lee, E. B., Shinohara, R. T., and Li, M. (2021). Spagcn: Integrating gene expression, spatial location and histology to identify spatial domains and spatially variable genes by graph convolutional network. *Nature methods*, 18(11):1342–1351. 87, 100

[80] Hu, Z., Wang, L., Lan, Y., Xu, W., Lim, E.-P., Bing, L., Xu, X., Poria, S., and Lee, R. K.-W. (2023). Llm-adapters: An adapter family for parameter-efficient fine-tuning of large language models. In *The 2023 Conference on Empirical Methods in Natural Language Processing*. 26

[81] Huang, K., Zhang, S., Wang, H., Qu, Y., Lu, Y., Roohani, Y., Li, R., Qiu, L., Li, G., Zhang, J., et al. (2025a). Biomni: A general-purpose biomedical ai agent. *biorxiv*. 133, 143, 180

[82] Huang, Q., Liu, Y., Du, Y., and Garmire, L. X. (2021). Evaluation of cell type annotation r packages on single-cell rna-seq data. *Genomics, proteomics & bioinformatics*, 19(2):267–281. 30

[83] Huang, T., Liu, T., Babadi, M., Jin, W., and Ying, R. (2025b). Scalable generation of spatial transcriptomics from histology images via whole-slide flow matching. In *Forty-second International Conference on Machine Learning*. 93, 102

[84] Huang, T., Liu, T., Babadi, M., Ying, R., and Jin, W. (2025c). Stpath: A generative foundation model for integrating spatial transcriptomics and whole slide images. *bioRxiv*, pages 2025–04. 27

[85] Hwang, B., Lee, J. H., and Bang, D. (2018). Single-cell rna sequencing technologies and bioinformatics pipelines. *Experimental & molecular medicine*, 50(8):1–14. 2

[86] Imani, S., Du, L., and Shrivastava, H. (2023). MathPrompter: Mathematical reasoning using large language models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 5: Industry Track)*, pages 37–42, Toronto, Canada. Association for Computational Linguistics. 2

[87] Jackson, H. W., Fischer, J. R., Zanotelli, V. R., Ali, H. R., Mechera, R., Soysal, S. D., Moch, H., Muenst, S., Varga, Z., Weber, W. P., et al. (2020). The single-cell pathology landscape of breast cancer. *Nature*, 578(7796):615–620. 6

[88] Jain, A., Ong, S. P., Hautier, G., Chen, W., Richards, W. D., Dacek, S., Cholia, S., Gunter, D., Skinner, D., Ceder, G., et al. (2013). Commentary: The materials project: A materials genome approach to accelerating materials innovation. *APL Materials*, 1(1):011002. 152, 169, 170, 171, 172

[89] Jain, S. and Wallace, B. C. (2019). Attention is not Explanation. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 3543–3556, Minneapolis, Minnesota. Association for Computational Linguistics. 27

[90] Jaume, G., Doucet, P., Song, A. H., Lu, M. Y., Pérez, C. A., Wagner, S. J., Vaidya, A. J., Chen, R. J., Williamson, D. F., Kim, A., and Mahmood, F. (2024). HEST-1k: A dataset for spatial transcriptomics and histology image analysis. In *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*. 87, 89, 96, 102

[91] Jhi, S.-H., Ihm, J., Louie, S. G., and Cohen, M. L. (1999). Electronic mechanism of hardness enhancement in transition-metal carbonitrides. *Nature*, 399(6732):132–134. 140

[92] Ji, Y., Deng, Y., Gong, Y., Peng, Y., Niu, Q., Zhang, L., Ma, B., and Li, X. (2023). Exploring the impact of instruction data scaling on large language models: An empirical study on real-world use cases. *arXiv preprint arXiv:2303.14742*. 26

[93] Jiao, R., Huang, W., Lin, P., Han, J., Chen, P., Lu, Y., and Liu, Y. (2023). Crystal structure prediction by joint equivariant diffusion. *Advances in Neural Information Processing Systems*, 36:17464–17497. 170, 171

[94] Kahneman, D. (2011). Thinking, fast and slow. *Farrar, Straus and Giroux*. 148

[95] Kang, J. B., Shen, A. Z., Gurajala, S., Nathan, A., Rumker, L., Aguiar, V. R., Valencia, C., Lagattuta, K. A., Zhang, F., Jonsson, A. H., et al. (2023). Mapping the dynamic genetic regulatory architecture of hla genes at single-cell resolution. *Nature genetics*, 55(12):2255–2268. 18

[96] Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., and Amodei, D. (2020). Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*. 21, 28, 34

[97] Katzman, J. L., Shaham, U., Cloninger, A., Bates, J., Jiang, T., and Kluger, Y. (2018). Deepsurv: personalized treatment recommender system using a cox proportional hazards deep neural network. *BMC medical research methodology*, 18:1–12. 92, 101

[98] Kingma, D. and Ba, J. (2015). Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, San Diega, CA, USA. 9

[99] Kiselev, V. Y., Kirschner, K., Schaub, M. T., Andrews, T., Yiu, A., Chandra, T., Natarajan, K. N., Reik, W., Barahona, M., Green, A. R., et al. (2017). Sc3: consensus clustering of single-cell rna-seq data. *Nature methods*, 14(5):483–486. 87, 104

[100] Kläser, K., Banaszewski, B., Maddrell-Mander, S., McLean, C., Müller, L., Parviz, A., Huang, S., and Fitzgibbon, A. (2024). `MiniMol`: A parameter-efficient foundation model for molecular learning. 165

[101] Korsunsky, I., Millard, N., Fan, J., Slowikowski, K., Zhang, F., Wei, K., Baglaenko, Y., Brenner, M., Loh, P.-r., and Raychaudhuri, S. (2019). Fast, sensitive and accurate integration of single-cell data with harmony. *Nature methods*, 16(12):1289–1296. 8

[102] Kresse, G. and Furthmüller, J. (1996a). Efficiency of ab-initio total energy calculations for metals and semiconductors using a plane-wave basis set. *Computational Materials Science*, 6(1):15–50. 172

[103] Kresse, G. and Furthmüller, J. (1996b). Efficient iterative schemes for ab initio total-energy calculations using a plane-wave basis set. *Physical Review B*, 54(16):11169. 172

[104] Kuzman, T., Ljubešić, N., and Mozetič, I. (2023). Chatgpt: beginning of an end of manual annotation? use case of automatic genre identification. *arXiv preprint arXiv:2303.03953*. 2

[105] Labianca, R., Beretta, G. D., Kildani, B., Milesi, L., Merlin, F., Mosconi, S., Pessi, M. A., Prochilo, T., Quadri, A., Gatta, G., et al. (2010). Colon cancer. *Critical reviews in oncology/hematology*, 74(2):106–133. 90

[106] Lal, A., Garfield, D., Biancalani, T., and Eraslan, G. (2024). Designing realistic regulatory dna with autoregressive language models. *Genome Research*, 34(9):1411–1420. 143, 153, 178, 180

[107] Lan, W., He, G., Liu, M., Chen, Q., Cao, J., and Peng, W. (2024). Transformer-based single-cell language model: A survey. *arXiv preprint arXiv:2407.13205*. 2

[108] Lee, K., Jang, J., Seo, S., Lim, J., and Kim, W. Y. (2022). Drug-likeness scoring based on unsupervised learning. *Chemical Science*, 13(2):554–565. 166

[109] Lee, Y., Liu, X., Hao, M., Liu, T., and Regev, A. (2024). Pathomclip: Connecting tumor histology with spatial gene expression via locally enhanced contrastive learning of pathology and single-cell foundation model. *bioRxiv*, pages 2024–12. 93

[110] Leek, J. T., Scharpf, R. B., Bravo, H. C., Simcha, D., Langmead, B., Johnson, W. E., Geman, D., Baggerly, K., and Irizarry, R. A. (2010). Tackling the widespread and critical impact of batch effects in high-throughput data. *Nature Reviews Genetics*, 11(10):733–739. 29

[111] LemaÃŽtre, G., Nogueira, F., and Aridas, C. K. (2017). Imbalanced-learn: A python toolbox to tackle the curse of imbalanced datasets in machine learning. *Journal of machine learning research*, 18(17):1–5. 91

[112] Li, B., Zhang, W., Guo, C., Xu, H., Li, L., Fang, M., Hu, Y., Zhang, X., Yao, X., Tang, M., et al. (2022). Benchmarking spatial and single-cell transcriptomics integration methods for transcript distribution prediction and cell type deconvolution. *Nature methods*, 19(6):662–670. 27, 29

[113] Li, H., Zhou, J., Li, Z., Chen, S., Liao, X., Zhang, B., Zhang, R., Wang, Y., Sun, S., and Gao, X. (2023). A comprehensive benchmarking with practical guidelines for cellular deconvolution of spatial transcriptomics. *Nature Communications*, 14(1):1548. 19, 32

[114] Li, W. V. and Li, J. J. (2018). An accurate and robust imputation method scimpute for single-cell rna-seq data. *Nature communications*, 9(1):997. 32

[115] Li, X.-h., Wang, Z.-x., Lu, T.-y., and Che, X.-j. (2009). Modelling immune system: Principles, models, analysis and perspectives. *Journal of Bionic Engineering*, 6(1):77–85. 19

[116] Liang, W., Tadesse, G. A., Ho, D., Fei-Fei, L., Zaharia, M., Zhang, C., and Zou, J. (2022). Advances, challenges and opportunities in creating data for trustworthy ai. *Nature Machine Intelligence*, 4(8):669–677. 27

[117] Lin, Y., Wu, T.-Y., Wan, S., Yang, J. Y., Wong, W. H., and Wang, Y. R. (2022). scjoint integrates atlas-scale single-cell rna-seq and atac-seq data with transfer learning. *Nature biotechnology*, 40(5):703–710. 10

[118] Liou, C.-Y., Cheng, W.-C., Liou, J.-W., and Liou, D.-R. (2014). Autoencoder for words. *Neurocomputing*, 139:84–96. 29

[119] Litinetskaya, A., Shulman, M., Hediyeh-zadeh, S., Moinfar, A. A., Curion, F., Szalata, A., Omidi, A., Lotfollahi, M., and Theis, F. J. (2024). Multimodal weakly supervised learning to identify disease-specific changes in single-cell atlases. *bioRxiv*, pages 2024–07. 101

[120] Liu, H., Li, Z., Hall, D. L. W., Liang, P., and Ma, T. (2024). Sophia: A scalable stochastic second-order optimizer for language model pre-training. In *The Twelfth International Conference on Learning Representations*. 9

[121] Liu, T., Chen, T., Zheng, W., Luo, X., and Zhao, H. (2023a). scelmo: Embeddings from language models are good learners for single-cell data analysis. *bioRxiv*, pages 2023–12. 84, 87, 99

[122] Liu, T., De Brouwer, E., Kuo, T., Diamant, N., Missarova, A., Wang, H., Hao, M., Bravo, H. C., Scalia, G., Regev, A., et al. (2025a). Learning multi-cellular representations of single-cell transcriptomics data enables characterization of patient-level disease states. In *International Conference on Research in Computational Molecular Biology*, pages 303–306. Springer. 92

[123] Liu, T., Huang, T., Wang, L., Lin, Y., Ying, R., and Zhao, H. (2025b). Unicorn: Towards universal cellular expression prediction with a multi-task learning framework. *Nature Communications*, 16(1):9455. 141

[124] Liu, T., Li, K., Wang, Y., Li, H., and Zhao, H. (2023b). Evaluating the utilities of foundation models in single-cell data analysis. *bioRxiv*, pages 2023–09. 1, 84, 102

[125] Liu, Y., Cavanagh, J. M., Sun, K., Toney, J., Yuan, C.-Y., Smith, A., II, R. S. M., Graggs, P. A., Toste, F. D., Kulik, H., and Head-Gordon, T. (2025c). Exploring transition metal complexes with large language models. *ChemRxiv*. 133, 138

[126] Loeffler, H. H., He, J., Tibo, A., Janet, J. P., Voronov, A., Mervin, L. H., and Engkvist, O. (2024). Reinvent 4: modern ai–driven generative molecule design. *Journal of Cheminformatics*, 16(1):20. xxii, 133, 161, 167

[127] Long, Y., Ang, K. S., Sethi, R., Liao, S., Heng, Y., van Olst, L., Ye, S., Zhong, C., Xu, H., Zhang, D., et al. (2024). Deciphering spatial domains from spatial multi-omics with spatialglue. *Nature Methods*, pages 1–10. 88, 104

[128] Lopez, R., Regier, J., Cole, M. B., Jordan, M. I., and Yosef, N. (2018). Deep generative modeling for single-cell transcriptomics. *Nature methods*, 15(12):1053–1058. 23

[129] Lotfollahi, M., Klimovskaia Susmelj, A., De Donno, C., Hetzel, L., Ji, Y., Ibarra, I. L., Srivatsan, S. R., Naghipourfar, M., Daza, R. M., Martin, B., et al. (2023). Predicting cellular responses to complex perturbations in high-throughput screens. *Molecular Systems Biology*, page e11517. 27

[130] Lu, C., Lu, C., Lange, R. T., Foerster, J., Clune, J., and Ha, D. (2024a). The ai scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*. 133

[131] Lu, M. Y., Chen, B., Williamson, D. F., Chen, R. J., Liang, I., Ding, T., Jaume, G., Odintsov, I., Le, L. P., Gerber, G., et al. (2024b). A visual-language foundation model for computational pathology. *Nature Medicine*, 30(3):863–874. 87

[132] Luecken, M. D., Büttner, M., Chaichoompu, K., Danese, A., Interlandi, M., Müller, M. F., Strobl, D. C., Zappia, L., Dugas, M., Colomé-Tatché, M., et al. (2022). Benchmarking atlas-level data integration in single-cell genomics. *Nature methods*, 19(1):41–50. 17, 29, 30, 102

[133] Lun, A. T., McCarthy, D. J., and Marioni, J. C. (2016). A step-by-step workflow for low-level analysis of single-cell rna-seq data with bioconductor. *F1000Research*, 5. 20

[134] Lundberg, S. M. and Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in neural information processing systems*, 30. 27

[135] Luo, C., Keown, C. L., Kurihara, L., Zhou, J., He, Y., Li, J., Castanon, R., Lucero, J., Nery, J. R., Sandoval, J. P., et al. (2017). Single-cell methylomes identify neuronal subtypes and regulatory elements in mammalian cortex. *Science*, 357(6351):600–604. 2

[136] M. Bran, A., Cox, S., Schilter, O., Baldassari, C., White, A. D., and Schwaller, P. (2024). Augmenting large language models with chemistry tools. *Nature Machine Intelligence*, 6(5):525–535. 133

[137] Ma, J., Guo, Z., Zhou, F., Wang, Y., Xu, Y., Cai, Y., Zhu, Z., Jin, C., Lin, Y., Jiang, X., Han, A., Liang, L., Chan, R. C. K., Wang, J., Cheng, K.-T., and Chen, H. (2024). Towards a generalizable pathology foundation model via unified knowledge distillation. 87, 96, 100

[138] Mann, V., Sales-Cruz, M., Gani, R., and Venkatasubramanian, V. (2024). esfiles: Intelligent process flowsheet synthesis using process knowledge, symbolic ai, and machine learning. *Computers & Chemical Engineering*, 181:108505. 187

[139] Mansouri Tehrani, A., Oliynyk, A. O., Parry, M., Rizvi, Z., Couper, S., Lin, F., Miyagi, L., Sparks, T. D., and Brgoch, J. (2018). Machine learning directed search for ultraincompressible, superhard materials. *Journal of the American Chemical Society*, 140(31):9844–9853. 140, 151

[140] Marco Salas, S., Czarnewski, P., Kuemmerle, L. B., Helgadottir, S., Mattsson Langseth, C., Tiesmeyer, S., Avenel, C., Rehman, H., Tiklova, K., Andersson, A., et al. (2023). Optimizing xenium in situ data utility by quality assessment and best practice analysis workflows. *bioRxiv*, pages 2023–02. 32

[141] Marx, V. (2021). Method of the year: spatially resolved transcriptomics. *Nature methods*, 18(1):9–14. 25

[142] Mathys, H., Davila-Velderrain, J., Peng, Z., Gao, F., Mohammadi, S., Young, J. Z., Menon, M., He, L., Abdurrob, F., Jiang, X., et al. (2019). Single-cell transcriptomic analysis of alzheimer's disease. *Nature*, 570(7761):332–337. 2

[143] Maynard, K. R., Collado-Torres, L., Weber, L. M., Uytingco, C., Barry, B. K., Williams, S. R., Catallini, J. L., Tran, M. N., Besich, Z., Tippani, M., et al. (2021). Transcriptome-scale spatial gene expression in the human dorsolateral prefrontal cortex. *Nature neuroscience*, 24(3):425–436. 86

[144] McInnes, L., Healy, J., Saul, N., and Großberger, L. (2018a). Umap: Uniform manifold approximation and projection. *Journal of Open Source Software*, 3(29). 8

[145] McInnes, L., Healy, J., Saul, N., and Großberger, L. (2018b). Umap: Uniform manifold approximation and projection. *Journal of Open Source Software*, 3(29):861. 90

[146] Merchant, A., Batzner, S., Schoenholz, S. S., Aykol, M., Cheon, G., and Cubuk, E. D. (2023). Scaling deep learning for materials discovery. *Nature*, 624(7990):80–85. 138

[147] Meyer, D., C. Aguiar, V. R., Bitarello, B. D., C. Brandt, D. Y., and Nunes, K. (2018). A genomic perspective on hla evolution. *Immunogenetics*, 70:5–27. 18

[148] Mitchener, L., Yiu, A., Chang, B., Bourdenx, M., Nadolski, T., Sulovari, A., Landsness, E. C., Barabasi, D. L., Narayanan, S., Evans, N., et al. (2025). Kosmos: An ai scientist for autonomous discovery. *arXiv preprint arXiv:2511.02824*. 133

[149] Moses, L. and Pachter, L. (2022). Museum of spatial transcriptomics. *Nature methods*, 19(5):534–546. 84

[150] Moyers, B. A., Partridge, E. C., Mackiewicz, M., Betti, M. J., Darji, R., Meadows, S. K., Newberry, K. M., Brandsmeier, L. A., Wold, B. J., Mendenhall, E. M., et al. (2023). Characterization of human transcription factor function and patterns of gene regulation in hepg2 cells. *Genome Research*, 33(11):1879–1892. 141

[151] Mueller, A. C. (2023). Wordcloud. xvi, 111

[152] Mulqueen, R. M., Pokholok, D., Norberg, S. J., Torkenczy, K. A., Fields, A. J., Sun, D., Sinnamon, J. R., Shendure, J., Trapnell, C., O'Roak, B. J., et al. (2018). Highly scalable generation of dna methylation profiles in single cells. *Nature biotechnology*, 36(5):428–431. 2

[153] Nguyen, E., Poli, M., Faizi, M., Thomas, A. W., Wornow, M., Birch-Sykes, C., Massaroli, S., Patel, A., Rabideau, C. M., Bengio, Y., Ermon, S., Re, C., and Baccus, S. (2023). HyenaDNA: Long-range genomic sequence modeling at single nucleotide resolution. In *Thirty-seventh Conference on Neural Information Processing Systems*. 180

[154] Novikov, A., Vũ, N., Eisenberger, M., Dupont, E., Huang, P.-S., Wagner, A. Z., Shirobokov, S., Kozlovskii, B., Ruiz, F. J., Mehrabian, A., et al. (2025). Alphaevolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*. 133

[155] Ong, S. P., Richards, W. D., Jain, A., Hautier, G., Kocher, M., Cholia, S., Gunter, D., Chevrier, V. L., Persson, K. A., and Ceder, G. (2013). Python materials genomics (pymatgen): A robust, open-source python library for materials analysis. *Computational Materials Science*, 68:314–319. 153, 170, 172, 173

[156] OpenAI (2023). Gpt-4 technical report. 2

[157] Palla, G., Spitzer, H., Klein, M., Fischer, D., Schaar, A. C., Kuemmerle, L. B., Rybakov, S., Ibarra, I. L., Holmberg, O., Virshup, I., et al. (2022). Squidpy: a scalable framework for spatial omics analysis. *Nature methods*, 19(2):171–178. 88

[158] Pangilinan, L. E., Hu, S., Hamilton, S. G., Tolbert, S. H., and Kaner, R. B. (2021). Hardening effects in superhard transition-metal borides. *Accounts of Materials Research*, 3(1):100–109. 140

[159] Park, H., Li, Z., and Walsh, A. (2024). Has generative artificial intelligence solved inverse materials design? *Matter*, 7(7):2355–2367. 138

[160] Pearl, J. and Mackenzie, D. (2018). *The book of why: the new science of cause and effect*. Basic books. 32

[161] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., et al. (2011). Scikit-learn: Machine learning in python. *the Journal of machine Learning research*, 12:2825–2830. 11, 101, 103

[162] Perdew, J. P., Burke, K., and Ernzerhof, M. (1996). Generalized gradient approximation made simple. *Physical Review Letters*, 77(18):3865. 172

[163] Perdew, J. P. and Wang, Y. (1992). Accurate and simple analytic representation of the electron-gas correlation energy. *Physical Review B*, 45(23):13244. 172

[164] Peters, M. E., Neumann, M., Iyyer, M., Gardner, M., Clark, C., Lee, K., and Zettlemoyer, L. (2018). Deep contextualized word representations. In Walker, M., Ji, H., and Stent, A., editors, *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, pages 2227–2237, New Orleans, Louisiana. Association for Computational Linguistics. 27

[165] Pliner, H. A., Shendure, J., and Trapnell, C. (2019). Supervised classification enables rapid annotation of cell atlases. *Nature methods*, 16(10):983–986. 30

[166] Pullin, J. M. and McCarthy, D. J. (2024). A comparison of marker gene selection methods for single-cell rna sequencing data. *Genome Biology*, 25(1):56. 30

[167] Ravindra, N., Sehanobish, A., Pappalardo, J. L., Hafler, D. A., and van Dijk, D. (2020). Disease state prediction from single-cell data using graph attention networks. In *Proceedings of the ACM conference on health, inference, and learning*, pages 121–130. 30

[168] Reddy, A. J., Herschl, M. H., Geng, X., Kolli, S., Lu, A. X., Kumar, A., Hsu, P. D., Levine, S., and Ioannidis, N. M. (2024). Strategies for effectively modelling promoter-driven gene expression using transfer learning. *bioRxiv*, pages 2023–02. 178

[169] Roohani, Y., Huang, K., and Leskovec, J. (2023). Predicting transcriptional outcomes of novel multigene perturbations with gears. *Nature Biotechnology*, pages 1–9. 15, 32

[170] Rosen, Y., Brbić, M., Roohani, Y., Swanson, K., Li, Z., and Leskovec, J. (2024). Toward universal cell embeddings: integrating single-cell rna-seq datasets across species with saturn. *Nature Methods*, pages 1–9. 21

[171] Rosen, Y., Roohani, Y., Agrawal, A., Samotorcan, L., Consortium, T. S., Quake, S. R., and Leskovec, J. (2023). Universal cell embeddings: A foundation model for cell biology. *bioRxiv*, pages 2023–11. 3

[172] Ruder, S. (2016). An overview of gradient descent optimization algorithms. *arXiv preprint arXiv:1609.04747*. 9

[173] Rupprecht, S., Gao, Q., Karia, T., and Schweidtmann, A. M. (2025). Multi-agent systems for chemical engineering: A review and perspective. *arXiv preprint arXiv:2508.07880*. 144

[174] Saelens, W., Cannoodt, R., Todorov, H., and Saeys, Y. (2019). A comparison of single-cell trajectory inference methods. *Nature biotechnology*, 37(5):547–554. viii, 7

[175] Sakaue, S., Gurajala, S., Curtis, M., Luo, Y., Choi, W., Ishigaki, K., Kang, J. B., Rumker, L., Deutsch, A. J., Schönherr, S., et al. (2023). Tutorial: a statistical genetics guide to identifying hla alleles driving complex disease. *Nature Protocols*, pages 1–17. 18

[176] Saliba, A.-E., Westermann, A. J., Gorski, S. A., and Vogel, J. (2014). Single-cell rna-seq: advances and future challenges. *Nucleic acids research*, 42(14):8845–8860. 2

[177] Salzberg, S. L. (2018). Open questions: How many genes do we have? *BMC biology*, 16(1):1–3. 31

[178] Sandelin, A., Alkema, W., Engström, P., Wasserman, W. W., and Lenhard, B. (2004). Jaspar: an open-access database for eukaryotic transcription factor binding profiles. *Nucleic acids research*, 32(suppl_1):D91–D94. 144, 179

[179] Sati, H., Carrara, E., Savoldi, A., Hansen, P., Garlasco, J., Campagnaro, E., Boccia, S., Castillo-Polo, J. A., Magrini, E., Garcia-Vello, P., Wool, E., Gigante, V., Duffy, E., Cassini, A., Huttner, B., Pilar Ramon Pardo, Naghavi, M., Mirzayev, F., Zignol, M., Cameron, A., Tacconelli, E., and WHO Bacterial Priority Pathogens List Advisory Group (2025). The who bacterial priority pathogens list 2024: a prioritisation study to guide research, development, and public health strategies against antimicrobial resistance. *The Lancet Infectious Diseases*, 25(9):1033–1043. Epub 2025-04-14. 135

[180] Satija, R., Farrell, J. A., Gennert, D., Schier, A. F., and Regev, A. (2015). Spatial reconstruction of single-cell gene expression data. *Nature biotechnology*, 33(5):495–502. 20

[181] Schaar, A. C., Tejada-Lapuerta, A., Palla, G., Gutgesell, R., Halle, L., Minaeva, M., Vornholz, L., Dony, L., Drummer, F., Bahrami, M., et al. (2024). Nicheformer: a foundation model for single-cell and spatial omics. *bioRxiv*, pages 2024–04. 84

[182] Schmidt, R., Ehmki, E. S., Ohm, F., Ehrlich, H.-C., Mashychev, A., and Rarey, M. (2019). Comparing molecular patterns using the example of smarts: theory and algorithms. *Journal of Chemical Information and Modeling*, 59(6):2560–2571. 166

[183] Sebastian, S., Roy, S., and Kalita, J. (2023). A generic parallel framework for inferring large-scale gene regulatory networks from expression profiles: application to alzheimer's disease network. *Briefings in Bioinformatics*, 24(1):bbac482. 17

[184] Selva Birunda, S. and Kanniga Devi, R. (2021). A review on word embedding techniques for text classification. *Innovative Data Communication Technologies and Application: Proceedings of ICIDCA 2020*, pages 267–281. 2

[185] Shaikovski, G., Casson, A., Severson, K., Zimmermann, E., Wang, Y. K., Kunz, J. D., Retamero, J. A., Oakley, G., Klimstra, D., Kanan, C., et al. (2024). Prism: A multi-modal generative foundation model for slide-level histopathology. *arXiv preprint arXiv:2405.10254*. 95

[186] Shen, H., Liu, J., Hu, J., Shen, X., Zhang, C., Wu, D., Feng, M., Yang, M., Li, Y., Yang, Y., et al. (2023). Generative pretraining from large-scale transcriptomes for single-cell deciphering. *Iscience*, 26(5). 3

[187] Shiraki, T., Kondo, S., Katayama, S., Waki, K., Kasukawa, T., Kawaji, H., Kodzius, R., Watahiki, A., Nakamura, M., Arakawa, T., et al. (2003). Cap analysis gene expression for high-throughput analysis of transcriptional starting point and

identification of promoter usage. *Proceedings of the National Academy of Sciences*, 100(26):15776–15781. 144

[188] Shivanyuk, A. N., Ryabukhin, S. V., Tolmachev, A., Bogolyubsky, A., Mykytenko, D., Chupryna, A., Heilman, W., and Kostyuk, A. (2007). Enamine real database: Making chemical diversity real. *Chemistry today*, 25(6):58–59. 151, 165

[189] Singhal, K., Azizi, S., Tu, T., Mahdavi, S. S., Wei, J., Chung, H. W., Scales, N., Tanwani, A., Cole-Lewis, H., Pfohl, S., et al. (2023). Large language models encode clinical knowledge. *Nature*, pages 1–9. 2

[190] Sohail, Y., Zhang, C., Xue, D., Zhang, J., Zhang, D., Gao, S., Yang, Y., Fan, X., Zhang, H., Liu, G., et al. (2025). Machine-learning design of ductile fenicoalta alloys with high strength. *Nature*, pages 1–6. 138

[191] Soldaini, L. and Goharian, N. (2016). Quickumls: a fast, unsupervised approach for medical concept extraction. 94, 105

[192] Song, A. H., Jaume, G., Williamson, D. F., Lu, M. Y., Vaidya, A., Miller, T. R., and Mahmood, F. (2023a). Artificial intelligence for digital and computational pathology. *Nature Reviews Bioengineering*, 1(12):930–949. 84

[193] Song, D., Wang, Q., Yan, G., Liu, T., Sun, T., and Li, J. J. (2023b). scdesign3 generates realistic in silico data for multimodal single-cell and spatial omics. *Nature Biotechnology*, pages 1–6. 20

[194] Song, Y., Miao, Z., Brazma, A., and Papatheodorou, I. (2023c). Benchmarking strategies for cross-species integration of single-cell rna sequencing data. *Nature Communications*, 14(1):6495. 21

[195] Stoeckius, M., Hafemeister, C., Stephenson, W., Houck-Loomis, B., Chattopadhyay, P. K., Swerdlow, H., Satija, R., and Smibert, P. (2017). Simultaneous epitope and transcriptome measurement in single cells. *Nature methods*, 14(9):865–868. 2

[196] Stops, L., Leenhouts, R., Gao, Q., and Schweidtmann, A. M. (2023). Flowsheet generation through hierarchical reinforcement learning and graph neural networks. *AIChE Journal*, 69(1):e17938. 144

[197] Stuart, T., Butler, A., Hoffman, P., Hafemeister, C., Papalexi, E., Mauck, W. M., Hao, Y., Stoeckius, M., Smibert, P., and Satija, R. (2019). Comprehensive integration of single-cell data. *Cell*, 177(7):1888–1902. 2, 30

[198] Stubbington, M. J., Rozenblatt-Rosen, O., Regev, A., and Teichmann, S. A. (2017). Single-cell transcriptomics to explore the immune system in health and disease. *Science*, 358(6359):58–63. 2

[199] Su, C., Xu, Z., Shan, X., Cai, B., Zhao, H., and Zhang, J. (2023). Cell-type-specific co-expression inference from single cell rna-sequencing data. *Nature Communications*, 14(1):4846. 32

[200] Sun, K., Bagni, D., Cavanagh, J. M., Wang, Y., Sawyer, J. M., Zhou, B., Gritsevskiy, A., Zhang, O., and Head-Gordon, T. (2025). Synllama: Generating synthesizable molecules and their analogs with large language models. *ACS Central Science*, 11(11):2108–2120. 133

[201] Sun, W., Shi, Z., Gao, S., Ren, P., de Rijke, M., and Ren, Z. (2023). Contrastive learning reduces hallucination in conversations. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 13618–13626. 27

[202] Sundararajan, M., Taly, A., and Yan, Q. (2017). Axiomatic attribution for deep networks. In *International conference on machine learning*, pages 3319–3328. PMLR. 27

[203] Swanson, K., Wu, W., Bulaong, N. L., Pak, J. E., and Zou, J. (2025). The virtual lab of ai agents designs new sars-cov-2 nanobodies. *Nature*, 646(8085):716–723. 133

[204] Teichmann, S. and Efremova, M. (2020). Method of the year 2019: single-cell multimodal omics. *Nat. Methods*, 17(1):2020. 2

[205] Theodoris, C. V., Xiao, L., Chopra, A., Chaffin, M. D., Al Sayed, Z. R., Hill, M. C., Mantineo, H., Brydon, E. M., Zeng, Z., Liu, X. S., et al. (2023). Transfer learning enables predictions in network biology. *Nature*, pages 1–9. 3, 27, 31, 97

[206] Thirunavukarasu, A. J., Ting, D. S. J., Elangovan, K., Gutierrez, L., Tan, T. F., and Ting, D. S. W. (2023). Large language models in medicine. *Nature medicine*, 29(8):1930–1940. 2

[207] Thomas, P. D., Ebert, D., Muruganujan, A., Mushayahama, T., Albou, L.-P., and Mi, H. (2022). Panther: Making genome-scale phylogenetics accessible to all. *Protein Science*, 31(1):8–22. 17

[208] Tian, Y., Xu, B., and Zhao, Z. (2012). Microscopic theory of hardness and design of novel superhard crystals. *International Journal of Refractory Metals and Hard Materials*, 33:93–106. 173

[209] Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., et al. (2023). Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*. 2

[210] Traag, V. A., Waltman, L., and Van Eck, N. J. (2019). From louvain to leiden: guaranteeing well-connected communities. *Scientific reports*, 9(1):5233. 17

[211] Tran, H. T. N., Ang, K. S., Chevrier, M., Zhang, X., Lee, N. Y. S., Goh, M., and Chen, J. (2020). A benchmark of batch-effect correction methods for single-cell rna sequencing data. *Genome biology*, 21:1–32. 29

[212] Tsuneki, M. and Kanavati, F. (2022). Inference of captions from histopathological patches. In *International Conference on Medical Imaging with Deep Learning*, pages 1235–1250. PMLR. 94

[213] Turton, R., Bailie, R. C., Whiting, W. B., and Shaeiwitz, J. A. (2008). *Analysis, synthesis and design of chemical processes*. Pearson Education. 144, 152

[214] van den Broek, R. L., Patel, S., van Westen, G. J., Jespers, W., and Sherman, W. (2025). In search of beautiful molecules: a perspective on generative modeling for drug design. *Journal of chemical information and modeling*, 65(18):9383–9397. 133, 135

[215] Van Steen, K. (2012). Travelling the world of gene–gene interactions. *Briefings in bioinformatics*, 13(1):1–19. 2

[216] Vogel, G., Balhorn, L. S., and Schweidtmann, A. M. (2023a). Learning from flowsheets: A generative transformer model for autocompletion of flowsheets. *Computers & Chemical Engineering*, 171:108162. 187

[217] Vogel, G., Hirtreiter, E., Schulze Balhorn, L., and Schweidtmann, A. M. (2023b). Sfiles 2.0: an extended text-based flowsheet representation. *Optimization and Engineering*, 24(4):2911–2933. 187

[218] Vorontsov, E., Bozkurt, A., Casson, A., Shaikovski, G., Zelechowski, M., Severson, K., Zimmermann, E., Hall, J., Tenenholtz, N., Fusi, N., et al. (2024). A foundation model for clinical-grade computational pathology and rare cancers detection. *Nature Medicine*, pages 1–12. 97

[219] Wang, X., Yang, S., Zhang, J., Wang, M., Zhang, J., Yang, W., Huang, J., and Han, X. (2022a). Transformer-based unsupervised contrastive learning for histopathological image classification. *Medical image analysis*, 81:102559. 97

[220] Wang, X., Zhao, J., Marostica, E., Yuan, W., Jin, J., Zhang, J., Li, R., Tang, H., Wang, K., Li, Y., et al. (2024). A pathology foundation model for cancer diagnosis and prognosis prediction. *Nature*, 634(8035):970–978. 87

[221] Wang, Y., Liu, T., and Zhao, H. (2022b). Respan: a powerful batch correction model for scrna-seq data through residual adversarial networks. *Bioinformatics*, 38(16):3942–3949. 8

[222] Wang, Y., Si, S., Li, D., Lukasik, M., Yu, F., Hsieh, C.-J., Dhillon, I. S., and Kumar, S. (2022c). Preserving in-context learning ability in large language model fine-tuning. *arXiv preprint arXiv:2211.00635*. 26

[223] Wei, J., Bosma, M., Zhao, V., Guu, K., Yu, A. W., Lester, B., Du, N., Dai, A. M., and Le, Q. V. (2022). Finetuned language models are zero-shot learners. In *International Conference on Learning Representations*. 27

[224] Weinstein, J. N., Collisson, E. A., Mills, G. B., Shaw, K. R., Ozenberger, B. A., Ellrott, K., Shmulevich, I., Sander, C., and Stuart, J. M. (2013). The cancer genome atlas pan-cancer analysis project. *Nature genetics*, 45(10):1113–1120. 92

[225] Wen, H., Tang, W., Dai, X., Ding, J., Jin, W., Xie, Y., and Tang, J. (2023). Cellplm: Pre-training of cell language model beyond single cells. In *The Twelfth International Conference on Learning Representations*. 3

[226] Wheeler, D. L., Barrett, T., Benson, D. A., Bryant, S. H., Canese, K., Chetvernin, V., Church, D. M., DiCuccio, M., Edgar, R., Federhen, S., et al. (2007). Database resources of the national center for biotechnology information. *Nucleic acids research*, 35(suppl_1):D5–D12. 84

[227] Wiegreffe, S. and Pinter, Y. (2019). Attention is not not explanation. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 11–20, Hong Kong, China. Association for Computational Linguistics. 27

[228] Wittkopp, P. J. and Kalay, G. (2012). Cis-regulatory elements: molecular mechanisms and evolutionary processes underlying divergence. *Nature Reviews Genetics*, 13(1):59–69. 152

[229] Wolf, F. A., Angerer, P., and Theis, F. J. (2018). Scanpy: large-scale single-cell gene expression data analysis. *Genome biology*, 19:1–5. 20, 25, 99

[230] Xia, Y., Jin, P., Xie, S., He, L., Cao, C., Luo, R., Liu, G., Wang, Y., Liu, Z., Chen, Y.-J., Guo, Z., Bai, Y., Deng, P., Min, Y., Lu, Z., Hao, H., Yang, H., Li, J., Liu, C., Zhang, J., Zhu, J., Bi, R., Wu, K., Zhang, W., Gao, K., Pei, Q., Wang, Q., Liu, X., Li, Y., Zhu, H., Lu, Y., Ma, M., Wang, Z., Xie, T., Maziarz, K., Segler, M., Yang, Z., Chen, Z., Shi, Y., Zheng, S., Wu, L., Hu, C., Dai, P., Liu, T.-Y., Liu, H., and Qin, T. (2025). Nature language model: Deciphering the language of nature for scientific discovery. 167

[231] Xiang, J., Wang, X., Zhang, X., Xi, Y., Eweje, F., Chen, Y., Li, Y., Bergstrom, C., Gopaulchan, M., Kim, T., et al. (2025). A vision–language foundation model for precision oncology. *Nature*, pages 1–10. 87

[232] Xiao, C., Liu, M., Yao, K., Zhang, Y., Zhang, M., Yan, M., Sun, Y., Liu, X., Cui, X., Fan, T., et al. (2025). Ultrabroadband and band-selective thermal meta-emitters by machine learning. *Nature*, 643(8070):80–88. 138

[233] Xu, C., Lopez, R., Mehlman, E., Regier, J., Jordan, M. I., and Yosef, N. (2021a). Probabilistic harmonization and annotation of single-cell transcriptomics data with deep generative models. *Molecular systems biology*, 17(1):e9620. 21

[234] Xu, H., Usuyama, N., Bagga, J., Zhang, S., Rao, R., Naumann, T., Wong, C., Gero, Z., González, J., Gu, Y., et al. (2024a). A whole-slide foundation model for digital pathology from real-world data. *Nature*, pages 1–8. 87, 97

[235] Xu, H., Woicik, A., Poon, H., Altman, R. B., and Wang, S. (2023). Multilingual translation for zero-shot biomedical classification using biotranslator. *Nature Communications*, 14(1):738. 2

[236] Xu, R., Luo, F., Zhang, Z., Tan, C., Chang, B., Huang, S., and Huang, F. (2021b). Raise a child in large language model: Towards effective and generalizable fine-tuning. In Moens, M.-F., Huang, X., Specia, L., and Yih, S. W.-t., editors, *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 9514–9528, Online and Punta Cana, Dominican Republic. Association for Computational Linguistics. 26

[237] Xu, Y., Wang, Y., Zhou, F., Ma, J., Yang, S., Lin, H., Wang, X., Wang, J., Liang, L., Han, A., et al. (2024b). A multimodal knowledge-enhanced whole-slide pathology foundation model. *arXiv preprint arXiv:2407.15362*. 99

[238] Xue, F., Fu, Y., Zhou, W., Zheng, Z., and You, Y. (2023). To repeat or not to repeat: Insights from scaling LLM under token-crisis. In *Thirty-seventh Conference on Neural Information Processing Systems*. 9

[239] Yamada, Y., Lange, R. T., Lu, C., Hu, S., Lu, C., Foerster, J., Clune, J., and Ha, D. (2025). The ai scientist-v2: Workshop-level automated scientific discovery via agentic tree search. *arXiv preprint arXiv:2504.08066*. 133

[240] Yang, F., Wang, W., Wang, F., Fang, Y., Tang, D., Huang, J., Lu, H., and Yao, J. (2022). scbert as a large-scale pretrained deep language model for cell type annotation of single-cell rna-seq data. *Nature Machine Intelligence*, 4(10):852–866. 3

[241] Yang, H., Hu, C., Zhou, Y., Liu, X., Shi, Y., Li, J., Li, G., Chen, Z., Chen, S., Zeni, C., et al. (2024). MatterSim: A deep learning atomistic model across elements, temperatures and pressures. *arXiv preprint arXiv:2405.04967*. 152, 170

[242] Yang, X., Liu, G., Feng, G., Bu, D., Wang, P., Jiang, J., Chen, S., Yang, Q., Zhang, Y., Man, Z., et al. (2023). Genecompass: Deciphering universal gene regulatory mechanisms with knowledge-informed cross-species foundation model. *bioRxiv*, pages 2023–09. 3

[243] Yao, Z., Lum, Y., Johnston, A., Mejia-Mendoza, L. M., Zhou, X., Wen, Y., Aspuru-Guzik, A., Sargent, E. H., and Seh, Z. W. (2023). Machine learning for a sustainable energy future. *Nature Reviews Materials*, 8(3):202–215. 138

[244] Ye, Q., Xu, H., Xu, G., Ye, J., Yan, M., Zhou, Y., Wang, J., Hu, A., Shi, P., Shi, Y., et al. (2023). mplug-owl: Modularization empowers large language models with multimodality. *arXiv preprint arXiv:2304.14178*. xxiv, 5

[245] Yuan, X., Zhu, B., Zhang, C., Zheng, Q., Gao, E., and Shao, Q. (2025). Accelerated discovery of ultraincompressible, superhard materials via physics-enhanced active learning. *Materials Horizons*. 140

[246] Yuan, Z., Zhao, F., Lin, S., Zhao, Y., Yao, J., Cui, Y., Zhang, X.-Y., and Zhao, Y. (2024). Benchmarking spatial clustering methods with spatially resolved transcriptomics data. *Nature Methods*, 21(4):712–722. 98

[247] Yuksekgonul, M., Bianchi, F., Boen, J., Liu, S., Lu, P., Huang, Z., Guestrin, C., and Zou, J. (2025). Optimizing generative ai by backpropagating language model feedback. *Nature*, 639(8055):609–616. 140, 143, 167, 171, 180

[248] Zappia, L., Phipson, B., and Oshlack, A. (2017). Splatter: simulation of single-cell rna sequencing data. *Genome biology*, 18(1):174. 20, 33

[249] Zeng, T., Badrinarayanan, S., Ock, J., Lai, C.-K., and Barati Farimani, A. (2025a). Llm-guided chemical process optimization with a multi-agent approach. *Machine Learning: Science and Technology*. 146

[250] Zeng, Y., Xie, J., Shangguan, N., Wei, Z., Li, W., Su, Y., Yang, S., Zhang, C., Zhang, J., Fang, N., et al. (2025b). Cellfm: a large-scale foundation model pre-trained on transcriptomics of 100 million human cells. *Nature Communications*, 16(1):4679. 3

[251] Zeni, C., Pinsler, R., Zügner, D., Fowler, A., Horton, M., Fu, X., Wang, Z., Shysheya, A., Crabbé, J., Ueda, S., et al. (2025). A generative model for inorganic materials design. *Nature*, pages 1–3. 133, 138, 140, 151, 153, 170, 171

[252] Zhang, B., Liu, Z., Cherry, C., and Firat, O. (2024a). When scaling meets LLM finetuning: The effect of data, model and finetuning method. In *The Twelfth International Conference on Learning Representations*. 21, 28, 34

[253] Zhang, D., Schroeder, A., Yan, H., Yang, H., Hu, J., Lee, M. Y., Cho, K. S., Susztak, K., Xu, G. X., Feldman, M. D., et al. (2024b). Inferring super-resolution tissue architecture by integrating spatial transcriptomics with histology. *Nature biotechnology*, 42(9):1372–1377. 87, 104

[254] Zhang, L., He, C. H., Coffey, S., Yin, D., Hsu, I.-U., Su, C., Ye, Y., Zhang, C., Spurrier, J., Nicholson, L., et al. (2023). Single-cell transcriptomic atlas of alzheimer's disease middle temporal gyrus reveals region, cell type and sex specificity of gene expression with novel genetic risk for mertk in female. *medRxiv*, pages 2023–02. 2

[255] Zhang, S. and Metaxas, D. (2024). On the challenges and perspectives of foundation models for medical image analysis. *Medical image analysis*, 91:102996. 84

[Zhang et al.] Zhang, T., Kishore, V., Wu, F., Weinberger, K. Q., and Artzi, Y. Bertscore: Evaluating text generation with bert. 94, 105

[257] Zhang, X., Li, F., Nordin, A. N., Tarbell, J., and Voiculescu, I. (2015). Toxicity studies using mammalian cells and impedance spectroscopy method. *Sensing and bio-sensing research*, 3:112–121. 165

[258] Zhao, S., Zhang, J., and Nie, Z. (2023a). Large-scale cell representation learning via divide-and-conquer contrastive learning. *arXiv preprint arXiv:2306.04371*. 3

[259] Zhao, W. X., Zhou, K., Li, J., Tang, T., Wang, X., Hou, Y., Min, Y., Zhang, B., Zhang, J., Dong, Z., et al. (2023b). A survey of large language models. *arXiv preprint arXiv:2303.18223*. 2

[260] Zheng, G. X., Terry, J. M., Belgrader, P., Ryvkin, P., Bent, Z. W., Wilson, R., Ziraldo, S. B., Wheeler, T. D., McDermott, G. P., Zhu, J., et al. (2017). Massively parallel digital transcriptional profiling of single cells. *Nature communications*, 8(1):1–12. 2

[261] Zheng, Z., Zhang, O., Nguyen, H. L., Rampal, N., Alawadhi, A. H., Rong, Z., Head-Gordon, T., Borgs, C., Chayes, J. T., and Yaghi, O. M. (2023). Chatgpt research group for optimizing the crystallinity of mofs and cofs. *ACS Central Science*, 9(11):2161–2170. 133

[262] Zimmermann, E., Vorontsov, E., Viret, J., Casson, A., Zelechowski, M., Shaikovski, G., Tenenholtz, N., Hall, J., Fuchs, T., Fusi, N., et al. (2024). Virchow 2: Scaling self-supervised mixed magnification models in pathology. *arXiv preprint arXiv:2408.00738*. 87

ProQuest Number: 32449965

INFORMATION TO ALL USERS
The quality and completeness of this reproduction is dependent on the quality
and completeness of the copy made available to ProQuest.

Distributed by
ProQuest LLC a part of Clarivate (2026).
Copyright of the Dissertation is held by the Author unless otherwise noted.

This work is protected against unauthorized copying under Title 17,
United States Code and other applicable copyright laws.

This work may be used in accordance with the terms of the Creative Commons license
or other rights statement, as indicated in the copyright statement or in the metadata
associated with this work. Unless otherwise specified in the copyright statement
or the metadata, all rights are reserved by the copyright holder.

ProQuest LLC
789 East Eisenhower Parkway
Ann Arbor, MI 48108 USA